

Arrays & Matrices

1. Two Sum

A top-of-the-band L5 or L6 engineer at Google doesn't just jump straight into writing the optimal code. At that level, the interview is an evaluation of clear communication, your ability to articulate tradeoffs, and your foresight into how an algorithm scales in real-world systems.

Here is exactly how a senior engineer would break down and solve "Two Sum" in an interview setting.

1. Problem explanation

The Prompt: Given an array of integers `nums` and an integer `target`, return the indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice.

In Simple Terms: We have a list of numbers. We need to find exactly two numbers in that list that add up to a specific goal number (the target). Once we find them, we return their positions (indices) in the list.

Constraints to Clarify with the Interviewer:

- Are there negative numbers? (Yes, usually).
- Can the array be modified? (Usually yes, but we should avoid it if possible).
- Is the array sorted? (No, if it were, we could use a different approach).

2. Solution explanation

We will discuss two approaches: the baseline Brute Force method, and the optimal Hash Map method. An L5 engineer always states the brute force first to establish a baseline before optimizing.

Approach A: The Brute Force (Baseline) The most intuitive way is to check every single pair of numbers in the array. We take the first number, add it to the second, the third, the fourth, and so on. Then we take the second number and add it to the third, the fourth, etc., until we hit the target.

Approach B: The One-Pass Hash Map (Optimal) We can solve this in a single sweep. As we iterate through the list, for every number we encounter, we can calculate its "complement" — the number needed to reach the target.

$\text{Complement} = \text{Target} - \text{Current Number}$

If we can remember the numbers we've seen so far, we can just look back and ask, "Have I already seen the complement?" To do this efficiently, we use a

Hash Map (a Dictionary in Python or an Object/Map in JavaScript) to store the numbers we've seen and their exact index.

Visualizing the Hash Map Approach (ASCII Diagram): Let's trace an example: `nums = [2, 7, 11, 15], target = 9`

Initial State:

Array: `[2, 7, 11, 15]`

Target: 9

Memory (Hash Map): `{}` <-- Empty to start

Step 1:

Pointer at index 0. Current Value = 2

What do we need? (Target 9 - Value 2) = 7

Is 7 in our Memory?

No.

Action: Save current value and its index into Memory.

Memory State:

```
+-----+-----+
| Value | Index |
+-----+-----+
|   2   |   0   |
+-----+-----+
```

Step 2:

Pointer at index 1. Current Value = 7

What do we need? (Target 9 - Value 7) = 2

Is 2 in our Memory?

Yes! We look at our memory and see 2 is at index 0.

Action: We found our pair!

Return: `[0, 1]` (The index of 2 from memory, and the current index)

3. Time and Space complexity analysis

Here is the breakdown of why the Hash Map is better, derived visually.

Derivation Diagram:

Variables: `n` = number of elements in the array

```
+-----+
| APPROACH A: BRUTE FORCE |
```

```

+=====+
| Outer Loop runs 'n' times. |
| Inner Loop runs 'n' times for each outer loop iteration. |
| |
| Operations:  $n * n = n^2$  |
| TIME COMPLEXITY:  $O(n^2)$  |
| |
| We don't store anything extra, just a couple of variables for the pointers. |
| SPACE COMPLEXITY:  $O(1)$  (Constant space) |
+=====+

+=====+
| APPROACH B: ONE-PASS HASH MAP |
+=====+
| Array Iteration: |
| [ Item 0 ] -> [ Item 1 ] -> ... -> [ Item n-1 ] |
| | | |
| Look up in Look up in Look up in |
| Map:  $O(1)$  Map:  $O(1)$  Map:  $O(1)$  |
| |
| Total Time: n iterations *  $O(1)$  lookup time =  $O(n)$  |
| TIME COMPLEXITY:  $O(n)$  |
| |
| In the absolute worst-case scenario (the pair is at the very end of the |
| array), we have to store almost all 'n' elements in our Hash Map memory. |
| SPACE COMPLEXITY:  $O(n)$  (Linear space) |
+=====+

```

4. Solution Code

Here are both the Brute Force (good to know) and the Optimized solutions in both Python and JavaScript.

Python Solutions

```

# APPROACH A: BRUTE FORCE (Not optimal, but establishes baseline)
def twoSum_brute_force(nums, target):
    n = len(nums)
    # Loop through each element
    for i in range(n):
        # Loop through the rest of the array after element 'i'
        for j in range(i + 1, n):
            # Check if they add up to the target
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

```

```

# -----

# APPROACH B: OPTIMIZED ONE-PASS HASH MAP (L5/L6 Expectation)
def twoSum_optimized(nums, target):
    # This dictionary will map: { array_value : array_index }
    seen_numbers = {}

    # Enumerate gives us both the index (i) and the value (num) as we loop
    for i, num in enumerate(nums):
        complement = target - num

        # O(1) check to see if we've encountered the complement before
        if complement in seen_numbers:
            # If yes, return the index of the complement and the current index
            return [seen_numbers[complement], i]

        # If not, store the current number and its index in the dictionary
        seen_numbers[num] = i

    return []

```

JavaScript Solutions

```

// APPROACH A: BRUTE FORCE (Not optimal, but establishes baseline)
function twoSumBruteForce(nums, target) {
    const n = nums.length;
    // Loop through each element
    for (let i = 0; i < n; i++) {
        // Loop through the rest of the array after element 'i'
        for (let j = i + 1; j < n; j++) {
            // Check if they add up to the target
            if (nums[i] + nums[j] === target) {
                return [i, j];
            }
        }
    }
    return [];
}

// -----

// APPROACH B: OPTIMIZED ONE-PASS HASH MAP (L5/L6 Expectation)
function twoSumOptimized(nums, target) {
    // This Map will store key-value pairs of: array_value => array_index
    const seenNumbers = new Map();

```

```

for (let i = 0; i < nums.length; i++) {
  const currentNum = nums[i];
  const complement = target - currentNum;

  // O(1) check to see if we've encountered the complement before
  if (seenNumbers.has(complement)) {
    // If yes, return the index of the complement and the current index
    return [seenNumbers.get(complement), i];
  }

  // If not, store the current number and its index in the Map
  seenNumbers.set(currentNum, i);
}

return [];
}

```

Note 1: Terms and Techniques Used

Hash Map (Dictionary in Python / Map in JS): A data structure that stores data in key-value pairs. Think of it like a real-life dictionary: you have a word (the key) and you instantly flip to its definition (the value) without having to read the book from page 1.

- **Why it helps here:** Looking up whether a number exists in an array takes $O(n)$ time because you have to check every slot. Looking up whether a key exists in a Hash Map takes $O(1)$ (instant) time. By trading space (memory to store the map) for time, we turn an $O(n^2)$ sluggish algorithm into a lightning-fast $O(n)$ algorithm.

Note 2: Real-World & Interview Variations at Top Tech Companies

For a senior/staff level interview, the interviewer will often start with Two Sum and immediately pivot to a complex system design or constrained-environment variation.

1. Google: The Distributed Massive Dataset Variation

- **The Problem:** You have a dataset of 100 billion transactions spread across hundreds of servers. Find if any two transactions sum exactly to a specific fraud detection threshold (target).
- **The L5/L6 Solution:** You cannot load 100 billion records into a single Hash Map (it will run out of memory). You must use a MapReduce approach.

- *Map Phase:* You partition the data. A clever way is to route numbers to specific “Reducer” machines based on their value. For example, if the target is 1000, numbers 0-500 are sent to Machine A, and numbers 501-1000 are sent to Machine A as well.
- *Reduce Phase:* Each machine now has a manageable subset of data and its potential complements. The machine can run the standard $O(n)$ Hash Map Two Sum algorithm in memory.

2. Meta (Facebook): The Binary Search Tree Variation (Two Sum IV)

- **The Problem:** Instead of an array, you are given a Binary Search Tree (BST) and a target. Find if two elements in the BST add up to the target.
- **The L5/L6 Solution:** Since it’s a BST, we know it’s sorted. We can use a “Two Pointer” approach to save space. We create two iterators: a forward iterator starting at the smallest element (bottom left of tree) and a backward iterator starting at the largest element (bottom right).
- If `left_val + right_val == target`, we win.
- If the sum is too small, we move the left iterator forward.
- If the sum is too large, we move the right iterator backward.
- This gives an $O(n)$ time complexity and $O(h)$ space complexity (where h is the height of the tree), which is vastly superior to dumping the whole tree into an array first.

3. Bloomberg: The Real-Time Financial Stream Variation

- **The Problem:** Design a `TwoSum` class with two functions: `add(number)` which receives numbers from a real-time market data stream, and `find(target)` which instantly returns true if any two previously received numbers add up to the target.
- **The L5/L6 Solution:** This is an object-oriented design question disguised as an algorithm.
- We maintain a Hash Map of frequencies: `{ number : count }`.
- `add(number)`: Takes $O(1)$ time. Just increment the count of the number in the Hash Map.
- `find(target)`: Iterates through the keys in the Hash Map. For each key, `complement = target - key`. If `complement == key`, we check if the frequency count is > 1 . If `complement != key`, we just check if `complement` exists in the Hash Map. This takes $O(n)$ time relative to the number of *unique* numbers stored, making it highly efficient for continuous streaming data.

121. Best Time to Buy and Sell Stock

Here is how a top-of-the-band L5 (Senior) or L6 (Staff) engineer would approach and communicate this problem in an interview setting. At these levels, the expectation is not just to spit out the optimal code, but to demonstrate a clear

progression of thought, impeccable communication, consideration of trade-offs, and production-ready code with strong fundamentals.

1. Problem Explanation

We are given an array named `prices` where every element `prices[i]` represents the price of a specific stock on the i -th day.

The Goal: We need to calculate the maximum possible profit we can achieve by choosing exactly *one day* to buy the stock and choosing exactly *one day in the future* to sell it. If no profit can be made (e.g., the price goes down every single day), we must return 0.

Constraints & Edge Cases to Consider Early:

- **Chronology:** You cannot sell before you buy. The sell index j must be strictly greater than the buy index i (i.e., $j > i$).
- **Edge cases:** * The array has fewer than 2 elements (impossible to buy and sell, return 0).
- The prices continually decrease (e.g., `[5, 4, 3, 2, 1]`). Profit would be negative, so we choose not to buy at all, returning 0.

Example: Input: `prices = [7, 1, 5, 3, 6, 4]`

- If we buy on Day 2 (price = 1) and sell on Day 5 (price = 6), profit = $6 - 1 = 5$.
- Note: Buying on Day 2 and selling on Day 1 is not allowed.

2. Solution Explanation

A senior engineer will briefly mention the brute-force approach to establish a baseline before diving into the optimal solution.

Approach A: The Brute Force (Baseline) The most naive way to solve this is to check every possible combination of buying and selling days. We would use a nested loop: the outer loop picks a buy day, and the inner loop checks every subsequent day as a potential sell day. We keep track of the maximum profit seen.

While correct, this does a lot of redundant work. If we have 100,000 days of data, calculating every pair will result in billions of operations, which is too slow.

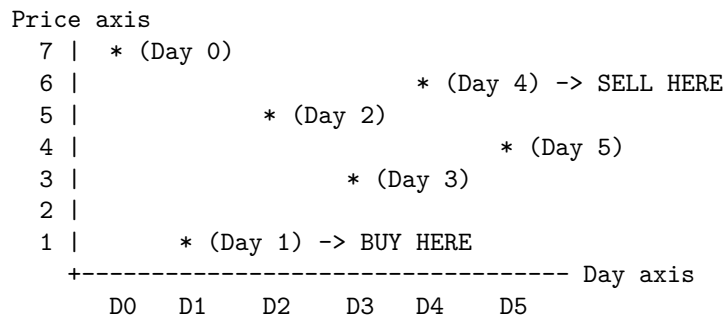
Approach B: The Optimal One-Pass (State Tracking) Instead of looking at pairs, let's flip our perspective. If we are standing on day j and we *must* sell today, how do we maximize our profit? Answer: We would need to have bought the stock at the absolute lowest price available on any day *prior* to day j .

Therefore, as we iterate through the array from left to right, we only need to keep track of two pieces of state:

1. **Lowest Price Seen So Far:** The minimum stock price encountered up to the current day.
2. **Maximum Profit Seen So Far:** The largest difference between the current day's price and the "Lowest Price Seen So Far".

Let's visualize this with an ASCII chart and a step-by-step state breakdown.

ASCII Visualization of the Array: [7, 1, 5, 3, 6, 4]



Step-by-Step State Execution: Initialize variables: `min_price = Infinity`
`max_profit = 0`

Looping through prices:

Day 0: Price = 7

-> Is 7 < `min_price` (Inf)? Yes. `min_price` becomes 7.
-> Profit if sold today: 7 - 7 = 0.
-> `max_profit` remains 0.
State: [`min_price`: 7, `max_profit`: 0]

Day 1: Price = 1

-> Is 1 < `min_price` (7)? Yes. `min_price` becomes 1.
-> Profit if sold today: 1 - 1 = 0.
-> `max_profit` remains 0.
State: [`min_price`: 1, `max_profit`: 0]

Day 2: Price = 5

-> Is 5 < `min_price` (1)? No. `min_price` stays 1.
-> Profit if sold today: 5 - `min_price`(1) = 4.
-> Is 4 > `max_profit` (0)? Yes. `max_profit` becomes 4.
State: [`min_price`: 1, `max_profit`: 4]

Day 3: Price = 3

-> Is 3 < `min_price` (1)? No. `min_price` stays 1.
-> Profit if sold today: 3 - `min_price`(1) = 2.

-> Is 2 > max_profit (4)? No. max_profit stays 4.
 State: [min_price: 1, max_profit: 4]

Day 4: Price = 6
 -> Is 6 < min_price (1)? No. min_price stays 1.
 -> Profit if sold today: 6 - min_price(1) = 5.
 -> Is 5 > max_profit (4)? Yes. max_profit becomes 5.
 State: [min_price: 1, max_profit: 5]

Day 5: Price = 4
 -> Is 4 < min_price (1)? No. min_price stays 1.
 -> Profit if sold today: 4 - min_price(1) = 3.
 -> Is 3 > max_profit (5)? No. max_profit stays 5.
 State: [min_price: 1, max_profit: 5]

Final Output: max_profit = 5

3. Time and Space Complexity Analysis

Brute Force Complexity:

- **Time:** $O(N^2)$ where N is the length of the prices array.
- **Space:** $O(1)$

Optimal Solution Complexity Derivation:

Time Complexity Analysis:

=====

Input: Array of N elements.

[Start Iteration]

```

      |
      v
+-----+
| For each element (N times): | -> N
|   1. min(min_price, current) | -> O(1) operations
|   2. current - min_price      | -> O(1) operations
|   3. max(max_profit, diff)   | -> O(1) operations
+-----+

```

```

      |
      v
Total Time = N steps * (O(1) work per step)
Total Time = O(N)

```

Space Complexity Analysis:

=====

Variables stored in memory:

1. min_price (integer/float) -> $O(1)$ space
2. max_profit (integer) -> $O(1)$ space
3. loop iterator 'i' -> $O(1)$ space

Total extra space required does not grow with input size N.
Total Space = $O(1)$

4. Solution Code

Here are both the Brute Force (for comprehensive interview communication) and Optimal implementations in Python and JavaScript.

Python Snippets

```
import math

# Approach 1: Brute Force (Not recommended for production, used to establish baseline)
def maxProfit_bruteForce(prices: list[int]) -> int:
    """
    Evaluates every possible buy and sell pair to find the maximum profit.
    Time Complexity:  $O(N^2)$ 
    Space Complexity:  $O(1)$ 
    """
    n = len(prices)
    max_profit = 0

    for i in range(n):
        for j in range(i + 1, n):
            current_profit = prices[j] - prices[i]
            if current_profit > max_profit:
                max_profit = current_profit

    return max_profit

# Approach 2: Optimal One-Pass (The expected interview solution)
def maxProfit_optimal(prices: list[int]) -> int:
    """
    Calculates the max profit by keeping track of the historical minimum
    price and the maximum profit possible at each step.
    Time Complexity:  $O(N)$ 
    Space Complexity:  $O(1)$ 
    """
    # Guard against empty or invalid inputs
    if not prices or len(prices) < 2:
        return 0
```

```

min_price = float('inf')
max_profit = 0

for current_price in prices:
    # Update the lowest price seen so far
    if current_price < min_price:
        min_price = current_price

    # Calculate profit if we sell today and update max_profit if it's the highest
    elif current_price - min_price > max_profit:
        max_profit = current_price - min_price

return max_profit

```

JavaScript Snippets

```

// Approach 1: Brute Force (Not recommended for production, used to establish baseline)
/**
 * Evaluates every possible buy and sell pair to find the maximum profit.
 * Time Complexity:  $O(N^2)$ 
 * Space Complexity:  $O(1)$ 
 */
function maxProfitBruteForce(prices) {
    let maxProfit = 0;
    const n = prices.length;

    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            const currentProfit = prices[j] - prices[i];
            if (currentProfit > maxProfit) {
                maxProfit = currentProfit;
            }
        }
    }

    return maxProfit;
}

// Approach 2: Optimal One-Pass (The expected interview solution)
/**
 * Calculates the max profit by keeping track of the historical minimum
 * price and the maximum profit possible at each step.
 * Time Complexity:  $O(N)$ 
 * Space Complexity:  $O(1)$ 
 */

```

```

function maxProfitOptimal(prices) {
    // Guard against empty or invalid inputs
    if (!prices || prices.length < 2) {
        return 0;
    }

    let minPrice = Infinity;
    let maxProfit = 0;

    for (let i = 0; i < prices.length; i++) {
        const currentPrice = prices[i];

        // Update the lowest price seen so far
        if (currentPrice < minPrice) {
            minPrice = currentPrice;
        }
        // Calculate profit if we sell today and update maxProfit if it's the highest
        else if (currentPrice - minPrice > maxProfit) {
            maxProfit = currentPrice - minPrice;
        }
    }

    return maxProfit;
}

```

Note 1: Underlying Techniques Used

1. Greedy Approach: This solution essentially uses a Greedy logic. At every step, we make a localized, greedy choice: “Record the absolute lowest price seen up to this exact moment.” By always knowing the cheapest historical buy point, we guarantee that the subtraction (`current_price - min_price`) yields the locally optimal profit for that specific day.

2. Dynamic Programming (State Reduction): You can also view this as a heavily space-optimized 1D Dynamic Programming problem.

- **State:** The state at day `i` depends on the minimum price from day 0 to `i-1`.
 - **Transition:** `DP_profit[i] = max(DP_profit[i-1], prices[i] - min_price_so_far)` Instead of keeping a whole DP array of size `N` (which would be $O(N)$ space), we realize we only ever need the *previous day's* maximum profit and the historical minimum price. We reduce the state to just two variables, achieving $O(1)$ space.
-

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

At the L5/L6 level, interviewers rarely ask this question in isolation. They will wrap it in real-world constraints or system design flavors.

Variation 1: The Data Stream API (Meta/Bloomberg)

- **The Prompt:** “Instead of an array, you are processing a live stream of prices from a ticker API. You have a class with a `process_next_price(price)` method. Return the maximum possible profit at any given moment.”
- **The Solution:** You cannot iterate backwards or use nested loops because data is arriving in real-time. The optimal array solution translates perfectly here. You maintain `self.min_price` and `self.max_profit` as class-level instance variables. Every time `process_next_price` is called, you execute the same conditional logic (`if price < self.min_price...`) in $O(1)$ time per tick.

Variation 2: Returning the Buy/Sell Indices (Google)

- **The Prompt:** “Knowing the profit is nice, but the trading desk needs to know *when* to execute the trades. Return the actual days (indices) you would buy and sell.”
- **The Solution:** You need to track indices alongside the values.
- Keep variables for `best_buy_day`, `best_sell_day`, and a temporary `current_potential_buy_day`.
- When you update `min_price`, you also update `current_potential_buy_day = i`.
- When you find a new `max_profit`, you update `best_buy_day = current_potential_buy_day` and `best_sell_day = i`.
- This retains $O(N)$ time and $O(1)$ space.

Variation 3: Maximum Drawdown (Bloomberg)

- **The Prompt:** “Risk management wants to know the worst possible historical loss. Find the maximum drawdown (the largest drop from a peak to a subsequent trough).”
- **The Solution:** This is the exact inverse of Best Time to Buy and Sell Stock. Instead of tracking the *minimum price seen so far*, you track the *maximum price seen so far* (the peak). As you iterate, you calculate the difference between the historical peak and the current price. You track the maximum of these differences.

217. Contains Duplicate

As an L5 or L6 engineer, the approach to a problem like “Contains Duplicate” isn’t just about finding *a* solution; it’s about evaluating trade-offs, understand-

ing the underlying data structures, and being ready to scale the solution for production environments.

Here is how I would break down, solve, and explain this problem in an interview setting.

1. Problem Explanation

The Core Question: Given an array of integers, does any value appear more than once?

- If we see any number at least twice, we return **true**.
- If every number is unique, we return **false**.

Examples & Visualizations:

Example A: Contains a Duplicate Input: `nums = [1, 2, 3, 1]` Output: `true`

Visualizing the scan:

```
[ 1 , 2 , 3 , 1 ]
      ^
```

First '1' seen. Keep going.

```
[ 1 , 2 , 3 , 1 ]
      ^
```

'2' seen. Keep going.

```
[ 1 , 2 , 3 , 1 ]
      ^
```

'3' seen. Keep going.

```
[ 1 , 2 , 3 , 1 ]
      ^
```

Wait, I've seen '1' before! -> Return TRUE immediately.

Example B: All Unique Input: `nums = [7, 8, 9, 10]` Output: `false`

Visualizing the scan:

```
[ 7 , 8 , 9 , 10 ] -> Scan 7 (new)
```

```
[ 7 , 8 , 9 , 10 ] -> Scan 8 (new)
```

```
[ 7 , 8 , 9 , 10 ] -> Scan 9 (new)
```

```
[ 7 , 8 , 9 , 10 ] -> Scan 10 (new)
```

End of list reached. No repeats -> Return FALSE.

2. Solution Explanation

When assessing this, a senior engineer looks at three primary approaches: Brute Force, Sorting, and Hashing. We discard Brute Force (comparing every number to every other number) immediately because it scales poorly. We will focus on **Sorting** (for memory-constrained environments) and **Hashing** (the optimal standard for speed).

Approach 1: The Sorting Method (Optimizing for Space) If we sort the array first, any duplicate numbers will be pushed right next to each other. We can then just walk through the array and check if the current number is the same as the next number.

ASCII Walkthrough (Sorting):
Unsorted Input: [4, 1, 2, 1]

Step 1: Sort the array
Sorted Output: [1, 1, 2, 4]

Step 2: Linear Scan for adjacent matches
[1, 1, 2, 4]
 ^__^
 1 == 1 ? YES -> Return True

Trade-off: We save space, but sorting takes extra time.

Approach 2: The Hash Set Method (Optimizing for Time) This is the gold standard for this problem. We use a “Set” data structure. A Set operates like a highly efficient VIP checklist. As we iterate through the array, we ask the Set: “Is this number already on the list?”

- If no, we add it to the Set and move on.
- If yes, we found our duplicate!

ASCII Walkthrough (Hash Set):
Input: [4, 5, 6, 4]
Set: { } (Empty initially)

Step 1: Current = 4
Is 4 in Set? No.
Add 4 to Set.
Set state: { 4 }

Step 2: Current = 5
Is 5 in Set? No.
Add 5 to Set.
Set state: { 4, 5 }

```

Step 3: Current = 6
Is 6 in Set? No.
Add 6 to Set.
Set state: { 4, 5, 6 }

Step 4: Current = 4
Is 4 in Set? YES!
Duplicate found -> Return True.

```

3. Time and Space Complexity Analysis

Here is the breakdown using standard algorithmic analysis.

For Approach 1: Sorting

- **Time Complexity:** $O(N \log N)$
- **Space Complexity:** $O(1)$ or $O(N)$ depending on the sorting algorithm the language uses under the hood (e.g., Python's Timsort uses $O(N)$ memory, whereas a true in-place Heapsort uses $O(1)$).

Time Complexity Derivation Diagram (Sorting)

```

-----
Operation 1: Sort Array      ==> Cost:  $N * \log(N)$  operations
Operation 2: Scan Array Once ==> Cost:  $N$  operations
-----

```

Total Time = $O(N \log N) + O(N)$.

The $N \log N$ dominates, so final Time Complexity = $O(N \log N)$

For Approach 2: Hash Set (The Optimal Approach)

- **Time Complexity:** $O(N)$
- **Space Complexity:** $O(N)$

Time & Space Complexity Derivation Diagram (Hash Set)

Input Size = N elements

TIME DERIVATION:

```

Array Elements:  [ Element1, Element2, ..., ElementN ]
                  |           |           |
Set Lookup Cost:   $O(1)$     +    $O(1)$     + ... +  $O(1)$     =  $O(N)$  total lookups
Set Insert Cost:   $O(1)$     +    $O(1)$     + ... +  $O(1)$     =  $O(N)$  total inserts
-----

```

Total Time = $O(N) + O(N) = O(N)$ overall time.

SPACE DERIVATION:

Worst case scenario: No duplicates exist.
We must store EVERY element in the Set.
Set Memory Growth: [1 item] -> [2 items] -> ... -> [N items]

Total Space = $O(N)$ memory required to hold the Set.

4. Solution Code

Here are both approaches implemented in Python and JavaScript.

Python 3

```
# Approach 1: Sorting (Useful if memory is strictly limited and in-place sorting is allowed.)
def containsDuplicate_sorting(nums: list[int]) -> bool:
    # Sort the array (Note: Python's sort is  $O(N \log N)$  time and  $O(N)$  space)
    nums.sort()

    # Iterate through the array up to the second-to-last element
    for i in range(len(nums) - 1):
        # If the current element matches the next one, we found a duplicate
        if nums[i] == nums[i + 1]:
            return True

    return False

# Approach 2: Hash Set (The Optimal Production Choice)
def containsDuplicate_hashset(nums: list[int]) -> bool:
    # Initialize an empty set to keep track of numbers we've seen
    seen = set()

    for num in nums:
        #  $O(1)$  average time complexity lookup
        if num in seen:
            return True
        #  $O(1)$  average time complexity insertion
        seen.add(num)

    return False

# Approach 2B: Pythonic One-Liner Hash Set
def containsDuplicate_oneliner(nums: list[int]) -> bool:
    # A set automatically removes duplicates.
    # If the length of the set is smaller than the array, duplicates existed.
    return len(set(nums)) != len(nums)
```

JavaScript

```
// Approach 1: Sorting
function containsDuplicateSorting(nums) {
    // Sort numerically in ascending order (O(N log N) time)
    nums.sort((a, b) => a - b);

    // Check adjacent elements
    for (let i = 0; i < nums.length - 1; i++) {
        if (nums[i] === nums[i + 1]) {
            return true;
        }
    }
    return false;
}

// Approach 2: Hash Set (The Optimal Production Choice)
function containsDuplicateHashSet(nums) {
    // Initialize a new Set object
    const seen = new Set();

    // Loop through each number in the array
    for (const num of nums) {
        // Set.has() provides O(1) lookup time
        if (seen.has(num)) {
            return true;
        }
        // Add the number to our tracked set
        seen.add(num);
    }

    return false;
}
```

Note 1: Terminology & Techniques Used

- **Algorithm/Data Structure:** Hash Set (backed by a Hash Table).
- **What it is:** A collection of unique items. Under the hood, it uses a mathematical “hash function” to convert the item (like the number 42) into a specific memory address index.
- **Why it helps:** Because the hash function tells the computer *exactly* where the item should live in memory, checking if an item exists takes $O(1)$ (constant) time. You don’t have to scan through a list; you just check that one specific memory slot.
- **How it applies here:** By reducing the “have I seen this number?” ques-

tion from an $O(N)$ list-scan down to an $O(1)$ instant-check, we reduce the entire algorithm's time complexity from $O(N^2)$ to $O(N)$.

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

At the senior/staff level, the basic LeetCode question is just a warm-up. Interviewers will quickly pivot to scale, distributed systems, or specialized constraints.

1. Google: Distributed “Contains Duplicate”

- **The Scenario:** The array is 500 Terabytes of user log IDs. It cannot fit on one machine's memory or disk. How do you find if there is a duplicate?
- **The L5/L6 Solution:** * *Approach A (MapReduce):* We hash each ID. We use a modulo operator based on the number of servers we have (e.g., $\text{hash}(\text{ID}) \% 100$). This ensures that if duplicate IDs exist, they are routed to the *exact same server*. Each individual server then runs a local Hash Set in memory (or on disk via external sorting) to find duplicates.
- *Approach B (Bloom Filter):* If the interviewer says, “We just need to know if a duplicate *probably* exists, and we have almost zero memory,” I would implement a Bloom Filter. It uses multiple hash functions to set bits in a tiny bit-array. It guarantees no false negatives (if it says “no duplicates”, it's 100% right), but might have false positives.

2. Meta: Contains Duplicate II (Proximity Constraint)

- **The Scenario:** Given an array `nums` and an integer `k`, return true if there are two *duplicate numbers* that are located within `k` distance of each other in the array.
- **The L5/L6 Solution:** Sliding Window + Hash Set. We maintain a Hash Set, but we cap its size at `k`. As we iterate, we add items to the set. If the set's size exceeds `k`, we remove the oldest element (the one at index `i - k`). If we ever try to add a number that is already in our sliding window set, we return true. Time: $O(N)$, Space: $O(K)$.

3. Bloomberg: High-Frequency Ticker Duplicates (Streaming Data)

- **The Scenario:** You are receiving a real-time stream of millions of stock ticker symbols per second. Alert us if the same ticker is sent twice within a 5-second sliding time window.
- **The L5/L6 Solution:** Queue + Hash Set. We cannot just use a sliding window of size `K` because the window is based on *time*, not *count*.
- We use a Queue to store objects like `[TickerSymbol, Timestamp]`.
- We use a Hash Set to store just the `TickerSymbol` for $O(1)$ lookups.
- When a new tick arrives, we look at the front of the Queue. While the front item's timestamp is older than `CurrentTime - 5 seconds`, we pop

it from the Queue AND remove it from the Hash Set.

- Then, we check if the new ticker is in the Hash Set. If yes, trigger alert. If no, add to Queue and add to Hash Set.

238. Product of Array Except Self

Here is how a senior engineer (L5/L6) approaches “Product of Array Except Self.” At this level, the focus shifts from just getting the code to work, to deeply understanding the “why” behind the algorithmic pattern, optimizing for memory hierarchy (cache locality), and handling edge cases cleanly.

1. Problem Explanation

The Goal: You are given an array of integers, `nums`. You need to return a new array, `answer`, where `answer[i]` is the product of every number in `nums` *except* `nums[i]`.

The Catch: 1. You must do this in $O(N)$ time. 2. You cannot use the division operator (`/`).

Why the “no division” constraint? In the real world, division is computationally more expensive than multiplication. More importantly, division introduces the “Divide by Zero” exception. If the array contains a 0, the total product is 0. Dividing $0 / 0$ is undefined, which breaks the naive approach of calculating the total product and dividing by each element.

Example: Input: `nums = [1, 2, 3, 4]` Output: `answer = [24, 24, 8, 6]`

- For index 0: $2 * 3 * 4 = 24$
 - For index 1: $1 * 3 * 4 = 12$
 - For index 2: $1 * 2 * 4 = 8$
 - For index 3: $1 * 2 * 3 = 6$
-

2. Solution Explanation

To solve this without division, we need to think about what constitutes the “product except self.” For any given index `i`, the product of the rest of the array is simply: **(The product of all elements to the left of i) * (The product of all elements to the right of i)**

This is the non-trivial core of the problem: breaking a single exclusion into two independent inclusion groups (left and right).

Approach 1: The Intuitive Two-Array Method (Good for interviews, but not optimal space) We can precompute two arrays: `left_products` and `right_products`.

three different arrays in memory.

ASCII Visualization of Approach 2:

Input array: nums = [1, 2, 3, 4]

Step 1: Build left products directly into 'answer'
answer: [1, 1, 2, 6]

Step 2: Iterate backwards with a running 'right_product'
Initialize right_product = 1

Iteration 1 (i = 3):
answer[3] = answer[3] * right_product => 6 * 1 = 6
right_product = right_product * nums[3] => 1 * 4 = 4
State: answer = [1, 1, 2, 6]

Iteration 2 (i = 2):
answer[2] = answer[2] * right_product => 2 * 4 = 8
right_product = right_product * nums[2] => 4 * 3 = 12
State: answer = [1, 1, 8, 6]

Iteration 3 (i = 1):
answer[1] = answer[1] * right_product => 1 * 12 = 12
right_product = right_product * nums[1] => 12 * 2 = 24
State: answer = [1, 12, 8, 6]

Iteration 4 (i = 0):
answer[0] = answer[0] * right_product => 1 * 24 = 24
right_product = right_product * nums[0] => 24 * 1 = 24
State: answer = [24, 12, 8, 6] <-- Final Result!

3. Time and Space Complexity Analysis

Derivation via ASCII Table:

Operation	Time Cost	Space Cost
Array Initialization	$O(N)$	$O(N)$ (Output)
Left-to-Right Pass	$O(N)$	$O(1)$ extra
Right-to-Left Pass	$O(N)$	$O(1)$ extra
TOTAL COMPLEXITY	$O(N) + O(N)$	Output Array
	$= O(N)$ Time	$= O(1)$ Aux Space

- **Time Complexity: $O(N)$.** We iterate through the array of size N exactly twice (once forward, once backward). $2N$ reduces to N .
 - **Space Complexity: $O(1)$ (Auxiliary).** The problem statement specifies that the output array does not count towards space complexity. We only use a single integer variable (`right_product`) for the second pass.
-

4. Solution Code

Python

Approach 1: Non-Optimized ($O(N)$ Space) - Good for explaining the concept

```
def productExceptSelf_TwoArrays(nums):
    n = len(nums)
    left_products = [0] * n
    right_products = [0] * n
    answer = [0] * n

    # Build left products
    left_products[0] = 1
    for i in range(1, n):
        left_products[i] = nums[i - 1] * left_products[i - 1]

    # Build right products
    right_products[n - 1] = 1
    for i in reversed(range(n - 1)):
        right_products[i] = nums[i + 1] * right_products[i + 1]

    # Multiply left and right
    for i in range(n):
        answer[i] = left_products[i] * right_products[i]

    return answer
```

Approach 2: Optimized ($O(1)$ Auxiliary Space) - The L5/L6 Standard

```
def productExceptSelf_Optimized(nums):
    n = len(nums)
    answer = [0] * n

    # Step 1: Compute left products directly into the answer array
    # answer[i] will hold the product of all elements to the left of i
    answer[0] = 1
    for i in range(1, n):
        answer[i] = nums[i - 1] * answer[i - 1]

    # Step 2: Compute right products on the fly and multiply into answer
```

```

# right_running_product keeps track of the product of elements to the right
right_running_product = 1
for i in reversed(range(n)):
    # Multiply the pre-calculated left product with the running right product
    answer[i] = answer[i] * right_running_product
    # Update the running right product for the next iteration (which is i-1)
    right_running_product = right_running_product * nums[i]

return answer

```

JavaScript

```

// Approach 1: Non-Optimized (O(N) Space)
function productExceptSelf_TwoArrays(nums) {
    const n = nums.length;
    const left_products = new Array(n);
    const right_products = new Array(n);
    const answer = new Array(n);

    left_products[0] = 1;
    for (let i = 1; i < n; i++) {
        left_products[i] = nums[i - 1] * left_products[i - 1];
    }

    right_products[n - 1] = 1;
    for (let i = n - 2; i >= 0; i--) {
        right_products[i] = nums[i + 1] * right_products[i + 1];
    }

    for (let i = 0; i < n; i++) {
        answer[i] = left_products[i] * right_products[i];
    }

    return answer;
}

// Approach 2: Optimized (O(1) Auxiliary Space) - The L5/L6 Standard
function productExceptSelf_Optimized(nums) {
    const n = nums.length;
    const answer = new Array(n);

    // Step 1: Compute left products directly into the answer array
    answer[0] = 1;
    for (let i = 1; i < n; i++) {
        answer[i] = nums[i - 1] * answer[i - 1];
    }
}

```



```

// Step 2: Compute right products on the fly
let right_running_product = 1;
for (let i = n - 1; i >= 0; i--) {
    // answer[i] currently holds the left product.
    // Multiply it by the right product.
    answer[i] = answer[i] * right_running_product;
    // Update right product for the next step leftward
    right_running_product = right_running_product * nums[i];
}

return answer;
}

```

Note 1: Foundational Techniques Used

Prefix / Suffix Precomputation Pattern: This algorithm relies heavily on the “Prefix Array” (or Prefix Sum/Product) pattern.

- **What it is:** It is a technique where you create an array that stores the cumulative result of an operation (like sum, product, or XOR) from the start of the array up to each index.
 - **Why it helps:** It reduces the time complexity of range queries from $O(N)$ to $O(1)$. If you need to know the product of a specific sub-segment of an array repeatedly, precomputing the prefix means you only do the heavy lifting once.
 - **Application here:** By precomputing the “prefix” (everything to the left) and the “suffix” (everything to the right), we avoid an $O(N^2)$ nested loop approach.
-

Note 2: Real World / Interview Variations at Top Tech Companies

1. Bloomberg: The “Division is Allowed, but deal with Zeroes” variation

- **The Problem:** You are given the same array, and division IS allowed, but you must handle edge cases gracefully and efficiently.
- **How to solve it:** You calculate the total product of the array while simultaneously counting the number of zeroes.
- *Case A: Zero count is 0.* Simply iterate through `nums` and set `answer[i] = total_product / nums[i]`.
- *Case B: Zero count is exactly 1.* For the element that is 0, its answer is the total product of all *other* (non-zero) elements. For every other

element, the answer is 0 (because multiplying by that single zero nullifies everything).

- *Case C: Zero count is > 1 .* The answer array is entirely 0s. If you remove any one element, there is still at least one other 0 left in the array, making every product 0.

2. Google: The Graph/Tree Sub-product variation (Telemetry / Factor impact)

- **The Problem:** Instead of an array, you are given a tree structure representing microservices. Each service returns a reliability multiplier (e.g., 0.99). You want to know the total reliability of the system if node X is removed, for *every* node in the tree.
- **How to solve it:** This is a generalization of the prefix/suffix pattern to trees (often called “In-out Dynamic Programming on Trees”).

1. *First Pass (Bottom-Up):* Run a Depth-First Search (DFS) to compute the “subtree product” for every node (the product of all nodes below it, including itself).
2. *Second Pass (Top-Down):* Run a second DFS to pass down an “outside product”. When visiting a child, the “outside product” for that child is the parent’s outside product multiplied by the subtree products of all the *other* siblings. This effectively gives you the product of the whole tree excluding a specific node, running in $O(V)$ time where V is the number of vertices.

3. Meta: The Massive Scale “Modulo” Variation

- **The Problem:** The same problem as LC 238, but the array is extremely large and contains large integers. The product will cause integer overflow. You must return the answers modulo $10^9 + 7$.
- **How to solve it:** The prefix/suffix logic remains exactly the same. The key difference is that at *every single multiplication step*, you apply the modulo operator. $(A * B) \% M = ((A \% M) * (B \% M)) \% M$.
- *Follow up constraint:* If division *was* allowed in this scenario, you cannot just use standard division with modulo. You would have to use Modular Multiplicative Inverses (using Fermat’s Little Theorem) to “divide”, which requires computing $\text{num}^{(M-2)} \% M$. This is why the Prefix/Suffix approach is far superior here—it strictly uses multiplication, side-stepping the complex math required for modular division.

53. Maximum Subarray

When assessing candidates for senior or staff engineering roles, a question like “Maximum Subarray” isn’t just about getting the right answer. It is about how clearly you can articulate the transition from a naive approach to an optimal one, how well you handle edge cases (like an array of entirely negative numbers),

and whether you can identify the underlying pattern to apply it to more complex system variations.

Let's break this down the way a senior engineer would in a whiteboard or system design session.

1. Problem Explanation

The Core Task: You are given an array of integers (which can be positive, negative, or zero). Your goal is to find a contiguous sequence of numbers within that array that adds up to the largest possible sum, and return that sum.

Key Definitions:

- **Subarray:** A contiguous, unbroken slice of the original array.
- *Example:* In [1, 2, 3], [1, 2] is a subarray. [1, 3] is NOT (it skips 2).
- **Subsequence:** Elements in the same relative order, but not necessarily contiguous. (Do not confuse subarray with subsequence!).

Why is this tricky? If all numbers were positive, the answer would just be the sum of the entire array. The negative numbers are what make this a puzzle. A negative number reduces your sum, but you might want to include it anyway if it acts as a “bridge” to an even larger positive number later on.

2. Solution Explanation

We will look at two approaches: an optimized brute-force approach to establish our baseline, and the optimal approach (Kadane's Algorithm).

Approach A: The Baseline (Optimized Brute Force) Before jumping to the trick, we must establish the baseline. We can find all possible subarrays, calculate their sums, and keep the highest one.

- Pick a starting index i .
- Pick an ending index j .
- Sum the elements from i to j .

If we dynamically keep track of the sum as we expand j , we avoid doing a third loop, taking this from a terrible $O(N^3)$ to a passable $O(N^2)$.

Approach B: The Optimal Solution (Kadane's Algorithm) Kadane's algorithm is built on a very simple, greedy realization: **A negative prefix is never worth keeping.** Imagine you are walking through the array accumulating a sum.

- If your current running sum drops below zero, it becomes a liability. Any future number you add it to will be dragged down.
- Therefore, if your running sum becomes negative, you should drop it completely and start a new running sum from the very next element.

The Golden Rules of Kadane's:

1. **Current Sum:** At each step, decide: Do I add the current number to my existing running sum, or do I start a brand new sum with just the current number? (You start fresh if the old sum was dragging you down).
2. **Max Sum:** Always keep track of the highest "Current Sum" you've ever seen.

ASCII Visualization: Let's trace the array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Initialize:

Max Sum Seen So Far: -Infinity

Current Running Sum: 0

Step-by-Step Execution:

Index	Value	Action (Current Sum + Value) vs Value	Current Sum	Max Sum Seen
0	-2	(-2) vs -2 -> Start fresh or add? Current sum was 0, so $0 + -2 = -2$.	-2	-2
1	1	$(-2 + 1) = -1$ VS (1) -1 is worse than 1. DROP the past! Start new subarray here.	1	1
2	-3	$(1 + -3) = -2$ VS (-3) -2 is better than -3. Keep going.	-2	1
3	4	$(-2 + 4) = 2$ VS (4) 4 is better than 2. DROP the past! Start new subarray here.	4	4
4	-1	$(4 + -1) = 3$ VS (-1) 3 is better. It's a dip, but keep it in case a big number is next.	3	4
5	2	$(3 + 2) = 5$ VS (2) 5 is better. (The dip was worth it!)	5	5
6	1	$(5 + 1) = 6$ VS (1) 6 is better.	6	6

7		-5		(6 + -5) = 1	VS	(-5)		1		6	
				1 is better. Another dip.							
8		4		(1 + 4) = 5	VS	(4)		5		6	

Final Output: 6 (The subarray was [4, -1, 2, 1])

3. Time and Space Complexity Analysis

Time Complexity Derivation:

Array: [A, B, C, D, E, F]

| | | | | |
 v v v v v v

Steps: 1 2 3 4 5 6 -> Total operations = N

Time Complexity = $O(N)$ where N is the length of the array.

We process each element in the array exactly once. We do a constant time check (addition and comparison) at each step.

Space Complexity Derivation:

Memory Blocks Used:

```

+-----+-----+
| Variable 1      | Variable 2      |
| current_sum     | max_sum         |
+-----+-----+

```

| Overwritten repeatedly (No array allocations)

Space Complexity = $O(1)$

We only need two variables to keep track of our state (`current_sum` and `max_sum`), regardless of how large the input array gets.

4. Solution Code

Python Snippets

```

# APPROACH 1: OPTIMIZED BRUTE FORCE ( $O(N^2)$  Time,  $O(1)$  Space)
# Useful to demonstrate you understand the fundamental combinations
def maxSubArray_brute(nums):
    max_sum = float('-inf')
    n = len(nums)

    for i in range(n):

```

```

        current_sum = 0
        for j in range(i, n):
            current_sum += nums[j]
            if current_sum > max_sum:
                max_sum = current_sum

    return max_sum

# APPROACH 2: KADANE'S ALGORITHM (O(N) Time, O(1) Space)
# The industry standard optimal solution
def maxSubArray_kadane(nums):
    # Initialize both to the first element to handle arrays with all negative numbers properly
    max_so_far = nums[0]
    current_sum = nums[0]

    # Iterate through the array starting from the second element
    for i in range(1, len(nums)):
        num = nums[i]

        # Core Kadane's logic:
        # Does the past dragging me down?
        # If current_sum + num is less than num itself, drop the past!
        if current_sum + num > num:
            current_sum = current_sum + num
        else:
            current_sum = num

        # Update the global maximum if our current streak is the best we've seen
        if current_sum > max_so_far:
            max_so_far = current_sum

    return max_so_far

```

JavaScript Snippets

```

// APPROACH 1: OPTIMIZED BRUTE FORCE (O(N^2) Time, O(1) Space)
function maxSubArrayBrute(nums) {
    let maxSum = -Infinity;
    const n = nums.length;

    for (let i = 0; i < n; i++) {
        let currentSum = 0;
        for (let j = i; j < n; j++) {
            currentSum += nums[j];
            if (currentSum > maxSum) {

```

```

        maxSum = currentSum;
    }
}

return maxSum;
}

// APPROACH 2: KADANE'S ALGORITHM (O(N) Time, O(1) Space)
function maxSubArrayKadane(nums) {
    let maxSoFar = nums[0];
    let currentSum = nums[0];

    for (let i = 1; i < nums.length; i++) {
        let num = nums[i];

        // Math.max is a cleaner way to write the logic:
        // "Start fresh from 'num', or append 'num' to the running sum"
        currentSum = Math.max(num, currentSum + num);
        maxSoFar = Math.max(maxSoFar, currentSum);
    }

    return maxSoFar;
}

```

Note 1: Terms and Techniques

Kadane's Algorithm:

- **What it is:** A dynamic programming approach that operates greedily. It solves the maximum subarray problem by breaking it down into smaller subproblems: "What is the maximum subarray ending at index i ?"
- **Why it helps:** It prevents recalculating the sums of subarrays repeatedly. By utilizing the answer from step $i - 1$, we can find the answer for step i in $O(1)$ time.
- **Application here:** Instead of looking at all combinations, we only make one decision at each index—extend the previous subarray, or start a new one.

Note 2: Real-World Interview Variations at Top Tech Companies

When targeting senior roles, interviewers at places like Google or Meta will rarely ask the basic Maximum Subarray problem directly. They use it as a building block for harder variations.

1. Google Variation: Maximum Circular Subarray (LeetCode 918)

- **The Problem:** What if the array wraps around? e.g., [5, -3, 5] -> the subarray could be the last 5 connecting to the first 5 for a sum of 10.
- **How to solve it:** This is a brilliant test of whether you actually understand Kadane's.
 1. Run standard Kadane's to find the normal **Max Sum**.
 2. Run Kadane's again, but inverted, to find the **Min Sum** (the most negative subarray).
 3. The circular maximum is the **Total Sum of Array** minus the **Min Sum** (effectively removing the worst part of the array to leave the wrapped-around best part).
 4. Return the maximum of either the normal **Max Sum** or the wrapped circular sum.

2. Meta Variation: Return the Indices of the Maximum Subarray

- **The Problem:** Instead of just returning 6, return the indices [3, 6] that correspond to the start and end of that maximum subarray.
- **How to solve it:** Modify Kadane's to carry state.
- Maintain variables `temp_start`, `best_start`, and `best_end`.
- Whenever `current_sum` resets (i.e., you decide to start fresh), update `temp_start = current_index`.
- Whenever `current_sum > max_so_far`, update `best_start = temp_start` and `best_end = current_index`.

3. Bloomberg Variation: Best Time to Buy and Sell Stock (LeetCode 121)

- **The Problem:** Given an array of stock prices, find the maximum profit you can achieve from one buy and one sell.
- **How to solve it:** This is essentially Kadane's algorithm in disguise. If you convert the array of *prices* into an array of *daily price differences* (e.g., Prices [7, 1, 5, 3] -> Differences [-6, 4, -2]), finding the max profit is exactly the same as finding the maximum contiguous subarray of those differences. (Though typically, keeping track of `min_price_seen` is an easier $O(N)$ way to code this specific problem, understanding the mathematical parity with Kadane's is a massive green flag in an interview).

152. Maximum Product Subarray

Approaching a problem like “152. Maximum Product Subarray” at a senior or staff level (L5/L6) means going beyond just finding a solution that passes the tests. An L5/L6 engineer focuses heavily on state management, edge cases (like zeros and negative numbers), code readability, and communicating the “why” behind the chosen algorithm.

Here is how a top-of-the-band engineer would break down, explain, and solve this problem.

1. Problem Explanation

The Goal: Given an integer array `nums`, we need to find a contiguous non-empty subarray that yields the largest mathematical product.

Why is this non-trivial? If the array only contained positive numbers, the solution would be trivial: just multiply all the numbers together. However, the introduction of negative numbers and zeros fundamentally changes the problem's behavior.

- **The Zero Effect:** Multiplying by zero resets your running product to zero. A zero effectively acts as a wall, splitting the array into distinct sub-problems.
- **The Negative Effect:** Multiplying a positive running product by a negative number instantly turns it into a very small (negative) number. However, if you hit *another* negative number later, that highly negative product flips back into a highly positive product.

Example of the Negative Effect: Look at the array: [2, 3, -2, 4, -1]

- Subarray [2, 3] -> Product is 6 (Current Max)
- Subarray [2, 3, -2] -> Product is -12 (Suddenly very small!)
- Subarray [2, 3, -2, 4] -> Product is -48 (Even smaller)
- Subarray [2, 3, -2, 4, -1] -> Product is 48 (The two negatives cancel out, making a massive positive number!)

To solve this, we cannot just track the maximum product. We *must* also track the minimum (most negative) product, because a minimum product is just one negative multiplier away from becoming the new maximum product.

2. Solution Explanation

We will explore two approaches: the Brute Force (baseline) and the Optimized Dynamic Programming approach (the expected L5/L6 solution).

Approach A: Brute Force (Baseline) The simplest way to solve this is to evaluate every single contiguous subarray possible, calculate its product, and keep track of the highest one seen so far.

Array: [2, 3, -2, 4]

Subarrays starting at index 0:

```
[2]           = 2
[2, 3]        = 6    <-- Max so far
[2, 3, -2]    = -12
[2, 3, -2, 4] = -48
```

Subarrays starting at index 1:

```
[3]           = 3
[3, -2]       = -6
[3, -2, 4]    = -24
```

...and so on.

Approach B: Optimized State Tracking (Modified Kadane's) Instead of recalculating, we walk through the array exactly once (from left to right). At every single number, we ask ourselves: “What is the maximum product that *ends exactly at this current number?*” and “What is the minimum product that *ends exactly at this current number?*”

At any index *i*, the new max product and new min product can only come from three possibilities:

1. The current number itself (starting a fresh subarray).
2. The previous maximum product multiplied by the current number.
3. The previous minimum product multiplied by the current number (crucial if the current number is negative).

ASCII Execution Trace & Visualization:

Let's trace `nums = [2, 3, -2, 4]`

INITIAL STATE:

```
Array:      [ 2,  3, -2,  4 ]
              ^
```

```
Current:      2
```

Variables:

```
    curr_max = 2
    curr_min = 2
    global_max = 2
```

STEP 1: Move to 3

```
Array:      [ 2,  3, -2,  4 ]
              ^
```

```
Current:      3
```

Options to calculate new max/min:

- a) Current number: 3
- b) `curr_max * 3`: `2 * 3 = 6`
- c) `curr_min * 3`: `2 * 3 = 6`

New state:

```
curr_max = max(3, 6, 6) = 6
curr_min = min(3, 6, 6) = 3
global_max = max(2, 6) = 6
```

STEP 2: Move to -2 (The flip!)

Array: [2, 3, -2, 4]
 ^

Current: -2

Options to calculate new max/min:

- a) Current number: -2
- b) $\text{curr_max} * -2: 6 * -2 = -12$
- c) $\text{curr_min} * -2: 3 * -2 = -6$

New state:

```
curr_max = max(-2, -12, -6) = -2
curr_min = min(-2, -12, -6) = -12  <-- We keep this highly negative number!
global_max = max(6, -2) = 6
```

STEP 3: Move to 4

Array: [2, 3, -2, 4]
 ^

Current: 4

Options to calculate new max/min:

- a) Current number: 4
- b) $\text{curr_max} * 4: -2 * 4 = -8$
- c) $\text{curr_min} * 4: -12 * 4 = -48$

New state:

```
curr_max = max(4, -8, -48) = 4
curr_min = min(4, -8, -48) = -48
global_max = max(6, 4) = 6
```

FINAL RESULT: 6

3. Time and Space Complexity Analysis

Time Complexity: $O(N)$, where N is the length of the array. **Space Complexity:** $O(1)$, constant space.

Complexity Derivation Diagram:

TIME COMPLEXITY (TC) DERIVATION:

Array length = N elements.

We process each element exactly one time in a single forward pass.

```
[ e_0 ] -> [ e_1 ] -> [ e_2 ] -> ... -> [ e_{N-1} ]
      |         |         |                   |
      0(1)      0(1)      0(1)               0(1)  <-- Work per element
```

Total Time = $N * O(1) = O(N)$

SPACE COMPLEXITY (SC) DERIVATION:

Memory allocated outside of the input array:

Variable 1: curr_max (integer) -> $O(1)$

Variable 2: curr_min (integer) -> $O(1)$

Variable 3: global_max (integer) -> $O(1)$

Variable 4: temp_max (integer) -> $O(1)$

Total Space = $O(1) + O(1) + O(1) + O(1) = O(1)$

No arrays or scaling data structures are created.

4. Solution Code

Here are both the Brute Force ($O(N^2)$) and Optimized ($O(N)$) solutions in Python and JavaScript.

Python Implementation

```
class Solution:
    # -----
    # APPROACH 1: Brute Force (Good to mention, bad to submit)
    # TC:  $O(N^2)$  | SC:  $O(1)$ 
    # -----
    def maxProduct_brute_force(self, nums: list[int]) -> int:
        if not nums: return 0

        global_max = nums[0]

        for i in range(len(nums)):
            current_product = 1
            for j in range(i, len(nums)):
                current_product *= nums[j]
                global_max = max(global_max, current_product)
```

```

        return global_max

# -----
# APPROACH 2: Optimized State Tracking (L5/L6 Standard)
# TC: O(N) | SC: O(1)
# -----
def maxProduct(self, nums: list[int]) -> int:
    if not nums: return 0

    # Initialize max, min, and the global result with the first element.
    curr_max = nums[0]
    curr_min = nums[0]
    global_max = nums[0]

    # Iterate starting from the second element
    for i in range(1, len(nums)):
        num = nums[i]

        # Since we will overwrite curr_max, we need to store it temporarily
        # to use it when calculating the new curr_min.
        temp_max = curr_max * num

        # The new max is either:
        # 1. The current number itself (starting over)
        # 2. The previous max * current number
        # 3. The previous min * current number (in case of double negatives)
        curr_max = max(num, temp_max, curr_min * num)

        # The new min follows the exact same logic.
        curr_min = min(num, temp_max, curr_min * num)

        # Update the global maximum found so far
        global_max = max(global_max, curr_max)

    return global_max

```

JavaScript Implementation

```

/**
 * @param {number[]} nums
 * @return {number}
 */

// -----
// APPROACH 1: Brute Force

```

```

// TC:  $O(N^2)$  | SC:  $O(1)$ 
// -----
var maxProductBruteForce = function(nums) {
  if (nums.length === 0) return 0;

  let globalMax = nums[0];

  for (let i = 0; i < nums.length; i++) {
    let currentProduct = 1;
    for (let j = i; j < nums.length; j++) {
      currentProduct *= nums[j];
      globalMax = Math.max(globalMax, currentProduct);
    }
  }

  return globalMax;
};

// -----
// APPROACH 2: Optimized State Tracking (L5/L6 Standard)
// TC:  $O(N)$  | SC:  $O(1)$ 
// -----
var maxProduct = function(nums) {
  if (nums.length === 0) return 0;

  let currMax = nums[0];
  let currMin = nums[0];
  let globalMax = nums[0];

  for (let i = 1; i < nums.length; i++) {
    let num = nums[i];

    // Temporarily hold the calculated max product before overwriting currMax
    let tempMax = currMax * num;

    // Calculate new boundaries dynamically
    currMax = Math.max(num, tempMax, currMin * num);
    currMin = Math.min(num, tempMax, currMin * num);

    // Update the absolute highest product seen
    globalMax = Math.max(globalMax, currMax);
  }

  return globalMax;
};

```

Note 1: Terminology and Techniques

Dynamic Programming (State Tracking): * **What it is:** Solving a complex problem by breaking it down into a collection of simpler subproblems, solving each of those just once, and storing their solutions.

- **How it applies:** Here, our “state” at any step i is simply the `curr_max` and `curr_min`. We use the state of step $i-1$ to calculate the state of step i . We don’t need a whole array to store previous states (which would be $O(N)$ space), just $O(1)$ variables.

Kadane’s Algorithm:

- **What it is:** A famous algorithm originally designed for the “Maximum Subarray Sum” problem. It looks at each element and asks: “Am I better off adding myself to the previous sum, or starting a new sum entirely with just myself?”
- **How it applies:** Our solution is a *modified* Kadane’s algorithm. Standard Kadane’s only tracks the maximum. Because we are dealing with multiplication (where negatives change everything), we modified the algorithm to track both the upper bound (max) and lower bound (min).

Note 2: Real-World / Interview Variations (Google, Meta, Bloomberg)

At the senior/staff level, interviewers rarely ask Leetcode 152 verbatim. They mask it as a real-world system or product requirement.

1. Bloomberg: The “Asset Growth Multiplier”

- **The Prompt:** “You are given an array of daily performance multipliers for a stock (e.g., 1.05 means a 5% gain, 0.98 means a 2% loss, -1.5 means an inverse leveraged drop). Find the contiguous period that yields the highest compound growth.”
- **How to solve:** This is identical to Maximum Product Subarray. The only difference is that the inputs are floating-point numbers instead of integers. The Modified Kadane’s $O(N)$ logic holds perfectly. You track the `max_compound` and `min_compound` at each day.

2. Meta: “Maximum Product Subarray with One Optional Deletion”

- **The Prompt:** “Find the maximum product of a contiguous subarray, but you are allowed to delete at most one element from the array to maximize the product.”
- **How to solve:** This tests your ability to use Prefix and Suffix arrays.

1. Do a forward pass (left-to-right) tracking the max product ending at `i`. Store this in an array `left_max[]`.
2. Do a backward pass (right-to-left) tracking the max product starting at `i`. Store this in `right_max[]`.
3. Iterate through the array. For any element at index `i` that you might delete, the max product if you delete `nums[i]` is simply `left_max[i-1] * right_max[i+1]`.
4. Keep the global maximum. TC: $O(N)$, SC: $O(N)$.

3. Google: “Max Product over a Sliding Window in a Data Stream”

- **The Prompt:** “You are receiving a stream of sensor readings (integers). At any given time, find the maximum product of a contiguous sequence, but the sequence cannot be longer than `K` elements.”
- **How to solve:** The $O(N)$ Kadane’s modification fails here because it assumes unbounded subarray lengths. To enforce a max length `K`, you need a Sliding Window approach combined with a Monotonic Queue or a Segment Tree. If the window grows beyond `K`, you must slide the left pointer and recalculate the boundaries inside the current window, which introduces significantly more state management overhead.

153. Find Minimum in Rotated Sorted Array

Here is how a top-of-the-band Senior (L5) or Staff (L6) engineer would break down and solve LeetCode 153. At this level, the focus is not just on writing the code, but on clearly communicating the insights, visualizing the state space, and handling edge cases systematically.

1. Problem Explanation

The Core Concept: What is a Rotated Sorted Array? Imagine taking a perfectly sorted array of distinct numbers and shifting all elements to the right by one position, taking the last element and wrapping it around to the front.

Original Array: [1, 2, 3, 4, 5, 6, 7] (Minimum is 1, at index 0)
 Rotated 1 time: [7, 1, 2, 3, 4, 5, 6] (Minimum is 1, at index 1)
 Rotated 3 times: [5, 6, 7, 1, 2, 3, 4] (Minimum is 1, at index 3)
 Rotated 7 times: [1, 2, 3, 4, 5, 6, 7] (Back to original, effectively 0 rotations)

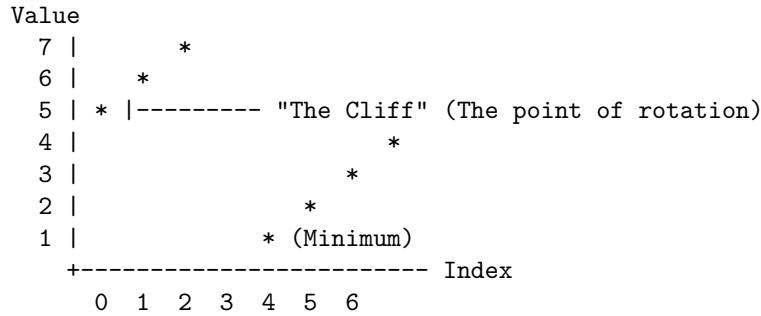
The Goal: Given one of these rotated arrays, find the minimum value. The brute force way is to check every single element. However, the problem strictly demands an algorithm that runs faster than checking every element, specifically in $O(\log N)$ time. This time complexity constraint is the classic signal that we must use **Binary Search**.

2. Solution Explanation

To achieve $O(\log N)$ time, we cannot look at every element. We must look at a middle element and definitively rule out half of the array.

A top-tier engineer will visualize the array not as a flat list, but as a graph of values plotted against their indices.

Visualizing “The Cliff” Let’s plot the array [5, 6, 7, 1, 2, 3, 4]:



Notice that a rotated sorted array is simply **two sorted segments**.

1. The left segment [5, 6, 7] consists of larger numbers.
2. The right segment [1, 2, 3, 4] consists of smaller numbers.
3. The minimum element is always the very first element of the right segment.

The Binary Search Logic: We use three pointers: `left`, `right`, and `mid`. How do we know which half to throw away? We compare the value at our `mid` pointer to the value at our `right` pointer.

- **Case A:** `nums[mid] > nums[right]`

Array: [5, 6, 7, 1, 2, 3, 4]

Indices: 0 1 2 3 4 5 6

Left=0, Right=6, Mid=3 (Value: 1... wait, let's pick a different mid to show this case)

Let's say Mid=1 (Value: 6), Right=6 (Value: 4)

6 > 4.

If the middle element is strictly greater than the rightmost element, the middle element **MUST** belong to the “left segment” (the higher numbers). The “cliff” and the minimum value *must* be somewhere to the right of `mid`. **Action:** Discard the left half and `mid` by setting `left = mid + 1`. • **Case B:** `nums[mid] <= nums[right]`

Array: [5, 6, 7, 1, 2, 3, 4]

Let's say Mid=5 (Value: 3), Right=6 (Value: 4)

3 <= 4.

If the middle element is less than or equal to the rightmost element, it means the elements from `mid` to `right` are perfectly sorted. Therefore, the “cliff” cannot

be between `mid` and `right`. The minimum is either at `mid` itself or somewhere to the left of `mid`. **Action:** Discard the right half by setting `right = mid`. (We keep `mid` in the search space because it could be the minimum).

The loop continues until `left` equals `right`, which will point directly to the minimum element.

3. Time and Space Complexity Analysis

Time Complexity: $O(\log N)$ Where N is the number of elements in the array.

Search Space Halving Derivation:

Step 1:	[. N elements]	Size = N
Step 2:	[. N/2 elements .]	Size = $N/2$
Step 3:	[N/4]	Size = $N/4$
	...	
Step k:	[1]	Size = $1 = N / (2^k)$

Because we divide the search space by 2 at every step, it takes k steps to reach 1 element.

$$N / (2^k) = 1$$

$$N = 2^k$$

$$k = \log_2(N)$$

Since we perform a constant number of operations (comparisons and assignments) at each step, the total time complexity is strictly bounded by the number of halves, which is $O(\log N)$.

Space Complexity: $O(1)$

Variables in memory:

[left] -> stores 1 integer

[right] -> stores 1 integer

[mid] -> stores 1 integer

Total extra memory = 3 integers, which is constant.

We do not create any new arrays or use the call stack for recursion. Thus, space complexity is $O(1)$.

4. Solution Code

Here are both the Brute Force (Linear Scan) and the Optimal (Binary Search) approaches. An L5/L6 engineer will often quickly state the brute force as a baseline before diving into the optimal solution.

Python Snippets

```
# =====
# APPROACH 1: Linear Scan (Brute Force)
# Useful for baseline comparison, but fails the  $O(\log N)$  requirement.
# =====
def findMin_brute_force(nums: list[int]) -> int:
    """
    Scans the entire array to find the minimum.
    Time:  $O(N)$ , Space:  $O(1)$ 
    """
    # Start by assuming the first element is the minimum
    current_min = nums[0]
    for num in nums:
        if num < current_min:
            current_min = num
    return current_min

# =====
# APPROACH 2: Binary Search (Optimal L5/L6)
# =====
def findMin_optimal(nums: list[int]) -> int:
    """
    Uses binary search by comparing the middle element with the rightmost element
    to determine which half of the array contains the 'cliff' (the minimum).
    Time:  $O(\log N)$ , Space:  $O(1)$ 
    """
    left = 0
    right = len(nums) - 1

    # We loop until left and right converge to a single index.
    # When left == right, we have isolated the minimum element.
    while left < right:
        # Calculate mid safely to prevent integer overflow
        # (more relevant in languages like Java/C++, but a good habit in Python)
        mid = left + (right - left) // 2

        # If mid is greater than the rightmost element, the cliff is to the right
        if nums[mid] > nums[right]:
```

```

        left = mid + 1
        # Otherwise, the right side is properly sorted, so the min is at mid or to its left
    else:
        right = mid

    # At the end of the loop, left == right, pointing to the minimum
    return nums[left]

```

JavaScript Snippets

```

// =====
// APPROACH 1: Linear Scan (Brute Force)
// =====
/**
 * Scans the entire array to find the minimum.
 * Time: O(N), Space: O(1)
 * @param {number[]} nums
 * @return {number}
 */
function findMinBruteForce(nums) {
    let currentMin = nums[0];
    for (let i = 1; i < nums.length; i++) {
        if (nums[i] < currentMin) {
            currentMin = nums[i];
        }
    }
    return currentMin;
}

// =====
// APPROACH 2: Binary Search (Optimal L5/L6)
// =====
/**
 * Uses binary search by comparing the middle element with the rightmost element
 * to isolate the segment containing the minimum value.
 * Time: O(log N), Space: O(1)
 * @param {number[]} nums
 * @return {number}
 */
function findMinOptimal(nums) {
    let left = 0;
    let right = nums.length - 1;

    while (left < right) {
        // Bitwise operator or Math.floor handles integer division in JS

```

```

    let mid = left + Math.floor((right - left) / 2);

    // If the middle element is strictly greater than the rightmost,
    // we are on the larger segment. The minimum is to the right.
    if (nums[mid] > nums[right]) {
        left = mid + 1;
    }
    // Otherwise, we are on the smaller segment (or the array isn't rotated).
    // The minimum is at mid or to its left.
    else {
        right = mid;
    }
}

// left and right point to the same element now
return nums[left];
}

```

Note 1: Algorithmic Concepts Used

Binary Search / Decrease and Conquer

- **What it is:** An algorithm that repeatedly divides the search interval in half. It is a subset of the “Decrease and Conquer” paradigm because at each step, we decrease the problem size by a fraction (in this case, by half) and conquer the remaining subproblem.
- **Why it helps:** It reduces the time complexity from scanning every element (linear time) to only checking a logarithmic number of elements.
- **How it applies here:** Even though the array is not perfectly sorted from index 0 to N-1, it retains enough structural sorting (two sorted halves) that comparing `mid` to `right` allows us to confidently discard half the elements at every step.

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

Companies rarely ask LeetCode questions verbatim at the Senior/Staff level. They frame them as system or domain problems.

1. Google: “The Rotated Server Logs”

- **The Problem:** You have a massive cluster log file consisting of timestamps. However, the system writes cyclically to an allocated block of disk space. When it reaches the end, it loops back to the beginning of the block. Given an array of these timestamps, find the oldest log entry.

- **The Solution:** This is exactly “Find Minimum in Rotated Sorted Array”. Timestamps continuously increase until the disk wraps around, creating a “cliff” where the newest timestamp sits right next to the oldest timestamp. You map the timestamps to the binary search logic above.

2. Bloomberg: “The Stock Ticker Reset”

- **The Problem:** A specific financial instrument’s price strictly increases throughout the day. At midnight, the price resets to a baseline lower value and starts increasing again. You are given a snapshot of price pings taken at regular intervals over exactly 24 hours, but you don’t know when the snapshot started. Find the lowest price of the day.
- **The Solution:** The array of prices is effectively rotated based on when the snapshot began. The “reset” is the rotation pivot. Apply the $O(\log N)$ binary search to find the minimum value, which corresponds to the reset price.

3. Meta: “Finding the Bug in a Deployed Service”

- **The Problem:** A service has multiple versions deployed across a ring of servers [S1, S2, S3... Sn]. Due to a rolling deployment strategy that wraps around the ring, the versions are sequentially increasing, but wrap around at some unknown server. For example, server versions might look like [v4, v5, v6, v1, v2, v3]. Find the server running the oldest version.
- **The Solution:** Again, a direct map to LeetCode 153. The array of servers is the rotated array. You poll the servers using binary search indices to find the server with the lowest version number in $O(\log N)$ network calls instead of $O(N)$ network calls, saving massive time in an emergency incident response.

33. Search in Rotated Sorted Array

This is a classic problem that separates junior engineers from senior engineers. A junior engineer tries to memorize the different **if/else** conditions. An L5 or L6 engineer approaches this by identifying the **core invariant**—the unchanging truth that holds no matter how the array is rotated.

Let’s break this down exactly how you would discuss it in a senior-level interview.

1. Problem Explanation

You are given an array of unique integers that was originally sorted in strictly increasing order. However, before it was given to you, it was “rotated” at an unknown pivot index.

Think of rotation like cutting a deck of cards and putting the bottom half on top.

Original: [0, 1, 2, 4, 5, 6, 7] **Rotated at index 3:** [4, 5, 6, 7, 0, 1, 2]

You need to find the index of a **target** value. If it's not there, return -1. The catch? You must do it in **$O(\log N)$** time. This time complexity constraint is a massive hint: you cannot look at every element (which would be $O(N)$). You *must* eliminate half of the remaining search space at every step. This demands a variation of Binary Search.

2. Solution Explanation

In a normal sorted array, Binary Search works because you look at the middle element, compare it to your target, and instantly know which half to throw away.

In a rotated array, the entire array isn't strictly sorted. However, the L5 insight is this: **If you slice a rotated sorted array perfectly in half, at least ONE of those halves will ALWAYS be perfectly, continuously sorted.**

Let's visualize the two possible realities when you pick a middle point.

Reality A: The Left Half is Sorted

Array: [4, 5, 6, 7, 8, 1, 2]
 $\hat{\quad}$ $\hat{\quad}$ $\hat{\quad}$
 Left Mid Right

Values: `nums[Left] = 4, nums[Mid] = 8`

Condition: `nums[Left] <= nums[Mid]` (`4 <= 8`) is TRUE.

- Because `4 <= 8`, we absolutely know the left side [4, 5, 6, 7, 8] is strictly increasing. The "cliff" (the rotation point) must be on the right side.
- **The Decision:** Since we know the exact bounds of the left side (from 4 to 8), we ask: *Is our target between 4 and 8?*
- If Target = 5: Yes, it's between 4 and 8. Throw away the right half.
- If Target = 2: No, it's not between 4 and 8. Throw away the left half.

Reality B: The Right Half is Sorted

Array: [7, 8, 1, 2, 3, 4, 5]
 $\hat{\quad}$ $\hat{\quad}$ $\hat{\quad}$
 Left Mid Right

Values: `nums[Left] = 7, nums[Mid] = 2`

Condition: `nums[Left] <= nums[Mid]` (`7 <= 2`) is FALSE.

- Because `7 <= 2` is false, the left side contains the "cliff". Therefore, the right side [2, 3, 4, 5] MUST be strictly increasing.

- **The Decision:** Since we know the exact bounds of the right side (from 2 to 5), we ask: *Is our target between 2 and 5?*
- If Target = 4: Yes, it's between 2 and 5. Throw away the left half.
- If Target = 8: No, it's not between 2 and 5. Throw away the right half.

By identifying the sorted half first, we can safely check if the target belongs in it.

3. Time and Space Complexity Analysis

Time Complexity: $O(\log N)$ We are halving the search space with every check. Here is the visual derivation of the time complexity:

```

Step 1: Search Space = N elements
===== (N)
      |
      Calculate Mid
      Discard half the elements
      |
Step 2: Search Space = N / 2 elements
===== (N/2)
      |
      Calculate Mid
      Discard half the elements
      |
Step 3: Search Space = N / 4 elements
===== (N/4)
      |
      ...
      |
Step K: Search Space = 1 element
===== (1)

```

The number of steps (K) it takes to reach 1 element is determined by the equation: $N / (2^K) = 1$. Solving for K gives us $K = \log_2(N)$. We drop the base for Big-O notation, giving us $O(\log N)$.

Space Complexity: $O(1)$ We are only allocating three integer variables for our pointers: `left`, `right`, and `mid`. No matter how large the input array gets, the memory used remains exactly the same.

4. Solution Code

As a senior engineer, it is great to present two approaches.

- **Approach 1 (Two-Pass):** Find the index of the smallest element first (the pivot). Then, do a standard binary search on either the left of the pivot or the right of the pivot. This is incredibly clean, easy to read, and less prone to off-by-one bugs in an interview setting.
- **Approach 2 (One-Pass):** The highly optimized version we discussed above, doing it all in a single loop.

Python Snippets

```
# =====
# Approach 1: Two-Pass (Find Pivot, then Search)
# =====
def search_two_pass(nums, target):
    if not nums: return -1

    # Helper to find the index of the minimum element (the rotation pivot)
    def find_pivot_index(left, right):
        while left < right:
            mid = left + (right - left) // 2
            # If mid element is greater than rightmost element,
            # the cliff is to the right of mid.
            if nums[mid] > nums[right]:
                left = mid + 1
            else:
                right = mid
        return left

    # Standard binary search helper
    def binary_search(left, right):
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target: return mid
            elif nums[mid] < target: left = mid + 1
            else: right = mid - 1
        return -1

    n = len(nums)
    pivot = find_pivot_index(0, n - 1)

    # Decide which half to perform the standard binary search on
    if pivot == 0: # Array wasn't actually rotated
        return binary_search(0, n - 1)
    elif target >= nums[0]: # Target is in the left, higher-valued segment
        return binary_search(0, pivot - 1)
    else: # Target is in the right, lower-valued segment
        return binary_search(pivot, n - 1)
```

```

# =====
# Approach 2: One-Pass Modified Binary Search
# =====
def search_one_pass(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Check if the LEFT half is sorted
        if nums[left] <= nums[mid]:
            # Is target within the bounds of this sorted left half?
            if nums[left] <= target < nums[mid]:
                right = mid - 1 # Target is here, discard right half
            else:
                left = mid + 1 # Target is NOT here, discard left half

        # Otherwise, the RIGHT half must be sorted
        else:
            # Is target within the bounds of this sorted right half?
            if nums[mid] < target <= nums[right]:
                left = mid + 1 # Target is here, discard left half
            else:
                right = mid - 1 # Target is NOT here, discard right half

    return -1

```

JavaScript Snippets

```

// =====
// Approach 1: Two-Pass (Find Pivot, then Search)
// =====
function searchTwoPass(nums, target) {
    if (!nums || nums.length === 0) return -1;

    // Helper to find the index of the minimum element (the rotation pivot)
    const findPivotIndex = (left, right) => {
        while (left < right) {
            const mid = left + Math.floor((right - left) / 2);
            if (nums[mid] > nums[right]) {
                left = mid + 1;
            }
        }
    }
}

```

```

        } else {
            right = mid;
        }
    }
    return left;
};

// Standard binary search helper
const binarySearch = (left, right) => {
    while (left <= right) {
        const mid = left + Math.floor((right - left) / 2);
        if (nums[mid] === target) return mid;
        if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
};

const n = nums.length;
const pivot = findPivotIndex(0, n - 1);

// Decide which segment to search
if (pivot === 0) {
    return binarySearch(0, n - 1);
} else if (target >= nums[0]) {
    return binarySearch(0, pivot - 1);
} else {
    return binarySearch(pivot, n - 1);
}
}

// =====
// Approach 2: One-Pass Modified Binary Search
// =====
function searchOnePass(nums, target) {
    let left = 0;
    let right = nums.length - 1;

    while (left <= right) {
        const mid = left + Math.floor((right - left) / 2);

        if (nums[mid] === target) return mid;

        // Check if the LEFT half is sorted
        if (nums[left] <= nums[mid]) {
            // Is target within the bounds of this sorted left half?

```

```

        if (nums[left] <= target && target < nums[mid]) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
        // Otherwise, the RIGHT half must be sorted
    } else {
        // Is target within the bounds of this sorted right half?
        if (nums[mid] < target && target <= nums[right]) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
}
return -1;
}

```

Note 1: Terminology and Techniques

- **Modified Binary Search:** This is the core algorithm used here. It takes the standard binary search (which relies on a fully monotonic/sorted array) and adapts the condition for pruning the search space. Instead of simply checking `nums[mid] < target`, it introduces a preliminary check to establish a local sorted invariant (`nums[left] <= nums[mid]`).
 - **Search Space Pruning:** This is the technique of systematically eliminating portions of your data structure where the answer cannot possibly exist. In L5/L6 system design and algorithmic thinking, pruning is critical for scaling.
 - **Invariant:** A property of a data structure or algorithm that holds true before and after every step. Finding the invariant (that one half is *always* sorted) is the key to unlocking this problem without memorization.
-

Note 2: Real-World & Interview Variations

Top tech companies rarely ask this question word-for-word anymore; they frame it in systems contexts or add constraints.

1. Meta (Facebook): “Find Minimum in Rotated Sorted Array”

- **The Variation:** Instead of searching for a target, just find the smallest number (the pivot point).
- **How to solve:** This is exactly the `find_pivot_index` helper function from Approach 1 above. You compare `nums[mid]` to `nums[right]`. If `mid`

is greater than `right`, the minimum must be to the right. Otherwise, `mid` itself or something to its left is the minimum.

2. Google: “Search in Rotated Sorted Array II” (Handling Duplicates)

- **The Variation:** The array can contain duplicates (e.g., `[3, 1, 2, 3, 3, 3, 3]`).
- **How to solve:** The invariant breaks! If `nums[left] == nums[mid] == nums[right]`, you cannot tell which half is strictly sorted.
- **The L5 Fix:** When this duplicate condition hits, you gracefully degrade the time complexity by incrementing `left` by 1 and decrementing `right` by 1 (`left += 1, right -= 1`) to shrink the window until you can re-establish the sorted halves. Worst-case time complexity degrades to $O(N)$.

3. Bloomberg: “Time-Series Log Search” (Real-world Application)

- **The Variation:** You are given a massive array of server logs. The logs are sorted by timestamp. However, at midnight, the logging server resets the timestamp counter to 0, effectively rotating the sorted logs. Given a specific timestamp, find the log entry.
- **How to solve:** This is identical to the main problem, but dressed up. The “midnight reset” is the rotation pivot. You use the exact One-Pass Modified Binary Search logic provided above, comparing timestamp values.

81. Search in Rotated Sorted Array II

A senior or staff-level engineer approaches a problem not just by rushing to the most optimal code, but by identifying the edge cases, explaining the trade-offs, and communicating the “why” behind the algorithm clearly. For a problem like this, the crux is handling ambiguity caused by duplicate values.

Here is how we would break down and solve “81. Search in Rotated Sorted Array II”.

1. Problem Explanation

Imagine a standard sorted array: `[0, 1, 2, 4, 4, 4, 5, 6]`. If we “rotate” it, we take a portion from the back and move it to the front. A rotation might look like this: `[4, 5, 6, 0, 1, 2, 4, 4]`.

The Goal: Given this rotated array (which contains duplicate numbers) and a `target` number, return `true` if the target exists in the array, and `false` otherwise.

Why is this non-trivial? In a perfectly sorted array, you use Binary Search to find a target in $O(\log N)$ time. Even in a *rotated* sorted array with *unique* elements (Leetcode 33), you can still use Binary Search because you can always tell which half of the array is perfectly sorted.

However, the introduction of **duplicates** breaks the standard binary search logic.

Look at this ambiguous scenario: Array A: [1, 0, 1, 1, 1] Array B: [1, 1, 1, 0, 1]

If we look at the ends and the middle of both arrays:

- Left value = 1
- Middle value = 1
- Right value = 1

If our target is 0, how do we know whether to search the left half or the right half? In Array A, it is on the left. In Array B, it is on the right. Because **Left == Middle == Right**, we are completely blind. This is the exact edge case the algorithm must solve.

2. Solution Explanation

To solve this, we will use a **Modified Binary Search**.

Step 1: The Baseline Rule of Rotated Arrays If you pick any middle point in a rotated sorted array, at least one half of the array (either left-to-mid or mid-to-right) **must** be perfectly sorted.

Step 2: Locating the Sorted Half We keep track of a **left** pointer and a **right** pointer. We calculate the **mid**.

- If `nums[left] < nums[mid]`: The left half is sorted.
- If `nums[mid] < nums[right]`: The right half is sorted.

Step 3: The Target Check Once we know which half is sorted, we simply check if our **target** falls within the boundary of that sorted half.

- If the left is sorted and `nums[left] <= target < nums[mid]`, we throw away the right half.
- Otherwise, we throw away the left half.

Step 4: The Tricky Part (Handling Duplicates) What happens when `nums[left] == nums[mid] == nums[right]`? We cannot safely discard half the array. The only safe operation is to shrink our search window by one on both sides. We do this by moving **left** one step to the right, and **right** one step to the left. We repeat this until we can identify a sorted half.

ASCII Visualizations Example 1: Clear Sorted Halves (No tricky duplicates blocking the view) Array: [4, 5, 6, 0, 1, 2, 4] | Target: 0

Initial State:

L = 0, R = 6, Mid = 3

```
[ 4 , 5 , 6 , 0 , 1 , 2 , 4 ]
  ^           ^           ^
  L           Mid          R
```

1. Is Mid (0) == Target (0)? YES! We found it immediately. Return True.

Example 2: The Tricky Case (Shrinking the window) Array: [1, 1, 1, 0, 1] | Target: 0

Iteration 1:

L = 0, R = 4, Mid = 2

```
[ 1 , 1 , 1 , 0 , 1 ]
  ^   ^   ^   ^
  L   Mid  R
```

1. Is Mid (1) == Target (0)? No.
2. nums[L] == nums[Mid] == nums[R] (1 == 1 == 1).
We cannot tell where the '0' is hiding.
ACTION: Shrink window. L becomes L+1, R becomes R-1.

Iteration 2:

L = 1, R = 3, Mid = 2

```
[ 1 , 1 , 1 , 0 , 1 ]
  ^   ^   ^   ^
    L Mid R
```

1. Is Mid (1) == Target (0)? No.
2. Now, nums[L] (1) <= nums[Mid] (1). The left half [1, 1] is sorted.
3. Is target (0) within the sorted left half? (Is 1 <= 0 < 1?) No.
ACTION: Target must be in the right half. Move L to Mid + 1.

Iteration 3:

L = 3, R = 3, Mid = 3

```
[ 1 , 1 , 1 , 0 , 1 ]
                ^
                L,M,R
```

1. Is Mid (0) == Target (0)? YES! Return True.

Time Complexity: * **Best / Average Case:** $O(\log N)$. When duplicates don't mask the boundaries, we cut the search space in half every step, exactly like standard binary search.

- **Worst Case:** $O(N)$. If the array is filled with duplicates (e.g., [1, 1, 1, 1, 1, 1]) and the target is not present, the `nums[left] == nums[mid] == nums[right]` condition triggers every time. We only shrink the window by 2 elements per step, looking at every element.

Time Complexity Derivation (Worst Case Scenario):

Target: 2

Array length N = 7

```
[ 1 , 1 , 1 , 1 , 1 , 1 , 1 ]  -> N elements to check. L=0, R=6
      ^             ^             ^
      L             M             R
```

Condition triggered: shrink boundaries!

[1 , 1 , 1 , 1 , 1 , 1 , 1] -> N-2 elements left. L=1, R=5

L
M
R

Condition triggered: shrink boundaries!

```
[ 1 , 1 , 1 , 1 , 1 , 1 , 1 ]  -> N-4 elements left. L=2, R=4
      ^       ^       ^
      L       M       R
```

As we can see, the search space decreases by a constant amount (2) rather than dividing by 2. Total operations = $N / 2$, which simplifies to $O(N)$ Time.

Space Complexity: $O(1)$. We are only using a few pointers (`left`, `right`, `mid`) which take constant space regardless of the input array size.

Space Complexity Derivation:

Variables used:

- L (integer pointer)
- R (integer pointer)
- Mid (integer pointer)

Total extra space = 3 variables.

Since 3 is a constant number, Space = $O(1)$.

4. Solution Code

As a senior engineer, I would present the optimized Binary Search solution in an interview, but briefly mention that a simple for loop (Linear Search) runs in $O(N)$ time and $O(1)$ space, which technically matches our worst-case scenario. However, the Modified Binary Search is required to achieve the $O(\log N)$ average-case performance.

Python Snippet

```
def search(nums, target):  
    """  
    Searches for a target in a rotated sorted array containing duplicates.  
    Average TC:  $O(\log N)$ , Worst TC:  $O(N)$ . SC:  $O(1)$ .  
    """  
    left = 0  
    right = len(nums) - 1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
  
        # Check if we found the target  
        if nums[mid] == target:  
            return True  
  
        # The Tricky Case: Duplicates at all three pointers  
        # We can't determine the sorted half, so we shrink the search space safely.  
        if nums[left] == nums[mid] and nums[mid] == nums[right]:  
            left += 1  
            right -= 1  
            continue  
  
        # Left half is strictly sorted  
        if nums[left] <= nums[mid]:  
            # Check if target is inside this sorted left half  
            if nums[left] <= target and target < nums[mid]:  
                right = mid - 1 # It is here, discard right half  
            else:  
                left = mid + 1 # It is not here, discard left half  
  
        # Right half is strictly sorted  
        else:  
            # Check if target is inside this sorted right half  
            if nums[mid] < target and target <= nums[right]:  
                left = mid + 1 # It is here, discard left half  
            else:
```

```

        right = mid - 1 # It is not here, discard right half

    return False

```

JavaScript Snippet

```

/**
 * Searches for a target in a rotated sorted array containing duplicates.
 * Average TC:  $O(\log N)$ , Worst TC:  $O(N)$ . SC:  $O(1)$ .
 * * @param {number[]} nums
 * * @param {number} target
 * * @return {boolean}
 */
function search(nums, target) {
    let left = 0;
    let right = nums.length - 1;

    while (left <= right) {
        // Use Math.floor to avoid float division in JS
        let mid = left + Math.floor((right - left) / 2);

        // Check if we found the target
        if (nums[mid] === target) {
            return true;
        }

        // The Tricky Case: Duplicates at all three pointers
        // We can't determine the sorted half, so we shrink the search space safely.
        if (nums[left] === nums[mid] && nums[mid] === nums[right]) {
            left++;
            right--;
            continue;
        }

        // Left half is strictly sorted
        if (nums[left] <= nums[mid]) {
            // Check if target is inside this sorted left half
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1; // It is here, discard right half
            } else {
                left = mid + 1; // It is not here, discard left half
            }
        }

        // Right half is strictly sorted
        else {
            // Check if target is inside this sorted right half

```

```

        if (nums[mid] < target && target <= nums[right]) {
            left = mid + 1; // It is here, discard left half
        } else {
            right = mid - 1; // It is not here, discard right half
        }
    }

    return false;
}

```

Note 1: Terms & Techniques Used

- **Modified Binary Search:** Standard binary search divides a sorted array in half. The “modified” version adds conditional checks to handle structural anomalies (like the rotation) before deciding which half to discard.
 - **Search Space Pruning (or Window Shrinking):** This is the technique used when `nums[left] == nums[mid] == nums[right]`. Instead of discarding a massive chunk of data blindly, we “prune” the absolute minimum amount of data (one element from each side) to safely reveal the underlying structure of the array without accidentally deleting our target.
-

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

Companies rarely ask Leetcode questions verbatim at the L5/L6 level. They wrap them in real-world system design micro-scenarios.

1. Google Variation: The Shifted Log File

- *Prompt:* You are parsing a massive server log file where timestamps are normally sorted. However, due to a rotating buffer write mechanism, the file starts from an older date halfway through, loops around, and continues. The server had an idle period where it logged the exact same “idle” timestamp thousands of times. Given a massive array of these timestamps, find if a specific error timestamp exists.
- *Solution:* This is identically “Search in Rotated Sorted Array II”. The “rotating buffer” is the rotated array. The “idle timestamps” are the duplicates. You would implement the exact Modified Binary Search above. To make it L5-level, you would mention that instead of loading the whole file into an array, you would use file pointers (`fseek` or similar byte-offsets) as your `left` and `right` indices to avoid memory exhaustion (optimizing Space Complexity for massive files).

2. Meta Variation: Clock Synchronization Skew

- *Prompt:* We have an array of user engagement events. They are supposed to be chronologically sorted. Due to a regional clock sync error, the events from the APAC region were appended at the start of the array instead of the end, shifting the timeline. Furthermore, a bot attack generated thousands of events with the exact same millisecond timestamp. Can you quickly check if a specific user timestamp exists?
- *Solution:* Again, same core problem. The clock skew creates the rotation. The bot attack creates the worst-case $O(N)$ duplicate scenario. In the interview, I would emphasize that if the bot attack is severe, our $O(\log N)$ algorithm degenerates to $O(N)$. If I know the bot timestamp in advance, I might add an early exit or skip logic specifically for that bot timestamp to optimize the worst-case scenario.

3. Bloomberg Variation: Rolling Stock Tick Data

- *Prompt:* You are querying an in-memory rolling array of stock prices for a specific ticker over the last 24 hours. The array is circular (rotated). A stock might be suspended from trading, causing its price to remain exactly the same for hours (duplicates). Find if the stock ever hit a specific target price.
- *Solution:* Use the Modified Binary Search. For Bloomberg, execution speed is everything. I would point out that moving pointers by 1 during the duplicate phase (`left += 1`, `right -= 1`) is slow. A slight optimization could be a `while` loop that rapidly increments `left` as long as `nums[left] == nums[left + 1]`, skipping whole blocks of identical prices quickly, before resuming the binary search.

15. 3Sum

Here is how a senior engineer approaches 3Sum. We will start by breaking down the core problem, isolating the trickiest parts (handling duplicates), and then building up from a conceptually sound but suboptimal approach to the optimal solution.

1. Problem Explanation

The Goal: Given an array of integers `nums`, find all unique triplets (`nums[i]`, `nums[j]`, `nums[k]`) such that their sum equals 0.

The Constraints & Non-Trivial Aspects:

1. **Indices must be distinct:** $i \neq j$, $i \neq k$, and $j \neq k$. You cannot reuse the exact same element from the same position twice in a single triplet.
2. **Triplets must be unique:** This is the most non-trivial part of the problem. If your array is `[-1, 0, 1, 2, -1, -4]`, the triplet `[-1, 0, 1]`

might be found twice because there are two -1s in the array. The final output must only contain `[[-1, 0, 1]]` once.

3. **Order does not matter:** `[-1, 0, 1]` is considered the exact same triplet as `[0, 1, -1]`.

If we just use three nested loops (Brute Force), we will check every combination. It will take $O(N^3)$ time, and we would still have the massive headache of filtering out duplicate triplets at the end. We need a structural way to avoid evaluating duplicates in the first place.

2. Solution Explanation

To solve this optimally, we must address the duplicate problem head-on. The easiest way to identify and skip duplicates is to group them together. We achieve this by **Sorting the array first**.

Sorting unlocks two massive advantages:

1. Duplicate values are now adjacent, so we can skip them with a simple `while` loop or `if` condition.
2. The array is ordered, which means we can use the **Two Pointers** technique to find pairs efficiently.

The Strategy: Reduce 3Sum to 2Sum Once the array is sorted, we iterate through the array, picking one number at a time to be our “first” number (`nums[i]`). Let’s call this our **anchor**. If our anchor is A, we need to find two other numbers, B and C, such that $A + B + C = 0$. This means $B + C = -A$. The problem has now been reduced to: “Find two numbers in the rest of the array that sum up to a specific target.”

Visualizing the Two Pointers approach: Let’s trace the optimal approach with `nums = [-1, 0, 1, 2, -1, -4]`.

Step 1: Sort the array Sorted array: `[-4, -1, -1, 0, 1, 2]`

Step 2: Iterate and use Two Pointers

Iteration 1: `anchor = nums[0] = -4` Target sum for the remaining two numbers: $0 - (-4) = 4$ Place a **left** pointer just after the anchor, and a **right** pointer at the end.

```
[-4, -1, -1, 0, 1, 2]
 ^   ^   ^
 i   left      right
```

Target: 4

Current Sum (`left + right`): $-1 + 2 = 1$

Logic:

1 is LESS than the Target (4).

Because the array is sorted, moving 'right' to the left will only make the sum smaller. To increase the sum, we MUST move 'left' to the right.

```
[-4, -1, -1, 0, 1, 2]
  ^     ^     ^
  i     left   right
```

Target: 4

Current Sum: $-1 + 2 = 1$ --> Still less than 4. Move left.

(We continue this for $i=0$ but will never hit 4. Moving on.)

Iteration 2: $anchor = nums[1] = -1$ Target sum: $0 - (-1) = 1$

```
[-4, -1, -1, 0, 1, 2]
      ^     ^     ^
      i     left   right
```

Target: 1

Current Sum: $-1 + 2 = 1$

Logic:

MATCH! We found a triplet: $[-1, -1, 2]$.

We record it.

Now, move BOTH pointers inward to look for more pairs for this anchor.

```
[-4, -1, -1, 0, 1, 2]
      ^     ^     ^
      i     left  right
```

Target: 1

Current Sum: $0 + 1 = 1$

Logic:

MATCH! We found another triplet: $[-1, 0, 1]$.

Record it. Move both pointers inward.

$left > right$. Loop ends for $i=1$.

Iteration 3: $anchor = nums[2] = -1$

```
[-4, -1, -1, 0, 1, 2]
      ^
      i
```

CRITICAL SKIP LOGIC: We notice $nums[2]$ is exactly the same as $nums[1]$. If we run the two pointers here, we will just find the exact same triplets we already found. So, we skip $i=2$.

We continue this until i reaches the end.

3. Time and Space Complexity Analysis

```
=====
                        TIME COMPLEXITY DERIVATION
=====
```

Algorithm Steps:

1. Sorting the input array $\Rightarrow O(N * \log(N))$
2. Outer loop (Iterating 'i' from 0 to N-2) $\Rightarrow$ Executes N times
|
+-- 3. Inner loop (Two Pointers) $\Rightarrow$ In the worst case, 'left' and 'right' traverse the remaining array once.
 $\Rightarrow$ Executes (N - i) times
 $\Rightarrow$ Average N/2 operations per outer loop step.

Total Time = Sorting Time + (Outer Loop * Inner Loop)

Total Time = $N * \log(N)$ + $(N * N)$

Total Time = $O(N * \log(N)) + O(N^2)$

Since N^2 grows much faster than $N * \log(N)$ for large N:

Dominant Term = $O(N^2)$

FINAL TIME COMPLEXITY: $O(N^2)$

```
=====
                        SPACE COMPLEXITY DERIVATION
=====
```

Space used by variables: 'i', 'left', 'right' $\Rightarrow O(1)$ constant space

Space used by output array $\Rightarrow$ Not counted towards auxiliary space complexity per standard interview rules.

Space used by Sorting Algorithm $\Rightarrow$ Depending on the language:
- Python's Timsort: $O(N)$
- JS V8 Engine (TimSort/Quick): $O(N)$ or $O(\log N)$

FINAL AUXILIARY SPACE COMPLEXITY: $O(1)$ or $O(N)$ depending on the in-built sorting algorithm's memory usage.

4. Solution Code

I will provide two solutions in both languages.

- **Solution A (Sub-optimal but valid): Hash Set for 2Sum.** This uses sorting, an outer loop, and a Hash Set for the inner 2Sum search. It

is $O(N^2)$ time but uses $O(N)$ auxiliary space for the set. It is great to mention in an interview to show you know multiple ways to solve 2Sum.

- **Solution B (Optimal): Two Pointers.** $O(N^2)$ time, $O(1)$ auxiliary space.

Python Code

```
# =====
# SOLUTION A: Sort + Hash Set (Sub-optimal Space)
# =====
def threeSum_hashset(nums):
    nums.sort() # Sorting helps us avoid duplicate anchors
    res = []

    for i in range(len(nums)):
        # Skip duplicate anchors to prevent duplicate triplets in the result
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        target = -nums[i]
        seen = set()
        j = i + 1

        while j < len(nums):
            complement = target - nums[j]
            if complement in seen:
                res.append([nums[i], complement, nums[j]])
                # Skip duplicate elements for the 2Sum part to avoid dupes
                while j + 1 < len(nums) and nums[j] == nums[j + 1]:
                    j += 1
            seen.add(nums[j])
            j += 1

    return res

# =====
# SOLUTION B: Sort + Two Pointers (Optimal Space)
# =====
def threeSum_twopointers(nums):
    nums.sort() #  $O(N \log N)$  time,  $O(N)$  space for Python's TimSort
    res = []

    for i in range(len(nums) - 2):
        # Optimization: If the smallest number is > 0, sum can never be 0
        if nums[i] > 0:
            break
```



```

    # Skip duplicate anchors
    if i > 0 and nums[i] == nums[i - 1]:
        continue

    left = i + 1
    right = len(nums) - 1

    while left < right:
        total = nums[i] + nums[left] + nums[right]

        if total > 0:
            right -= 1 # Sum too big, move right pointer to a smaller number
        elif total < 0:
            left += 1 # Sum too small, move left pointer to a larger number
        else:
            # Found a valid triplet
            res.append([nums[i], nums[left], nums[right]])

            # Advance both pointers
            left += 1
            right -= 1

            # CRITICAL: Skip duplicate 'left' values to avoid duplicate triplets
            while left < right and nums[left] == nums[left - 1]:
                left += 1

            # CRITICAL: Skip duplicate 'right' values
            while left < right and nums[right] == nums[right + 1]:
                right -= 1

    return res

```

JavaScript Code

```

// =====
// SOLUTION A: Sort + Hash Set (Sub-optimal Space)
// =====
function threeSumHashset(nums) {
    // JS sort() converts to strings by default, MUST provide comparator for ints
    nums.sort((a, b) => a - b);
    const res = [];

    for (let i = 0; i < nums.length; i++) {
        // Skip duplicate anchors
        if (i > 0 && nums[i] === nums[i - 1]) continue;
    }
}

```

```

    let target = -nums[i];
    let seen = new Set();
    let j = i + 1;

    while (j < nums.length) {
        let complement = target - nums[j];
        if (seen.has(complement)) {
            res.push([nums[i], complement, nums[j]]);
            // Skip duplicate second elements
            while (j + 1 < nums.length && nums[j] === nums[j + 1]) {
                j++;
            }
        }
        seen.add(nums[j]);
        j++;
    }
    return res;
}

// =====
// SOLUTION B: Sort + Two Pointers (Optimal Space)
// =====
function threeSumTwoPointers(nums) {
    nums.sort((a, b) => a - b);
    const res = [];

    for (let i = 0; i < nums.length - 2; i++) {
        // If anchor is positive, we can never sum to 0 with sorted elements
        if (nums[i] > 0) break;

        // Skip duplicate anchors
        if (i > 0 && nums[i] === nums[i - 1]) continue;

        let left = i + 1;
        let right = nums.length - 1;

        while (left < right) {
            let total = nums[i] + nums[left] + nums[right];

            if (total > 0) {
                right--; // Sum too large, shrink from the right
            } else if (total < 0) {
                left++; // Sum too small, grow from the left
            } else {

```

```

        // Match found
        res.push([nums[i], nums[left], nums[right]]);

        left++;
        right--;

        // Skip duplicate elements to avoid duplicate triplets in output
        while (left < right && nums[left] === nums[left - 1]) {
            left++;
        }
        while (left < right && nums[right] === nums[right + 1]) {
            right--;
        }
    }
}
return res;
}

```

Note 1: Terms and Techniques Explained

- **Two Pointers Algorithm:** This is a technique where you use two references (indices) to iterate over a data structure, usually from opposite ends heading towards the middle.
 - *Why it helps:* It dramatically reduces time complexity. On a sorted array, finding a pair that sums to a target using nested loops takes $O(N^2)$. The Two Pointers technique reduces this exact same search to $O(N)$ because the sorted nature allows us to make intelligent decisions (if sum is too big, move right pointer down; if too small, move left pointer up) instead of blindly checking every combination.
 - *How it applies:* After fixing our anchor element in 3Sum, we use Two Pointers on the remainder of the sorted array to execute a lightning-fast $O(N)$ search for the final two numbers.
-

Note 2: Real-World & Interview Variations

1. Meta (Facebook): “3Sum Closest” or “3Sum Target”

- **The Query:** Instead of finding a sum exactly equal to 0, find a triplet whose sum is *closest* to a given target integer T.
- **How to solve:** The structure is identical. Sort the array, use a for loop, and set up `left` and `right` pointers. Instead of checking if `total === 0`, you keep a running variable `min_diff` and `closest_sum`. For every iteration, calculate `current_diff = absolute_value(target - total)`. If

`current_diff < min_diff`, update your closest sum. Then move pointers based on `total > target` or `total < target`.

2. Bloomberg: “Contextualized 3Sum returning Original Indices”

- **The Query:** “Given a list of stock prices and a budget B, find three distinct stocks that spend exactly B. Return their original array indices.”
- **The Catch:** If you sort the array, you lose the original indices, which breaks the standard Two Pointer approach.
- **How to solve:** Before sorting, map the values to their original indices by creating an array of tuples or objects: `[(value_0, index_0), (value_1, index_1), ...]`. Sort *this* new array by the values. Perform the exact same Two Pointers logic looking at the values, but when a match is found, extract and return the saved original indices. This bumps the Space Complexity strictly to $O(N)$ to store the tuples.

3. Google: “3Sum Across Multiple Arrays”

- **The Query:** Given three arrays A, B, and C, find `A[i] + B[j] + C[k] == 0`.
- **How to solve:** You cannot just sort one array and run Two Pointers easily.
- *Approach 1 (Space Optimized):* Sort array C. Iterate through every element in A (nested with every element in B). For each pair, calculate `target = -(A[i] + B[j])`. Run a Binary Search on array C to find the target. Time: $O(N^2 * \log N)$. Space: $O(1)$.
- *Approach 2 (Time Optimized):* Put all elements of C into a Hash Map (recording their frequencies if duplicates matter). Iterate through A and B with nested loops, checking if `-(A[i] + B[j])` exists in the Hash Map. Time: $O(N^2)$. Space: $O(N)$.

18. 4Sum

Here is how a top-of-the-band L5 or L6 engineer approaches 4Sum. At a senior level, we do not just look for the solution that passes the test cases; we look for a solution that is extensible, clean, and demonstrates a deep understanding of algorithmic reduction.

While solving 4Sum specifically using three nested loops and a two-pointer approach is standard, a top-tier engineer will immediately recognize this as a specific instance of a broader problem: **K-Sum**. I will walk you through the standard optimal approach first, and then the generalized K-Sum approach that truly signals senior-level thinking.

1. Problem Explanation

You are given an array of integers `nums` of length `n` and an integer `target`. You need to find all unique quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that:

- The indices `a`, `b`, `c`, and `d` are all distinct (you cannot reuse the same element at a specific index).
- The sum of the four elements equals the `target`.
- The final array of quadruplets must not contain any duplicate sets of numbers.

The Non-Trivial Part: Handling duplicates. If your array is `[2, 2, 2, 2, 2]` and target is 8, the only valid quadruplet is `[2, 2, 2, 2]`. If you aren't careful, your loops will evaluate the different index combinations of those 2s and output multiple `[2, 2, 2, 2]` arrays. Preventing duplicates without relying on a slow, memory-heavy `HashSet` is the primary challenge here.

2. Solution Explanation

To solve this efficiently, we build upon the classic **2Sum** and **3Sum** problems.

The Progression:

- **Brute Force:** 4 nested loops. Time complexity is $O(N^4)$. Space complexity is $O(1)$. Too slow.
- **Hash Map:** 2 nested loops to store pairs in a hash map, then 2 more loops to find complements. Time complexity is $O(N^2)$ on average, but handling duplicates gracefully makes the implementation messy and space complexity jumps to $O(N^2)$.
- **Sorting + Two Pointers (The Standard Optimal):** We sort the array first. This allows us to skip duplicates easily. We fix the first two numbers using two nested loops, and then use the "Two Pointer" technique to find the remaining two numbers in $O(N)$ time. Time complexity is $O(N^3)$. Space complexity is $O(1)$ auxiliary.

The Strategy (Sorting + Two Pointers):

1. **Sort the array:** This puts duplicates adjacent to each other, making them trivial to skip (`if nums[i] == nums[i-1] then skip`). It also orders the numbers so we can use directional pointers.
2. **Loop 1 (Index i):** Iterate from 0 to `n-3`. This fixes the first number.
3. **Loop 2 (Index j):** Iterate from `i+1` to `n-2`. This fixes the second number.
4. **Two Pointers (left and right):** Set `left = j + 1` and `right = n - 1`.
 - Calculate the sum: `nums[i] + nums[j] + nums[left] + nums[right]`.

- If the sum is too small, increment **left** (to get a bigger number).
- If the sum is too large, decrement **right** (to get a smaller number).
- If it's an exact match, record the quadruplet, then move both pointers inward, explicitly skipping any duplicate values to avoid redundant answers.

Visualizing the Two Pointer execution:

Target: 0

Input Array: [1, 0, -1, 0, -2, 2]

Step 1: Sort!

Array: [-2, -1, 0, 0, 1, 2]

Step 2 & 3: Fix i and j

i = 0 (value -2)

j = 1 (value -1)

Step 4: Two Pointers

[-2, -1, 0, 0, 1, 2]

i j L R

Current Sum = (-2) + (-1) + 0 + 2 = -1

-1 is less than Target (0). We need a bigger sum.

Since the array is sorted, moving L to the right gives a bigger number.

Move L ->

[-2, -1, 0, 0, 1, 2]

i j L R

Current Sum = (-2) + (-1) + 0 + 2 = -1

Still -1. Move L ->

[-2, -1, 0, 0, 1, 2]

i j L R

Current Sum = (-2) + (-1) + 1 + 2 = 0

Match found! Add [-2, -1, 1, 2] to results.

Now move L right and R left. (End of inner loop for this j).

3. Time and Space Complexity Analysis

```

+-----+
| Time Complexity Derivation: O(N^3) |
+-----+
| Array Length = N |

```

```

| 1. Array Sorting:  $O(N * \log N)$ 
|
| 2. Nested Iterations:
|
|     [Loop i] iterates N times
|     |
|     +-- [Loop j] iterates N times
|     |
|     +-- [Two Pointers L & R] scan remaining array
|         They meet in the middle, covering at most
|         N elements together. ->  $O(N)$  work.
|
| Total Iteration Work:  $N * N * N = O(N^3)$ 
|
| Overall Time:  $O(N * \log N) + O(N^3) \Rightarrow O(N^3)$ 
+-----+

+-----+
| Space Complexity Derivation:  $O(1)$  to  $O(N)$ 
+-----+

| 1. Results Array: Not counted towards auxiliary space
|    in standard algorithmic analysis.
|
| 2. Pointers (i, j, left, right):  $O(1)$  memory
|
| 3. Sorting overhead:
|    - Python (Timsort):  $O(N)$  space
|    - JS (V8 TimSort):  $O(N)$  space
|    - C++ (Introsort):  $O(\log N)$  space
|
| Overall Space:  $O(1)$  conceptually, but practically bounded
| by  $O(N)$  depending on the language's native sorting engine.
+-----+

```

4. Solution Code

As a senior engineer, I will provide you with two implementations in each language.

1. **Iterative 4Sum:** The standard, highly optimized, non-recursive solution expected in an interview.
2. **Generalized K-Sum:** The scalable solution. If a product manager suddenly asks for “5Sum” tomorrow, this code handles it without needing another nested loop.

Python Solutions Solution A: Standard Optimal Iterative ($O(N^3)$ Time, $O(1)$ Space)

```
def fourSum(nums, target):  
    """  
    Standard Iterative approach using Sorting + Two Pointers.  
    """  
  
    nums.sort()  
    n = len(nums)  
    results = []  
  
    # Early exit if array is too small  
    if n < 4:  
        return results  
  
    for i in range(n - 3):  
        # Skip duplicate 'i' to prevent duplicate quadruplets  
        if i > 0 and nums[i] == nums[i - 1]:  
            continue  
  
        for j in range(i + 1, n - 2):  
            # Skip duplicate 'j'  
            if j > i + 1 and nums[j] == nums[j - 1]:  
                continue  
  
            left = j + 1  
            right = n - 1  
  
            while left < right:  
                current_sum = nums[i] + nums[j] + nums[left] + nums[right]  
  
                if current_sum == target:  
                    results.append([nums[i], nums[j], nums[left], nums[right]])  
                    left += 1  
                    right -= 1  
  
                    # Skip duplicates for 'left'  
                    while left < right and nums[left] == nums[left - 1]:  
                        left += 1  
                    # Skip duplicates for 'right'  
                    while left < right and nums[right] == nums[right + 1]:  
                        right -= 1  
  
                elif current_sum < target:  
                    left += 1  
                else:  
                    right -= 1
```



```
right -= 1
```

```
return results
```

Solution B: Generalized K-Sum (The Staff Engineer Flex)

```
def fourSum_ksum(nums, target):  
    """  
    Generalized K-Sum approach. Reduces any K-Sum problem to 2-Sum.  
    Highly modular and reusable.  
    """  
  
    def kSum(nums, target, k):  
        res = []  
  
        # Base cases and early pruning  
        if not nums:  
            return res  
  
        # Calculate average value needed.  
        # If the smallest value in array is greater than average,  
        # or largest value is smaller than average, it's impossible.  
        average_value = target // k  
        if average_value < nums[0] or nums[-1] < average_value:  
            return res  
  
        # Base case: reduce down to 2Sum  
        if k == 2:  
            return twoSum(nums, target)  
  
        # Recursive step: reduce K to K-1  
        for i in range(len(nums)):  
            # Skip duplicates  
            if i == 0 or nums[i - 1] != nums[i]:  
                # Recurse for k-1, adjusting the target  
                subsets = kSum(nums[i + 1:], target - nums[i], k - 1)  
                for subset in subsets:  
                    res.append([nums[i]] + subset)  
        return res  
  
    def twoSum(nums, target):  
        res = []  
        left, right = 0, len(nums) - 1  
  
        while left < right:  
            curr_sum = nums[left] + nums[right]  
            if curr_sum < target:  
                left += 1
```

```

        elif curr_sum > target:
            right -= 1
        else:
            res.append([nums[left], nums[right]])
            left += 1
            right -= 1
            # Skip duplicates to maintain uniqueness
            while left < right and nums[left] == nums[left - 1]:
                left += 1
    return res

nums.sort()
return kSum(nums, target, 4)

```

JavaScript Solutions Solution A: Standard Optimal Iterative

```

/**
 * Standard Iterative approach using Sorting + Two Pointers.
 * Time:  $O(N^3)$ , Space:  $O(1)$  auxiliary.
 */
function fourSum(nums, target) {
    // Sort array numerically (JS default sort is string-based, so a-b is required)
    nums.sort((a, b) => a - b);
    const n = nums.length;
    const results = [];

    if (n < 4) return results;

    for (let i = 0; i < n - 3; i++) {
        // Skip duplicate 'i'
        if (i > 0 && nums[i] === nums[i - 1]) continue;

        for (let j = i + 1; j < n - 2; j++) {
            // Skip duplicate 'j'
            if (j > i + 1 && nums[j] === nums[j - 1]) continue;

            let left = j + 1;
            let right = n - 1;

            while (left < right) {
                // Prevent integer overflow logic implicitly handled by JS (safe up to  $2^{53}$ )
                const sum = nums[i] + nums[j] + nums[left] + nums[right];

                if (sum === target) {
                    results.push([nums[i], nums[j], nums[left], nums[right]]);
                    left++;
                }
            }
        }
    }
    return results;
}

```

```

        right--;

        // Skip duplicates for pointers
        while (left < right && nums[left] === nums[left - 1]) left++;
        while (left < right && nums[right] === nums[right + 1]) right--;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
    }
}
}
return results;
}

```

Solution B: Generalized K-Sum

```

/**
 * Generalized K-Sum approach.
 * Shows architectural forethought for extensibility.
 */
function fourSum_ksum(nums, target) {
    nums.sort((a, b) => a - b);

    function kSum(start_index, target, k) {
        const res = [];

        // Base case pruning
        if (start_index === nums.length) return res;

        const average_value = Math.floor(target / k);
        if (nums[start_index] > average_value || average_value > nums[nums.length - 1]) {
            return res;
        }

        if (k === 2) {
            return twoSum(start_index, target);
        }

        for (let i = start_index; i < nums.length; i++) {
            if (i === start_index || nums[i - 1] !== nums[i]) {
                const subsets = kSum(i + 1, target - nums[i], k - 1);
                for (const subset of subsets) {
                    res.push([nums[i], ...subset]);
                }
            }
        }
    }
}

```

```

    }
    return res;
}

function twoSum(start_index, target) {
    const res = [];
    let left = start_index;
    let right = nums.length - 1;

    while (left < right) {
        const sum = nums[left] + nums[right];
        if (sum < target || (left > start_index && nums[left] === nums[left - 1])) {
            left++;
        } else if (sum > target || (right < nums.length - 1 && nums[right] === nums[right + 1])) {
            right--;
        } else {
            res.push([nums[left], nums[right]]);
            left++;
            right--;
        }
    }
    return res;
}

return kSum(0, target, 4);
}

```

Note 1: Terms and Techniques Used

- **Two Pointers:** A technique where two variables (pointers) traverse an array, usually from opposite ends toward the middle.
- *Why it helps:* In a sorted array, it allows you to dynamically adjust the sum of two elements by either moving the left pointer up (to increase the sum) or the right pointer down (to decrease the sum). It reduces a brute force $O(N^2)$ pair-finding loop into an elegant $O(N)$ scan.
- **Generalized K-Sum Reduction:** A divide-and-conquer recursion technique.
- *Why it helps:* Instead of writing separate functions for 3Sum, 4Sum, and 5Sum, you write a recursive function that strips away one number at a time (reducing K by 1 and subtracting that number from the **target**), eventually bottoming out at the base case of 2Sum. It shows scalable system design thinking within an algorithmic context.

Note 2: Real-World Interview Variations at FAANG

Top companies rarely ask “Solve 4Sum” outright. They disguise it in business logic. Here is how they frame it and how you map it back to the exact code above.

Google: The Bin-Packing / Resource Allocation Variant

- **The Prompt:** “You have a list of virtual machines with varying idle RAM capacities. You need to assign exactly 4 VMs to a new distributed database cluster such that their combined RAM exactly equals a `target` allocation. Return all unique combinations of 4 VMs.”
- **How to solve:** This is identical to 4Sum. Sort the array of RAM capacities and use the exact Two Pointer or K-Sum logic. The conversation here should focus on scaling—mentioning that if the number of VMs is massive, the $O(N^3)$ time complexity might require distributing the sorting and subset generation across multiple machines (MapReduce).

Meta: The Financial Fraud / Zero-Sum Transaction Variant

- **The Prompt:** “Given a ledger of financial transactions (some positive, some negative), we are looking for a specific cyclic fraud pattern where 4 transactions perfectly net out to \$0. Find all unique groups of 4 transactions that sum to 0.”
- **How to solve:** This is literally 4Sum with `target = 0`. Meta interviewers will probe deeply on duplicate handling here. Since a fraudster might make multiple transactions of the exact same amount to throw off algorithms, you must explicitly explain the `if nums[i] == nums[i-1] then skip` logic to ensure you aren’t outputting identical fraud cycles repeatedly.

Bloomberg: The Portfolio Balancing Variant

- **The Prompt:** “Given a list of historical daily yields for different stock assets, find a bundle of exactly 4 unique assets whose historical yields sum up to a specific target yield, `Y`. Return all possible asset bundles.”
- **How to solve:** Again, 4Sum. Sort the yields and apply the algorithm. Because Bloomberg deals with financial data, the interviewer might ask about floating-point precision issues (e.g., yields like 1.045). You would solve this by clarifying that floating-point addition can result in errors (like $0.1 + 0.2 = 0.30000000000000004$). You would adapt the standard 4Sum by either multiplying all yields by 10,000 to convert them to integers before running the algorithm, or by using an epsilon check `abs(sum - target) < 0.000001` instead of `sum == target`.

454. 4Sum II

As a senior engineer, the way I look at interview problems isn't just about jumping to the most optimal code immediately; it is about breaking down the problem, establishing a baseline, and systematically optimizing it while communicating the trade-offs.

Let's dive deep into LeetCode 454: 4Sum II, an absolute classic that tests your ability to trade space for time.

1. Problem Explanation

The Core Request: You are given four integer arrays (`nums1`, `nums2`, `nums3`, `nums4`), all of the exact same length, `n`. You need to find the total number of unique combinations of indices (`i`, `j`, `k`, `l`) such that picking one number from each array results in a sum of exactly zero.

Mathematically, we are looking for: `nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0`

Visualizing the Input: Imagine four buckets of numbers. You must draw exactly one number from each bucket.

Bucket 1 (nums1)	Bucket 2 (nums2)	Bucket 3 (nums3)	Bucket 4 (nums4)
+-----+	+-----+	+-----+	+-----+
1	-2	-1	0
2	-1	2	2
+-----+	+-----+	+-----+	+-----+
+-----+			
Sum == 0 ? Count it!			

2. Solution Explanation

The Baseline (Brute Force): The most trivial way to solve this is to use four nested loops, checking every single combination. *Is this good?* No. For arrays of length `N`, checking every combination takes $O(N^4)$ time. If `N` is 200 (a common LeetCode constraint), 200^4 is 1.6 billion operations. This will result in a Time Limit Exceeded (TLE) error.

The Evolution ($O(N^3)$ Sub-optimal but useful): How can we do better? We can rewrite our target equation: `nums1[i] + nums2[j] + nums3[k] = -nums4[l]`

This means if we store all elements of `nums4` in a Hash Map (recording how many times each number appears), we only need three nested loops for the first three arrays. We calculate the sum of three numbers, negate it, and check if

that negated value exists in our `nums4` Hash Map in $O(1)$ time. This reduces our time complexity by a whole factor of N , bringing it to $O(N^3)$.

The Optimal Insight ($O(N^2)$ “Meet in the Middle”): If breaking off one array saved us a factor of N , what if we break the problem exactly in half? `nums1[i] + nums2[j] == -(nums3[k] + nums4[l])`

Instead of looking at four arrays, we treat them as two pairs:

1. **Pair A:** `nums1` and `nums2`
2. **Pair B:** `nums3` and `nums4`

Step-by-Step Logic:

1. Create a Hash Map to store the frequencies of all possible sums from Pair A.
2. Use two nested loops to iterate through `nums1` and `nums2`. For every combination, calculate the sum and record it in the Hash Map.
3. Use two nested loops to iterate through `nums3` and `nums4`. For every combination, calculate the sum, negate it, and look it up in the Hash Map.
4. If the negated sum exists in the Hash Map, it means we found a valid 4-way combination. Add the frequency stored in the Hash Map to our total count.

Extensive ASCII Trace: Let’s trace this with a real example. `nums1 = [1, 2]`
`nums2 = [-2, -1]` `nums3 = [-1, 2]` `nums4 = [0, 2]`

PHASE 1: Build the Hash Map from `nums1` and `nums2`

<code>nums1 \ nums2</code>	<code>-2</code>	<code>-1</code>	
<code>1</code>	<code>1-2=-1</code>	<code>1-1=0</code>	
<code>2</code>	<code>2-2=0</code>	<code>2-1=1</code>	

Hash Map State:

```
{
  -1: 1,    (appeared 1 time)
   0: 2,    (appeared 2 times)
   1: 1     (appeared 1 time)
}
```

PHASE 2: Search using `nums3` and `nums4`

<code>nums3 \ nums4</code>	<code>0</code>	<code>2</code>	
<code>-1</code>	<code>-1+0=-1</code>	<code>-1+2=1</code>	
<code>2</code>	<code>2+0=2</code>	<code>2+2=4</code>	

Now, we take each Phase 2 sum, negate it, and look for it in the Hash Map:

```
Sum = -1 -> Target = 1 -> In Map? YES! (freq: 1) -> Total Combinations = 1
Sum = 1 -> Target = -1 -> In Map? YES! (freq: 1) -> Total Combinations = 1 + 1 = 2
Sum = 2 -> Target = -2 -> In Map? NO. -> Total Combinations = 2
Sum = 4 -> Target = -4 -> In Map? NO. -> Total Combinations = 2
```

Final Answer: 2 combinations sum to zero.

3. Time and Space Complexity Analysis

Time Complexity: $O(N^2)$ Where N is the length of the arrays.

Time Complexity Derivation Diagram:

[Total Time] = [Phase 1 Time] + [Phase 2 Time]

[Phase 1 Time]: Nested loop over nums1 and nums2

```
Loop nums1 (N iterations)
|
+---> Loop nums2 (N iterations)
|
+---> Hash Map Insert (O(1) time)
Result = N * N * 1 = O(N^2)
```

[Phase 2 Time]: Nested loop over nums3 and nums4

```
Loop nums3 (N iterations)
|
+---> Loop nums4 (N iterations)
|
+---> Hash Map Lookup (O(1) time)
Result = N * N * 1 = O(N^2)
```

[Total Time] = $O(N^2) + O(N^2) = O(N^2)$
(Constants are dropped in Big-O analysis)

Space Complexity: $O(N^2)$ In the worst-case scenario, every single combination of `nums1[i] + nums2[j]` produces a completely unique sum. Since there are $N * N$ combinations, our Hash Map will need to store N^2 unique key-value pairs.

Space Complexity Derivation Diagram:

Hash Map Size = Number of unique sums from (`nums1 + nums2`)
Worst Case: Every sum is unique.
Max elements = `length(nums1) * length(nums2)`

Max elements = $N * N$

Therefore, Space = $O(N^2)$

4. Solution Code

As promised, I will provide two solutions per language: the $O(N^3)$ stepping-stone (good for showing iterative problem-solving in an interview) and the $O(N^2)$ optimal solution.

Python Snippets

```
# --- SUB-OPTIMAL BUT INSTRUCTIVE SOLUTION:  $O(N^3)$  Time ---
def fourSumCount_ON3(nums1: list[int], nums2: list[int], nums3: list[int], nums4: list[int])
    # Step 1: Cache the last array's frequencies to save one loop layer
    # Time to build map:  $O(N)$ , Space:  $O(N)$ 
    freq_map4 = {}
    for num in nums4:
        freq_map4[num] = freq_map4.get(num, 0) + 1

    total_count = 0

    # Step 2: Iterate through the first three arrays
    # Time:  $O(N^3)$ 
    for a in nums1:
        for b in nums2:
            for c in nums3:
                # We need  $(a + b + c + d) == 0$ , which means  $d == -(a + b + c)$ 
                target = -(a + b + c)

                #  $O(1)$  lookup instead of a fourth  $O(N)$  loop
                if target in freq_map4:
                    total_count += freq_map4[target]

    return total_count

# --- OPTIMAL SOLUTION:  $O(N^2)$  Time ---
def fourSumCount_ON2(nums1: list[int], nums2: list[int], nums3: list[int], nums4: list[int])
    # Step 1: Group nums1 and nums2 together
    # This hash map stores the sum of pairs as the key, and their frequency as the value
    pair_sums_map = {}

    # Iterate through all combinations of nums1 and nums2
    for a in nums1:
        for b in nums2:
            current_sum = a + b
```

```

        pair_sums_map[current_sum] = pair_sums_map.get(current_sum, 0) + 1

total_count = 0

# Step 2: Group nums3 and nums4 together and check against the map
for c in nums3:
    for d in nums4:
        # We need (a + b) + (c + d) == 0, meaning (a + b) == -(c + d)
        target = -(c + d)

        # If the negated sum exists in our map, we've found valid combinations
        if target in pair_sums_map:
            total_count += pair_sums_map[target]

return total_count

```

JavaScript Snippets

```

// --- SUB-OPTIMAL BUT INSTRUCTIVE SOLUTION: O(N^3) Time ---
var fourSumCount_ON3 = function(nums1, nums2, nums3, nums4) {
    // Cache frequencies of nums4 in a Map for O(1) lookups
    const freqMap4 = new Map();
    for (let num of nums4) {
        freqMap4.set(num, (freqMap4.get(num) || 0) + 1);
    }

    let totalCount = 0;

    // 3 nested loops: O(N^3)
    for (let i = 0; i < nums1.length; i++) {
        for (let j = 0; j < nums2.length; j++) {
            for (let k = 0; k < nums3.length; k++) {
                const target = -(nums1[i] + nums2[j] + nums3[k]);

                if (freqMap4.has(target)) {
                    totalCount += freqMap4.get(target);
                }
            }
        }
    }

    return totalCount;
};

// --- OPTIMAL SOLUTION: O(N^2) Time ---
var fourSumCount_ON2 = function(nums1, nums2, nums3, nums4) {

```

```

// Map to store frequency of sums from nums1 and nums2
const pairSumsMap = new Map();

// Phase 1: Populate the map. Time:  $O(N^2)$ , Space:  $O(N^2)$ 
for (let i = 0; i < nums1.length; i++) {
    for (let j = 0; j < nums2.length; j++) {
        const sum = nums1[i] + nums2[j];
        pairSumsMap.set(sum, (pairSumsMap.get(sum) || 0) + 1);
    }
}

let totalCount = 0;

// Phase 2: Check combinations from nums3 and nums4 against the map
// Time:  $O(N^2)$ 
for (let k = 0; k < nums3.length; k++) {
    for (let l = 0; l < nums4.length; l++) {
        const target = -(nums3[k] + nums4[l]);

        // Look for the required value to reach exactly 0
        if (pairSumsMap.has(target)) {
            totalCount += pairSumsMap.get(target);
        }
    }
}

return totalCount;
};

```

Note 1: Terminology & Techniques Used

Meet in the Middle: This is the core algorithm pattern used here. Whenever you have a search space that is too large (like N^4), you split the problem into two distinct halves. You compute all possibilities for the first half and store them in a fast-lookup data structure (like a Hash Map). Then, you compute the second half and look up the complement in the data structure. It turns multiplicative complexity ($N * N * N * N$) into additive complexity over smaller multiplications ($N^2 + N^2$), drastically reducing overall runtime.

Space-Time Tradeoff: By choosing to allocate $O(N^2)$ extra memory for the Hash Map, we purchased a massive reduction in Time Complexity (from $O(N^4)$ to $O(N^2)$). In software engineering, memory is often cheaper than compute time, making this a highly favorable trade-off.

Note 2: Real-World & Interview Variations at Top Tech Companies

As an L5/L6, your interviewer won't just ask you the basic LeetCode question. They will mask it in a real-world scenario to see if you can identify the underlying pattern.

1. Bloomberg: Financial Net-Zero Trade Reconciliation

- **The Problem:** You are given four massive ledgers from four different global exchanges. You need to find combinations of exactly four trades (one from each ledger) that perfectly net to a value of \$0.00 to identify potential multi-leg arbitrage cycles. However, instead of just the *count*, you need to return the *actual lists of transaction IDs*.
- **How to solve it:** The algorithmic skeleton is identical (Meet in the Middle). However, your Hash Map can no longer just store integers (frequencies). The key remains the sum, but the value must be a **List** of tuples representing the transaction IDs (`trade_id_A`, `trade_id_B`). In Phase 2, when you find a matching target, you iterate through the list of tuples stored at that key and append (`trade_id_A`, `trade_id_B`, `trade_id_C`, `trade_id_D`) to your result array.
- **Trade-off discussion:** You must inform the interviewer that while the time complexity remains $O(N^2)$ to find matches, the worst-case space complexity blows up to $O(N^4)$ if many trades sum to the same amount and you have to store all their index combinations.

2. Google: Distributed Log Correlation / Latency Anomaly Detection

- **The Problem:** In a massive microservices architecture, a request passes through Services A, B, C, and D. You have the latency drift logs (deviation from expected time, which can be negative or positive) for thousands of requests across all four services. Find how many request combinations result in exactly zero overall latency drift. The datasets are too large to fit in the memory of a single machine.
- **How to solve it:** You can't use a single Hash Map in RAM. You solve this using the **MapReduce paradigm** while keeping the Meet in the Middle logic.
- **Map Phase 1:** Spin up workers to take partitions of Service A and B logs, calculate sums, and emit **Key: (Sum), Value: (1)**.
- **Reduce Phase 1:** Aggregate the values to get total frequencies for A+B sums. Store this in a distributed key-value store (like Bigtable or Redis).
- **Map Phase 2:** Spin up workers to process Service C and D logs, calculating **Target = -(C+D)**.
- **Reduce Phase 2:** The workers query the distributed key-value store for **Target**. If found, they emit the count. Finally, aggregate all counts.

3. Meta (Facebook): Social Budgeting / Target Sum Variation

- **The Problem:** You have four lists of friend groups. Each person in a group has a specific "social score" contribution to a party. You want to

invite exactly one person from each of the four groups such that their combined social score perfectly matches a specific target K (rather than 0).

- **How to solve it:** The logic remains exactly the same, but the algebra shifts slightly. Instead of $A + B = -(C + D)$, the equation is $A + B + C + D = K$. You rearrange this to $A + B = K - (C + D)$.
- Phase 1 is identical: store all $A + B$ sums in a map.
- Phase 2 changes slightly: for every combination of C and D , you calculate $\text{Target} = K - (C + D)$ and look that target up in the map.

11. Container With Most Water

An L5 or L6 engineer at Google approaches a problem not just by blurting out the optimal code, but by demonstrating a deep understanding of the problem space, proving why the optimal solution works, and communicating the trade-offs clearly. Here is exactly how a senior engineer would break down and solve “Container With Most Water”.

1. Problem explanation

Imagine you have a series of vertical lines standing on a flat surface. You are given an array of positive integers called `height`, where each number represents the height of a vertical line. The distance between each line is exactly 1 unit (represented by the array indices).

Your goal is to pick exactly two lines to act as the walls of a container. You want this container to hold the maximum possible amount of water.

The amount of water a container can hold (its area) is determined by two factors:

- **Width:** The distance between the two lines (Index of right line - Index of left line).
- **Height:** The water level cannot go higher than the shorter of the two lines, otherwise it spills over. So, the effective height is the minimum of the two line heights.

Formula: $\text{Area} = (\text{Right Index} - \text{Left Index}) * \min(\text{height}[\text{Left Index}], \text{height}[\text{Right Index}])$

Visual Example: Consider the array `height = [1, 5, 4, 3]`

Y-Axis (Heights)

```

5 |   |
4 |   | |
3 |   | | |
2 |   | | |
1 | | | | |

```

```

0 +----- X-Axis (Indices)
  0  1  2  3

```

If we choose the line at index 1 (height 5) and index 3 (height 3):

- Width = $3 - 1 = 2$
- Height = $\text{minimum}(5, 3) = 3$
- Area = $2 * 3 = 6$

2. Solution explanation

A senior engineer will always acknowledge the brute force method first to establish a baseline before optimizing.

Approach A: The Brute Force (Baseline) The simplest way to solve this is to test every single possible pair of lines, calculate the area for each pair, and keep track of the largest area found. While this guarantees finding the right answer, it is incredibly inefficient. For 100,000 lines, you would have to check nearly 5 billion pairs.

Approach B: The Optimized “Two Pointers” Strategy To find a better way, we need a logical rule to eliminate pairs without explicitly checking them. We can do this using two pointers starting at the absolute extremes of the array (one at the beginning, one at the end).

Why start at the extremes? Because this gives us the maximum possible **width**. From this starting point, the only way to potentially find a larger area is to find **taller** lines to compensate for the width we will lose as we move the pointers inward.

The Golden Rule of Moving Pointers: Calculate the area with your current left and right pointers. Now, look at the heights of the two lines. You must move the pointer pointing to the **shorter** line inward.

Why does this work? (The Proof) If you move the pointer of the taller line, the width decreases. Furthermore, the maximum height of the new container is still limited by that original shorter line you left in place. Decreasing the width without any possibility of increasing the height guarantees that the area will decrease or stay the same. Therefore, it is completely useless to move the taller line. We move the shorter line hoping to find a taller line that makes up for the lost width.

Step-by-Step Visualization (Array: [1, 5, 4, 3])

INITIAL STATE:

Left Pointer (L) at index 0, height 1

Right Pointer (R) at index 3, height 3

```

5 |   |
4 |   | |

```

```

3 |   |   |   R
2 |   |   |   |
1 | L |   |   |
0 +-----+
   0  1  2  3

```

Calculation:

Width = $3 - 0 = 3$

Height = $\min(1, 3) = 1$

Area = $3 * 1 = 3$

Current Max Area = 3

Decision:

Line at L (1) is shorter than line at R (3).

Moving R inward would just make width smaller while height is still bottlenecked by L(1).

Therefore, move L inward ($L = L + 1$).

STEP 2:

Left Pointer (L) at index 1, height 5

Right Pointer (R) at index 3, height 3

```

5 |   L
4 |   |   |
3 |   |   |   R
2 |   |   |   |
1 | |   |   |   |
0 +-----+
   0  1  2  3

```

Calculation:

Width = $3 - 1 = 2$

Height = $\min(5, 3) = 3$

Area = $2 * 3 = 6$

Current Max Area = $\max(3, 6) = 6$

Decision:

Line at R (3) is shorter than line at L (5).

Move R inward ($R = R - 1$).

STEP 3:

Left Pointer (L) at index 1, height 5

Right Pointer (R) at index 2, height 4

```

5 |   L
4 |   |   R
3 |   |   |   |
2 |   |   |   |

```

```

1 | | | | |
0 +-----+
   0  1  2  3

```

Calculation:

Width = 2 - 1 = 1

Height = min(5, 4) = 4

Area = 1 * 4 = 4

Current Max Area = max(6, 4) = 6

Decision:

Pointers are next to each other. The next move will cause them to cross, meaning we have ev

Final Answer: 6

3. Time and Space complexity analysis

Optimized Two Pointers Approach:

Time Complexity: $O(N)$ Where N is the number of elements in the height array.

Time Complexity Derivation Diagram:

Array size = N

```

[ L . . . . . R ] -> 1 calculation, move 1 step
  [ L . . . . . R ] -> 1 calculation, move 1 step
    [ L . . . . R ] -> 1 calculation, move 1 step
      ...
        [ L R ] -> Final calculation

```

Total Steps taken = N - 1 steps.

Since we only visit each element at most once, the time scales linearly with the input size.

Result = $O(N)$

Space Complexity: $O(1)$

Space Complexity Derivation Diagram:

Memory allocated:

- Variable `left\_pointer`: 1 integer
- Variable `right\_pointer`: 1 integer
- Variable `max\_area`: 1 integer
- Variable `current\_area`: 1 integer

Total extra memory = 4 integers.

Does this memory grow if N becomes 1,000,000? No, it remains 4 integers.

Since memory usage is constant regardless of input size, it is $O(1)$.

4. Solution Code

Python Snippets

```
# Approach A: Brute Force (Not recommended for final answer, but good to discuss)
# Time: O(N^2), Space: O(1)
def maxArea_brute(height):
    max_area = 0
    n = len(height)

    # Check every combination of left and right lines
    for i in range(n):
        for j in range(i + 1, n):
            width = j - i
            current_height = min(height[i], height[j])
            current_area = width * current_height

            if current_area > max_area:
                max_area = current_area

    return max_area

# Approach B: Two Pointers (Optimal)
# Time: O(N), Space: O(1)
def maxArea_optimal(height):
    max_area = 0
    left = 0
    right = len(height) - 1

    # Continue until the two pointers meet
    while left < right:
        width = right - left

        # Calculate area based on the shorter line
        if height[left] < height[right]:
            current_area = width * height[left]
            # Move the pointer pointing to the shorter line
            left += 1
        else:
            current_area = width * height[right]
            # Move the pointer pointing to the shorter line
            right -= 1

        # Update max_area if we found a bigger container
        if current_area > max_area:
            max_area = current_area
```

```
    return max_area
```

JavaScript Snippets

```
// Approach A: Brute Force (Not recommended for final answer, but good to discuss)  
// Time: O(N^2), Space: O(1)
```

```
function maxAreaBrute(height) {  
    let maxArea = 0;  
    const n = height.length;  
  
    // Check every combination of left and right lines  
    for (let i = 0; i < n; i++) {  
        for (let j = i + 1; j < n; j++) {  
            const width = j - i;  
            const currentHeight = Math.min(height[i], height[j]);  
            const currentArea = width * currentHeight;  
  
            if (currentArea > maxArea) {  
                maxArea = currentArea;  
            }  
        }  
    }  
    return maxArea;  
}
```

```
// Approach B: Two Pointers (Optimal)  
// Time: O(N), Space: O(1)
```

```
function maxAreaOptimal(height) {  
    let maxArea = 0;  
    let left = 0;  
    let right = height.length - 1;  
  
    // Continue until the two pointers meet  
    while (left < right) {  
        const width = right - left;  
        let currentArea = 0;  
  
        // Calculate area based on the shorter line and move that pointer  
        if (height[left] < height[right]) {  
            currentArea = width * height[left];  
            left++;  
        } else {  
            currentArea = width * height[right];  
            right--;  
        }  
    }  
}
```

```

        // Update maxArea if we found a bigger container
        if (currentArea > maxArea) {
            maxArea = currentArea;
        }
    }
    return maxArea;
}

```

Note 1: Terms and Techniques

Algorithm Technique: Two Pointers Strategy This is a pattern where two references (pointers) are used to iterate through a data structure, usually an array or string. In this problem, we used opposite-directional pointers (starting at both ends and moving toward the center).

Why it helps: It reduces time complexity dramatically. By applying a logical rule (always moving the shorter line), we safely eliminate large portions of the search space without having to explicitly test them. This turns a slow $O(N^2)$ brute force search into a blazing fast $O(N)$ single-pass algorithm.

Note 2: Real World / Interview Variations at FAANG

Top companies frequently modify this question to ensure candidates are not just reciting memorized LeetCode answers, but actually understand the underlying geometry and array manipulation concepts.

Variation 1: “Trapping Rain Water” (Frequently asked by Meta, Bloomberg, Google)

- **The Twist:** Instead of finding the area between two infinitely thin lines, the array represents the elevation of terrain blocks. You need to calculate how much water gets trapped *on top* of the terrain after it rains. (This is LeetCode 42).
- **How to solve it:** This also uses a Two Pointers approach, but the logic flips. Instead of calculating the area *between* pointers, you track the **left_max** height seen so far and the **right_max** height seen so far. The water trapped at any specific index *i* is determined by the shorter of the maximum bounds on its left and right, minus its own height. You move the pointer whose corresponding **max** is smaller.

Variation 2: X-Coordinates Provided (Frequently asked by Google)

- **The Twist:** In the standard problem, lines are evenly spaced by 1 unit (indices 0, 1, 2). Google interviewers often alter this by providing two arrays: **heights** and **x_coordinates**. The lines are placed at varying distances.

- **How to solve it:** The core Two Pointers logic remains completely intact. The only change is in calculating the width. Instead of `width = right - left`, you calculate it as `width = x_coordinates[right] - x_coordinates[left]`. The rule to move the shorter line still holds mathematically true.

Variation 3: Largest Rectangle in Histogram (Asked in senior loops)

- **The Twist:** Find the largest solid rectangle that can fit inside a bar chart. Unlike water, a solid rectangle cannot have gaps; it is limited by the shortest bar *anywhere between* its left and right bounds, not just the two endpoints.
- **How to solve it:** Two Pointers will fail here. A senior engineer recognizes that the bottleneck is no longer just the endpoints, but every element in between. This requires a Monotonic Stack strategy to keep track of bars in increasing order of height, calculating the area whenever a shorter bar is encountered that forces the rectangle to end.

88. Merge Sorted Array

Here is how a senior engineer approaches this problem. At the L5/L6 level, the focus is not just on getting the code to pass, but on demonstrating a deep understanding of memory management, optimal algorithms, and how such a pattern scales to real-world system design scenarios.

1. Problem Explanation

We are given two integer arrays, `nums1` and `nums2`, both sorted in ascending order. We are also given two integers, `m` and `n`, representing the number of valid elements in `nums1` and `nums2`, respectively.

The core challenge is to merge `nums2` into `nums1` so that the resulting array is sorted in ascending order.

The constraints that define the strategy:

- We must do this **in-place**. We cannot create a third array of size `m + n` to store the result and then copy it back. This tests our ability to manipulate memory safely.
- `nums1` is cleverly sized to be exactly `m + n`. The first `m` elements are the actual numbers, and the last `n` elements are initialized to 0 (serving as empty buffer space).

2. Solution Explanation

When approaching array manipulation where we must operate in-place, overwriting existing, unprocessed data is the primary trap.

The Unoptimized Approach (Append and Sort) The easiest, brute-force way is to simply copy the elements of `nums2` into the empty spaces (the zeroes) at the end of `nums1`, and then use the built-in sorting function on `nums1`.

While this works and is easy to implement, it throws away the most valuable piece of information we were given: **both arrays are already sorted**. Sorting from scratch ignores this and wastes computational cycles.

The Optimized Approach (Three Pointers, Backwards Merge) To achieve peak efficiency, we must leverage the fact that both arrays are pre-sorted. Normally, to merge sorted arrays, we use two pointers starting from the beginning (index 0). However, if we do that here, placing a small element from `nums2` into the front of `nums1` will overwrite a valid element in `nums1` before we have a chance to evaluate it.

The breakthrough insight is to **work backward**.

Because the end of `nums1` is filled with zeroes (empty space), we can safely write the *largest* elements to the back of `nums1` without overwriting any valid, unmerged elements.

We will use three pointers:

1. `p1`: Points to the last valid element in `nums1` (index $m - 1$).
2. `p2`: Points to the last element in `nums2` (index $n - 1$).
3. `p`: Points to the very end of `nums1` (index $m + n - 1$), which is our write position.

We compare the elements at `p1` and `p2`. The larger element gets written to `p`. Then, we decrement `p` and the pointer of the array we just pulled from (`p1` or `p2`).

Visual Walkthrough: Let `nums1` = [1, 2, 3, 0, 0, 0], $m = 3$ Let `nums2` = [2, 5, 6], $n = 3$

STEP 1: Initial state. Compare `nums1[p1]` (3) and `nums2[p2]` (6). 6 is larger.

```
nums1: [ 1, 2, 3, 0, 0, 0 ]
          ^       ^
        p1      p
```

```
nums2: [ 2, 5, 6 ]
          ^
        p2
```

STEP 2: Write 6 at `p`. Decrement `p2` and `p`. Compare 3 and 5. 5 is larger.

```
nums1: [ 1, 2, 3, 0, 0, 6 ]
          ^       ^
        p1      p
```

```
nums2: [ 2, 5, 6 ]
```

^
p2

STEP 3: Write 5 at p. Decrement p2 and p. Compare 3 and 2. 3 is larger.
 nums1: [1, 2, 3, 0, 5, 6]

^ ^
p1 p

nums2: [2, 5, 6]
 ^
 p2

STEP 4: Write 3 at p. Decrement p1 and p. Compare 2 and 2. Tie! Pick nums2[p2] (2).
 nums1: [1, 2, 3, 3, 5, 6]

^ ^
p1 p

nums2: [2, 5, 6]
 ^
 p2

STEP 5: Write 2 at p. Decrement p2 and p. p2 is now less than 0. We are done!
 nums1: [1, 2, 2, 3, 5, 6]

^
p1/p

nums2: [2, 5, 6]
 ^
 p2 (-1)

If p1 runs out before p2, we just copy the rest of nums2 into nums1. If p2 runs out first, we are completely done because the remaining elements in nums1 are already in their correct, sorted positions.

3. Time and Space Complexity Analysis

```
+-----+
|  TIME COMPLEXITY DERIVATION  |
+-----+
|  Total elements to position = m + n  |
|  |                                  |
|  While loop execution:           |
|  - Each iteration places exactly one element into `nums1`. |
|  - The loop runs at most m + n times. |
|  - Comparisons and assignments are constant time operations. |
|  |                                  |
```

```

| Mathematical breakdown:
| T(m, n) = c * (m + n) where c is a constant.
|
| Result: O(m + n) Time
+-----+

+-----+
| SPACE COMPLEXITY DERIVATION
+-----+
| Memory allocated for pointers:
| - p1 (integer)
| - p2 (integer)
| - p (integer)
|
| Memory allocated for arrays:
| - None. All modifications happen in-place within the
|   existing `nums1` array structure.
|
| Result: O(1) Space
+-----+

```

For the brute force approach, Time Complexity is $O((m+n) * \log(m+n))$ due to sorting, and Space Complexity is $O(1)$ or $O(\log(m+n))$ depending on the language's under-the-hood sorting implementation (e.g., Timsort in Python uses extra space).

4. Solution Code

Python Snippets

```

# =====
# APPROACH 1: Unoptimized (Append and Sort)
# Good to mention in an interview to establish a baseline.
# =====
def merge_naive(nums1: list[int], m: int, nums2: list[int], n: int) -> None:
    """
    Do not return anything, modify nums1 in-place instead.
    """
    # Replace the trailing zeros in nums1 with the elements of nums2
    for i in range(n):
        nums1[m + i] = nums2[i]

    # Sort the combined array
    nums1.sort()

# =====

```

```

# APPROACH 2: Optimized (Three Pointers Backwards)
# The expected L5/L6 solution.
# =====
def merge_optimal(nums1: list[int], m: int, nums2: list[int], n: int) -> None:
    """
    Do not return anything, modify nums1 in-place instead.
    """
    # Initialize pointers to the last valid elements and the last insert position
    p1 = m - 1
    p2 = n - 1
    p = m + n - 1

    # While there are still elements in nums2 to evaluate
    while p2 >= 0:
        # If nums1 still has valid elements AND the current nums1 element is larger
        if p1 >= 0 and nums1[p1] > nums2[p2]:
            nums1[p] = nums1[p1]
            p1 -= 1
        else:
            # Otherwise, the nums2 element is larger (or p1 is exhausted)
            nums1[p] = nums2[p2]
            p2 -= 1

    # Move the insert pointer backwards regardless of which element we chose
    p -= 1

```

JavaScript Snippets

```

// =====
// APPROACH 1: Unoptimized (Append and Sort)
// =====
var merge_naive = function(nums1, m, nums2, n) {
    // Overwrite the zeros at the end of nums1 with nums2
    for (let i = 0; i < n; i++) {
        nums1[m + i] = nums2[i];
    }
    // Sort in-place. Note: JS sort() converts to strings by default,
    // so a comparator function is strictly required for integers.
    nums1.sort((a, b) => a - b);
};

// =====
// APPROACH 2: Optimized (Three Pointers Backwards)
// =====
var merge_optimal = function(nums1, m, nums2, n) {

```



```

let p1 = m - 1;
let p2 = n - 1;
let p = m + n - 1;

// Process backwards until all elements from nums2 are merged
while (p2 >= 0) {
    if (p1 >= 0 && nums1[p1] > nums2[p2]) {
        nums1[p] = nums1[p1];
        p1--;
    } else {
        nums1[p] = nums2[p2];
        p2--;
    }
    p--;
}
};

```

Note 1: Algorithm Concepts Used

Reverse Traversal / Backwards Two-Pointer Technique: Normally, iteration processes data from left to right (index 0 to N). This technique flips that paradigm. It helps specifically in in-place array transformations where the front of the array contains critical data you are actively reading, but the back of the array contains garbage or buffer space. By filling the buffer space first from right to left, you guarantee that your write operations will never collide with your unread read operations.

Note 2: Real-World & Interview Variations at Top Tech Companies

At the senior engineering level, interviewers often abstract this exact algorithmic pattern into system design or domain-specific coding questions.

1. Google: Distributed Log Merging (Observability / CMS)

- **The Scenario:** You have a massive Content Management System distributed across multiple clusters. Two different servers generate localized, chronological (sorted) logs of system events. The logs are too large to fit into memory all at once. How do you merge them into a single chronological log file?
- **The Solution:** This is a variation of “Merge Sorted Arrays” applied to files, often called an **External Sort** merge phase. Instead of pointers in an array, you use file pointers. You read a small chunk (buffer) of logs from File A and File B into memory. You compare the earliest timestamps, write the oldest log to the final Output File, and advance the pointer.

When a memory buffer empties, you stream the next chunk from the disk. This applies the exact same $O(m+n)$ time logic, but adapted to optimize disk I/O and strict memory limits.

2. Meta: Merging Feed Timelines

- **The Scenario:** You need to merge a user’s “organic” post timeline (Array A) with an “ad/sponsored” timeline (Array B) to serve to the client. Both are sorted by a ranking score. You want to merge them in-place or into a unified stream with the highest scores first.
- **The Solution:** You apply the two-pointer technique. If the prompt requires descending order (highest score first), you start your pointers from the front (index 0) rather than the back, assuming you are pushing to a new stream or API response rather than doing an in-place array mutation. If there are ties in the score, you implement a tie-breaker (e.g., if scores are equal, the organic post wins over the ad).

3. Bloomberg: Merging Real-Time Market Tick Data

- **The Scenario:** You receive sorted arrays of stock tick data (timestamp, price) from two different exchanges (e.g., NYSE and NASDAQ) for the same ticker. You need to merge them.
- **The Solution:** You use the two-pointer technique. However, Bloomberg interviewers will often add a constraint: “What if there are exact timestamp duplicates? We only want to keep one.” The solution requires modifying the standard `if/else` logic to include an `else if (nums1[p1] == nums2[p2])`. In that block, you write one of the values to your output, decrement *both* `p1` and `p2`, and gracefully handle the deduplication on the fly in $O(m+n)$ time.

189. Rotate Array

Here is how a senior engineer (L5/L6) at a top tech company approaches “189. Rotate Array”. At this level, the focus is not just on getting an accepted answer, but on code maintainability, space-time trade-offs, handling edge cases gracefully (like when `k` is larger than the array itself), and explaining the “why” behind the code.

1. Problem Explanation

The Goal: We are given an array of numbers. We need to shift every element to the right by `k` positions. When an element reaches the end of the array, it “wraps around” and moves to the beginning.

The Edge Case (Crucial L5 Observation): What happens if the array has 5 elements, and `k = 7`? Rotating an array of length 5 by 5 steps results in the exact same original array. Therefore, rotating it 7 times is mathematically identical to rotating it 2 times (because $7 \bmod 5 = 2$).

Before doing any work, we must normalize k: $\text{Effective Rotations} = k \% \text{Length of Array}$

Visualizing the Problem: Let's use an array `nums = [1, 2, 3, 4, 5, 6, 7]` and `k = 3`.

Initial Array:

Index: 0 1 2 3 4 5 6
Value: [1, 2, 3, 4, 5, 6, 7]

We want to shift everything 3 steps to the right.

The last 3 elements (5, 6, 7) will fall off the edge and wrap to the front.

Final Expected Array:

Index: 0 1 2 3 4 5 6
Value: [5, 6, 7, 1, 2, 3, 4]

2. Solution Explanation

We will look at two solutions. The first is a reliable baseline that is easy to reason about. The second is the optimal, elegant solution expected of a strong candidate.

Solution A: The Extra Array (The Pragmatic Baseline) If we aren't restricted by memory, the easiest way to solve this is to create a brand new array. We calculate exactly where each element *should* go and place it there.

- Formula: The new index for any element at `current_index` will be $(\text{current_index} + k) \% \text{array_length}$.

Example: `nums = [1, 2, 3]`, `k = 1`, `length = 3`

Original:

[1, 2, 3]
 [^] [^] [^]

i=0 i=1 i=2

Calculating new positions:

Value 1 at i=0 moves to: $(0 + 1) \% 3 = 1$

Value 2 at i=1 moves to: $(1 + 1) \% 3 = 2$

Value 3 at i=2 moves to: $(2 + 1) \% 3 = 0$ <-- Wraps around beautifully!

New Array:

[3, 1, 2]

Solution B: The In-Place Reversal (The Optimal Choice) Creating a new array requires $O(N)$ extra memory. In systems with massive arrays, allocating that memory is expensive. We can do this “in-place” (modifying the original array directly) using a brilliant mathematical trick involving array reversals.

The Reversal Strategy:

1. **Reverse the entire array:** This moves the elements that were at the end (which need to be at the front) to the front, but they are backwards.
2. **Reverse the first k elements:** This fixes the order of the elements that wrapped around.
3. **Reverse the remaining $n - k$ elements:** This fixes the order of the original front elements that were pushed to the back.

Step-by-Step Reversal Visualization: `nums = [1, 2, 3, 4, 5, 6, 7],`
`k = 3`

START: `[ 1, 2, 3, 4, 5, 6, 7 ]`

STEP 1: Reverse the whole array (indices 0 to 6)

Operation: Swap ends moving inwards.

Result 1: `[ 7, 6, 5, 4, 3, 2, 1 ]`
                               ~~~~~

Notice how 5,6,7 are now at the front, but backwards!

STEP 2: Reverse the first  $k$  elements (indices 0 to 2)

Operation: Reverse [7, 6, 5] into [5, 6, 7]

Result 2:                `[ 5, 6, 7, 4, 3, 2, 1 ]`  
                                                       ~~~~~

Now the front is perfect. The back is still backwards.

STEP 3: Reverse the remaining elements (indices 3 to 6)

Operation: Reverse [4, 3, 2, 1] into [1, 2, 3, 4]

Result 3: `[ 5, 6, 7, 1, 2, 3, 4 ] <-- Done!`

3. Time and Space Complexity Analysis

Here is the derivation using ASCII to avoid complex formatting:

Reversal Solution Analysis

TIME COMPLEXITY DERIVATION: $O(N)$

Let N = total number of elements in the array.

Let K = number of rotation steps.

Task 1: Reverse all N elements.

Every swap processes 2 elements.

[+ + + + +] ---> Takes roughly $N/2$ swaps ---> N operations

Task 2: Reverse first K elements.

[* * *] - - - ---> Takes $K/2$ swaps ---> K operations

Task 3: Reverse remaining $(N - K)$ elements.

- - - [* * * * *] ---> Takes $(N-K)/2$ swaps ---> $(N - K)$ operations

Total Time Operations = $N + K + (N - K)$
= $N + N$
= $2 * N$

Dropping the constant multiplier (2), we get a Time Complexity of $O(N)$.

SPACE COMPLEXITY DERIVATION: $O(1)$

Original Array in Memory:

| 1 | 2 | 3 | 4 | 5 |

Pointers used for swapping (Variables: start, end, temp):

[p1] [p2] [temp] ---> Takes exactly 3 extra slots of memory, regardless of array size.

Since the memory used does not grow with N , Space Complexity = $O(1)$ (Constant Space).

For the **Extra Array Solution**, Time is $O(N)$ because we iterate through the array once. Space is $O(N)$ because we create a completely new array of size N to hold the shifted values.

4. Solution Code (Python and JavaScript)

Python Snippets

```
# OPTIMIZED SOLUTION:  $O(N)$  Time,  $O(1)$  Space
class SolutionOptimized:
    def rotate(self, nums: list[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        # Handle edge case where k is greater than array length
        k = k % n

        # Helper function to reverse a portion of the array in-place
        def reverse(start: int, end: int) -> None:
            while start < end:
```

```

        # Python allows easy swapping without a explicit temp variable
        nums[start], nums[end] = nums[end], nums[start]
        start += 1
        end -= 1

    # Step 1: Reverse the whole array
    reverse(0, n - 1)
    # Step 2: Reverse the first k elements
    reverse(0, k - 1)
    # Step 3: Reverse the remaining elements
    reverse(k, n - 1)

# SUB-OPTIMAL (BUT USEFUL) SOLUTION: O(N) Time, O(N) Space
class SolutionExtraArray:
    def rotate(self, nums: list[int], k: int) -> None:
        n = len(nums)
        k = k % n

        # Create a new array filled with zeros
        rotated_array = [0] * n

        # Place each element in its new calculated position
        for i in range(n):
            new_index = (i + k) % n
            rotated_array[new_index] = nums[i]

        # Copy the new array back into the original array reference
        for i in range(n):
            nums[i] = rotated_array[i]

```

JavaScript Snippets

```

// OPTIMIZED SOLUTION: O(N) Time, O(1) Space
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var rotateOptimized = function(nums, k) {
    const n = nums.length;
    // Normalize k to prevent unnecessary rotations
    k = k % n;

    // Helper function to reverse elements between two indices
    const reverse = (start, end) => {

```

```

        while (start < end) {
            let temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    };

    // Execute the 3-step reversal algorithm
    reverse(0, n - 1);      // Reverse all
    reverse(0, k - 1);      // Reverse front part
    reverse(k, n - 1);      // Reverse back part
};

// SUB-OPTIMAL (BUT USEFUL) SOLUTION: O(N) Time, O(N) Space
var rotateExtraArray = function(nums, k) {
    const n = nums.length;
    k = k % n;

    // Allocate new array memory
    const rotatedArray = new Array(n);

    // Calculate new positions using modulo
    for (let i = 0; i < n; i++) {
        rotatedArray[(i + k) % n] = nums[i];
    }

    // Mutate the original array to hold the new values
    for (let i = 0; i < n; i++) {
        nums[i] = rotatedArray[i];
    }
};

```

Note 1: Techniques & Terminology Used

1. **Modulo Arithmetic (%):** This is the mathematical operation that finds the remainder of division.
 - *Why it helps:* It creates a “circular” or “wrap-around” effect. If you have an array of length 5, and index 4 moves right by 1, $4 + 1 = 5$. Index 5 is out of bounds. But $(4 + 1) \% 5 = 0$, successfully wrapping it back to the start.

2. **In-Place Algorithm:** An algorithm that transforms data without using auxiliary data structures (ignoring a small constant number of variables like pointers).
- *Why it helps:* It saves massive amounts of memory (RAM) and avoids the CPU overhead of allocating and garbage-collecting new arrays.
3. **Two-Pointer Reversal:** Setting one pointer at the start and one at the end of a segment, swapping their values, and moving them toward the center until they meet.

Note 2: Real-World & Interview Variations (Google, Meta, Bloomberg)

Top companies rarely ask the raw LeetCode question directly. They disguise it.

1. Meta Variation: “String Shifting / Caesar Cipher”

- **The Problem:** You are given a string *s* and a shift value *k*. Shift all characters in the string *k* positions to the right.
- **How to solve it:** The logic is identical, but the *data type* matters. In languages like Python and JavaScript, strings are immutable (cannot be changed in-place). Therefore, an L5 engineer must point out that the $O(1)$ space reversal algorithm is impossible for strings in these languages. You *must* allocate an $O(N)$ character array, build the shifted string using the Extra Array logic (or string slicing), and join it back.

2. Google Variation: “Circular Log Buffer / PagerDuty On-Call Rotation”

- **The Problem:** You have a fixed-size list of engineers on call: ["Alice", "Bob", "Charlie", "Dave"]. A system outage pauses the rotation schedule for *X* weeks. Given the current list, generate the new on-call list shifted by *X* weeks.
- **How to solve it:** This is exactly the array rotation problem. Because it represents a real-world system, Google interviewers will push hard on the edge case: What if *X* is negative (rewinding the schedule)?
- *Handling Negative K:* If $k < 0$, shifting left by *k* is mathematically exactly the same as shifting right by $n - (\text{abs}(k) \% n)$. You normalize negative *k* using this formula, and then apply the standard Reversal Algorithm perfectly.

3. Bloomberg Variation: “Stock Ticker Rolling Window”

- **The Problem:** We receive a stream of stock prices and keep the last 24 hours of hourly data in an array of size 24. A network delay caused 3 new hours of data to arrive at once. We need to shift the old array to make room for the new data.

- **How to solve it:** This is a partial rotation/shift. You use the Reversal logic to shift the valid data to the right by 3 spots, overwriting the oldest 3 elements, and then you insert the 3 new stock prices into the newly freed indices 0, 1, and 2. This proves you understand *how* to manipulate memory addresses efficiently without creating garbage arrays that a high-frequency trading system (like Bloomberg's) would choke on.

3349. Adjacent Increasing Subarrays Detection I

As an L5/L6 engineer, when I look at a problem like this, I immediately think about how it scales. While the “Part I” version of this problem might allow for a less efficient approach, a top-tier engineer will aim for the most optimal solution right out of the gate—one that processes the data in a single pass with minimal memory footprint.

Let's break down exactly how to understand, visualize, and solve this problem from the ground up.

1. Problem Explanation

The Goal: You are given an array of numbers and a target length, k . You need to find if there are **two** subarrays (contiguous chunks of the array) that meet all three of these conditions:

1. **Length:** Both subarrays must be exactly of length k .
2. **Adjacency:** They must sit exactly next to each other. The second subarray must start immediately after the first one ends.
3. **Strictly Increasing:** Inside each subarray, every number must be strictly greater than the number before it. (e.g., 1, 2, 3 is valid. 1, 2, 2 is invalid).

Visualizing the Constraints:

Let's say $k = 3$. We need a total block of $2 * k = 6$ elements divided into two halves.

Valid Example:

```
Array: [ 5,  8, 12,  2,  9, 15,  1,  3 ]
Index:  0   1   2   3   4   5   6   7
```

Subarray 1: [5, 8, 12] --> Strictly increasing? YES. Length 3? YES.

Subarray 2: [2, 9, 15] --> Strictly increasing? YES. Length 3? YES.

Are they adjacent? YES (Subarray 1 ends at index 2, Subarray 2 starts at index 3).

Result: TRUE

Invalid Example:

```
Array: [ 5,  8, 12,  9, 10, 11 ]
```

$k = 3$

Subarray 1: [5, 8, 12] --> YES
 Subarray 2: [9, 10, 11] --> YES
 Are they adjacent? YES.
 Wait, let's look closer. If Subarray 1 is indices 0,1,2 and Subarray 2 is 3,4,5...
 Wait! 5->8->12 is increasing. 9->10->11 is increasing. This IS valid.

Let's look at a TRULY invalid one:
 Array: [5, 8, 12, 10, 9, 11]

Subarray 1: [5, 8, 12] --> YES
 Subarray 2: [10, 9, 11] --> NO (10 to 9 is a decrease).
 Result: FALSE

2. Solution Explanation

There are two main ways to solve this. I will explain both, as discussing the brute force first shows good problem-solving progression.

Approach A: The Brute Force (Non-Optimized) The most intuitive way is to act like a scanner. You take a “window” of size $2 * k$ and slide it across the array from left to right. At every step, you split the window down the middle. You check if the left half is strictly increasing, and if the right half is strictly increasing. If both are, you’re done!

- **Why it’s not ideal:** If k is large, you are re-checking the same numbers over and over again. It does unnecessary overlapping work.

Approach B: Contiguous Block Tracking (Optimal L5/L6 Solution)
 Instead of sliding a window, let’s change our perspective. Any array can be broken down into “Blocks” of strictly increasing numbers.

Whenever a number drops or stays the same (e.g., going from 12 to 2), that is a “boundary” that breaks a block.

If we just read the array from left to right and track the **length of the current increasing block** and the **length of the previous increasing block**, we can solve this in one pass.

There are exactly **two cases** where we can find our answer:

Case 1: Both subarrays are inside a single massive block. If we have a block that has been strictly increasing for a very long time, we don’t even need two separate blocks. We can just slice one big block in half! *Condition:* `current_block_length >= 2 * k`

Array: [10, 20, 30, 40, 50, 60]
k = 3

```
[ 10 -> 20 -> 30 -> 40 -> 50 -> 60 ]
+-----+
|          CURRENT BLOCK LENGTH = 6          |
+-----+
```

Since $6 \geq (2 * 3)$, we can split it internally!

Subarray 1: 10, 20, 30

Subarray 2: 40, 50, 60

Case 2: The subarrays span across a boundary. If the current block isn't big enough on its own, maybe we can combine the end of the *previous* block with the beginning of the *current* block. Because the previous block ended exactly where the current block started, taking a chunk from the end of the previous and a chunk from the start of the current guarantees they are adjacent! *Condition:* $\text{previous_block_length} \geq k$ AND $\text{current_block_length} \geq k$

Array: [5, 6, 7, 8, 1, 2, 3]
k = 3

Block 1: [5, 6, 7, 8] --> length 4 (Previous)

Block 2: [1, 2, 3] --> length 3 (Current)

Take the LAST k elements of Previous: [6, 7, 8]

Take the FIRST k elements of Current: [1, 2, 3]

Because they meet at the boundary between 8 and 1, they are adjacent!

If we track just those two variables (*prev_len* and *curr_len*) as we loop through the array exactly once, we can spot either Case 1 or Case 2 immediately.

3. Time and Space Complexity Analysis

We will evaluate the Optimal Approach (Approach B).

+-----+		
TIME COMPLEXITY DERIVATION: O(N)		
+-----+		
Step	Operation	Cost
+-----+		
1.	Initialize variables (prev_len, curr_len)	1 operation
2.	Loop through array elements exactly once	N steps
3.	Inside loop: Compare with previous element	1 op per step
4.	Inside loop: Update prev/curr variables	1 op per step
5.	Inside loop: Check Case 1 & Case 2 conditions	2 ops per step

+-----+ TOTAL TIME: roughly 4 * N operations. Drops constants -> O(N) time +-----+		
+-----+ SPACE COMPLEXITY DERIVATION: O(1) +-----+		
Memory Allocated	Purpose	Size
prev_len	Stores an integer	O(1)
curr_len	Stores an integer	O(1)
i (iterator)	Stores current index	O(1)
n (length cache)	Stores array length	O(1)
+-----+		
TOTAL SPACE: Only primitive variables used -> O(1) auxiliary space +-----+		

4. Solution Code (Python & JavaScript)

Here are the implementations for both languages, showcasing both the Brute Force (good for explaining your thought process) and the Optimal approach (what you write to pass the interview).

Python Solutions

```
class Solution:
    # -----
    # APPROACH 1: Brute Force (Sliding Window)
    # Useful to explain to an interviewer before writing optimal code.
    # Time: O(N * K), Space: O(1)
    # -----
    def hasIncreasingSubarrays_BruteForce(self, nums: list[int], k: int) -> bool:
        n = len(nums)
        if n < 2 * k:
            return False

        # Helper function to check if a specific chunk is strictly increasing
        def is_strictly_increasing(start, end):
            for i in range(start, end - 1):
                if nums[i] >= nums[i + 1]:
                    return False
            return True

        # Slide a window of size 2*k across the array
        for i in range(n - 2 * k + 1):
```

```

        # Check left half and right half
        if is_strictly_increasing(i, i + k) and is_strictly_increasing(i + k, i + 2 * k):
            return True

    return False

# -----
# APPROACH 2: Optimal Contiguous Block Tracking
# Time: O(N), Space: O(1)
# -----
def hasIncreasingSubarrays(self, nums: list[int], k: int) -> bool:
    n = len(nums)
    if n < 2 * k:
        return False

    prev_len = 0
    curr_len = 1

    for i in range(1, n):
        # If the current element strictly increases, extend the current block
        if nums[i] > nums[i - 1]:
            curr_len += 1
        else:
            # The increasing streak broke.
            # Current block becomes the previous block, and we start a new current block
            prev_len = curr_len
            curr_len = 1

        # Case 1: Can we form two subarrays inside the current block alone?
        if curr_len >= 2 * k:
            return True

        # Case 2: Can we form one subarray from the end of the previous block
        # and one from the start of the current block?
        if prev_len >= k and curr_len >= k:
            return True

    return False

```

JavaScript Solutions

```

/**
 * -----
 * APPROACH 1: Brute Force (Sliding Window)
 * Time: O(N * K), Space: O(1)
 * -----

```

```

*/
function hasIncreasingSubarrays_BruteForce(nums, k) {
    const n = nums.length;
    if (n < 2 * k) return false;

    // Helper function to check if a slice is strictly increasing
    const isStrictlyIncreasing = (start, end) => {
        for (let i = start; i < end - 1; i++) {
            if (nums[i] >= nums[i + 1]) {
                return false;
            }
        }
        return true;
    };

    // Slide window
    for (let i = 0; i <= n - 2 * k; i++) {
        if (isStrictlyIncreasing(i, i + k) && isStrictlyIncreasing(i + k, i + 2 * k)) {
            return true;
        }
    }

    return false;
}

/**
 * -----
 * APPROACH 2: Optimal Contiguous Block Tracking
 * Time: O(N), Space: O(1)
 * -----
 */
function hasIncreasingSubarrays(nums, k) {
    const n = nums.length;
    if (n < 2 * k) return false;

    let prevLen = 0;
    let currLen = 1;

    for (let i = 1; i < n; i++) {
        // Continue the current streak
        if (nums[i] > nums[i - 1]) {
            currLen++;
        } else {
            // Streak broken, shift state
            prevLen = currLen;
            currLen = 1;
        }
    }

```

```

    }

    // Case 1: A single massive block
    if (currLen >= 2 * k) {
        return true;
    }

    // Case 2: Spanning across the boundary of two blocks
    if (prevLen >= k && currLen >= k) {
        return true;
    }
}

return false;
}

```

Note 1: The Underlying Technique

Algorithm Name: Contiguous State Tracking (or Run-Length Encoding of Monotonicity).

- **What it is:** Instead of looking at individual elements, you abstract the array into a series of “states” or “blocks” (in this case, segments of strictly increasing numbers) and only track the metadata of those blocks (their lengths).
 - **Why it helps:** It condenses the history of the array traversal into just one or two integer variables (`prev_len`, `curr_len`), completely eliminating the need for nested loops or secondary arrays.
 - **How it applies here:** By proving that valid adjacent subarrays must either exist entirely within one block or perfectly span the exact boundary of two adjacent blocks, tracking the lengths of just the last two blocks is mathematically sufficient to find any valid answer.
-

Note 2: Real-World & Big Tech Interview Variations

Companies rarely ask Leetcode questions verbatim. They disguise them in business logic. Here is how this exact problem is disguised at top tier companies:

1. Google: Server Cascading Failure Detection

- **The Prompt:** “We are monitoring CPU load metrics over time. A ‘compounding surge’ is defined as a period of exactly K minutes of strictly increasing CPU load, followed immediately by another exactly K minutes of strictly increasing load (often after a brief system reset/drop). Given

an array of minute-by-minute CPU loads, return true if a compounding surge occurred.”

- **How to solve:** This is literally the exact same problem. The “brief system reset” is the boundary between `prev_len` and `curr_len`. You would implement the $O(N)$ Block Tracking solution exactly as written above, iterating through the `cpu_loads` array.

2. Meta: Viral Engagement Streaks

- **The Prompt:** “We track Daily Active Users (DAU) for a new feature. We want to reward creators who achieve a ‘Double Growth Week’. This means 7 consecutive days of strictly increasing views, immediately followed by another 7 consecutive days of strictly increasing views. Write a function to detect this in an array of DAU counts.”
- **How to solve:** Here, `k` is hardcoded to 7. You apply the exact same algorithm. If `currLen >= 14` (14 days of straight growth) or `prevLen >= 7` and `currLen >= 7` (streak broke but restarted strongly), you flag the creator for the reward.

3. Bloomberg: Stock Market “Double Breakout” Pattern

- **The Prompt:** “A ‘Double Breakout’ is a technical trading pattern. It occurs when a stock’s closing price rises strictly for `K` days, experiences a sudden drop or plateau, and then immediately rises strictly for another `K` days. Detect if a stock ticker history contains this pattern.”
- **How to solve:** The logic is identical, but Bloomberg interviewers often ask you to return the *starting index* of the pattern rather than just a boolean.
- **Modification:** To return the starting index, you just track where your blocks started.
- If Case 1 hits (`curr_len >= 2 * k`), the start index is `i - 2 * k + 1`.
- If Case 2 hits (`prev_len >= k` and `curr_len >= k`), the start index is `i - curr_len - k + 1` (because we step back the current block length, and then step back `k` into the previous block).

3350. Adjacent Increasing Subarrays Detection II

This is a fantastic problem to dissect for a top-tier Google L5/L6 software engineering interview. As a senior or staff-level candidate, your interviewer expects you to swiftly bypass the brute-force $O(N * N * N)$ traps, confidently identify the $O(N * \log N)$ optimization, and ultimately land on the most elegant $O(N)$ single-pass approach.

Let us break down exactly how you would drive this discussion in a real Google interview.

1. Problem explanation

You are given an array of numbers. Your goal is to find the maximum length k such that you can pull out **two** contiguous subarrays from the array that meet three specific rules:

1. **Strictly Increasing:** Every number in the first subarray must be greater than the one before it. The same applies to the second subarray.
2. **Length k :** Both subarrays must be exactly k elements long.
3. **Adjacent:** The second subarray must start exactly one index after the first subarray ends.

Let us visualize this with an example. Suppose `nums = [2, 5, 7, 8, 9, 2, 3, 4, 3, 1]`.

INDEX:	0	1	2	3	4	5	6	7	8	9
ARRAY:	[2,	5,	7,	8,	9,	2,	3,	4,	3,	1]

We can extract two adjacent subarrays of length $k = 3$:

Subarray A:	[7,	8,	9]	(Starts at index 2, strictly increasing)	
Subarray B:		[2,	3,	4]	(Starts at index 5, strictly increasing)

Are they adjacent? Yes, B starts immediately after A.

Both are strictly increasing.

Both have length 3.

Therefore, $k = 3$ is our answer.

2. Solution explanation

A junior engineer might try to test every possible length k from 1 to $N / 2$ and scan the array for valid subarrays. That is far too slow.

As an L5/L6 candidate, you should immediately recognize that this problem is simply asking us to analyze “runs” (continuous segments) of strictly increasing numbers.

If we break the entire array into maximal increasing segments, we only have two possible ways to form our target adjacent subarrays:

Scenario A: Splitting a single long run If we have a single continuous increasing sequence of length L , we can slice it down the middle to form two adjacent increasing subarrays of length $L / 2$.

Array:	[10,	20,	30,	40,	50,	60]

	Current Run Length (L) = 6					

We can split this exactly in half to get $k = L / 2 = 3$:

[10,	20,	30]	[40,	50,	60]
-------	-----	------	-------	-----	------



$k = 3$



$k = 3$

Scenario B: Combining two adjacent runs When one increasing run ends and the next increasing run begins, these two runs are inherently adjacent. To get two subarrays of the same length k , we can grab the “tail” (suffix) of the first run and the “head” (prefix) of the second run. The maximum length we can extract is limited by the shorter of the two runs.

```

Array: [ 2, 5, 7, 8, 9, 2, 3, 4 ]
        |-----| |-----|
        Prev Run = 5   Curr Run = 3

```

Since we need equal lengths, we take the minimum of both runs: $\min(5, 3) = 3$. Take the last 3 of Prev, and the first 3 of Curr:

$$\begin{bmatrix} 7 & 8 & 9 \end{bmatrix}$$

$$|-----|$$

$$k = 3$$

$$\begin{bmatrix} 2 & 3 & 4 \end{bmatrix}$$

$$|-----|$$

$$k = 3$$

The Single-Pass Strategy: We do not need to precompute or store all the segments. We can simply iterate through the array once, keeping track of exactly two variables:

- `curr_len`: The length of the increasing run we are currently traversing.
- `prev_len`: The length of the immediately preceding increasing run.

At every single step of the loop, we check:

1. What is the max k if we split `curr_len`? $\rightarrow \text{curr_len} / 2$
2. What is the max k if we combine `prev_len` and `curr_len`? $\rightarrow \min(\text{prev_len}, \text{curr_len})$ We keep a running maximum of these values, and by the end of the array, we have our answer.

3. Time and Space complexity analysis

```
| TIME COMPLEXITY DERIVATION: O(N)
+-----+
| Input Array Size = N
|
| Index: 0    1    2    3    4    5    6    7
| Array: [*,  *,  *,  *,  *,  *,  *,  *]
|         \_--\_--\_--\_--\_--\_--\_-- Single linear scan
|
| Work done per element:
| 1. Compare nums[i] with nums[i-1]           => O(1) constant time
| 2. Increment counter or reset counter        => O(1) constant time
| 3. Calculate Math.max and Math.min            => O(1) constant time
```

```

|
| Total Time = N iterations * O(1) work = O(N) overall time.
|
+-----+

+-----+
| SPACE COMPLEXITY DERIVATION: O(1)
+-----+
| Data structures used:
| - integer `ans`      => 1 unit of memory
| - integer `curr_len` => 1 unit of memory
| - integer `prev_len` => 1 unit of memory
|
| Regardless of how large N grows (even N = 200,000), we strictly
| use exactly 3 integer variables.
|
| Total Space = 3 variables = O(1) auxiliary space.
|
+-----+

```

4. Solution Code

As a senior candidate, it is highly recommended to mention the Suboptimal approach ($O(N * \log N)$) first to show you understand the search space, before gracefully pivoting into writing the optimal ($O(N)$) code. I will provide both.

Python Snippets

```

# --- OPTIMAL SOLUTION (O(N) Time, O(1) Space) ---
# This is the code you want to write on the whiteboard.
class Solution:
    def maxIncreasingSubarrays(self, nums: list[int]) -> int:
        ans = 0
        curr_len = 1
        prev_len = 0

        # Start from the second element
        for i in range(1, len(nums)):
            if nums[i] > nums[i - 1]:
                # The run continues to increase
                curr_len += 1
            else:
                # The run broke. Current becomes previous, start a new current run.
                prev_len = curr_len
                curr_len = 1

        # At each step, update our global max based on the two scenarios:
        # 1. Splitting the current run: curr_len // 2

```

```

        # 2. Combining previous and current: min(prev_len, curr_len)
        ans = max(ans, curr_len // 2, min(prev_len, curr_len))

    return ans

# --- SUBOPTIMAL SOLUTION (O(N * log N) Time, O(N) Space) ---
# Useful to discuss if the interviewer asks: "What if we wanted to binary search the answer"
class SolutionSuboptimal:
    def maxIncreasingSubarrays(self, nums: list[int]) -> int:
        n = len(nums)
        # inc[i] will store the length of the strictly increasing subarray STARTING at index i
        inc = [1] * n
        for i in range(n - 2, -1, -1):
            if nums[i] < nums[i + 1]:
                inc[i] = inc[i + 1] + 1

        # Helper to validate if a specific k is possible
        def can_form(k: int) -> bool:
            # We iterate up to n - 2*k to ensure we have room for two adjacent k-length arrays
            for i in range(n - 2 * k + 1):
                if inc[i] >= k and inc[i + k] >= k:
                    return True
            return False

        # Binary search for the maximum k
        low, high = 1, n // 2
        ans = 1

        while low <= high:
            mid = (low + high) // 2
            if can_form(mid):
                ans = mid # mid is valid, try to find a larger k
                low = mid + 1
            else:
                high = mid - 1 # mid is invalid, search for smaller k

        return ans

```

JavaScript Snippets

```

// --- OPTIMAL SOLUTION (O(N) Time, O(1) Space) ---
var maxIncreasingSubarrays = function(nums) {
    let ans = 0;
    let currLen = 1;
    let prevLen = 0;

```

```

    for (let i = 1; i < nums.length; i++) {
        if (nums[i] > nums[i - 1]) {
            currLen++;
        } else {
            prevLen = currLen;
            currLen = 1;
        }

        // Scenario 1: Split current -> Math.floor(currLen / 2)
        // Scenario 2: Combine prev & current -> Math.min(prevLen, currLen)
        ans = Math.max(ans, Math.floor(currLen / 2), Math.min(prevLen, currLen));
    }

    // Fallback for arrays of length 2 that are decreasing (ans initialized to 0, but min k
    // Actually, our loop handles this gracefully because Math.min(1, 1) = 1.
    return ans;
};

// --- SUBOPTIMAL SOLUTION (O(N * log N) Time, O(N) Space) ---
var maxIncreasingSubarraysBinarySearch = function(nums) {
    const n = nums.length;
    const inc = new Array(n).fill(1);

    // Precompute the strictly increasing lengths starting from index i
    for (let i = n - 2; i >= 0; i--) {
        if (nums[i] < nums[i + 1]) {
            inc[i] = inc[i + 1] + 1;
        }
    }

    // Helper to check if size k is feasible
    const canForm = (k) => {
        for (let i = 0; i <= n - 2 * k; i++) {
            if (inc[i] >= k && inc[i + k] >= k) {
                return true;
            }
        }
        return false;
    };

    let low = 1;
    let high = Math.floor(n / 2);
    let ans = 1;

    // Binary Search on the length of k

```

```

while (low <= high) {
    let mid = Math.floor((low + high) / 2);
    if (canForm(mid)) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

return ans;
};

```

Note 1: Techniques Used The optimal solution employs a technique known as **Single-Pass State Tracking** (often related to Run-Length Encoding / RLE concepts). Instead of storing the data or doing lookbacks, we distill the historical state of the array into a single variable (**prev_len**). This helps immensely in array problems because it reduces auxiliary space overhead from $O(N)$ to strictly $O(1)$, which is a key signal of senior engineering maturity.

Note 2: Real-World / Interview Variations This exact core algorithm is highly requested in FAANG interviews under different narrative wrappers. Here is how Google, Meta, and Bloomberg frame it:

1. **Google - “Server Load Spikes” (Infrastructure):**

- *Prompt:* “You are given an array of CPU utilization percentages over time. Find the longest duration D where we had two consecutive back-to-back periods of strictly increasing server load.”
- *How to solve:* It is identical. The CPU load is **nums**, and D is **k**. You use the exact same $O(N)$ single-pass tracker to monitor the increasing load runs.

2. **Meta - “User Engagement Streaks” (Product):**

- *Prompt:* “Given an array representing a user’s daily active minutes on Instagram, what is the maximum window of days k such that the user had two back-to-back k -day periods where their daily minutes strictly grew?”
- *How to solve:* Again, track **prev_len** and **curr_len** of the engagement streaks. Split long streaks and combine adjacent streaks to find the max k .

3. **Bloomberg - “Stock Price Momentum Phases” (FinTech):**

- *Prompt:* “Given an array of daily closing prices for a ticker, find the maximum trading days k such that there are two consecutive k -day windows of strictly bullish (increasing) prices.”

- *How to solve:* Bloomberg likes to see edge-case management. You would apply the $O(N)$ approach, but ensure you explicitly clarify how you handle days where the stock price is flat (e.g., `nums[i] == nums[i-1]`). The logic remains that a flat day breaks the run, triggering `prev_len = curr_len` and `curr_len = 1`.

3346. Maximum Frequency of an Element After Performing Operations I

This is NOT a DP problem. Dynamic Programming relies on optimal substructure (breaking a problem into sequential, smaller subproblems). In this problem, the elements don't depend on each other sequentially, and keeping track of the "state" of operations across 10^5 elements would cause a combinatorial explosion. Because it isn't a DP problem, concepts like "Top-Down" and "Bottom-Up" don't strictly apply here.

However, you are right on the money with your intuition: **Sweep Line is absolutely one of the best ways to solve this.** Since you asked for multiple approaches, I am going to fulfill your request by providing the two absolute best approaches an L5/L6 engineer would weigh during an interview: **Approach 1: Sweep Line** and **Approach 2: Sliding Window & Binary Search.**

Let's break this down step-by-step.

1. Problem explanation

You have an array of numbers, a budget of `numOperations`, and a range `k`. For any number in the array, you can spend 1 operation to change it to **any number** within `k` units of its original value.

Your goal is to pick a **Target Value**, and then use your operations to pull as many elements as possible to that Target Value, maximizing the frequency of that number.

The Golden Rule of the Problem: If you pick a Target Value:

1. Some elements are **already** equal to the Target. You get these for free! (0 operations used).
2. Some elements are **within reach** (distance $\leq k$). You can spend 1 operation per element to change them, up to your `numOperations` limit.
3. Some elements are **out of reach** (distance $> k$). You can't use them at all.

Let's visualize this with an example. `nums = [1, 4, 5]`, `k = 1`, `numOperations = 2`

Each number can stretch left by 1, and right by 1.

Sweep Line Explanation

For every number x , the reachable interval is inclusive: $[x - k, x + k]$.

- An interval **starts** at $x - k$ (adds +1 to overlap).
- An interval **ends** at $x + k + 1$ (subtracts -1 from overlap).

Let's look at the exact intervals:

- `nums[0] = 1`: Reachable range is $[0, 2]$. Events: Start at **0** (+1), End at **3** (-1).
- `nums[1] = 4`: Reachable range is $[3, 5]$. Events: Start at **3** (+1), End at **6** (-1).
- `nums[2] = 5`: Reachable range is $[4, 6]$. Events: Start at **4** (+1), End at **7** (-1).

```
=====
                        SWEEP LINE EVENTS
=====
nums = [1, 4, 5], k = 1

Number Line:          0   1   2   3   4   5   6   7
-----
Interval for 1:  [+1]----->[-1]
                  (start 0)    (end 3)

Interval for 4:                      [+1]----->[-1]
                                      (start 3)    (end 6)

Interval for 5:                                [+1]----->[-1]
                                                (start 4)    (end 7)
-----
Net Event:          +1   0   0   0  +1   0  -1  -1
Running Sum:         1   1   1   1   2   2   1   0
(Total Overlap)
```

Notice what happens exactly at **Point 3**:

- The interval for 1 is no longer active at 3, so we process its end event: **-1**.
- The interval for 4 becomes active at 3, so we process its start event: **+1**.
- The Net Event at Point 3 is $-1 + 1 = 0$.
- The Running Sum (total overlap) at Point 3 remains **1**. (Only the number 4 can reach point 3).

Notice what happens exactly at **Point 4**:

- The interval for 4 is still active (no event).
- The interval for 5 becomes active at 4, so we process its start event: **+1**.
- The Net Event at Point 4 is **+1**.

- The Running Sum (total overlap) at Point 4 becomes $1 + 1 = 2$. (Both the numbers 4 and 5 can reach point 4).

At point 4, our running overlap is 2. The number 4 is already at point 4 (1 free element), and the number 5 needs 1 operation. If `numOperations` is at least 1, our max frequency here is 2.

If we choose 4 as our Target Value:

- 1 can only reach up to 2. (Cannot reach 4)
- 4 is already 4. (Free! 0 operations)
- 5 can reach 4. (Costs 1 operation)

Total frequency at Target 4 = 1 (free) + 1 (transformed) = 2.

2. Solution explanation

We have two incredibly elegant ways to solve this.

Approach 1: The Sweep Line Algorithm Imagine scanning a vertical line from left to right across the number line to see how many intervals overlap at any given point.

For every number x , its reachable interval is $[x - k, x + k]$. We can represent this as “Events” on a timeline:

- At $x - k$, an interval **starts** (+1 to overlap).
- At $x + k + 1$, an interval **ends** (-1 to overlap).

If we sort these events and keep a running sum, we instantly know exactly how many elements can reach any point P . The maximum frequency at point P is: $\min(\text{Total Overlap at } P, \text{Elements already equal to } P + \text{numOperations})$

“If we sort these events and keep a running sum, we instantly know exactly how many elements can reach any point P .”

Imagine you are walking along the number line from left to right. Every time you hit a +1 event, you step *onto* a new rug. Every time you hit a -1 event, you step *off* a rug.

If you sort the events by their position on the number line, you are guaranteed to process them in the correct left-to-right order. The “Running Sum” is just you counting: *“How many rugs am I currently standing on at this exact position P ?”*

Because each “rug” represents a number’s reachable range ($x - k$ to $x + k$), the number of rugs you are standing on is exactly the number of elements in the array that can physically be transformed into P .

At $P = 4$, you are standing on 2 rugs. This means exactly 2 numbers from the original array can physically reach the number 4.

“The maximum frequency at point P is: $\min(\text{Total Overlap at P}, \text{Elements already equal to P} + \text{numOperations})$ ”

This is the core logic that prevents us from cheating. We have two separate limitations, and our final answer is bottlenecked by whichever limitation is stricter (hence the $\min()$ function).

Let’s define the terms in the formula:

- **Total Overlap at P:** The number of “rugs” we are standing on. The total physical amount of numbers that can reach P.
- **Elements already equal to P:** Numbers in our array that are exactly P right now. These cost **0 operations** to turn into P. We get them for free!
- **numOperations:** Our budget to change *other* numbers into P.

When we add (Elements already equal to P) + (numOperations), we are calculating: *“What is the absolute maximum number of elements I can legally afford to have at P, regardless of how many can physically reach it?”*

Let’s look at the two scenarios that prove why the $\min()$ function is necessary.

Scenario A: You are bottlenecked by Physics (Overlap)

- Array: [4, 5]
- $k = 1$
- Target $P = 4$
- $\text{numOperations} = 100$ (Huge budget!)

```
=====
SCENARIO A: UNLIMITED BUDGET, LIMITED OVERLAP
=====
At P = 4:
- Elements already equal to 4: 1 (the number 4)
- Other numbers that can reach 4: 1 (the number 5)
- Total Overlap at 4: 2
```

Formula:

```
max_frequency = min( Total Overlap , Free Elements + Budget )
max_frequency = min(      2      ,      1      + 100 )
max_frequency = min( 2, 101 )
max_frequency = 2
=====
```

Why it works: Even though we have 100 operations to spend, there are physically only 2 numbers in the entire array that can reach $P = 4$. We cannot magically invent 99 more numbers. The physical overlap restricts us.

Scenario B: You are bottlenecked by your Budget (Operations)

- Array: [1, 2, 3, 4, 5, 6, 7]
- k = 5 (Everything can reach everywhere)
- Target P = 4
- numOperations = 2 (Tiny budget!)

```
=====
SCENARIO B: UNLIMITED OVERLAP, LIMITED BUDGET
=====
```

At P = 4:

- Elements already equal to 4: 1 (the number 4)
- Other numbers that can reach 4: 6 (all the others)
- Total Overlap at 4: 7

Formula:

```
max_frequency = min( Total Overlap , Free Elements + Budget )
max_frequency = min(      7      ,      1      +      2      )
max_frequency = min( 7, 3 )
max_frequency = 3
=====
```

Why it works: 7 numbers can physically reach P = 4. But transforming them costs 1 operation each. We get one 4 for free, and we only have the budget to pull in 2 extra numbers. Therefore, the absolute maximum frequency we can achieve at P = 4 is 1 + 2 = 3. Our budget restricts us.

- The Sweep Line (Running Sum) answers the question: *“How many elements are close enough?”*
- The Formula (min(...)) answers the question: *“Of the ones that are close enough, how many can I actually afford?”*

Approach 2: Sliding Window + Binary Search An L6 engineer might realize there’s an even simpler mathematical truth here. There are only two scenarios for the best Target Value:

Case A: The target IS an existing number in the array. If we pick an existing number `nums[i]`, we just need to know how many elements fall in the range `[nums[i] - k, nums[i] + k]`. Because the array is sorted, we can use Binary Search to find the left and right boundaries of this range instantly.

Case B: The target is NOT in the array. If the target isn’t in the array, we get **zero** free elements. Every single element we transform costs 1 operation. Therefore, our frequency can NEVER exceed `numOperations`. We just need to find the maximum number of elements that fit inside ANY window of size `2 * k`. We can use a standard Sliding Window to find this!

```
=====
SLIDING WINDOW VISUALIZATION
=====
```

Looking for max elements in any window of size $2k$.
nums = [5, 11, 20, 20], k = 5 (Window size = 10)

Window [5, 15] --> Covers [5, 11] (Length = 2)
Window [11, 21] --> Covers [11, 20, 20] (Length = 3)

Max elements in ANY window of size 10 is 3.
If we pick a target not in the array, we can transform
at most $\min(3, \text{numOperations})$.

3. Time and Space complexity analysis

```
+-----+
| TIME & SPACE COMPLEXITY DERIVATION |
+-----+
| APPROACH 1: Sweep Line |
| -> Create events map:  $O(N)$  |
| -> Sort unique event keys:  $O(N \log N)$  |
| -> Sweep through keys:  $O(N)$  |
| TOTAL TIME:  $O(N \log N)$  |
| SPACE:  $O(N)$  for Hash Maps tracking frequencies and events |
+-----+
| APPROACH 2: Sliding Window + Binary Search |
| -> Sort array:  $O(N \log N)$  |
| -> Sliding Window (Case B):  $O(N)$  |
| -> Binary Search (Case A):  $O(N)$  loops *  $O(\log N)$  search |
| TOTAL TIME:  $O(N \log N)$  |
| SPACE:  $O(N)$  for counting frequencies |
+-----+
```

4. Solution Code (Javascript and Python snippets)

Here are both approaches, fully commented and production-ready.

Python Code

```
from typing import List
from collections import defaultdict, Counter
import bisect
```

```
class Solution:
```

```
    # -----
    # APPROACH 1: Sweep Line
    # -----
```

```

def maxFrequencySweepLine(self, nums: List[int], k: int, numOperations: int) -> int:
    events = defaultdict(int)
    freq = Counter(nums)

    for num in nums:
        # Mark the start and end of the reachable interval
        events[num - k] += 1
        events[num + k + 1] -= 1
        # Ensure the exact number itself is registered as an event point to be checked
        if num not in events:
            events[num] = 0

    running_overlap = 0
    max_freq = 0

    # Sort the critical points and sweep across the number line
    for pos in sorted(events.keys()):
        running_overlap += events[pos]

        # max_freq is bounded by overlap, and how many ops we can afford over our free elements
        current_freq = min(running_overlap, freq[pos] + numOperations)
        max_freq = max(max_freq, current_freq)

    return max_freq

# -----
# APPROACH 2: Sliding Window + Binary Search
# -----
def maxFrequencySlidingWindow(self, nums: List[int], k: int, numOperations: int) -> int:
    nums.sort()
    freq = Counter(nums)
    n = len(nums)
    max_freq = 0

    # CASE B: Target is NOT an existing array element.
    # Find the maximum number of elements packed into ANY window of size 2*k
    left = 0
    max_window_len = 0
    for right in range(n):
        while nums[right] - nums[left] > 2 * k:
            left += 1
        max_window_len = max(max_window_len, right - left + 1)

    # If we pick a target not in nums, we get NO free elements.
    max_freq = min(max_window_len, numOperations)

```

```

    # CASE A: Target IS an existing array element.
    for num in freq:
        # Find how many elements can physically reach 'num'
        l_idx = bisect.bisect_left(nums, num - k)
        r_idx = bisect.bisect_right(nums, num + k)
        total_reachable = r_idx - l_idx

        current_freq = min(total_reachable, freq[num] + numOperations)
        max_freq = max(max_freq, current_freq)

    return max_freq

```

Javascript Code

```

// -----
// APPROACH 1: Sweep Line
// -----
function maxFrequencySweepLine(nums, k, numOperations) {
    let events = new Map();
    let freq = new Map();

    for (let num of nums) {
        freq.set(num, (freq.get(num) || 0) + 1);

        events.set(num - k, (events.get(num - k) || 0) + 1);
        events.set(num + k + 1, (events.get(num + k + 1) || 0) - 1);

        if (!events.has(num)) {
            events.set(num, 0);
        }
    }

    // Sort keys numerically
    let sortedPositions = Array.from(events.keys()).sort((a, b) => a - b);
    let runningOverlap = 0;
    let maxFreq = 0;

    for (let pos of sortedPositions) {
        runningOverlap += events.get(pos);
        let countPos = freq.get(pos) || 0;
        let currentFreq = Math.min(runningOverlap, countPos + numOperations);
        maxFreq = Math.max(maxFreq, currentFreq);
    }

    return maxFreq;
}

```

```

// -----
// APPROACH 2: Sliding Window + Binary Search
// -----
function maxFrequencySlidingWindow(nums, k, numOperations) {
  nums.sort((a, b) => a - b);
  let freq = new Map();
  for (let num of nums) {
    freq.set(num, (freq.get(num) || 0) + 1);
  }

  let n = nums.length;
  let maxFreq = 0;

  // CASE B: Target is NOT an existing array element
  let left = 0;
  let maxWindowLen = 0;
  for (let right = 0; right < n; right++) {
    while (nums[right] - nums[left] > 2 * k) {
      left++;
    }
    maxWindowLen = Math.max(maxWindowLen, right - left + 1);
  }
  maxFreq = Math.min(maxWindowLen, numOperations);

  // Binary Search Helpers
  const bisectLeft = (arr, target) => {
    let l = 0, r = arr.length;
    while (l < r) {
      let mid = Math.floor((l + r) / 2);
      if (arr[mid] < target) l = mid + 1;
      else r = mid;
    }
    return l;
  };
  const bisectRight = (arr, target) => {
    let l = 0, r = arr.length;
    while (l < r) {
      let mid = Math.floor((l + r) / 2);
      if (arr[mid] <= target) l = mid + 1;
      else r = mid;
    }
    return l;
  };

  // CASE A: Target IS an existing array element

```

```

for (let [num, count] of freq.entries()) {
    let lIdx = bisectLeft(nums, num - k);
    let rIdx = bisectRight(nums, num + k);
    let totalReachable = rIdx - lIdx;

    let currentFreq = Math.min(totalReachable, count + numOperations);
    maxFreq = Math.max(maxFreq, currentFreq);
}

return maxFreq;
}

```

Concept Definitions

- **Sweep Line Algorithm:** A geometric algorithmic technique where a conceptual line is “swept” across a space (in this case, a 1D number line) to track overlapping intervals efficiently. It converts a problem of comparing intervals into a problem of sorting points, drastically reducing time complexity.
- **Sliding Window:** A technique that involves creating a “window” (a sub-array) that slides over the underlying data to continuously process localized sets of data, preventing unnecessary nested loops.

Real World / Interview Variations

- **Google** often asks this exact logic wrapped in a “Server Load Balancing” or “Temperature Control” scenario: “You have N servers operating at different temperatures. You can adjust up to M servers by K degrees. Find the maximum number of servers you can sync to the exact same temperature.”
- **Meta (Facebook)** frames this around user locations: “You have users located along a highway. Users are willing to travel K miles. You have numOperations free bus tickets to give out. What is the maximum number of people you can gather at a single coordinate?”
- **Bloomberg** loves to ask this in a FinTech wrapper: “You have a list of stock bids. Traders are willing to negotiate their bid by \$K. You have a budget to subsidize numOperations trades. Maximize the number of matches at a single price.”

84. Largest Rectangle in Histogram

To tackle a problem like “Largest Rectangle in Histogram” at an L5 or L6 level, an interviewer isn’t just looking for the right code. They are evaluating your ability to deconstruct a problem, recognize inefficiencies in the naive approach,

and progressively build an optimal solution using a robust data structure, all while clearly communicating your thought process.

Here is how a senior engineer would break down and solve this problem.

1. Problem Explanation

The Goal: Given an array of integers representing the heights of consecutive bars in a histogram (where each bar has a uniform width of 1), we need to find the area of the largest rectangle that can be perfectly inscribed within these bars.

Let's visualize an example array: `heights = [2, 1, 5, 6, 2, 3]`

```

      ASCII HISTOGRAM VISUALIZATION
      -----
  6 |           [ ]
  5 |          [ ] [ ]
  4 |          [ ] [ ]
  3 |          [ ] [ ]           [ ]
  2 | [ ]      [ ] [ ] [ ]      [ ]
  1 | [ ] [ ] [ ] [ ] [ ] [ ] [ ]
  0 +-----
Idx:  0   1   2   3   4   5
Val:  2   1   5   6   2   3

```

The Core Insight: Any valid rectangle we can form will be bottlenecked by the *shortest* bar within that rectangle. Therefore, the height of the maximum rectangle will *always* be exactly equal to the height of at least one of the bars in the histogram.

If we treat every single bar as the “bottleneck” (the shortest bar in a potential rectangle), we just need to find out how far left and how far right we can extend a rectangle of that specific height.

- **Left limit:** The index of the first bar to the left that is *shorter* than our current bar.
- **Right limit:** The index of the first bar to the right that is *shorter* than our current bar.
- **Width:** (Right limit index) - (Left limit index) - 1
- **Area:** Current Bar Height * Width

2. Solution Explanation

Approach A: The Brute Force ($O(N^2)$) We can literally implement the core insight above. Iterate through every bar `i`. Then, use two while loops to expand a left pointer and a right pointer as long as the bars they point to are greater than or equal to `heights[i]`. Calculate the area, and keep a running maximum.

While this works, it does redundant work. If we have a sequence of ascending bars [1, 2, 3, 4, 5, 6], finding the left limit for 6 means scanning all the way back, and then finding the left limit for 5 means scanning all the way back again. We need to remember previous states.

Approach B: The Monotonic Stack (O(N) Optimal) To eliminate redundant scanning, we use a **Monotonic Increasing Stack**. We will store the *indices* of the bars in the stack. The invariant we maintain is: the heights corresponding to the indices in the stack will strictly increase from bottom to top.

How it works:

1. As we iterate from left to right, if the current bar is *taller* than the bar at the top of the stack, we push its index onto the stack. This means “this bar can still be extended to the right.”
2. If the current bar is *shorter* than the bar at the top of the stack, we have found the **right limit** for the bar at the top of the stack!
3. We pop the top of the stack. Let’s call its height H.
4. Because the stack is strictly increasing, the new top of the stack is the **left limit** for H (it’s the nearest shorter bar to the left).
5. We calculate the area using H and the distance between the current index (right limit) and the new stack top (left limit). We repeat this until the current bar is taller than the stack top, then push it.

Step-by-Step Visualization for [2, 1, 5, 6, 2, 3]:

Note: We push a dummy index -1 to the stack initially to handle the math elegantly when a bar extends all the way to index 0.

Idx 0 (Val 2): Taller than stack top (which is empty/-1). Push index 0.
Stack: [-1, 0]

Idx 1 (Val 1): 1 is LESS than 2 (height at stack top index 0).
--> BOTTLE-NECK FOUND for index 0!
--> Pop 0. Height = 2. Left limit = Stack top = -1. Right limit = 1.
--> Width = 1 - (-1) - 1 = 1. Area = 2 * 1 = 2. MaxArea = 2.
--> Push index 1.
Stack: [-1, 1]

Idx 2 (Val 5): 5 > 1. Push index 2.
Stack: [-1, 1, 2]

Idx 3 (Val 6): 6 > 5. Push index 3.
Stack: [-1, 1, 2, 3]

Idx 4 (Val 2): 2 is LESS than 6 (stack top index 3).
--> BOTTLE-NECK FOUND for index 3!

```

--> Pop 3. Height = 6. Left limit = 2. Right limit = 4.
--> Width = 4 - 2 - 1 = 1. Area = 6 * 1 = 6. MaxArea = 6.
--> 2 is also LESS than 5 (new stack top index 2).
--> BOTTLE-NECK FOUND for index 2!
--> Pop 2. Height = 5. Left limit = 1. Right limit = 4.
--> Width = 4 - 1 - 1 = 2. Area = 5 * 2 = 10. MaxArea = 10.
--> Push index 4.
Stack: [-1, 1, 4]

```

```

Idx 5 (Val 3): 3 > 2. Push index 5.
Stack: [-1, 1, 4, 5]

```

End of array reached. Now we pop remaining elements. The "right limit" for remaining element

```

--> Pop 5 (Height 3). Width = 6 - 4 - 1 = 1. Area = 3.
--> Pop 4 (Height 2). Width = 6 - 1 - 1 = 4. Area = 8.
--> Pop 1 (Height 1). Width = 6 - (-1) - 1 = 6. Area = 6.
Final MaxArea = 10.

```

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION (Total Time: $O(N)$)

Operation	Cost per Bar	Total Cost over N bars
Iterate array	$O(1)$ step	N steps $\rightarrow O(N)$
Push to Stack	$O(1)$	Every element pushed exactly 1x
Pop from Stack	$O(1)$	Every element popped exactly 1x
Calculate Area	$O(1)$	Done only during a pop

Because an element can only be pushed once and popped once, the inner while loop does NOT cause an $N*N$ scenario. The maximum total operations across the entire array traversal is $2N$ stack operations. Thus, Time Complexity is strictly $O(N)$.

SPACE COMPLEXITY DERIVATION (Total Space: $O(N)$)

In the worst case (a strictly increasing histogram like 1,2,3,4,5), we will push every single index onto the stack without popping until the very end.
Max Stack Size = $N + 1$ (including the dummy -1)

Thus, Space Complexity is $O(N)$.

4. Solution Code

JavaScript

```
/**
 * Approach 1: Brute Force (Acceptable for discussion, but will Time Out)
 * Time:  $O(N^2)$ , Space:  $O(1)$ 
 */
function largestRectangleAreaBruteForce(heights) {
    let maxArea = 0;
    for (let i = 0; i < heights.length; i++) {
        let minHeight = heights[i];
        // Expand to the right to find the max area using heights[i] as the starting point
        for (let j = i; j < heights.length; j++) {
            minHeight = Math.min(minHeight, heights[j]);
            const width = j - i + 1;
            maxArea = Math.max(maxArea, minHeight * width);
        }
    }
    return maxArea;
}

/**
 * Approach 2: Monotonic Stack (Optimal)
 * Time:  $O(N)$ , Space:  $O(N)$ 
 */
function largestRectangleArea(heights) {
    // Stack stores indices, not heights.
    // We initialize with -1 to represent the "ground" left boundary.
    const stack = [-1];
    let maxArea = 0;

    for (let i = 0; i < heights.length; i++) {
        // While current bar is shorter than the bar at stack top,
        // we process the stack top (we found its right boundary).
        while (stack[stack.length - 1] !== -1 && heights[stack[stack.length - 1]] > heights[i]) {
            const currentHeightIndex = stack.pop();
            const currentHeight = heights[currentHeightIndex];

            // i is the right boundary, new stack top is the left boundary
            const currentWidth = i - stack[stack.length - 1] - 1;
            maxArea = Math.max(maxArea, currentHeight * currentWidth);
        }
        stack.push(i);
    }
}
```

```

    // Process any remaining bars in the stack.
    // Their right boundary is the end of the array (heights.length).
    while (stack[stack.length - 1] !== -1) {
        const currentHeightIndex = stack.pop();
        const currentHeight = heights[currentHeightIndex];
        const currentWidth = heights.length - stack[stack.length - 1] - 1;
        maxArea = Math.max(maxArea, currentHeight * currentWidth);
    }

    return maxArea;
}

```

Python

Approach 1: Brute Force (Will result in Time Limit Exceeded on LeetCode)
Time: $O(N^2)$, Space: $O(1)$

```

def largestRectangleAreaBruteForce(heights: list[int]) -> int:
    max_area = 0
    n = len(heights)
    for i in range(n):
        min_height = heights[i]
        for j in range(i, n):
            min_height = min(min_height, heights[j])
            width = j - i + 1
            max_area = max(max_area, min_height * width)
    return max_area

```

Approach 2: Monotonic Stack (Optimal)

Time: $O(N)$, Space: $O(N)$

```

def largestRectangleArea(heights: list[int]) -> int:
    # Initialize stack with -1 to serve as the default left boundary
    stack = [-1]
    max_area = 0

    for i in range(len(heights)):
        # If we find a dip in height, the previous taller bars cannot extend further right.
        while stack[-1] != -1 and heights[stack[-1]] > heights[i]:
            current_height_index = stack.pop()
            current_height = heights[current_height_index]

            # The current index 'i' is the right limit.
            # The new top of the stack is the left limit.
            current_width = i - stack[-1] - 1
            max_area = max(max_area, current_height * current_width)

```

```

        stack.append(i)

    # Flush the remaining elements in the stack.
    # Their right boundary is considered to be just past the end of the array.
    while stack[-1] != -1:
        current_height_index = stack.pop()
        current_height = heights[current_height_index]
        current_width = len(heights) - stack[-1] - 1
        max_area = max(max_area, current_height * current_width)

    return max_area

```

Note 1: Concepts Used

Monotonic Stack: This is a stack whose elements are strictly increasing or decreasing. In this problem, we use a *Monotonic Increasing Stack*.

- **Why it helps:** It acts as a time-travel device. By keeping elements increasing, the moment we see a smaller element, we immediately know the boundaries for all the larger elements we've seen recently without having to scan backwards. It turns an $O(N^2)$ backward-scanning operation into an amortized $O(1)$ operation per element.
 - **Application here:** It tracks the left boundaries of bars inherently. The bottom of the stack holds the index of the shortest bar seen so far, and the top holds the tallest.
-

Note 2: Real-World & FAANG Interview Variations

1. Google & Meta: Maximal Rectangle (LeetCode 85)

- *The Setup:* You are given a 2D boolean grid (0s and 1s). Find the area of the largest rectangle containing only 1s.
- *How to solve it:* This is a brilliant extension of the Histogram problem. You iterate through the grid row by row. For each row, you maintain an array of heights. If a cell is '1', you add 1 to the height of that column. If a cell is '0', the height for that column drops to 0. After updating the heights for a row, you pass that heights array directly into our `largestRectangleArea` function. The maximum returned across all rows is the answer.

2. Bloomberg: Daily Temperatures / Next Greater Element (LeetCode 739, 496)

- *The Setup:* Given an array of temperatures, return an array answering how many days you have to wait until a warmer temperature.

- *How to solve it:* This tests the exact same Monotonic Stack foundation. You iterate through the array, pushing indices to a stack. If you encounter a temperature hotter than the stack top, you pop the stack. The difference between the current index and the popped index is the number of days waited.

3. Real-World Application (System Design / Front-End Rendering)

- *The Setup:* A UI rendering engine needs to allocate a contiguous block of empty space (e.g., placing a large dynamic widget on a dashboard).
- *How to solve it:* The UI grid can be mapped to a 2D matrix where occupied pixels are 0 and free pixels are 1. Using the histogram approach allows the rendering engine to find the largest contiguous rectangular free space in $O(\text{Rows} * \text{Cols})$ time, which is fast enough to do on the fly during browser reflow operations.

85. Maximal Rectangle

Here is how an L5/L6 engineer would approach, break down, and communicate the solution to the “Maximal Rectangle” problem.

1. Problem Explanation

Imagine you are looking at a top-down map of a city grid. Each cell is either an empty lot (1) or a building (0). Your goal is to find the largest single rectangular plot of land that consists entirely of empty lots (1s) and calculate its area.

Let’s look at an example matrix:

	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0:	1	0	1	0	0
Row 1:	1	0	1	1	1
Row 2:	1	1	1	1	1
Row 3:	1	0	0	1	0

In this grid, you can form several rectangles of 1s.

- A vertical rectangle in Col 0 from Row 0 to Row 3 (Area = $4 * 1 = 4$).
- A horizontal rectangle in Row 2 from Col 0 to Col 4 (Area = $1 * 5 = 5$).
- A block rectangle covering Rows 1 and 2, and Cols 2 to 4 (Area = $2 * 3 = 6$).

The maximum area here is **6**. Our algorithm needs to programmatically discover this.

2. Solution Explanation

To solve this efficiently, we must reframe the problem. Finding rectangles in a raw 2D grid is difficult because a rectangle can start and end anywhere.

Instead, we can build upon a simpler, foundational problem: **The Largest Rectangle in a Histogram.**

Prerequisite: The Histogram Concept (Non-Trivial Step) If we process the grid row by row, we can treat each row as the “ground” and count how many consecutive 1s stack up vertically above each cell. This creates a histogram for each row.

Let’s convert our matrix into a series of histograms:

Matrix Row	Consecutive 1s looking UP (The Histogram)	ASCII Visual of Histogram
Row 0:	[1, 0, 1, 0, 0]	# . # . .
Row 1:	[2, 0, 2, 1, 1]	# . # . .
		# . # # # <- ground (Row 1)
Row 2:	[3, 1, 3, 2, 2]	# . # . .
		# . # # #
		# # # # # <- ground (Row 2)
Row 3:	[4, 0, 0, 3, 0]	#
		# . . # .
		# . . # .
		# . . # . <- ground (Row 3)

By doing this, the “Maximal Rectangle in a 2D Matrix” problem becomes: **“Find the Largest Rectangle in the Histogram for Row 0, then Row 1, then Row 2, etc., and keep track of the absolute maximum.”**

Solving the Histogram Problem: The Monotonic Stack How do we find the largest rectangle in a single histogram like [3, 1, 3, 2, 2] (from Row 2)?

For every bar in the histogram, the largest rectangle that *includes that bar at its full height* is limited by the closest shorter bar to its left and the closest shorter bar to its right.

We use a **Monotonic Stack** (a stack that keeps elements in a strictly increasing order) to efficiently find these “left and right limits” for every bar in a single pass.

Let’s trace the histogram [3, 1, 3, 2, 2] from Row 2. We append a 0 at the end [3, 1, 3, 2, 2, 0] to force the stack to empty at the end of our loop.

Histogram: [3, 1, 3, 2, 2, 0]
Indices: 0 1 2 3 4 5

Step 1: i=0, height=3. Stack is empty.

Action: Push index 0.
Stack: [0]

Step 2: i=1, height=1. 1 is LESS than height at top of stack (histogram[0] = 3).
Action: We found the right boundary for the bar at index 0!
Pop index 0. Height = 3.
Left boundary is the new top of stack (none, so -1).
Width = current_index - left_boundary - 1 = 1 - (-1) - 1 = 1.
Area = 3 * 1 = 3. MaxArea = 3.
Action: Push index 1.
Stack: [1]

Step 3: i=2, height=3. 3 is GREATER than height at top of stack (histogram[1] = 1).
Action: Push index 2.
Stack: [1, 2]

Step 4: i=3, height=2. 2 is LESS than height at top of stack (histogram[2] = 3).
Action: Right boundary found for index 2!
Pop index 2. Height = 3.
Left boundary is new top of stack (1).
Width = 3 - 1 - 1 = 1. Area = 3 * 1 = 3. MaxArea = 3.
Action: Push index 3.
Stack: [1, 3]

Step 5: i=4, height=2. 2 is EQUAL to top of stack. We can push it.
Action: Push index 4.
Stack: [1, 3, 4]

Step 6: i=5, height=0. 0 is LESS than everything. This forces the stack to empty.
Action: Pop 4 (Height=2). Width = 5 - 3 - 1 = 1. Area = 2.
Pop 3 (Height=2). Width = 5 - 1 - 1 = 3. Area = 6. MaxArea = 6. -> NEW MAX!
Pop 1 (Height=1). Width = 5 - (-1) - 1 = 5. Area = 5.
Stack: []

The maximum area found in this row is 6, which matches our manual observation.

3. Time and Space Complexity Analysis

[TC & SC Derivation Flow]

Input: Matrix (R rows, C columns)

Loop over R rows: -----> O(R) multiplier
|

```

+-- Update Histogram Array for current row:
|   For each column C, if matrix[r][c]=='1', add 1.
|   Else, reset to 0. -----> O(C) operations
|
+-- Calculate Max Area in Histogram:
    Iterate through C columns.
    Push/Pop to stack. Every element is pushed
    at most once and popped at most once. -----> O(C) operations

Total Time Complexity  = O(R * (C + C))
                      = O(R * 2C)
                      = O(R * C)

Total Space Complexity = Array to store current histogram -> O(C)
                      + Stack to process histogram -----> O(C)
                      = O(C)

```

- **Time Complexity:** $O(R * C)$. We visit each cell in the matrix a constant number of times.
- **Space Complexity:** $O(C)$. We only need to store the histogram heights for the current row and a stack, both of which scale with the number of columns.

4. Solution Code

As requested, I will provide two excellent solutions.

1. **Solution A (Optimal Monotonic Stack):** The histogram approach detailed above.
2. **Solution B (Dynamic Programming - Left/Right bounds):** Another classic approach that avoids the stack entirely by maintaining **left** and **right** arrays to track the boundaries of rectangles. This is often preferred by interviewers who want to see your DP state-tracking skills.

JavaScript Snippets

```

/**
 * Solution A: Monotonic Stack (Histogram Approach)
 * Time: O(R * C) | Space: O(C)
 */
function maximalRectangleStack(matrix) {
    if (!matrix || matrix.length === 0 || matrix[0].length === 0) return 0;

    const cols = matrix[0].length;
    // Initialize histogram heights with an extra 0 at the end to flush the stack
    const heights = new Array(cols + 1).fill(0);

```

```

    let maxArea = 0;

    for (let row = 0; row < matrix.length; row++) {
        // Step 1: Update the histogram for the current row
        for (let col = 0; col < cols; col++) {
            if (matrix[row][col] === '1') {
                heights[col] += 1;
            } else {
                heights[col] = 0; // The continuous stack is broken
            }
        }

        // Step 2: Compute max area using Monotonic Stack
        const stack = []; // Will store indices
        for (let i = 0; i < heights.length; i++) {
            // While current bar is shorter than the bar at the top of the stack,
            // we have found the right boundary for the stack's top bar.
            while (stack.length > 0 && heights[i] < heights[stack[stack.length - 1]]) {
                const height = heights[stack.pop()];
                // Left boundary is the new top of the stack. If empty, it's -1.
                const leftBoundary = stack.length === 0 ? -1 : stack[stack.length - 1];
                const width = i - leftBoundary - 1;
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }

        return maxArea;
    }

    /**
     * Solution B: Dynamic Programming (Left, Right, Height arrays)
     * Time: O(R * C) | Space: O(C)
     */
    function maximalRectangleDP(matrix) {
        if (!matrix || matrix.length === 0) return 0;

        const cols = matrix[0].length;
        const heights = new Array(cols).fill(0);
        const leftBoundaries = new Array(cols).fill(0);
        const rightBoundaries = new Array(cols).fill(cols); // Default right bound is out of bounds

        let maxArea = 0;

        for (let row = 0; row < matrix.length; row++) {

```

```

    let currentLeft = 0;
    let currentRight = cols;

    // Compute Heights
    for (let j = 0; j < cols; j++) {
        if (matrix[row][j] === '1') heights[j]++;
        else heights[j] = 0;
    }

    // Compute Left Boundaries
    for (let j = 0; j < cols; j++) {
        if (matrix[row][j] === '1') {
            leftBoundaries[j] = Math.max(leftBoundaries[j], currentLeft);
        } else {
            leftBoundaries[j] = 0;
            currentLeft = j + 1; // Next possible left boundary
        }
    }

    // Compute Right Boundaries
    for (let j = cols - 1; j >= 0; j--) {
        if (matrix[row][j] === '1') {
            rightBoundaries[j] = Math.min(rightBoundaries[j], currentRight);
        } else {
            rightBoundaries[j] = cols;
            currentRight = j; // Next possible right boundary
        }
    }

    // Compute Area
    for (let j = 0; j < cols; j++) {
        const width = rightBoundaries[j] - leftBoundaries[j];
        maxArea = Math.max(maxArea, width * heights[j]);
    }

    return maxArea;
}

```

Python Snippets

```

# Solution A: Monotonic Stack (Histogram Approach)
# Time:  $O(R * C)$  | Space:  $O(C)$ 
def maximalRectangleStack(matrix: list[list[str]]) -> int:
    if not matrix or not matrix[0]:
        return 0

```

```

cols = len(matrix[0])
# Add a trailing 0 to flush the stack at the end of each row
heights = [0] * (cols + 1)
max_area = 0

for row in matrix:
    # Step 1: Update heights
    for col in range(cols):
        if row[col] == '1':
            heights[col] += 1
        else:
            heights[col] = 0

    # Step 2: Monotonic Stack to find max area in current histogram
    stack = []
    for i in range(len(heights)):
        while stack and heights[i] < heights[stack[-1]]:
            h = heights[stack.pop()]
            # If stack is empty after pop, the left boundary is -1
            left_boundary = stack[-1] if stack else -1
            w = i - left_boundary - 1
            max_area = max(max_area, h * w)
        stack.append(i)

return max_area

# Solution B: Dynamic Programming (Left, Right, Height arrays)
# Time: O(R * C) | Space: O(C)
def maximalRectangleDP(matrix: list[list[str]]) -> int:
    if not matrix or not matrix[0]:
        return 0

    cols = len(matrix[0])
    heights = [0] * cols
    left_boundaries = [0] * cols
    right_boundaries = [cols] * cols

    max_area = 0

    for row in matrix:
        current_left = 0
        current_right = cols

        # Compute Heights
        for j in range(cols):

```

```

    if row[j] == '1':
        heights[j] += 1
    else:
        heights[j] = 0

    # Compute Left Boundaries
    for j in range(cols):
        if row[j] == '1':
            left_boundaries[j] = max(left_boundaries[j], current_left)
        else:
            left_boundaries[j] = 0
            current_left = j + 1

    # Compute Right Boundaries
    for j in range(cols - 1, -1, -1):
        if row[j] == '1':
            right_boundaries[j] = min(right_boundaries[j], current_right)
        else:
            right_boundaries[j] = cols
            current_right = j

    # Compute Area
    for j in range(cols):
        width = right_boundaries[j] - left_boundaries[j]
        max_area = max(max_area, width * heights[j])

    return max_area

```

Terminology Glossary

- **Monotonic Stack:** A data structure technique where elements are kept in a strictly increasing or decreasing order. *Why it helps here:* It allows us to efficiently find the “next smaller element” to the left and right of any histogram bar in $O(N)$ time, which dictates how wide a rectangle can stretch.
 - **Dynamic Programming (State Arrays):** Using auxiliary arrays (`left`, `right`, `height`) to remember the state from the previous row. *Why it helps here:* Instead of scanning left and right for every cell repeatedly, we inherit the constraints from the row above us and merge them with the constraints of the current row, ensuring an $O(R * C)$ runtime without needing a stack.
-

Interview Variations at Top Tech Companies

1. Meta/Google: “Find the largest safe zone in a grid with obstacles”

- **The Prompt:** You are given a grid of a forest. 0 represents an obstacle (tree/rock), 1 represents flat land. Find the largest rectangular flat land to land a helicopter.
- **How to solve:** This is identically “Maximal Rectangle”. You would implement the Monotonic Stack solution exactly as written above.

2. Bloomberg: “Maximal Square”

- **The Prompt:** Given a binary matrix, find the largest *square* containing only 1s and return its area.
- **How to solve:** While you *could* use the rectangle logic and filter for `width == height`, that is overkill. The optimal solution here is a simpler Dynamic Programming approach:
- Create a 2D DP array where `dp[i][j]` represents the side length of the maximum square whose bottom-right corner is at cell `(i, j)`.
- Transition formula: If `matrix[i][j] == '1'`, then `dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1`.
- The answer is the square of the maximum value found in the DP array. Time Complexity: $O(R * C)$.

3. Meta: “Number of Submatrices That Sum to Target” / “2D Prefix Sums”

- **The Prompt:** Given a matrix of integers (not just 0s and 1s) and a target value, find how many rectangular submatrices sum to exactly `target`.
- **How to solve:** This requires a different technique entirely. You use a 1D technique (subarray sum equals K using a Hash Map) and expand it to 2D. You lock in a “top row” and a “bottom row”, collapse the columns between them into a 1D array by summing them, and then run the $O(C)$ Hash Map subarray sum logic. Total Time: $O(R^2 * C)$.

73. Set Matrix Zeroes

Stepping into a senior or staff-level interview loop means looking past the brute-force solutions and immediately hunting for the clever state management and memory optimizations that distinguish a seasoned engineer. “Set Matrix Zeroes” is a classic for exactly this reason. It isn’t testing a complex, obscure algorithm; it’s testing your ability to manage state cleanly in place without relying on a crutch of infinite memory.

Here is exactly how a top-of-the-band engineer breaks down, explains, and codes this problem.

1. Problem Explanation

The Core Task: You are given an $M \times N$ matrix. Whenever you find a 0, you must change every other element in that exact same row and column to 0. The critical constraint is that you must do this **in-place** (modifying the matrix directly).

The Trap (The “Cascading Zeroes” Problem): If you simply iterate through the matrix, find a 0, and immediately turn its row and column into 0s, you corrupt your own data. As you continue iterating, you will eventually hit one of those *new* 0s you just created. Your algorithm won’t know if it was an original 0 or a newly set one, leading to a chain reaction where the entire matrix incorrectly turns to 0s.

To solve this, we need a way to “remember” which rows and columns need to be zeroed *before* we actually start modifying the data.

2. Solution Explanation

We will look at two approaches. In a real interview, you should quickly mention Approach A to show you understand the cascading trap, and then immediately pivot to implement Approach B.

Approach A: The $O(M + N)$ Space Solution (Good, but not optimal)

We can use two separate arrays: a `row_flags` array of size M and a `col_flags` array of size N .

1. **Pass 1:** Iterate through the matrix. If `matrix[r][c] == 0`, set `row_flags[r] = true` and `col_flags[c] = true`.
2. **Pass 2:** Iterate through the matrix again. If `row_flags[r]` is true OR `col_flags[c]` is true, set `matrix[r][c] = 0`.

This safely bypasses the cascading trap, but uses extra memory. At scale, allocating extra arrays is wasteful.

Approach B: The $O(1)$ Space Solution (The L5/L6 Expectation) Instead of allocating *new* arrays, we can use the **first row** and the **first column** of the matrix itself to act as our `row_flags` and `col_flags`.

The Overlap Edge Case: The first row and first column intersect at `matrix[0][0]`. We cannot use `matrix[0][0]` to store both the flag for row 0 and the flag for column 0.

- We will use `matrix[0][0]` to track if **Row 0** needs to be zeroed.
- We will create a single, separate variable called `col0` to track if **Column 0** needs to be zeroed.

Step-by-Step Visualization:

Let's use the following input matrix:

```
Input Matrix:
[ 0, 1, 2, 0 ] <-- Row 0
[ 3, 4, 5, 2 ] <-- Row 1
[ 1, 3, 1, 5 ] <-- Row 2
```

Step 1: Top-Down Flagging Pass We iterate through the matrix. If we see a 0, we mark the top of its column and the start of its row with a 0.

- At (0, 0): It's a 0. We set `col0 = 0` (marking the first column), and `matrix[0][0] = 0` (marking the first row).
- At (0, 3): It's a 0. We set `matrix[0][3] = 0` (marking column 3) and `matrix[0][0] = 0` (marking the first row again).

State after Pass 1:

`col0 = 0` (Tells us column 0 needs to be completely zeroed later)

```
      c0 c1 c2 c3
+-----+
r0 |  0  1  2  0  <-- matrix[0][0] is 0 (Row 0 needs zeroing)
r1 |  3  4  5  2          matrix[0][3] is 0 (Col 3 needs zeroing)
r2 |  1  3  1  5
```

Step 2: Bottom-Up Resolution Pass We iterate through the matrix *ignoring the first row and first column for now* (from row M-1 down to 1, and col N-1 down to 1). We check the flags in the first row and column to decide if the current cell becomes 0.

Updating internal cells (Rows 1-2, Cols 1-3):

Since `matrix[0][3] == 0`, everything in column 3 becomes 0.

```
      c0 c1 c2 c3
+-----+
r0 |  0  1  2  0
r1 |  3  4  5  0 <-- (1, 3) becomes 0
r2 |  1  3  1  0 <-- (2, 3) becomes 0
```

Step 3: Resolve the First Row and First Column Finally, we apply our flags to the first row and first column themselves.

- Does the first row need to be zeroes? Look at `matrix[0][0]`. It is 0, so Row 0 becomes all 0s.
- Does the first column need to be zeroes? Look at `col0`. It is 0, so Column 0 becomes all 0s.

Final Matrix:

```
[ 0, 0, 0, 0 ]
[ 0, 4, 5, 0 ]
[ 0, 3, 1, 0 ]
```

3. Time and Space Complexity Analysis

```
+-----+
| TIME COMPLEXITY DERIVATION:  $O(M * N)$  |
+-----+
| The algorithm requires iterating over the matrix cells. |
| |
| M Rows total |
| [=====] |
| |
| +--> For each row, we visit N Columns |
|      [*, *, *, *, *, ..., N] |
| |
| Pass 1: Visit every cell once to set flags ->  $M * N$  operations |
| Pass 2: Visit every cell once to apply 0s ->  $M * N$  operations |
| |
| Total Work =  $(M * N) + (M * N) = 2 * (M * N)$  |
| Drop the constant 2 ->  $O(M * N)$  |
+-----+
```

Time Complexity: $O(M * N)$ where M is the number of rows and N is the number of columns. We iterate through the matrix essentially twice.

```
+-----+
| SPACE COMPLEXITY DERIVATION:  $O(1)$  |
+-----+
| Approach A Space (Suboptimal): |
|   Row Array: [0, 1, 0, ...] Size M |
|   Col Array: [1, 0, 0, ...] Size N |
|   Total Extra Memory =  $O(M + N)$  |
| |
| Approach B Space (Optimal): |
|   We reuse the input matrix itself for storage. |
|   Extra variables created: |
|   - col0 (1 integer) -> Size 1 |
|   - loop counters (r, c) -> Size 2 |
| |
|   Total Extra Memory = 3 integers =  $O(1)$  constant space. |
+-----+
```

Space Complexity: $O(1)$ auxiliary space. We modify the matrix in place and only use a single extra variable (col0) regardless of the matrix size.

4. Solution Code

Python Implementations

```
# =====  
# Python - Optimal O(1) Space Solution  
# =====  
def setZeroes_optimal(matrix):  
    """  
    Modifies the matrix in-place to set rows and columns to zero.  
    Uses the first row and first column as state-tracking arrays.  
    """  
  
    if not matrix:  
        return  
  
    rows = len(matrix)  
    cols = len(matrix[0])  
    col0 = 1 # 1 means true/keep data, 0 means false/zero it out  
  
    # Pass 1: Set flags in the first row and first col  
    for r in range(rows):  
        # Check if the first column needs to be zeroed  
        if matrix[r][0] == 0:  
            col0 = 0  
  
        # Check the rest of the row  
        for c in range(1, cols):  
            if matrix[r][c] == 0:  
                # Mark the corresponding first row and first col cells  
                matrix[r][0] = 0  
                matrix[0][c] = 0  
  
    # Pass 2: Iterate backwards and use flags to set zeroes  
    # We iterate backwards to prevent altering the first row/col flags  
    # before we are done referencing them for the inner matrix cells.  
    for r in range(rows - 1, -1, -1):  
        for c in range(cols - 1, 0, -1):  
            # If the flag in the row OR col is 0, set cell to 0  
            if matrix[r][0] == 0 or matrix[0][c] == 0:  
                matrix[r][c] = 0  
  
    # Finally, handle the first column based on the col0 flag  
    if col0 == 0:  
        matrix[r][0] = 0
```

```

# =====
# Python - Suboptimal O(M+N) Space Solution (For interview context)
# =====
def setZeroes_suboptimal(matrix):
    rows = len(matrix)
    cols = len(matrix[0])

    row_flags = [False] * rows
    col_flags = [False] * cols

    # Pass 1: Record zeroes
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] == 0:
                row_flags[r] = True
                col_flags[c] = True

    # Pass 2: Apply zeroes
    for r in range(rows):
        for c in range(cols):
            if row_flags[r] or col_flags[c]:
                matrix[r][c] = 0

```

JavaScript Implementations

```

// =====
// JavaScript - Optimal O(1) Space Solution
// =====
/**
 * @param {number[][]} matrix
 * @return {void} Do not return anything, modify matrix in-place instead.
 */
var setZeroes_optimal = function(matrix) {
    if (!matrix || matrix.length === 0) return;

    const rows = matrix.length;
    const cols = matrix[0].length;
    let col0 = 1; // Flag for the first column

    // Pass 1: Mark the zero states in the first row/col
    for (let r = 0; r < rows; r++) {
        if (matrix[r][0] === 0) col0 = 0;

        for (let c = 1; c < cols; c++) {
            if (matrix[r][c] === 0) {
                matrix[r][0] = 0;
            }
        }
    }

```

```

        matrix[0][c] = 0;
    }
}

// Pass 2: Update the matrix from bottom-right to top-left
for (let r = rows - 1; r >= 0; r--) {
    for (let c = cols - 1; c >= 1; c--) {
        if (matrix[r][0] === 0 || matrix[0][c] === 0) {
            matrix[r][c] = 0;
        }
    }

    // Apply the col0 flag to the first column
    if (col0 === 0) {
        matrix[r][0] = 0;
    }
}

};

// =====
// JavaScript - Suboptimal O(M+N) Space Solution
// =====
var setZeroes_suboptimal = function(matrix) {
    const rows = matrix.length;
    const cols = matrix[0].length;

    const rowFlags = new Set();
    const colFlags = new Set();

    // Pass 1
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (matrix[r][c] === 0) {
                rowFlags.add(r);
                colFlags.add(c);
            }
        }
    }

    // Pass 2
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (rowFlags.has(r) || colFlags.has(c)) {
                matrix[r][c] = 0;
            }
        }
    }
}

```

```

    }
}
};

```

Note 1: Terminology and Techniques Used

- **In-Place Algorithm:** An algorithm that transforms input using no auxiliary data structures. It implies modifying the data structure directly, relying only on a few extra variables for pointers or state tracking (resulting in $O(1)$ space). It's crucial for system design when handling massive datasets that cannot be duplicated in memory.
 - **State Array / Flag Array:** Using a structure to remember the state of an application or dataset at a given point in time. In our optimal solution, we used **Sentinel Mapping**, where we repurposed existing data boundaries (the first row/col) to act as our state arrays without destroying the overarching logic.
 - **Forward-Backward Pass (or Top-Down, Bottom-Up):** Notice how in Pass 1 we iterate forwards, but in Pass 2 we iterate *backwards* (`for r in range(rows - 1, -1, -1)`). This is a common technique when doing in-place grid modifications to ensure you don't overwrite the flags you placed at the "start" of the grid before you finish processing the "end" of the grid.
-

Note 2: Real-World Interview Variants (Google, Meta, Bloomberg)

In senior-level interviews, this exact problem is often disguised as a domain-specific scenario. Here is how top companies have recently framed it and how to pivot the underlying logic:

1. Bloomberg: The Financial Data Sector Invalidation

- **Prompt:** "You have a large grid of financial metrics. Rows are sectors, columns are time periods. If a cell contains a 'null' or '0' (indicating missing data), the entire sector's data is considered invalid for all periods, and the entire period is invalid for all sectors. Sanitize the grid."
- **How to solve:** This is identically Leetcode 73. However, Bloomberg interviewers will often add a constraint: *What if the matrix is too large to fit in memory and is highly sparse (mostly zeroes)?*
- **L5/L6 Pivot:** If it's a massive Sparse Matrix, the $O(1)$ array trick is terrible because you still iterate $O(M * N)$ times over millions of empty cells. Instead, you'd suggest storing only the coordinates of non-zero elements using a HashMap or a Coordinate List (COO). You would aggregate the invalid rows/cols into HashSets, and only process the non-zero list.

2. Meta (Facebook): The Spreadsheet Error Cascading

- **Prompt:** “Imagine you are building the backend for a simple Google Sheets/Excel clone. If a cell evaluates to a `#DIV/0!` error, we want to visually gray out its entire row and column in the UI.”
- **How to solve:** The logic is the same, but they are looking for production-level thinking.
- **L5/L6 Pivot:** Modifying a massive matrix sequentially blocks the main thread. You would explain the $O(1)$ algorithm, but then suggest an event-driven or chunked approach. You would emit events: `RowInvalidated(r)` and `ColInvalidated(c)`. The frontend UI would listen to these events and apply the gray-out CSS classes at the DOM level without ever actually mutating the underlying $M \times N$ data structure just for a visual update.

3. Google: The Server Rack Dependency Grid

- **Prompt:** “You have a 2D grid representing clusters of servers. If a server goes down (state 0), we must quarantine the entire horizontal and vertical subnet (set them to 0) to prevent a virus spread.”
- **How to solve:** Identical algorithmic logic.
- **L5/L6 Pivot:** The interviewer will likely ask how to handle continuous incoming streams of failures. The $O(1)$ in-place algorithm is a one-time batch job. For a live system, you would keep the `row_flags` and `col_flags` arrays permanently in memory ($O(M+N)$ space). When a new failure arrives at (x, y) , you check if `row[x]` or `col[y]` is already quarantined. If not, you flip the flag and broadcast the quarantine command. This shifts the focus from purely memory optimization to CPU optimization and fast $O(1)$ lookups in a distributed system context.

54. Spiral Matrix

Here is how a senior engineer approaches the “Spiral Matrix” problem. At the L5/L6 level, the focus shifts from just getting the code to work to managing state cleanly, avoiding off-by-one errors, and communicating the logic so clearly that it feels obvious to the interviewer.

1. Problem Explanation

We are given a 2D array (a matrix) of size m (rows) by n (columns). Our goal is to return all elements of the matrix in spiral order.

Spiral order means we start at the top-left corner, walk all the way to the right, then all the way down, then all the way left, then all the way up, peeling off the outer layer of the matrix. We repeat this process on the remaining inner matrix until we have visited every element.

Let us look at a standard 3×4 matrix:

```
[ 1,  2,  3,  4 ]
[ 5,  6,  7,  8 ]
```

[9, 10, 11, 12]

The spiral order for this would be: 1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7.

2. Solution Explanation

The most robust way to solve this is using the **Boundary Shrinking** approach. Instead of keeping track of every visited cell (which requires extra memory or modifying the original matrix), we maintain four boundary pointers: **top**, **bottom**, **left**, and **right**.

These boundaries act as “walls”. Every time we complete a line in our spiral, we move the corresponding wall inward.

The Non-Trivial Edge Case: The biggest trap in this problem happens when the matrix is not perfectly square (e.g., m is not equal to n). As you peel away layers, you might end up with a single row or a single column left in the middle. If you blindly execute all four steps (Right, Down, Left, Up) in a loop, you will count the remaining elements twice. To fix this, before moving Left or Up, we must check if the **top** and **bottom** walls, or the **left** and **right** walls, have crossed.

Step-by-Step Visualization:

Let’s trace the 3x4 matrix. Initial boundaries: **top** = 0, **bottom** = 2, **left** = 0, **right** = 3

```
      L           R
+-----+
T |  1   2   3   4 |
  |  5   6   7   8 |
B |  9  10  11  12 |
+-----+
```

Pass 1:

- **Move Right:** From L to R on the T row. Append [1, 2, 3, 4].
- **Shrink Top:** T becomes 1.

```
      L           R
..... (Row 0 is done)
T +-----+
  |  5   6   7   8 |
B |  9  10  11  12 |
+-----+
```

- **Move Down:** From T to B on the R column. Append [8, 12].
- **Shrink Right:** R becomes 2.

```
      L           R
..... (Row 0 done)
```



```

T +-----+ .
  | 5   6   7| . (Col 3 done)
B | 9  10  11| .
  +-----+ .

```

- **Move Left:** (Check $T \leq B$. Yes, $1 \leq 2$). From R to L on the B row. Append [11, 10, 9].
- **Shrink Bottom:** B becomes 1.

```

      L      R
.....
T +-----+ .
B | 5   6   7| .
..... (Row 2 done)

```

- **Move Up:** (Check $L \leq R$. Yes, $0 \leq 2$). From B to T on the L column. Append [5].
- **Shrink Left:** L becomes 1.

```

      L      R
.....
T/B . | 6   7| .
.....

```

Pass 2:

- **Move Right:** L to R on T row. Append [6, 7].
- **Shrink Top:** T becomes 2.
- Now $T(2) > B(1)$. The main loop terminates.
- **Final Output:** [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]

3. Time and Space Complexity Analysis

Complexity Derivation:

```

Total elements in matrix = m rows * n columns
Output array size        = m * n elements
Visited array needed?    = No (we use boundaries)
Recursion stack needed?  = No (we use a simple while loop)

```

Time Complexity (TC) derivation:

```

+-----+
| Every cell is pushed into the result array exactly |
| one time. No cell is visited twice.                |
| Operations per cell = 1 (constant time append)     |
| Total Operations = m * n                            |
+-----+
TC = O(m * n)

```

Space Complexity (SC) derivation:

```

+-----+
| Variables used: top, bottom, left, right (4 ints) |
| Output array: stores m * n elements              |
+-----+
SC = O(1) auxiliary space (excluding output array)
SC = O(m * n) total space (including output array)

```

4. Solution Code

Here are the highly readable, senior-level implementations.

Python Snippet:

```

def spiralOrder(matrix):
    # Edge case check: empty matrix
    if not matrix or not matrix[0]:
        return []

    result = []

    # Define the 4 boundaries
    top = 0
    bottom = len(matrix) - 1
    left = 0
    right = len(matrix[0]) - 1

    # Continue spiraling while boundaries are valid
    while top <= bottom and left <= right:

        # 1. Traverse from left to right along the top boundary
        for i in range(left, right + 1):
            result.append(matrix[top][i])
        # Shrink the top boundary downwards
        top += 1

        # 2. Traverse from top to bottom along the right boundary
        for i in range(top, bottom + 1):
            result.append(matrix[i][right])
        # Shrink the right boundary inwards
        right -= 1

        # Make sure we are now on a different row than the one we just processed top-to-bot.
        if top <= bottom:

```

```

    # 3. Traverse from right to left along the bottom boundary
    for i in range(right, left - 1, -1):
        result.append(matrix[bottom][i])
    # Shrink the bottom boundary upwards
    bottom -= 1

    # Make sure we are now on a different column than the one we just processed right-to-left
    if left <= right:
        # 4. Traverse from bottom to top along the left boundary
        for i in range(bottom, top - 1, -1):
            result.append(matrix[i][left])
        # Shrink the left boundary inwards
        left += 1

    return result

```

JavaScript Snippet:

```

function spiralOrder(matrix) {
    if (matrix.length === 0 || matrix[0].length === 0) {
        return [];
    }

    const result = [];

    let top = 0;
    let bottom = matrix.length - 1;
    let left = 0;
    let right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {

        // 1. Move left to right
        for (let i = left; i <= right; i++) {
            result.push(matrix[top][i]);
        }
        top++;

        // 2. Move top to bottom
        for (let i = top; i <= bottom; i++) {
            result.push(matrix[i][right]);
        }
        right--;

        // Ensure we haven't crossed rows
        if (top <= bottom) {
            // 3. Move right to left

```

```

        for (let i = right; i >= left; i--) {
            result.push(matrix[bottom][i]);
        }
        bottom--;
    }

    // Ensure we haven't crossed columns
    if (left <= right) {
        // 4. Move bottom to top
        for (let i = bottom; i >= top; i--) {
            result.push(matrix[i][left]);
        }
        left++;
    }
}

return result;
}

```

Note 1: Algorithm/Technique Breakdown

Technique Name: Boundary Shrinking (or Layer-by-Layer Matrix Simulation).

- **What it is:** A simulation approach that uses pointer limits to restrict traversal instead of keeping a separate boolean grid to track visited cells.
 - **Why it helps:** In system design and algorithms, state management is the root of most bugs. Keeping a full $m \times n$ visited array increases memory overhead and branching logic (checking bounds + checking visited state). Boundary Shrinking distills the state down to exactly 4 integers representing the current bounding box.
 - **Application to this problem:** It inherently models the concept of “peeling layers.” Once an outer layer is peeled, the core problem remains exactly the same, just bounded within a smaller coordinate plane.
-

Note 2: Real World & Interview Variations

In top-tier technical interviews (Google, Meta, Bloomberg), interviewers often test your ability to adapt this core logic.

1. Google: Spiral Matrix II (Generation)

- **The Problem:** Instead of reading a matrix, you are given an integer n . You must generate an $n \times n$ matrix filled with elements from 1 to $n \times n$ in spiral order.

- **The Solution:** You apply the exact same Boundary Shrinking algorithm. You initialize an empty matrix of zeros. Instead of `result.push(matrix[top][i])`, you keep a counter `val = 1`, and do `matrix[top][i] = val; val++;`. This tests if you memorized the problem or if you actually understand the boundary pointer concept.

2. Meta: Diagonal Traversal / Snake Traversal

- **The Problem:** Print a matrix diagonally or in a snake-like “S” shape.
- **The Solution:** This tests a different kind of state management. Instead of bounding boxes, you track direction indicators (`going_up`, `going_down`) and calculate the next coordinates (`r - 1, c + 1` for up-right). It requires similar careful edge-case checking when hitting the borders of the matrix, ensuring indices wrap or cap out correctly.

3. Bloomberg: Real-World Grid Routing (Drone / Mapping)

- **The Problem:** You have a drone or a mapping vehicle navigating a dense city grid (imagine programming a street sweeper or mapping camera through central London or another tier-1 city layout). Given a target coordinate in a bounded zone, plot a spiral search path outward from the center, or inward from the perimeter.
- **The Solution:** If moving inward, it’s identical to our solution. If moving *outward* from a center point, the logic is reversed. You start `top`, `bottom`, `left`, and `right` at the central coordinate. Your simulation loop expands the boundaries (`left -= 1, right += 1`, etc.) and tests limits against the max city grid bounds. This tests practical application and adaptability of the algorithm to simulated hardware behavior.

48. Rotate Image

Here is how a top-of-the-band L5 or L6 software engineer would break down, explain, and solve the “Rotate Image” problem.

1. Problem Explanation

We are given an $N \times N$ 2D matrix that represents an image. We need to rotate this entire image by 90 degrees clockwise.

The most critical constraint is that we must do this **in-place**. This means we cannot create a new $N \times N$ matrix to hold the rotated values and then copy them back. We must shift the elements around within the exact same matrix provided in the input, using only a tiny, constant amount of extra memory (like a few variables to hold temporary values).

Input:

```
[
  [1, 2, 3],
```

```

    [4, 5, 6],
    [7, 8, 9]
]

```

Desired Output:

```

[
    [7, 4, 1],
    [8, 5, 2],
    [9, 6, 3]
]

```

If you look closely at the output, the first row of the original matrix [1, 2, 3] has become the last column of the rotated matrix, reading from top to bottom.

2. Solution Explanation

At a senior level, it is best to discuss multiple valid approaches. We will look at two solutions: an incredibly elegant mathematical trick (Transpose and Reverse) and a highly optimized pointer manipulation technique (Layer-by-Layer 4-Way Swap). Both are excellent for an interview.

Approach A: Transpose and Reverse (The Elegant Solution) This approach breaks the complex 90-degree rotation into two simpler matrix operations.

Step 1: Transpose the Matrix Transposing a matrix means turning its rows into columns and its columns into rows. Visually, you flip the matrix over its main diagonal (the line from top-left to bottom-right).

Original Matrix:	Main Diagonal:	Swap across diagonal:
1 2 3	(1) 2 3	(1) 4 7
4 5 6	4 (5) 6	2 (5) 8
7 8 9	7 8 (9)	3 6 (9)

Notice how 2 and 4 swapped places. 3 and 7 swapped places. 6 and 8 swapped places. The diagonal elements (1, 5, 9) stayed exactly where they were.

Step 2: Reverse Each Row Now, we simply take the transposed matrix and reverse the elements in each individual row.

Transposed Matrix:	Reverse Row 1:	Reverse Row 2:	Reverse Row 3 (Final):
1 4 7	7 4 1	7 4 1	7 4 1
2 5 8 ->	2 5 8 ->	8 5 2 ->	8 5 2
3 6 9	3 6 9	3 6 9	9 6 3

We have successfully rotated the matrix 90 degrees clockwise!

Approach B: Layer-by-Layer 4-Way Swap (The “L5/L6 Pointer Mastery” Solution) Instead of doing two passes over the matrix, we can do it in a single pass by rotating the matrix layer by layer, moving from the outside perimeter inward.

Think of the matrix as an onion. For an $N \times N$ matrix, we rotate the outermost “square” of elements, then step inside to the next smaller square, and repeat until we hit the center.

For every element in the top row of our current layer, we can perform a 4-way swap. Let’s look at the corners of a 3x3 matrix:

- Top-Left (TL)
- Top-Right (TR)
- Bottom-Right (BR)
- Bottom-Left (BL)

To rotate 90 degrees:

1. Save TL in a temporary variable.
2. Move BL into TL.
3. Move BR into BL.
4. Move TR into BR.
5. Move the temporary variable (original TL) into TR.

Visualizing the 4-Way Swap (Corners first):

Step 0:	Step 1 (BL to TL):	Step 2 (BR to BL):	Step 3 (TR to BR):	Step 4 (Temp to TR):
1 2 3	7 2 3	7 2 3	7 2 1	7 2 1
4 5 6	4 5 6	4 5 6	4 5 6	4 5 6
7 8 9	7 8 9	9 8 9	9 8 9	9 8 3

Next, we shift one position to the right and do the same 4-way swap for the middle edge elements.

The Coordinate Math (Explained Simply): If our matrix size is N , and we are looking at an element at (row, column):

- It needs to move to (column, $N - 1 - \text{row}$).
- This mapping ensures that row 0 becomes the last column ($N - 1$), row 1 becomes the second to last column ($N - 2$), and so forth.

3. Time and Space Complexity Analysis

Time Complexity Derivation Diagram:

```

+-----+
| Row 0: * * * * * (N items)|
| Row 1: * * * * * (N items)|
| Row 2: * * * * * (N items)|

```

```

| ... | (N rows total)
| Row N: * * * * * (N items) |
+-----+

```

Time Complexity: $O(N * N)$ We must visit and move every single element in the $N \times N$ matrix at least once. Since there are $N * N$ total elements, the number of operations scales squarely with the dimension N . Thus, the time complexity is $O(N * N)$. Both approaches have the same time complexity, though the 4-way swap technically reads/writes fewer times overall.

Space Complexity: $O(1)$

Space Complexity Derivation Diagram:

Input Matrix (Modified In-Place)	Extra Memory Used
+-----+	+-----+
7 4 1	temp -> Just one variable!
8 5 2	+-----+
9 6 3	
+-----+	

Because the problem strictly requires us to rotate the image in-place, we do not create any new matrices or arrays that grow with N . We only use a few single-value variables (like a `temp` variable for swapping). Therefore, our space complexity is strictly $O(1)$, meaning constant extra space.

4. Solution Code

JavaScript

```

/**
 * Approach A: Transpose and Reverse
 * Time:  $O(N * N)$  | Space:  $O(1)$ 
 */
function rotateApproachA(matrix) {
    const n = matrix.length;

    // Step 1: Transpose the matrix
    // We iterate through the rows, and for columns, we start at j = i.
    // Starting at j = i prevents us from swapping elements back to their original places.
    for (let i = 0; i < n; i++) {
        for (let j = i; j < n; j++) {
            // Swap matrix[i][j] with matrix[j][i]
            let temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
}

```



```

    }

    // Step 2: Reverse each row
    for (let i = 0; i < n; i++) {
        // We only need to iterate to the middle of the row to reverse it
        for (let j = 0; j < Math.floor(n / 2); j++) {
            // Swap matrix[i][j] with matrix[i][n - 1 - j]
            let temp = matrix[i][j];
            matrix[i][j] = matrix[i][n - 1 - j];
            matrix[i][n - 1 - j] = temp;
        }
    }
}

/**
 * Approach B: Layer-by-Layer 4-Way Swap
 * Time:  $O(N * N)$  | Space:  $O(1)$ 
 */
function rotateApproachB(matrix) {
    const n = matrix.length;

    // We only need to process half the matrix layers.
    // E.g., for a 4x4, there are 2 layers. For a 5x5, there are 2 outer layers + 1 center
    for (let layer = 0; layer < Math.floor(n / 2); layer++) {
        let first = layer;
        let last = n - 1 - layer;

        // Loop through the elements in the top row of the current layer
        for (let i = first; i < last; i++) {
            // Offset helps us target the correct elements on the right, bottom, and left sides
            let offset = i - first;

            // Save the top-left element
            let top = matrix[first][i];

            // Move bottom-left to top-left
            matrix[first][i] = matrix[last - offset][first];

            // Move bottom-right to bottom-left
            matrix[last - offset][first] = matrix[last][last - offset];

            // Move top-right to bottom-right
            matrix[last][last - offset] = matrix[i][last];

            // Move saved top-left into top-right
            matrix[i][last] = top;
        }
    }
}

```

```

    }
}
}

```

Python

```

# Approach A: Transpose and Reverse
# Time: O(N * N) | Space: O(1)
def rotate_approach_a(matrix):
    n = len(matrix)

    # Step 1: Transpose the matrix
    for i in range(n):
        # j starts at i to only swap the top-right triangle with the bottom-left triangle
        for j in range(i, n):
            # Swap elements across the main diagonal
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Step 2: Reverse each row
    for i in range(n):
        # Python allows an elegant in-place reversal using two pointers natively
        # Alternatively: matrix[i].reverse()
        left, right = 0, n - 1
        while left < right:
            matrix[i][left], matrix[i][right] = matrix[i][right], matrix[i][left]
            left += 1
            right -= 1

# Approach B: Layer-by-Layer 4-Way Swap
# Time: O(N * N) | Space: O(1)
def rotate_approach_b(matrix):
    n = len(matrix)

    left, right = 0, n - 1

    # Process layer by layer, closing in on the center
    while left < right:
        # i represents the offset from the 'left' boundary
        for i in range(right - left):
            top, bottom = left, right

            # Save the top-left value
            top_left = matrix[top][left + i]

            # Move bottom-left into top-left
            matrix[top][left + i] = matrix[bottom - i][left]

```

```

# Move bottom-right into bottom-left
matrix[bottom - i][left] = matrix[bottom][right - i]

# Move top-right into bottom-right
matrix[bottom][right - i] = matrix[top + i][right]

# Move saved top-left into top-right
matrix[top + i][right] = top_left

left += 1
right -= 1

```

Note 1: Core Terms and Techniques

- **In-Place Algorithm:** An algorithm that transforms data using no auxiliary data structure. A small, constant amount of extra storage space is allowed for variables (like a `temp` variable). This is crucial for optimizing memory usage in systems with massive data sets.
- **Matrix Transposition:** A mathematical operation that flips a matrix over its diagonal. In software engineering, recognizing that complex 2D spatial transformations can be broken down into native, standard transformations (like transpose + reverse) demonstrates strong analytical problem-solving skills, allowing you to write cleaner, less error-prone code.

Note 2: Real-World / Interview Variations

Senior interviews at top tech companies frequently alter constraints to test adaptability. Here is how this problem is disguised or twisted:

1. “Rotate an image represented by a 1D array” (Google, Meta)

- **The Twist:** The image is stored as a flat array [1, 2, 3, 4, 5, 6, 7, 8, 9] instead of a 2D array to optimize cache locality.
- **How to solve it:** You must translate 2D coordinates to 1D indices. The formula to access (row, col) in a 1D array of width N is `index = (row * N) + col`. You would implement the exact same 4-way swap logic, but everywhere you would write `matrix[row][col]`, you write `array[(row * N) + col]`.

2. “Rotate the image Anti-Clockwise” or “Rotate 180 Degrees” (Bloomberg)

- **The Twist:** Changing the direction or magnitude of rotation.
- **How to solve it (Anti-Clockwise):** Instead of (Transpose -> Reverse Rows), you do (Reverse Rows -> Transpose) or (Transpose -> Reverse Columns).

- **How to solve it (180 Degrees):** A 180-degree rotation is simply reversing the matrix top-to-bottom, and then reversing every row left-to-right. No transposition is needed.

3. “Rotate a non-square (M x N) matrix” (Meta)

- **The Twist:** The matrix is not a perfect square, making an in-place rotation mathematically impossible without complex, non-standard memory shifting, because the dimensions change (an M x N becomes N x M).
- **How to solve it:** The interviewer expects you to immediately point out that an $O(1)$ space in-place rotation is impossible due to dimension shifting. You must propose an out-of-place solution. You create a new matrix `result` of size N x M. You then iterate through the original matrix: `result[col][M - 1 - row] = original[row][col]`.

79. Word Search

To solve the “Word Search” problem at a top-tier engineering level, you must not only produce a working algorithm but also demonstrate an understanding of edge cases, space optimization, and search space pruning. A standard passing solution uses basic Backtracking. A top-of-the-band solution adds heuristic pruning to drastically reduce the time spent in the worst-case scenarios.

Here is how to break down the problem, design the solution, and analyze it.

1. Problem Explanation

We are given an M (rows) by N (columns) grid of characters and a target `word`. We need to determine if the `word` can be spelled by moving sequentially from one cell to an adjacent cell (up, down, left, or right).

Constraints & Rules:

- You can start at any cell in the grid.
- You must move to an immediately adjacent cell (no diagonals).
- **Crucial Constraint:** You cannot reuse the exact same cell twice within a single attempt to form the word.

Visualization of a Match: Grid: [A, B, C, E] [S, F, C, S] [A, D, E, E]

Word: “ABCCED”

```
[A] -> [B] -> [C]   E
      S      F      |
                  [C] S
                  |
      A      [D] <- [E] E
```

The path connects perfectly without revisiting any coordinates.

2. Solution Explanation

To solve this, we will use **Depth-First Search (DFS) with Backtracking**.

The Basic Approach:

1. We iterate through every single cell in the $M \times N$ grid.
2. If the character in the current cell matches the first character of our target word, we launch a DFS from that cell.
3. The DFS looks at the 4 neighboring cells. If a neighbor matches the *next* character in the word, we move there and continue the search.
4. To ensure we don't revisit a cell in our current path, we temporarily mark the current cell as "visited" (often by changing its value to a special character like #).
5. If a path fails (we hit a dead end before finishing the word), we "backtrack"—meaning we restore the cell's original character and try a different path.

The Top-Tier (L5/L6) Optimizations (Pruning): While the basic approach is often accepted, top tech companies will test your system-level thinking by feeding your algorithm adversarial test cases. For example, a massive grid of 'A's with a single 'B' at the end, searching for a word with 50 'A's and one 'Z'. The standard DFS will Time Out.

To optimize:

1. **Character Frequency Pruning:** Before starting the DFS, count the characters in the grid. If the grid doesn't contain enough of the specific characters needed to form the word, return **false** immediately in $O(M * N)$ time without doing any heavy recursive searches.
2. **Path Reversal Pruning:** If the first character of the word appears *more frequently* in the grid than the last character of the word, we reverse the word. Searching from the character with the lower frequency heavily reduces the "branching factor" (the number of wrong paths the algorithm attempts) at the top of the search tree.

3. Time and Space Complexity Analysis

Time Complexity: $O(M * N * 3^L)$

- M = Number of rows in the board.
- N = Number of columns in the board.
- L = Length of the target word.

ASCII Derivation of Time Complexity:

Phase 1: Grid Traversal

We must potentially start a search from every cell.

Total starting points = $M * N$

For each starting point, we build a search tree of length L .

Because the first step has 4 choices, but every subsequent step has at most 3 choices (since one direction is the cell we just came from), the maximum number of paths explored is $4 * 3^{(L-1)}$. In Big-O notation, constants are dropped, leaving us with a branching factor of 3^L . Multiplying by the $M * N$ starting points yields $O(M * N * 3^L)$.

- The space complexity is determined by the maximum depth of the call stack due to recursion.
- The recursion depth will never exceed the length of the word L.
- We use $O(1)$ auxiliary space because we modify the board in-place to track visited cells rather than allocating a separate $M * N$ boolean visited matrix.

Python Implementations

1. Standard DFS Backtracking (Solid passing solution)

166

```

# Out of bounds or character doesn't match
if (r < 0 or c < 0 or r >= ROWS or c >= COLS or
    board[r][c] != word[i]):
    return False

# Mark cell as visited by temporarily modifying it
temp = board[r][c]
board[r][c] = '#'

# Explore all 4 adjacent directions
res = (dfs(r + 1, c, i + 1) or
       dfs(r - 1, c, i + 1) or
       dfs(r, c + 1, i + 1) or
       dfs(r, c - 1, i + 1))

# Backtrack: Restore the cell's original character
board[r][c] = temp

return res

# Iterate through every cell to find a valid starting point
for r in range(ROWS):
    for c in range(COLS):
        if dfs(r, c, 0):
            return True

return False

```

2. Optimized DFS with Pruning (Top-tier solution)

```

from collections import Counter

def exist_optimized(board, word):
    ROWS, COLS = len(board), len(board[0])

    # Optimization 1: Character frequency check (Fail fast)
    board_counts = Counter(char for row in board for char in row)
    word_counts = Counter(word)

    for char, count in word_counts.items():
        if board_counts[char] < count:
            return False # Board doesn't have enough of this character

    # Optimization 2: Reverse word based on frequency pruning
    # If the first character is more abundant than the last, reverse the word.
    # This ensures we start searching from the character with fewer occurrences,
    # drastically reducing dead-end branches.

```

```

if board_counts[word[0]] > board_counts[word[-1]]:
    word = word[::-1]

def dfs(r, c, i):
    if i == len(word): return True

    if r < 0 or c < 0 or r >= ROWS or c >= COLS or board[r][c] != word[i]:
        return False

    temp = board[r][c]
    board[r][c] = '#' # mark visited

    # Check directions. Short-circuiting 'or' ensures we stop on the first True
    found = (dfs(r + 1, c, i + 1) or
             dfs(r - 1, c, i + 1) or
             dfs(r, c + 1, i + 1) or
             dfs(r, c - 1, i + 1))

    board[r][c] = temp # backtrack
    return found

for r in range(ROWS):
    for c in range(COLS):
        if board[r][c] == word[0]:
            if dfs(r, c, 0): return True

return False

```

Javascript Implementations 1. Standard DFS Backtracking

```

function existStandard(board, word) {
    const ROWS = board.length;
    const COLS = board[0].length;

    function dfs(r, c, i) {
        // Base case: Word is fully found
        if (i === word.length) return true;

        // Check boundaries and character match
        if (r < 0 || c < 0 || r >= ROWS || c >= COLS || board[r][c] !== word[i]) {
            return false;
        }

        // Temporarily mark the cell to prevent revisiting in this path
        const temp = board[r][c];
        board[r][c] = '#';
    }
}

```



```

        // Traverse 4 directions
        const found = dfs(r + 1, c, i + 1) ||
                      dfs(r - 1, c, i + 1) ||
                      dfs(r, c + 1, i + 1) ||
                      dfs(r, c - 1, i + 1);

        // Backtrack: Restore state
        board[r][c] = temp;

        return found;
    }

    for (let r = 0; r < ROWS; r++) {
        for (let c = 0; c < COLS; c++) {
            if (dfs(r, c, 0)) return true;
        }
    }
    return false;
}

```

2. Optimized DFS with Pruning

```

function existOptimized(board, word) {
    const ROWS = board.length;
    const COLS = board[0].length;

    // Optimization 1: Frequency map check
    const boardCounts = new Map();
    for (let r = 0; r < ROWS; r++) {
        for (let c = 0; c < COLS; c++) {
            const char = board[r][c];
            boardCounts.set(char, (boardCounts.get(char) || 0) + 1);
        }
    }

    const wordCounts = new Map();
    for (const char of word) {
        wordCounts.set(char, (wordCounts.get(char) || 0) + 1);
    }

    // Fail fast if board lacks necessary characters
    for (const [char, count] of wordCounts.entries()) {
        if ((boardCounts.get(char) || 0) < count) return false;
    }

    // Optimization 2: Reverse word to minimize search space

```

```

let searchWord = word;
if ((boardCounts.get(word[0]) || 0) > (boardCounts.get(word[word.length - 1]) || 0)) {
    searchWord = word.split('').reverse().join('');
}

function dfs(r, c, i) {
    if (i === searchWord.length) return true;

    if (r < 0 || c < 0 || r >= ROWS || c >= COLS || board[r][c] !== searchWord[i]) {
        return false;
    }

    const temp = board[r][c];
    board[r][c] = '#'; // Mark visited

    const found = dfs(r + 1, c, i + 1) ||
                  dfs(r - 1, c, i + 1) ||
                  dfs(r, c + 1, i + 1) ||
                  dfs(r, c - 1, i + 1);

    board[r][c] = temp; // Backtrack
    return found;
}

// Start search only when first character matches
for (let r = 0; r < ROWS; r++) {
    for (let c = 0; c < COLS; c++) {
        if (board[r][c] === searchWord[0]) {
            if (dfs(r, c, 0)) return true;
        }
    }
}

return false;
}

```

Note 1: Terminology Explained

- **Depth-First Search (DFS):** An algorithm for traversing tree or graph data structures. It starts at a source node and explores as far as possible along each branch before retreating. *Why it helps here:* We need to trace continuous paths. DFS naturally follows a path to its conclusion (a full word or a dead end) before checking alternative paths.
- **Backtracking:** An algorithmic technique for solving problems recursively

by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point. *Why it helps here:* It allows us to simulate moving a piece on the board, but if we hit a dead end, we can “undo” our steps (unmark cells) and try a different fork in the road without needing to copy the entire board state for every move.

- **Search Space Pruning:** The process of eliminating sections of a search tree from consideration, reducing the number of calculations. *Why it helps here:* Checking character frequencies and reversing the string prunes massive subtrees of impossible or highly unlikely paths, changing worst-case scenarios into instant failures.

Note 2: Real-World Interview Variations (Google, Meta, Bloomberg)

Variation 1: Word Search II (Multiple Words)

- **Asked heavily by:** Google, Meta, Bloomberg.
- **The Problem:** Instead of one word, you are given a dictionary of thousands of words. Find all words present in the grid.
- **The Solution:** Do not run the single Word Search DFS thousands of times. Instead, insert all words into a **Trie (Prefix Tree)** data structure. You then iterate through the grid exactly once, performing DFS. At each cell, you step down the Trie. If a path in the grid doesn’t match any path in the Trie, you prune the DFS immediately.
- *Optimization detail:* When a word is found in the Trie, remove that word’s terminating marker to prevent duplicates, and ideally, prune that node from the Trie so future DFS paths don’t re-explore dead prefix trees.

Variation 2: Wrapping Grid (Pac-Man Topology)

- **Asked by:** Meta.
- **The Problem:** Same as Word Search, but going off the right edge of the board wraps you around to the left edge of the same row (and similarly for top/bottom edges).
- **The Solution:** The core backtracking algorithm remains identical. The only change is how you calculate the next row and column during DFS. Instead of returning **False** on an out-of-bounds check, you use Modulo Arithmetic to wrap the coordinates: `next_r = (r + 1) % ROWS` and `next_c = (c - 1 + COLS) % COLS`. You still need to track visited cells so you don’t loop endlessly.

Variation 3: Wildcard Grid (Boggle with Blanks)

- **Asked by:** Bloomberg.
- **The Problem:** Some cells in the grid contain a `*` character which can represent any letter.
- **The Solution:** Modify the DFS character matching condition. In the recursive check, instead of just `if board[r][c] == word[i]`, you eval-

uate if `board[r][c] == word[i]` or `board[r][c] == '*'`. You must still backtrack correctly, ensuring you restore the `*` and not the literal matched character.

34. Find First and Last Position of Element in Sorted Array

As an engineer preparing for or operating at an L5/L6 level, you know that solving a problem is only half the battle. The other half is communicating your thought process, identifying edge cases, and showing that you can write production-ready, highly readable code.

Here is how a Senior/Staff engineer at Google would break down and solve LeetCode 34: “Find First and Last Position of Element in Sorted Array.”

1. Problem Explanation

We are given an array of integers called `nums` that is sorted in ascending order. We are also given a `target` integer. Our goal is to find the starting and ending index (0-based) of that `target` in the array. If the `target` is not in the array, we must return `[-1, -1]`.

The Catch: The algorithm must run in $O(\log N)$ time complexity.

Examples to build intuition:

- **Example 1:** `nums = [5, 7, 7, 8, 8, 10]`, `target = 8`
 - The number 8 appears at index 3 and index 4.
 - Output: `[3, 4]`
- **Example 2:** `nums = [5, 7, 7, 8, 8, 10]`, `target = 6`
 - The number 6 is nowhere in the array.
 - Output: `[-1, -1]`
- **Example 3:** `nums = [8, 8, 8, 8]`, `target = 8`
 - The number 8 spans the whole array.
 - Output: `[0, 3]`

Why is this non-trivial? If we were allowed $O(N)$ time, we could just scan the array from left to right to find the first 8, and right to left to find the last 8. But $O(\log N)$ is a strict constraint. In computer science, an $O(\log N)$ search on a sorted array immediately dictates that we must use **Binary Search**.

2. Solution Explanation

To solve this in $O(\log N)$ time, we need to perform two separate binary searches:

1. **Search 1:** Find the *first* occurrence (the left boundary).
2. **Search 2:** Find the *last* occurrence (the right boundary).

How standard Binary Search works: You look at the middle element. If it's your target, you stop. If it's smaller than your target, you discard the left half. If it's larger, you discard the right half.

How we modify it for this problem: When we find the **target**, we *do not stop*.

- **Finding the First Position:** If `nums[mid] == target`, this *might* be the first occurrence. We record this index, but just to be sure, we continue searching in the **left** half (`right = mid - 1`) to see if there is an earlier occurrence.
- **Finding the Last Position:** If `nums[mid] == target`, we record this index, but we continue searching in the **right** half (`left = mid + 1`) to see if there is a later occurrence.

Visualization (ASCII Diagram):

Let's find the **first** position of `target = 8` in `[5, 7, 7, 8, 8, 10]`.

```
STEP 1: Initial state
[ 5 , 7 , 7 , 8 , 8 , 10 ]
  ^           ^           ^
  Left        Mid        Right
```

`nums[Mid]` is 7. `7 < 8`. So, the target must be on the right.
Action: `Left = Mid + 1`

```
STEP 2:
[ 5 , 7 , 7 , 8 , 8 , 10 ]
              ^     ^     ^
              Left  Mid  Right
```

`nums[Mid]` is 8. This **EQUALS** our target!
Action: We record index 4 as a potential answer.
But to find the **FIRST** occurrence, we must keep looking left.
Action: `Right = Mid - 1`

```
STEP 3:
[ 5 , 7 , 7 , 8 , 8 , 10 ]
                ^
                Left/Mid/Right
```

nums[Mid] is 8. This EQUALS our target!
Action: We update our recorded index to 3.
We must keep looking left.
Action: Right = Mid - 1

STEP 4:
Right is now less than Left. The loop breaks.
The earliest index we recorded is 3. First position = 3.
The process is mirrored for finding the last position (moving Left = Mid + 1
when a match is found to push the search space as far right as possible).

3. Time and Space Complexity Analysis

Time Complexity: $O(\log N)$ We run a standard binary search twice. Each binary search cuts the search space in half at every step.

ASCII Diagram for Time Complexity Derivation:

Search Space Reduction (N elements):

Level 0:	[===== N elements =====]	-> 1 operation
Level 1:	[===== N/2 =====]	-> 1 operation
Level 2:	[===== N/4 =====]	-> 1 operation
Level 3:	[===== N/8 =====]	-> 1 operation
...		
Level K:	[1]	-> 1 operation

Total operations = K.

Since we divide by 2 until we reach 1 element: $N / (2^K) = 1$

This means $2^K = N$, therefore $K = \log_2(N)$.

Two searches means $2 * \log_2(N)$. In Big-O notation, we drop constants, yielding $O(\log N)$.

Space Complexity: $O(1)$ We only use a few variables (left, right, mid, first_pos, last_pos) to keep track of our indices. We do not use any recursive call stacks or allocate any extra arrays that grow with the input size. Therefore, the memory footprint is constant.

4. Solution Code

Here are two approaches. In a senior-level interview, you should briefly mention Approach 1 (Linear) to show you understand the brute-force baseline, but immediately write Approach 2 (Binary Search) as your actual solution.

Approach 1: Linear Scan (Not optimized, $O(N)$ time) *Useful to acknowledge as the baseline.*

Python:

```
def searchRange_linear(nums, target):
    first, last = -1, -1
    for i in range(len(nums)):
        if nums[i] == target:
            if first == -1:
                first = i # Record first time we see it
            last = i # Keep updating last time we see it
    return [first, last]
```

JavaScript:

```
function searchRange_linear(nums, target) {
    let first = -1, last = -1;
    for (let i = 0; i < nums.length; i++) {
        if (nums[i] === target) {
            if (first === -1) first = i;
            last = i;
        }
    }
    return [first, last];
}
```

Approach 2: Double Binary Search (Optimized, $O(\log N)$ time) *This is the code an L5/L6 engineer writes.*

Python:

```
class Solution:
    def searchRange(self, nums, target):
        # Helper function to keep our code DRY (Don't Repeat Yourself)
        def findBound(isFirst):
            left, right = 0, len(nums) - 1
            bound = -1

            while left <= right:
                # mid = (left + right) // 2 can cause integer overflow in some languages.
                # left + (right - left) // 2 is the mathematically safer way.
                mid = left + (right - left) // 2

                if nums[mid] == target:
                    bound = mid
                    if isFirst:
                        # If looking for the first occurrence, discard the right half
```

```

        right = mid - 1
    else:
        # If looking for the last occurrence, discard the left half
        left = mid + 1
    elif nums[mid] > target:
        right = mid - 1
    else:
        left = mid + 1

    return bound

first_occurrence = findBound(True)
# Optimization: If the target isn't in the array at all, we don't need a second search
if first_occurrence == -1:
    return [-1, -1]

last_occurrence = findBound(False)

return [first_occurrence, last_occurrence]

```

JavaScript:

```

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var searchRange = function(nums, target) {

    // Helper function to find either the left or right bound
    const findBound = (isFirst) => {
        let left = 0;
        let right = nums.length - 1;
        let bound = -1;

        while (left <= right) {
            // Safe way to calculate mid to avoid overflow
            let mid = left + Math.floor((right - left) / 2);

            if (nums[mid] === target) {
                bound = mid; // Record the index
                if (isFirst) {
                    right = mid - 1; // Keep searching left
                } else {
                    left = mid + 1; // Keep searching right
                }
            } else if (nums[mid] > target) {

```



```

        right = mid - 1;
    } else {
        left = mid + 1;
    }
}
return bound;
};

const firstOccurrence = findBound(true);

// Early exit if the target doesn't exist
if (firstOccurrence === -1) {
    return [-1, -1];
}

const lastOccurrence = findBound(false);

return [firstOccurrence, lastOccurrence];
};

```

Note 1: Terminology Used

- **Binary Search:** A divide-and-conquer algorithm that finds the position of a target value within a sorted array. It helps because it drastically reduces the search time from linear (looking at every item) to logarithmic (halving the data set every step), making it highly scalable for massive datasets.
 - **Lower Bound / Upper Bound:** This is what we call the variation of Binary Search we used.
 - *Lower Bound* finds the first position where the element could be inserted without breaking the sorted order (our `first_occurrence`).
 - *Upper Bound* finds the last position where the element could be inserted (our `last_occurrence`).
 - **DRY Principle (Don't Repeat Yourself):** Notice how we used a helper function `findBound(isFirst)` in the optimized code. Writing two separate `while` loops that do 95% of the same thing is a red flag in senior interviews. A parameterized helper function demonstrates good software design.
-

Note 2: Real-World Interview Variations (Google, Meta, Bloomberg)

At the L5/L6 level, interviewers often disguise this LeetCode problem as a real-world system scenario.

1. Google: Distributed Systems Logging

- **The Prompt:** “You have a massive log file consisting of millions of log entries sorted by timestamp. The log entries contain error codes. Find the exact start and end time of an outage caused by **Error Code 500**.”
- **How to solve:** The log file is the sorted `nums` array. The **Error Code 500** is the `target`. You would apply the exact same lower bound and upper bound binary searches to find the first index of the error and the last index of the error, then return their respective timestamps.

2. Meta: User Activity Timeline

- **The Prompt:** “We track a user’s online status as a stream of sorted timestamped events (e.g., `['offline', 'offline', 'online', 'online', 'online', 'offline']`). Given a massive array of these states, find the duration the user was continuously online during a specific session.”
- **How to solve:** If the array is strictly grouped/sorted by state for a specific time window, you use the algorithm to find the first index of `'online'` and the last index of `'online'`. Subtract the timestamp of the first index from the timestamp of the last index to get the total duration.

3. Bloomberg: Financial Tick Data

- **The Prompt:** “You are given an array of stock prices recorded every millisecond for Apple (AAPL), sorted by time. The price naturally fluctuates, but sometimes plateaus. Given a target price (e.g., `$150.00`), find the start and end indices of the longest contiguous period the stock sat exactly at `$150.00` today.”
- **How to solve:** Since stock prices throughout a whole day go up and down (not entirely sorted), they might give you a strictly monotonically increasing/decreasing segment, or ask you to find local boundaries. If the segment is sorted, it’s the exact same Double Binary Search code.

27. Remove Element

This is a classic “Two Pointers” problem. While the logic seems simple, a top-tier engineer (L5/L6) focuses on **edge cases**, **in-place mutation constraints**, and **minimizing unnecessary writes**.

1. Problem Explanation

The goal is to remove all occurrences of a specific value (**val**) from an array (**nums**) **in-place**.

The Constraints:

- You cannot create a new array.
- The relative order of the elements can change.
- You must return the count of elements that are *not* equal to **val**.
- Only the first **k** elements of the array matter (where **k** is the count you return).

Why is this non-trivial?

In many programming languages, arrays have a fixed size. “Removing” an element usually involves shifting every single element after it to the left to fill the gap. If you do this for every match, you end up with an inefficient $O(n^2)$ solution. An L5 engineer looks for an $O(n)$ approach.

2. Solution Explanation: The “Slow & Fast” Pointer Strategy

We use two pointers:

1. **Fast Pointer (i):** Scans every element in the array from left to right.
2. **Slow Pointer (k):** Keeps track of the “write” position for the next valid element.

Visual Walkthrough

Input: `nums = [3, 2, 2, 3], val = 3`

Step 1: Initial State **k** starts at 0. **i** starts at 0.

[3, 2, 2, 3]

^

k, i (Both at index 0)

- `nums[i]` is 3. This matches our target **val**.
- **Action:** Do nothing. Move **i** forward.

Step 2: Find a valid number

[3, 2, 2, 3]

^

^
k i

- `nums[i]` is 2. This is NOT **val**.
- **Action:** Copy `nums[i]` to `nums[k]`. Then increment both.

```
[ 2, 2, 2, 3 ] (Index 0 updated from 3 to 2)
  ^  ^
  k  i
```

Step 3: Find another valid number

- `nums[i]` is 2. This is NOT `val`.
- **Action:** Copy `nums[i]` to `nums[k]`. Increment both.

```
[ 2, 2, 2, 3 ] (Index 1 updated from 2 to 2 - no visual change)
  ^  ^
  k  i
```

Step 4: Final match

- `nums[i]` is 3. Matches `val`.
- **Action:** Do nothing. Increment `i`.
- `i` reaches the end. We return `k` (which is 2).

3. Time and Space Complexity Analysis

Time Complexity: $O(n)$

We traverse the list exactly once. Each element is visited once by the fast pointer.

Complexity Derivation (Time):

Operation	Frequency	Cost
Pointer Comparison ($i < n$)	n times	$O(1)$
Value Comparison	n times	$O(1)$
Element Assignment	m times*	$O(1)$
Total		$O(n)$

* m is the number of elements not equal to `val`.

Since $m \leq n$, the total is linear.

Space Complexity: $O(1)$

We do not use any extra data structures (like HashMaps or additional Arrays) that scale with input size.

Complexity Derivation (Space):

Variable	Size	Type

k (Slow Pointer)	1 integer	0(1)
i (Fast Pointer)	1 integer	0(1)
----- ----- -----		
Total		0(1)

4. Solution Code

Python Implementation

```
def removeElement(nums, val):
    # k acts as the boundary for the 'valid' part of the array.
    # Everything to the left of k is guaranteed to not be 'val'.
    k = 0

    for i in range(len(nums)):
        # If the current element is not the one we want to remove
        if nums[i] != val:
            # Move the valid element to the front (at index k)
            nums[k] = nums[i]
            # Move the boundary forward
            k += 1

    return k
```

JavaScript Implementation

```
/**
 * @param {number[]} nums
 * @param {number} val
 * @return {number}
 */
var removeElement = function(nums, val) {
    // k tracks the position where the next non-val element should be placed
    let k = 0;

    for (let i = 0; i < nums.length; i++) {
        // When we find a value that is NOT the target
        if (nums[i] !== val) {
            // Overwrite the element at k with the current element
            nums[k] = nums[i];
            // Increment k to prepare for the next valid element
            k++;
        }
    }
}
```

```
    return k;
};
```

Key Terms & Techniques

- **Two Pointers:** A technique where two indices move through a data structure at different speeds or from different directions. It helps avoid nested loops, reducing complexity from $O(n^2)$ to $O(n)$.
 - **In-Place Mutation:** Modifying the input data structure directly without allocating extra space for a copy. This is crucial for memory-constrained environments (common in Bloomberg/Google systems).
-

Real-World Interview Variations

Google, Meta, and Bloomberg rarely ask the “easy” version. They use these variations:

1. **The “Rare Target” Variation (Google/Bloomberg):**
 - *Scenario:* What if the element to remove is very rare (e.g., removing 1 element from a 1-million-item array)?
 - *Optimization:* Instead of shifting everyone, swap the target with the **last** element and decrease the array size. This avoids unnecessary copies.
2. **The “Move Zeroes” Variation (Meta):**
 - *Scenario:* “Move all 0s to the end while maintaining the relative order of non-zero elements.”
 - *Solution:* This is the exact same logic as `removeElement` where `val = 0`. After the loop, you simply fill the remainder of the array (from `k` to `end`) with zeros.
3. **The “Log Filtering” Variation (System Design context):**
 - *Scenario:* You have a massive stream of logs and need to remove “Debug” level logs before sending them to a high-cost storage unit.
 - *Solution:* This uses the same “Filter-in-place” logic to ensure the memory buffer is reused efficiently without triggering heavy Garbage Collection.

26. Remove Duplicates from Sorted Array

An L5 or L6 engineer at Google doesn’t just look for “a” solution; they look for the most memory-efficient, readable, and “production-ready” approach. For

a problem like removing duplicates from a sorted array, the key is recognizing that the **sorted** property is your biggest advantage.

Here is how a top-tier engineer breaks this down.

1. Problem Explanation

The goal is to take an array that is already sorted in non-decreasing order and remove any duplicate elements **in-place**.

The Constraints:

- You cannot create a new array (Space complexity must be constant).
- The relative order of the elements must stay the same.
- You need to return the count of “unique” elements (k).
- The first k elements of the original array must contain these unique values.

The “Aha!” Moment

Since the array is sorted, any duplicates **must** be right next to each other. We don’t need to look across the whole array to find a duplicate; we only need to look at the immediate neighbor.

2. Solution Explanation: The Two-Pointer Strategy

An L6 engineer would immediately identify this as a **Two-Pointer** problem. We use two variables to track different “states” of the array simultaneously.

1. **The “Slow” Pointer (write_index):** This keeps track of where the next unique element should be written.
2. **The “Fast” Pointer (read_index):** This scans through the array to find the next unique value.

Step-by-Step Visualization

Imagine the array: [1, 1, 2, 2, 3]

Initial State: Both pointers start at index 1 (the second element), because the first element is always “unique” by default.

Index:	0	1	2	3	4
Array:	[1,	1,	2,	2,	3]
	$\wedge$	$\wedge$			
	Slow	Fast			

Iteration 1: Is Array[Fast] (1) different from Array[Slow - 1] (1)? **No**. They are the same. We just move **Fast** forward.

```

Index:      0      1      2      3      4
Array:      [ 1,    1,    2,    2,    3 ]
              ^          ^
            Slow        Fast

```

Iteration 2: Is Array[Fast] (2) different from Array[Slow - 1] (1)? **Yes!**
 We found a new unique value. We copy it to Slow and move both pointers.

Action: Array[Slow] = Array[Fast] -> Array[1] = 2

```

Index:      0      1      2      3      4
Array:      [ 1,    2,    2,    2,    3 ]
              ^          ^
            Slow        Fast

```

Iteration 3: Is Array[Fast] (2) different from Array[Slow - 1] (2)? **No.**
 Move Fast.

```

Index:      0      1      2      3      4
Array:      [ 1,    2,    2,    2,    3 ]
              ^          ^
            Slow        Fast

```

Iteration 4: Is Array[Fast] (3) different from Array[Slow - 1] (2)? **Yes!**
 Copy 3 to the Slow position.

Action: Array[Slow] = Array[Fast] -> Array[2] = 3

```

Index:      0      1      2      3      4
Array:      [ 1,    2,    3,    2,    3 ]
              ^          ^ (End)
            Slow        Fast

```

Final Result: The Slow pointer is at index 3. This means there are **3** unique elements. The array prefix is [1, 2, 3].

3. Time and Space Complexity Analysis

Time Complexity (TC)

We iterate through the array exactly once using the Fast pointer. Each element is visited one time.

```

Array size (n) -----> [ x, x, x, x, x ]
Operation per element: Comparison + Write (Constant time)
Total operations: n * 1
TC = O(n)

```


Space Complexity (SC)

We do not allocate any extra arrays or hash maps. We only use two integer variables (Slow and Fast).

Memory usage:

[Integer: Slow] -> 4 bytes

[Integer: Fast] -> 4 bytes

Total extra memory: 8 bytes (Constant)

SC = $O(1)$

4. Solution Code

Javascript Implementation

```
/**
 * Removes duplicates from a sorted array in-place.
 * Uses a two-pointer approach to maintain  $O(1)$  space.
 * * @param {number[]} nums
 * * @return {number}
 */
var removeDuplicates = function(nums) {
    if (nums.length === 0) return 0;

    // The 'write' pointer starts at 1 because the first
    // element is always unique.
    let writeIndex = 1;

    // The 'read' pointer scans the rest of the array.
    for (let readIndex = 1; readIndex < nums.length; readIndex++) {
        // If current element is different from the last unique one found
        if (nums[readIndex] !== nums[readIndex - 1]) {
            nums[writeIndex] = nums[readIndex];
            writeIndex++;
        }
    }

    return writeIndex;
};
```

Python Implementation

```
def remove_duplicates(nums):
    """
    Removes duplicates from a sorted array in-place.
    The function modifies the input list and returns the length
    """
```

```

of the modified list containing only unique elements.
"""
if not nums:
    return 0

# write_index keeps track of where to put the next unique element
write_index = 1

# Loop through the array starting from the second element
for read_index in range(1, len(nums)):
    # If the current element is not equal to the previous element,
    # it's a new unique element.
    if nums[read_index] != nums[read_index - 1]:
        nums[write_index] = nums[read_index]
        write_index += 1

return write_index

```

Note 1: Techniques Used

Two-Pointer Technique: This is a strategy where two pointers move through a data structure at different speeds or in different directions. In this problem, it helps us “collapse” the array by overwriting duplicates as we find new unique values. It is highly efficient because it avoids the $O(n^2)$ cost of deleting elements from the middle of an array (which requires shifting all subsequent elements).

Note 2: Real-World Interview Variations

Google, Meta, and Bloomberg often use this “foundation” to ask more complex system or data processing questions:

1. The “K-Duplicates” Variation (Meta/Google)

- **Question:** “Allow at most 2 occurrences of each element.”
- **Solution:** Instead of comparing `nums[read_index]` to `nums[read_index - 1]`, you compare it to `nums[write_index - 2]`. This ensures the “write” position only accepts a value if it hasn’t already been written twice.

2. Large Scale Log Deduplication (Google/Bloomberg)

- **Context:** “You have a massive stream of sorted timestamps or log IDs. How do you deduplicate them if the data is too large for memory?”

- **Solution:** Use the **Two-Pointer** logic in a **Streaming/External Sort** context. Since the data is sorted, you only need to keep the “Last Seen ID” in memory. If the incoming ID matches the last seen, discard it. This is the core logic behind **LSM Tree** compaction in databases like Bigtable.

3. Merging and Deduplicating (Meta)

- **Question:** “Merge two sorted arrays and remove duplicates.”
- **Solution:** Combine the **Merge Sort** merge-step with the **Two-Pointer** deduplication logic. As you pick the smaller element from the two arrays to add to the result, check if it is equal to the last element added to the result.

80. Remove Duplicates from Sorted Array II

Handling this problem at a Google L5 (Senior) or L6 (Staff) level isn’t just about finding the “correct” answer; it’s about identifying the **optimal pattern** that generalizes to similar problems. A top-tier engineer looks for the most “elegant” invariant—a rule that remains true throughout the entire execution of the algorithm.

1. Problem Explanation

The goal is to take a sorted array and remove “extra” duplicates. Unlike the basic version of this problem where you keep only one of each number, here you are allowed to keep **at most two** occurrences of each number.

You must do this **in-place**. This means you cannot create a new array. You must modify the existing one and return the length of the new “valid” portion of the array.

The “Mental Model”

Imagine you are a filter. Numbers are coming in from a conveyor belt (the array). You have a “Result Zone” at the front of the belt where you place the numbers you want to keep.

- If a number is “new” or only the “second” of its kind, you move it to the Result Zone.
 - If you’ve already seen that number twice, you discard it and move to the next one.
-

2. Solution Explanation

A Senior Engineer uses the **Two-Pointer Technique**. We maintain two markers:

1. **i (The Writer)**: This marks the position where the next “valid” number should be placed.
2. **n (The Reader/Scanner)**: This iterates through every element in the array.

The Invariant (The Secret Sauce)

The logic is simpler than most people think. Since the array is **sorted**, all duplicates are grouped together. To decide if the current number **n** can be kept, we only need to compare it with the element placed **two positions ago** in our “Result Zone.”

If **n** is greater than the element at **i - 2**, it means we haven’t filled our “quota” of two for that specific number yet.

ASCII Visualization of the Process

Example Array: [1, 1, 1, 2, 2, 3]

Step 1: Initialization We can always keep the first two elements because the limit is two. So, **i** starts at index 2.

```
Index:    0    1    2    3    4    5
Array:   [ 1,  1,  1,  2,  2,  3 ]
                ^
                i (Writer starts here)
                ^
                n (Reader starts here)
```

Step 2: Processing index 2 Check: Is `nums[n] (1) > nums[i-2] (1)`? Result: No (1 is not > 1). This is a third duplicate. Action: Increment **n**, but keep **i** where it is.

```
Array:   [ 1,  1,  1,  2,  2,  3 ]
                ^
                i (Waiting...)
                ^
                n (Moving to next)
```

Step 3: Processing index 3 Check: Is `nums[n] (2) > nums[i-2] (1)`? Result: Yes! 2 is greater than 1. Action: Copy `nums[n]` to `nums[i]`, then move both.

```
Array:   [ 1,  1,  2,  2,  2,  3 ]
                ^
                i (Moved)
                ^
```

n (Moving)

Step 4: Processing index 4 Check: Is `nums[n] (2) > nums[i-2] (1)`? Result: Yes! Action: Copy `nums[n]` to `nums[i]`, then move both.

Array: [1, 1, 2, 2, 2, 3]

i (Moved)

n (Moving)

Step 5: Processing index 5 Check: Is `nums[n] (3) > nums[i-2] (2)`? Result: Yes! Action: Copy `nums[n]` to `nums[i]`, then move both.

Array: [1, 1, 2, 2, 3, 3]

i (Final Length = 5)

n (Finished)

3. Time and Space Complexity Analysis

Time Complexity (TC)

We iterate through the array exactly once using the reader pointer `n`. Each element is visited one time, and each comparison/assignment is a constant time operation.

ASCII TC Derivation:

`n` = length of input array

Operation per element: Comparison $O(1)$ + Assignment $O(1)$

Total Operations = `n` * ($O(1)$ + $O(1)$)

Simplified = $O(n)$

Space Complexity (SC)

We do not use any extra data structures (like HashMaps or Sets). We only use two integer variables (`i` and `n`) to track indices regardless of how large the input array grows.

ASCII SC Derivation:

Variable '`i`': Takes constant space $O(1)$

Variable '`n`': Takes constant space $O(1)$

Extra Array: None (In-place modification)

Total Space = $O(1)$

4. Solution Code

Python Implementation

```
def removeDuplicates(nums):  
    # This function uses the Two-Pointer 'Slow-Fast' approach.  
    # The 'i' pointer tracks the position for the next valid element.  
    # The loop variable 'n' scans through every element in the array.  
  
    if len(nums) <= 2:  
        return len(nums)  
  
    i = 2  
    for n in range(2, len(nums)):  
        # If current element is different from the element  
        # placed two positions back, it's valid to keep.  
        if nums[n] > nums[i-2]:  
            nums[i] = nums[n]  
            i += 1  
  
    return i
```

JavaScript Implementation

```
/**  
 * Removes duplicates in-place such that each unique element appears at most twice.  
 * Uses a single-pass scan to maintain  $O(n)$  time and  $O(1)$  space.  
 * @param {number[]} nums  
 * @return {number}  
 */  
var removeDuplicates = function(nums) {  
    if (nums.length <= 2) {  
        return nums.length;  
    }  
  
    let i = 2;  
    for (let n = 2; n < nums.length; n++) {  
        // Since the array is sorted, comparing nums[n] with nums[i-2]  
        // ensures that we don't have more than two of the same value  
        // in our 'valid' prefix.  
        if (nums[n] > nums[i - 2]) {  
            nums[i] = nums[n];  
            i++;  
        }  
    }  
  
    return i;  
}
```

```
};
```

Terminology: The “Slow-Fast Pointer” Pattern

This is a variation of the **Two-Pointer technique**.

- **Why it helps:** It allows you to transform or filter data in a single pass without allocating new memory.
 - **Application:** In this problem, the “Fast” pointer explores the data, while the “Slow” pointer builds the result. By comparing the “Fast” pointer’s value to a relative position of the “Slow” pointer ($i-2$), we enforce a frequency constraint dynamically.
-

Real-World Interview Variations

1. The “Streaming Logs” Problem (Google/Bloomberg)

Scenario: You are receiving a stream of log IDs that are roughly sorted by timestamp. Because of retries, some logs appear multiple times. You need to store them in a buffer but limit any specific Log ID to 3 occurrences to prevent “log spamming” from a single failing service.

- **Solution:** Use the same logic, but change the check to `nums[n] > nums[i-3]`. In a real-world scenario, if the data isn’t perfectly sorted, you would use a “Sliding Window” with a `HashMap`, but if the window is small and sorted, the Two-Pointer approach is significantly faster.

2. The “In-Place Compressed Array” (Meta)

Scenario: Given an array of sorted user IDs, modify the array in-place so that if a user appears more than K times, the extras are removed.

- **Solution:** This is the **Generalized Version** of Leetcode 80. An L6 engineer would write a generic function where K is a parameter. The core logic remains: `if nums[n] > nums[i-K]: nums[i] = nums[n]`.

3. “Compact Storage for Sorted Sensors” (Bloomberg)

Scenario: Sensor data is stored in a fixed-size memory block. To save space, we only want to keep the first two readings of any specific temperature value if they occur consecutively.

- **Solution:** This is identical to the Leetcode problem but emphasizes **memory constraints**. In an interview, you would highlight that this approach is “Cache Friendly” because it accesses memory linearly, which is a key concern for high-performance systems at Bloomberg.

169. Majority Element

To solve “Majority Element” like a top-tier L5 or L6 engineer at Google, one must look beyond the basic “sorting” or “hash map” solutions. A senior engineer identifies the most efficient algorithm that minimizes space complexity to constant levels while maintaining linear time.

1. Problem Explanation

The problem asks us to find the **Majority Element** in an array of size n .

- **Definition:** The majority element is the element that appears **more than $n / 2$** times.
- **Guarantee:** The problem states that the majority element *always* exists in the array.

Visualizing the “More than Half” Rule

Imagine an array of 7 elements. For an element to be the majority, it must appear at least 4 times (since $7 / 2 = 3.5$, and we need more than that).

```
[ A, A, B, A, C, A, B ]  <-- Total 7 elements
  ^   ^   ^   ^         <-- "A" appears 4 times.
                        4 is > 3.5, so "A" is the majority.
```

If we “cancel out” every non-majority element with one instance of the majority element, the majority element will still have at least one survivor left over. This is the core intuition for the optimal solution.

2. Solution Explanation: Boyer-Moore Voting Algorithm

While a Hash Map solution is intuitive ($O(n)$ time, $O(n)$ space), a Staff-level engineer would provide the **Boyer-Moore Voting Algorithm**. It achieves $O(n)$ time but uses only $O(1)$ space.

The Logic

Think of this as a “Last Man Standing” competition.

1. We maintain a **candidate** and a **count**.
2. We walk through the array.
3. If **count** is 0, we pick the current element as our new **candidate**.
4. If the current element is the same as the **candidate**, we increment **count**.
5. If it’s different, we decrement **count** (they “cancel” each other out).

Since the majority element appears more than half the time, it will always survive the “cancellation” process.

Step-by-Step Visualization

Input Array: [2, 2, 1, 1, 1, 2, 2]

Step 1: Element = 2
Count was 0, so Candidate = 2, Count = 1
[(2), 2, 1, 1, 1, 2, 2]

Step 2: Element = 2
Same as Candidate! Count = 2
[2, (2), 1, 1, 1, 2, 2]

Step 3: Element = 1
Different! "1" battles "2". Count = 1
[2, 2, (1), 1, 1, 2, 2]

Step 4: Element = 1
Different! "1" battles "2". Count = 0
[2, 2, 1, (1), 1, 2, 2]

Step 5: Element = 1
Count was 0! New Candidate = 1, Count = 1
[2, 2, 1, 1, (1), 2, 2]

Step 6: Element = 2
Different! "2" battles "1". Count = 0
[2, 2, 1, 1, 1, (2), 2]

Step 7: Element = 2
Count was 0! New Candidate = 2, Count = 1
[2, 2, 1, 1, 1, 2, (2)]

Final Result: Candidate is 2.

3. Time and Space Complexity Analysis

Time Complexity (TC)

We iterate through the list exactly once. Each operation inside the loop (comparison and addition) takes constant time.

TC Derivation:

Iteration 1 + Iteration 2 + ... + Iteration N
= 1 + 1 + ... + 1 (N times)
= N
Result: $O(N)$

Space Complexity (SC)

We only store two variables: `candidate` and `count`. Regardless of how large the input array grows, we do not allocate more memory.

SC Derivation:

Variable 'candidate' : 4 bytes (int)

Variable 'count' : 4 bytes (int)

Total : 8 bytes

8 bytes = Constant

Result: $O(1)$

4. Solution Code

Solution 1: Boyer-Moore Voting Algorithm (Optimized)

Javascript

```
/**
 * Uses the Boyer-Moore Voting Algorithm to find the majority element.
 * This works by maintaining a candidate and a counter that increments
 * on matches and decrements on mismatches.
 */
var majorityElement = function(nums) {
    let candidate = null;
    let count = 0;

    for (let num of nums) {
        // If count hits zero, we start fresh with the current number
        if (count === 0) {
            candidate = num;
        }

        // Add to count if it matches candidate, else subtract
        count += (num === candidate) ? 1 : -1;
    }

    return candidate;
};
```

Python

```
class Solution:
    def majority_element(self, nums: list[int]) -> int:
        # candidate stores the potential majority element
        candidate = None
        # count tracks the "strength" of the current candidate
```

```

count = 0

for num in nums:
    # When count is 0, the previous candidate was 'cancelled out'
    if count == 0:
        candidate = num

    # Increment if we see the candidate again, decrement otherwise
    count += 1 if num == candidate else -1

return candidate

```

Solution 2: Hash Map (Good for Interview Discussion)

Javascript

```

/**
 * Uses a Hash Map to track frequencies.
 * Better for general cases where a majority might not exist.
 */
var majorityElementHash = function(nums) {
    let counts = {};
    let threshold = nums.length / 2;

    for (let num of nums) {
        counts[num] = (counts[num] || 0) + 1;
        if (counts[num] > threshold) {
            return num;
        }
    }
};

```

Python

```

class Solution:
    def majority_element_hash(self, nums: list[int]) -> int:
        counts = {}
        threshold = len(nums) // 2

        for num in nums:
            # Map the frequency of each number
            counts[num] = counts.get(num, 0) + 1
            # Check immediately if we passed the half-way mark
            if counts[num] > threshold:
                return num

```

Terminology: Boyer-Moore Voting Algorithm

- **What it is:** An algorithm for finding the majority of a sequence of elements using linear time and constant space.
 - **Why it helps:** It avoids the $O(n)$ memory overhead of a hash map or the $O(n \log n)$ time overhead of sorting.
 - **Application:** In this problem, because the majority is guaranteed to occupy $> 50\%$ of the slots, the algorithm effectively “filters” out the noise of minority elements.
-

Real-World / Interview Variations

1. The “Majority Element II” (Google/Meta)

- **Question:** Find all elements that appear more than $n / 3$ times.
- **L5/L6 Solution:** Use a generalized Boyer-Moore. Keep **two** candidates and two counts. If an element doesn’t match either, decrement both counts. This is often asked to see if you can scale an algorithm’s logic.

2. Distributed Log Processing (Bloomberg)

- **Question:** You have a massive stream of IP addresses hitting a server. How do you find the “heavy hitter” (an IP causing more than 50% of traffic) without storing millions of IPs in memory?
- **L5/L6 Solution:** This is a direct application of Boyer-Moore. Since we only need $O(1)$ space, we can process a stream of trillions of logs with just two variables in memory.

3. Fault-Tolerant Voting (Meta)

- **Question:** In a distributed system, multiple nodes vote on a value. If more than half must agree for a consensus, how do you find the winner efficiently?
- **L5/L6 Solution:** Use the voting algorithm. If the nodes are on different machines, you can perform local Boyer-Moore counts and then aggregate the candidates for a final verification pass.

392. Is Subsequence

1. Problem Explanation

The problem asks us to determine if a string **s** is a **subsequence** of another string **t**.

A **subsequence** is different from a **substring**.

- A **substring** must be contiguous (the characters must be right next to each other in the original order).
- A **subsequence** is formed by deleting zero or more characters from the original string without disturbing the relative positions of the remaining characters.

Visualizing the Difference

Original String: “abcde”

Is “ace” a subsequence? Yes. You can get “ace” by deleting ‘b’ and ‘d’. The order (a -> c -> e) is preserved.

Is “aec” a subsequence? No. While ‘a’, ‘e’, and ‘c’ all exist in the original string, the order in the original is a -> c -> e. To get “aec”, you would have to swap characters, which isn’t allowed.

2. Solution Explanation

A senior-level (L5/L6) approach focuses on two scenarios: the **Standard Interview Case** (one-off check) and the **Follow-up/Scale Case** (checking many **s** strings against one massive **t**).

Approach A: The Two-Pointer Technique (Standard)

This is the most efficient way to solve the problem for a single pair of strings. We use two markers (pointers) to scan both strings from left to right.

1. Place pointer **i** at the start of **s**.
2. Place pointer **j** at the start of **t**.
3. If **s[i]** matches **t[j]**, it means we found one character of our subsequence. Move **i** forward to look for the next character.
4. Always move **j** forward because we can only use each character in **t** once and we must move from left to right.
5. If **i** reaches the end of **s**, we found all characters in the correct order. Return **True**.

ASCII Visualization of Two-Pointer

s = "abc", **t** = "ahbgdc"

Step 1:

s: [a] b c
 ^ i=0

t: [a] h b g d c
 ^ j=0

Match! Move both **i** and **j**.

Step 2:
s: a [b] c
 [^] i=1
t: a [h] b g d c
 [^] j=1
No match. Move only j.

Step 3:
s: a [b] c
 [^] i=1
t: a h [b] g d c
 [^] j=2
Match! Move both i and j.

Step 4:
s: a b [c]
 [^] i=2
t: a h b [g] d c
 [^] j=3
No match. Move only j. (Repeat for 'd')

Step 5:
s: a b [c]
 [^] i=2
t: a h b g d [c]
 [^] j=5
Match! i reaches the end (index 3). Return True.

Approach B: Preprocessing with Hash Map (The “Google Scale” Solution)

In a real interview, an L5/L6 engineer will ask: “*What if we have 1 billion strings (s1, s2, ...) and we need to check them against the same t?*” The two-pointer approach would be $O(\text{number_of_strings} * \text{length_of_t})$. To optimize, we preprocess *t* into a dictionary/hash map where keys are characters and values are lists of indices where that character appears.

Example: *t* = "ahbgdc" Map: { 'a': [0], 'h': [1], 'b': [2], 'g': [3], 'd': [4], 'c': [5] }

For each *s*, we use **Binary Search** to find the smallest index in *t* that is greater than the index of the previously matched character.

3. Time and Space Complexity Analysis

For the standard two-pointer approach:

Time Complexity (TC)

Let N = length of string s
Let M = length of string t

Derivation:

We iterate through t at most once.

In each step, we do a constant time comparison.

Max operations = M

TC = $O(M)$

Space Complexity (SC)

Derivation:

We only store two integer pointers (i and j).

No extra data structures are created relative to input size.

SC = $O(1)$

4. Solution Code

Implementation 1: Two-Pointer (Optimal for single check)

Python

```
def isSubsequence(s, t):  
    # If s is empty, it's always a subsequence of any string  
    if not s:  
        return True  
  
    i = 0 # Pointer for s  
    j = 0 # Pointer for t  
  
    while j < len(t):  
        # If characters match, move the s pointer to the next character  
        if s[i] == t[j]:  
            i += 1  
  
        # If i reaches the end of s, we found all characters  
        if i == len(s):  
            return True  
  
        # Always move the t pointer forward
```

```
    j += 1

    return False
```

JavaScript

```
/**
 * Checks if s is a subsequence of t using two pointers.
 */
var isSubsequence = function(s, t) {
    if (s.length === 0) return true;

    let i = 0; // pointer for s
    let j = 0; // pointer for t

    while (j < t.length) {
        // When characters match, increment i to look for the next s char
        if (s[i] === t[j]) {
            i++;
        }

        // Check if we've completed the subsequence
        if (i === s.length) {
            return true;
        }

        // Move j forward every iteration to scan through t
        j++;
    }

    return false;
};
```

Algorithm Terms Used

- **Two-Pointer Technique:** A strategy where two indices move through data structures (often at different speeds or starting points). It helps avoid nested loops, reducing complexity from $O(N^2)$ to $O(N)$.
 - **Preprocessing:** Organizing data beforehand (like the Hash Map + Binary Search approach) to make future queries much faster.
-

Real World / Interview Variations

1. The “Big Data” Variation (Google/Meta)

The Problem: You have a dictionary of 1 million words. Given a long string t , which words in the dictionary are subsequences of t ? **L5/L6 Solution:** Instead of checking each word one by one, use a **Waitlist (Bucket) Approach**.

1. Create 26 buckets (one for each letter ‘a’-‘z’).
2. Store each word from the dictionary in the bucket corresponding to its *first* letter.
3. Iterate through string t once. For each character $char$ in t :
 - Take all words out of `bucket[char]`.
 - For each word, move it to the bucket corresponding to its *next* letter.
 - If a word has no more letters, it is a subsequence.

2. Minimum Window Subsequence (Bloomberg)

The Problem: Find the smallest substring of t that contains s as a subsequence. **L5/L6 Solution:** Use Dynamic Programming or a Two-Pointer “Slide and Shrink” method. After finding a match using the standard two-pointer method, work backward from the end of the match to find the start that gives the shortest possible length.

3. Log Analysis (Real World)

The Problem: Determine if a specific sequence of user actions (e.g., Login -> Add to Cart -> Checkout) occurred within a massive log file of all user events. **L5/L6 Solution:** This is exactly “Is Subsequence.” In a distributed system, you would use the Preprocessing approach. Since logs are indexed by timestamp, you can use the timestamps as the “indices” in your Binary Search to quickly find if “Checkout” happened *after* “Add to Cart.”

162. Find Peak Element

1. Problem Explanation

The problem asks us to find a **peak element** in an array. A peak element is an element that is strictly greater than its neighbors.

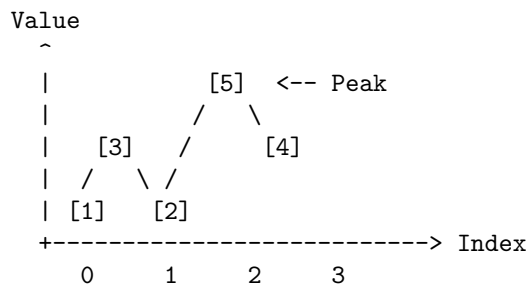
Key Rules:

- **Neighbors:** For an element at index i , the neighbors are at $i-1$ and $i+1$.
- **Boundaries:** The problem defines that `nums[-1]` and `nums[n]` (the elements outside the array) are equal to negative infinity. This is crucial because it means:

- If the first element is greater than the second, it's a peak.
- If the last element is greater than the second-to-last, it's a peak.
- **Strictly Greater:** $\text{nums}[i] > \text{nums}[i-1]$ and $\text{nums}[i] > \text{nums}[i+1]$.
- **Guarantee:** The array will always contain at least one peak.
- **Constraint:** You must write an algorithm that runs in $O(\log n)$ time.

Visualization of Peaks:

Imagine the array as a mountain range. A peak is any top point.



In the array $[1, 3, 2, 5, 4]$, both 3 (index 1) and 5 (index 3) are peaks. You only need to return **any one** of their indices.

2. Solution Explanation

A Senior (L5) or Staff (L6) engineer at Google would immediately recognize that an $O(\log n)$ constraint on an unsorted array almost always points to **Binary Search**.

While Binary Search is usually for sorted arrays, we can use it here because we are looking for a local property (a peak) rather than a specific value.

The “Slope” Logic

At any index mid , there are two possibilities:

1. **Descending Slope:** $\text{nums}[\text{mid}] > \text{nums}[\text{mid} + 1]$
 - This means a peak exists to the **left** (including mid itself). Why? Because even if the numbers keep decreasing to the left, the boundary is negative infinity, so the first element would be a peak.
2. **Ascending Slope:** $\text{nums}[\text{mid}] < \text{nums}[\text{mid} + 1]$
 - This means a peak exists to the **right**. Why? Because even if the numbers keep increasing to the right, the last element would be a peak.

Visualizing the Binary Search Decision:

Scenario A: Ascending Slope

```
Indices:  [ i ]   [mid]   [mid+1]   [ j ]
Values:    1       3       5         4
           ^       ^
           (mid) < (mid+1)
           GO RIGHT ->
```

Scenario B: Descending Slope

```
Indices:  [ i ]   [mid]   [mid+1]   [ j ]
Values:    1       6       4         2
           ^       ^
           (mid) > (mid+1)
           <- GO LEFT (including mid)
```

The Algorithm Step-by-Step:

1. Initialize `left = 0` and `right = nums.length - 1`.
 2. While `left < right`:
 - Find `mid`.
 - Compare `nums[mid]` with `nums[mid + 1]`.
 - If `nums[mid] < nums[mid + 1]`, we are on an upward slope. The peak must be on the right. Set `left = mid + 1`.
 - Else (if `nums[mid] > nums[mid + 1]`), we are on a downward slope. The peak could be `mid` or something to the left. Set `right = mid`.
 3. When `left == right`, you have found a peak index.
-

3. Time and Space Complexity Analysis

Time Complexity: $O(\log n)$

We divide the search space in half during every iteration.

```
Iteration 1: n elements
Iteration 2: n / 2 elements
Iteration 3: n / 4 elements
...
Iteration k: n / (2^k) elements
```

```
We stop when n / (2^k) = 1
n = 2^k
log2(n) = k
```

Total Steps = $\log_2(n)$
Time Complexity = $O(\log n)$

Space Complexity: $O(1)$

We only use a few variables (`left`, `right`, `mid`) regardless of the input size.

Memory Usage:

[`left`] -> 8 bytes

[`right`] -> 8 bytes

[`mid`] -> 8 bytes

Total : Constant Space $O(1)$

4. Solution Code

Optimized Solution (Binary Search)

Python Implementation

```
def findPeakElement(nums):
    # We use a standard binary search approach to find the peak in O(log n)
    left = 0
    right = len(nums) - 1

    while left < right:
        # Calculate mid. (left + right) // 2 is fine for Python as it handles large ints
        mid = (left + right) // 2

        # Check if we are on an ascending slope
        if nums[mid] < nums[mid + 1]:
            # Peak must be to the right, and mid cannot be the peak
            left = mid + 1
        else:
            # We are on a descending slope.
            # Mid could be the peak, or the peak is to the left.
            right = mid

    # When left == right, we've converged on a peak element index
    return left
```

JavaScript Implementation

```
/**
 * Finds a peak element in an array where nums[i] != nums[i+1].
 * Uses Binary Search to achieve O(log n) time complexity.
```

```

*/
var findPeakElement = function(nums) {
    let left = 0;
    let right = nums.length - 1;

    while (left < right) {
        // Using Math.floor for integer division
        let mid = Math.floor((left + right) / 2);

        // Compare mid element with its right neighbor to determine the slope
        if (nums[mid] < nums[mid + 1]) {
            // Ascending: peak is definitely to the right
            left = mid + 1;
        } else {
            // Descending: peak is mid or to the left
            right = mid;
        }
    }

    // Left and right converge to the peak index
    return left;
};

```

Note 1: Terms and Techniques

- **Binary Search on Answer Space:** Usually, binary search is used to find a value in a sorted array. Here, we use it on an unsorted array to find a specific property (a peak). We treat the array as a series of gradients (slopes).
 - **Local Maximum:** A peak is a local maximum. This algorithm works because if you are climbing “up,” you are guaranteed to either hit a peak or the end of the array (which is treated as a peak due to the negative infinity boundary).
-

Note 2: Real-World Interview Variations

1. **Google (Mountain Array Search):** * *Question:* You are given a “Mountain Array” (increases then decreases). Find a specific target value.
- *L5/L6 Solution:* Use `findPeakElement` logic first to find the “peak” (the top of the mountain) in $O(\log n)$. Then, perform two separate binary searches: one on the increasing side and one on the decreasing side. Total time: $O(\log n)$.

2. **Meta (Find Peak in 2D Matrix):** * *Question:* Find a peak element in a 2D grid where a peak is greater than Top, Bottom, Left, and Right neighbors.
 - *L5/L6 Solution:* Use a modified binary search. Select a middle column, find the maximum in that column ($O(\text{rows})$), then check its left/right neighbors to decide which half of the matrix to search next. Time: $O(\text{rows} * \log(\text{cols}))$.
3. **Bloomberg (Bitonic Array Rotation):** * *Question:* An array was a mountain array but then it was rotated. Find the peak.
 - *L5/L6 Solution:* This is trickier. You first identify the rotation point (the drop-off) using binary search, which effectively gives you the “mountain” structure back, then apply the peak-finding logic.

108. Convert Sorted Array to Binary Search Tree

For a Senior (L5) or Staff (L6) engineer at Google, the goal isn’t just to “solve” the problem, but to provide a solution that is robust, handles edge cases, and demonstrates a deep understanding of tree properties and recursion.

1. Problem Explanation

The task is to take a **sorted** array (ascending order) and convert it into a **height-balanced** Binary Search Tree (BST).

What is a Height-Balanced BST?

1. **BST Property:** For every node, all elements in the left subtree are smaller, and all elements in the right subtree are larger.
2. **Height-Balanced Property:** For every single node in the tree, the depth of the left and right subtrees must not differ by more than 1.

The Core Intuition

Since the array is already sorted, the middle element is the most logical choice for the **root**. Why? Because it perfectly splits the remaining elements into two equal (or near-equal) halves, ensuring the “balanced” requirement is met.

2. Solution Explanation

We use a **Divide and Conquer** approach (specifically, a recursive method).

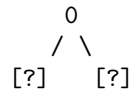
The Logic

1. Find the **middle** index of the current array segment.
2. Create a new **TreeNode** using the value at that middle index.
3. **Recursively** do the same for the left half of the array to build the **left** child.
4. **Recursively** do the same for the right half of the array to build the **right** child.

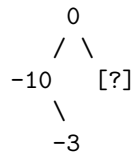
Step-by-Step Visualization

Input Array: [-10, -3, 0, 5, 9]

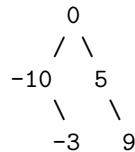
Step 1: Find Middle of [0, 4] Middle index = 2 (Value: 0). This is our Root.
Left segment: [-10, -3] | Right segment: [5, 9]



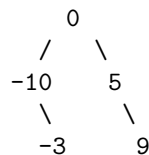
Step 2: Process Left Half [-10, -3] Middle index = 0 (Value: -10). Left segment: [] | Right segment: [-3]



Step 3: Process Right Half [5, 9] Middle index = 3 (Value: 5). Left segment: [] | Right segment: [9]



Final Resulting Tree:



(Note: Depending on how you pick the middle—rounding up or down—the tree might look slightly different, but it will still be balanced.)

3. Complexity Analysis

Time Complexity (TC)

We visit every element in the array exactly once to create a node.

Derivation:

Each recursive call does $O(1)$ work (finding mid, creating node).

Total calls = Number of elements (n).

Total Time = $O(n)$

Space Complexity (SC)

The space is determined by the recursion stack. Since we always split the array in half, the tree is balanced, meaning the height is logarithmic.

Derivation:

The recursion stack depth is equal to the height of the tree.

For a balanced tree with n nodes:

Height = $\log_2(n)$

Total Space = $O(\log n)$

4. Solution Code

Implementation Strategy

A top-tier engineer avoids creating “slices” of arrays in every recursive call (like `arr[:mid]`) because slicing creates new copies of the data, turning an $O(n)$ space complexity into something much worse. Instead, we pass **pointers** (indices).

Python

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def sortedArrayToBST(self, nums):
        # We use a helper function to avoid slicing the array,
        # which saves memory by passing only the index boundaries.
        def build_tree(left_idx, right_idx):
            # Base case: if the left boundary exceeds the right,
            # there are no elements to process.
```



```

    if left_idx > right_idx:
        return None

    # Choose the middle element to ensure height balance.
    # (left + right) // 2 is standard.
    mid = (left_idx + right_idx) // 2

    # Create the current node
    node = TreeNode(nums[mid])

    # Recursively build the left and right subtrees
    node.left = build_tree(left_idx, mid - 1)
    node.right = build_tree(mid + 1, right_idx)

    return node

return build_tree(0, len(nums) - 1)

```

JavaScript

```

function TreeNode(val, left, right) {
    this.val = (val===undefined ? 0 : val)
    this.left = (left===undefined ? null : left)
    this.right = (right===undefined ? null : right)
}

var sortedArrayToBST = function(nums) {
    /**
     * Helper function to build the tree using index boundaries.
     * This ensures we maintain O(log n) space complexity.
     */
    const buildTree = (left, right) => {
        // Base case: segment is empty
        if (left > right) return null;

        // Find the middle point.
        // Using Math.floor to handle integer division in JS.
        let mid = Math.floor((left + right) / 2);

        // Create the node with the middle element
        let node = new TreeNode(nums[mid]);

        // Build subtrees by splitting the range
        node.left = buildTree(left, mid - 1);
        node.right = buildTree(mid + 1, right);
    }
}

```

```

        return node;
    };

    return buildTree(0, nums.length - 1);
};

```

Terminology & Techniques

- **Divide and Conquer:** An algorithmic paradigm where you break a problem into smaller sub-problems, solve them, and combine them. Here, we divide the array into halves and “conquer” by making them subtrees.
 - **Base Case:** The condition that stops recursion. In this problem, it’s when our “left” pointer passes our “right” pointer, meaning we have run out of numbers to add to that branch.
-

Real-World Interview Variations

Google and Meta rarely ask the “easy” version anymore. They use variations that test your understanding of data structures:

1. Convert Sorted Linked List to BST (Google/Meta)

- **The Twist:** You don’t have random access (you can’t just jump to `nums[mid]`).
- **L5/L6 Solution:** Use a “Bottom-Up” approach. You keep a global pointer to the head of the list. You simulate an in-order traversal: build the left subtree, create the root from the current list node, move the list pointer, then build the right subtree. This maintains $O(n)$ time.

2. Flatten a BST to a Sorted List (Bloomberg)

- **The Twist:** The reverse of this problem.
- **L5/L6 Solution:** Use a modified Morris Traversal or a standard in-order traversal with a “prev” pointer to re-link the nodes in place without using extra space for a new list.

3. Balanced BST from a “Stream” of Data (Google)

- **The Twist:** The data isn’t all there at once; it’s coming in one by one.
- **L5/L6 Solution:** If you need to keep it balanced at all times, you would implement an AVL Tree or a Red-Black Tree. If you only need it balanced at the end, you collect the data, sort it (if not sorted), and then use the `sortedArrayToBST` logic.

36. Valid Sudoku

Solving a Sudoku board efficiently is less about complex algorithms and more about **optimized data representation** and **bitmasking**. A top-tier L5 or L6 engineer at Google looks for “clean code” that balances readability with maximum performance, often avoiding redundant passes over the data.

1. Problem Explanation

The goal is to determine if a 9x9 Sudoku board is “valid.” A valid board does not necessarily mean it is solvable; it just means the current numbers do not break Sudoku rules.

The Rules:

1. Each row must contain digits 1-9 without repetition.
2. Each column must contain digits 1-9 without repetition.
3. Each of the nine 3x3 sub-boxes must contain digits 1-9 without repetition.

Visualizing the 3x3 Sub-boxes

The trickiest part for many is mapping a coordinate (row, col) to one of the nine 3x3 boxes. We can index these boxes from 0 to 8:

	COL	0	1	2		3	4	5		6	7	8
ROW												
0												
1		Box 0				Box 1				Box 2		
2												
3												
4		Box 3				Box 4				Box 5		
5												
6												
7		Box 6				Box 7				Box 8		
8												

The Formula: Box Index = (row / 3) * 3 + (col / 3) *If row=4, col=5:*
(4/3)*3 + (5/3) = 1*3 + 1 = Index 4.

2. Solution Explanation

A Senior/Staff Engineer would likely use **Bitmasking** or **Hash Sets**. While Hash Sets are readable, Bitmasking is faster and uses less memory, which shows deep system-level understanding.

The Strategy: Single Pass

Instead of checking rows, then columns, then boxes (3 passes), we do it in **one single pass** through the 81 squares.

1. Create 3 arrays of integers, each of size 9 (one for rows, one for columns, one for boxes).
2. Each integer acts as a “bitset” where the Nth bit represents the number N.
3. For every cell (r, c) containing digit D:
 - Check if the D-th bit is already “on” in rows[r], cols[c], or boxes[box_idx].
 - If “on”, it’s a duplicate -> Return **False**.
 - If “off”, turn it “on” and move to the next cell.

ASCII Visualization of Bitmasking

Imagine we are checking a row and find the number **3**. The bitmask starts as all zeros: 000000000 (representing digits 987654321). To “mark” 3, we shift 1 left by (3-1) positions: $1 \ll 2 = 000000100$.

```
Row Mask (Initial): 0 0 0 0 0 0 0 0 0 (Decimal: 0)
Found '3':          0 0 0 0 0 0 1 0 0 (OR operation)
Row Mask (Updated): 0 0 0 0 0 0 1 0 0 (Decimal: 4)
```

Later, found another '3' in the same row:

```
Row Mask:           0 0 0 0 0 0 1 0 0
New '3' bit:        0 0 0 0 0 0 1 0 0
AND operation:      0 0 0 0 0 0 1 0 0 (Result > 0! Duplicate detected!)
```

3. Complexity Analysis

Time Complexity (TC)

We visit each cell of the 9x9 board exactly once.

Total Cells = 9 rows * 9 columns = 81 cells

Operations per cell:

- Calculate box index (O(1))
- Bitwise AND check (O(1))
- Bitwise OR update (O(1))

TC = $O(81)$ which simplifies to $O(1)$ because the board size is fixed.

Space Complexity (SC)

We store the “seen” numbers in three arrays of size 9.

Storage:

- rows_bits: 9 integers (each 32-bit)
- cols_bits: 9 integers (each 32-bit)
- boxes_bits: 9 integers (each 32-bit)

Total Space = 27 integers.

SC = $O(27)$ which simplifies to $O(1)$ because it does not scale with input.

4. Solution Code

Implementation: The Bitmask Approach (Optimized)

Python

```
class Solution:
```

```
    def isValidSudoku(self, board: list[list[str]]) -> bool:
```

```
        # We use 9 integers to store the bits for rows, cols, and 3x3 boxes
```

```
        rows = [0] * 9
```

```
        cols = [0] * 9
```

```
        boxes = [0] * 9
```

```
        for r in range(9):
```

```
            for c in range(9):
```

```
                val = board[r][c]
```

```
                if val == ".":
```

```
                    continue
```

```
                # Convert char "5" to integer index 4 (0-based)
```

```
                # This represents the (val-1)-th bit
```

```
                digit_bit = 1 << (int(val) - 1)
```

```
                box_idx = (r // 3) * 3 + (c // 3)
```

```
                # check_validity: uses bitwise AND to see if bit is already set
```

```
                if (rows[r] & digit_bit) or (cols[c] & digit_bit) or (boxes[box_idx] & digit_bit):
```

```
                    return False
```

```
                # record_presence: uses bitwise OR to set the bit to 1
```

```
                rows[r] |= digit_bit
```

```

        cols[c] |= digit_bit
        boxes[box_idx] |= digit_bit

    return True

```

JavaScript

```

/**
 * @param {character[][]} board
 * @return {boolean}
 */
var isValidSudoku = function(board) {
    const rows = new Int32Array(9);
    const cols = new Int32Array(9);
    const boxes = new Int32Array(9);

    for (let r = 0; r < 9; r++) {
        for (let c = 0; c < 9; c++) {
            const val = board[r][c];
            if (val === ".") continue;

            const digitBit = 1 << (parseInt(val) - 1);
            const boxIdx = Math.floor(r / 3) * 3 + Math.floor(c / 3);

            // check_if_seen: If the result of bitwise AND is non-zero,
            // it means this digit has appeared before in this row/col/box.
            if ((rows[r] & digitBit) || (cols[c] & digitBit) || (boxes[boxIdx] & digitBit))
                return false;
        }

        // mark_as_seen: Update the bitmask using OR.
        rows[r] |= digitBit;
        cols[c] |= digitBit;
        boxes[boxIdx] |= digitBit;
    }
    return true;
};

```

Technical Terms & Real World Applications

New Term: Bitmasking

What it is: Using the individual bits of an integer to store boolean (True/False) flags. **Why it helps:** It is incredibly memory efficient and performs “checks”

at the CPU instruction level, making it faster than looking up items in a Hash Set or List.

Real World / Interview Variations

1. The “Large Scale” Sudoku (Google/Meta)

- **Prompt:** “What if the board was 1,000,000 x 1,000,000 and sparse (mostly empty)?”
- **L6 Solution:** You can’t use a bitmask or a fixed array. You would use a **Map of Sets** or a **Coordinate List (COO)** format. You’d process only the non-empty cells and use a Distributed Hash Table (like BigTable) if the data is too large for one machine.

2. The “Cloud Logging” Validator (Bloomberg)

- **Prompt:** “Validate that in a stream of 1 billion logs, no ‘User ID’ appears twice within the same ‘Session’ and ‘Region’.”
- **L6 Solution:** This is just Valid Sudoku at scale. “Row” = Session, “Column” = Region, “Digit” = User ID. You would solve this using **Bloom Filters** (to save space) or a **Redis Bitset** to check for duplicates in real-time.

3. The “Conflict Checker” (Meta)

- **Prompt:** “Given a UI layout grid, ensure no two components overlap in the same grid box.”
- **L6 Solution:** Use a 2D segment tree or a spatial hash map to detect intersections in $O(\log N)$ time instead of checking every pair.

151. Reverse Words in a String

Solving a problem like “Reverse Words in a String” at an L5 (Senior) or L6 (Staff) level at Google isn’t just about getting the code to pass; it’s about identifying the most memory-efficient approach, handling edge cases gracefully (like multiple spaces), and demonstrating a deep understanding of how strings are handled in memory.

1. Problem Explanation

The goal is to take a string of words and reverse the order of those words. However, the input string may have leading spaces, trailing spaces, or multiple spaces between words. The output must be a single string where words are separated by exactly one space, with no extra spaces at the beginning or end.

The “Trick” to an L5/L6 Solution: In many languages (like Python or Java), strings are immutable. This means any “in-place” trick is technically a simu-

lation. However, a top-tier engineer will explain the **In-place Two-Pointer** approach, which is the gold standard for C++ or systems-level interviews because it achieves $O(1)$ extra space.

Visualization of the Input/Output

```
Input: " hello world "
Step 1: Trim and Clean (Remove extra spaces) -> "hello world"
Step 2: Reverse the entire string -> "dlrow olleh"
Step 3: Reverse each individual word -> "world hello"
Output: "world hello"
```

2. Solution Explanation

To solve this optimally, we use a three-step strategy. This is more “clever” than just using a built-in `split().reverse().join()` because it demonstrates how to manipulate data at a low level.

Step 1: Clean the Spaces

We use a two-pointer approach to shift characters to the left, skipping unnecessary spaces.

```
Initial: [ _ ][ _ ][ h ][ e ][ l ][ l ][ o ][ _ ][ _ ][ _ ][ w ][ o ][ r ][ l ][ d ][ _ ][ _ ][ _ ]
                                     ^
                                     Read pointer (i)
           [ h ][ e ][ l ][ l ][ o ][ _ ][ w ][ o ][ r ][ l ][ d ]
                                     ^
                                     Write pointer (j)
```

Step 2: Reverse the Whole String

By reversing everything, the last word moves to the front, but the characters within the words are also backwards.

```
Cleaned: "hello world"
Reversed: "dlrow olleh"
```

Step 3: Reverse Each Word

We iterate through the reversed string. Every time we find a full word, we reverse just those characters to bring them back to their original spelling.

```
"dlrow" -> "world"
"olleh" -> "hello"
Result: "world hello"
```

3. Time and Space Complexity Analysis

Time Complexity (TC)

We traverse the string exactly three times (once for cleaning, once for the full reverse, and once for the word-by-word reverse).

ASCII TC Derivation:

Operation	Passes	Complexity
1. Clean Spaces	1	$O(N)$
2. Reverse All	1	$O(N)$
3. Reverse Words	1	$O(N)$
Total Time	$3 * N$	$O(N)$

Space Complexity (SC)

In an ideal environment (or if we convert the string to a character array), we only use a few integer variables for pointers.

ASCII SC Derivation:

Component	Type	Complexity
Pointers (i, j, k)	Integers	$O(1)$
Output Array/Buffer	Array (Required)	$O(N)$
Total Extra Space	Constant Pointers	$O(1)*$

*Note: In JS/Python, strings are immutable, so we must create an array first, making it $O(N)$ in practice, but the logic is $O(1)$ "in-place" logic.

4. Solution Code

Implementation 1: The “High-Level” Functional Approach

This is what you write if you want to show you know the language’s power. It is clean and readable.

Javascript

```
const reverseWordsHighLevel = (s) => {  
  // 1. Trim leading/trailing spaces  
  // 2. Split by one or more spaces using Regex
```

```

    // 3. Reverse the array of words
    // 4. Join with a single space
    return s.trim().split(/\s+/).reverse().join(' ');
};

```

Python

```

def reverse_words_high_level(s: str) -> str:
    # .split() without arguments automatically handles
    # any amount of whitespace and trims the string.
    words = s.split()
    return " ".join(reversed(words))

```

Implementation 2: The “L5/L6” Two-Pointer Approach

This demonstrates manual memory management logic.

Javascript

```

/**
 * Helper to reverse a portion of an array in-place
 */
function reverse(arr, start, end) {
    while (start < end) {
        [arr[start], arr[end]] = [arr[end], arr[start]];
        start++;
        end--;
    }
}

const reverseWords = (s) => {
    // Convert to array because JS strings are immutable
    let chars = s.split('');
    let n = s.length;

    // Step 1: Reverse the whole string
    reverse(chars, 0, n - 1);

    // Step 2: Reverse each word & Clean spaces
    let result = [];
    let i = 0;
    while (i < n) {
        if (chars[i] !== ' ') {
            let start = i;
            // Find the end of the word
            while (i < n && chars[i] !== ' ') i++;
            let word = chars.slice(start, i);
            reverse(word, 0, word.length - 1);

```

```

        result.push(word.join(''));
    }
    i++;
}

return result.join(' ');
};

```

Python

```

class Solution:
    def reverseWords(self, s: str) -> str:
        # Convert string to list for "in-place" simulation
        chars = list(s)
        n = len(chars)

        # Helper: Reverse a section of the list
        def reverse_segment(l, r):
            while l < r:
                chars[l], chars[r] = chars[r], chars[l]
                l, r = l + 1, r - 1

        # 1. Reverse the entire string
        reverse_segment(0, n - 1)

        # 2. Reverse each individual word
        start = 0
        for end in range(n + 1):
            # If we hit a space or end of string, we found a word
            if end == n or chars[end] == ' ':
                reverse_segment(start, end - 1)
                start = end + 1

        # 3. Clean up extra spaces (L6 level detail)
        # We join the words while ignoring multiple spaces
        return ' '.join("".join(chars).split())

```

New Terms & Techniques

- **Two-Pointer Technique:** This uses two variables (usually *i* and *j*) to track indices. It helps process an array in a single pass without creating extra copies. In this problem, it's used to identify word boundaries.
- **In-Place Algorithms:** These are algorithms that transform input using no auxiliary data structures. It is crucial for “Staff” level discussions where system memory (RAM) is a bottleneck.

- **Tokenization:** The process of breaking a string into “tokens” (words). Knowing when to do this manually versus using a library like `.split()` shows you understand the underlying cost of regex and string splitting.
-

Real-World / Interview Variations

1. The Bloomberg “Streaming” Variation

The Problem: “What if the string is so large it doesn’t fit in memory, and you are receiving it as a stream of characters?” **The Solution:** Use a **Stack**. As characters come in, buffer them into a “Word” string. When you hit a space, push that word onto the stack. Once the stream ends, pop everything off the stack. This ensures you only store the words, not the full massive string at once.

2. The Google “In-Place” Constraint

The Problem: “Perform the reversal in a language like C with zero extra memory ($O(1)$ space).” **The Solution:** You must do the “Reverse All $\rightarrow$ Reverse Each Word” trick directly on the memory buffer. You would use a third pass to “shift” characters left to delete extra spaces, effectively resizing the string in-place.

3. The Meta “Sentence Integrity” Variation

The Problem: “Reverse the words but keep the punctuation in its original relative position.” **The Solution:** You would treat punctuation as “fixed” indices. You extract only the alphanumeric words, reverse that list, and then re-insert them into the original string at the non-punctuation gaps.

135. Candy

Targeting an L5/L6 level at Google means looking beyond just “making the code work.” It requires identifying the most efficient greedy approach, handling edge cases gracefully, and achieving the optimal $O(n)$ time and $O(1)$ space complexity.

The “Candy” problem is a classic “Two-Pass Greedy” or “Peak and Valley” challenge.

1. Problem Explanation

Imagine a row of children, each with a specific rating. You must give each child at least 1 candy. The catch: if a child has a higher rating than their immediate neighbor (left or right), they **must** receive more candies than that neighbor.

The Goal: Find the absolute minimum number of candies you can distribute while satisfying these rules.

The Core Constraints:

1. Every child gets at least 1 candy.
 2. If `rating[i] > rating[i-1]`, then `candy[i] > candy[i-1]`.
 3. If `rating[i] > rating[i+1]`, then `candy[i] > candy[i+1]`.
-

2. Solution Explanation

A high-level engineer recognizes that this problem has local dependencies that dictate global results. The “Two-Pass” approach is the most intuitive and interview-friendly way to solve this.

The Strategy: Two-Pass Greedy

We split the requirements into two independent directions.

Pass 1: Left-to-Right (The “Left Neighbor” Rule) We ensure every child has more candies than their left neighbor if their rating is higher.

- Start with 1 candy for everyone.
- If `ratings[i] > ratings[i-1]`, set `candies[i] = candies[i-1] + 1`.

Pass 2: Right-to-Left (The “Right Neighbor” Rule) We ensure every child has more candies than their right neighbor if their rating is higher, **without** breaking the rule we established in the first pass.

- If `ratings[i] > ratings[i+1]`, set `candies[i] = max(current_candies[i], candies[i+1] + 1)`.

Visualization of the Two-Pass logic

Let's use an example: `Ratings = [1, 2, 8, 8, 2, 1]`

Step 1: Initialization All children start with 1 candy.

`Ratings: [ 1, 2, 8, 8, 2, 1 ]`

`Candies: [ 1, 1, 1, 1, 1, 1 ]`

Step 2: Left-to-Right Pass Compare each child to their **left** neighbor.

- Index 1 (2 > 1): Give 2 candies.
- Index 2 (8 > 2): Give 3 candies.
- Index 3 (8 is not > 8): Keep 1 candy.
- Index 4 (2 is not > 8): Keep 1 candy.
- Index 5 (1 is not > 2): Keep 1 candy.

Ratings: [1, 2, 8, 8, 2, 1]
 L-to-R: [1, 2, 3, 1, 1, 1]
 $\hat{}$ $\hat{}$ $\hat{}$ $\phantom{\hat{}}$ $\phantom{\hat{}}$ $\phantom{\hat{}}$

Step 3: Right-to-Left Pass Compare each child to their **right** neighbor.

- Index 4 (2 > 1): Give $\max(1, 1+1) = 2$ candies.
- Index 3 (8 > 2): Give $\max(1, 2+1) = 3$ candies.
- Index 2 (8 is not > 8): Keep $\max(3, 1) = 3$ candies.
- Index 1 (2 is not > 8): Keep $\max(2, 1) = 2$ candies.

Ratings: [1, 2, 8, 8, 2, 1]
 L-to-R: [1, 2, 3, 1, 1, 1]
 R-to-L: [1, 2, 3, 3, 2, 1]
 $\phantom{\hat{}}$ $\phantom{\hat{}}$ $\hat{}$ $\hat{}$ $\hat{}$ $\hat{}$

Total Candies: 1 + 2 + 3 + 3 + 2 + 1 = 12

3. Time and Space Complexity Analysis

For an L5/L6 candidate, explaining why the complexity is what it is matters more than just stating it.

Time Complexity (TC)

We iterate through the array exactly twice.

Pass 1 (N steps) + Pass 2 (N steps) = 2N steps
 2N simplified in Big O notation is $O(n)$

Space Complexity (SC)

We store an extra array of size N to keep track of the candies.

Array of size N = $O(n)$ space

Note: There is a “One-Pass” $O(1)$ space solution using “Peak and Valley” counting, but in a Google interview, the Two-Pass $O(n)$ space solution is usually preferred for its readability and lower bug-risk unless specifically asked to optimize space.

4. Solution Code

Implementation 1: The Two-Pass Greedy (Recommended)

Python

```

def candy(ratings):
    n = len(ratings)
    # Every child must have at least 1 candy
    candies = [1] * n

    # Left-to-right pass: satisfy the left-neighbor condition
    # If I am better than the person to my left, I get one more than them
    for i in range(1, n):
        if ratings[i] > ratings[i-1]:
            candies[i] = candies[i-1] + 1

    # Right-to-left pass: satisfy the right-neighbor condition
    # If I am better than the person to my right, I need more than them.
    # We use max() to ensure we don't break the left-neighbor condition.
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i+1]:
            candies[i] = max(candies[i], candies[i+1] + 1)

    return sum(candies)

```

JavaScript

```

/**
 * @param {number[]} ratings
 * @return {number}
 */
var candy = function(ratings) {
    const n = ratings.length;
    // Initialize an array with 1 candy for each child
    const candies = new Array(n).fill(1);

    // Forward pass: Ensure a child has more candies than their left neighbor if they have a higher rating
    for (let i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            candies[i] = candies[i - 1] + 1;
        }
    }

    // Backward pass: Ensure a child has more than their right neighbor if they have a higher rating
    // We take the maximum of the current value and the new requirement to satisfy both sides
    for (let i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1]) {
            candies[i] = Math.max(candies[i], candies[i + 1] + 1);
        }
    }
}

```

```
// Sum up the candies array for the final result
return candies.reduce((total, val) => total + val, 0);
};
```

New Terms & Techniques

Greedy Algorithm: A greedy algorithm makes the locally optimal choice at each step with the hope of finding the global optimum. In this problem, we “greedily” satisfy one neighbor’s rule at a time. It works here because the “more than neighbor” requirement only depends on immediate neighbors, allowing local adjustments to eventually satisfy the global minimum.

Real-World Interview Variations

Google, Meta, and Bloomberg rarely ask “The Candy Problem” verbatim anymore. Instead, they use these variations:

1. The “Server Load” Problem (Google/Meta)

- **The Prompt:** You have a row of servers. Each has a “priority” score. You need to assign “worker threads.” High-priority servers must have more threads than adjacent lower-priority servers. What’s the minimum threads needed?
- **The Solution:** Identical to the Candy problem. It tests if you can map a physical resource (threads) to the greedy neighbor logic.

2. The “Gas Station Distance” / “Altitudes” (Bloomberg)

- **The Prompt:** You are building gas stations along a mountain range. The amount of fuel at each station must be higher than the fuel at an adjacent station if the altitude is higher.
- **The Solution:** This often adds a twist where `ratings[i] == ratings[i+1]` might have different rules (e.g., they *must* have the same amount of fuel). In the standard Candy problem, equal ratings don’t require equal candies.

3. The “Two-Sided Ranking” (Meta)

- **The Prompt:** Similar to Candy, but you are given a circular row of children.
- **The Solution:** This is much harder. You usually find the “valley” (the child with the lowest rating) and break the circle there to turn it into a linear array, then perform the two-pass logic.

4. Median of Two Sorted Arrays

1. Problem Explanation

The Core Concept: What is a Median? In a single sorted array, the median is the middle value. Its fundamental property is that it divides the array into two equal halves: * Everything in the **left half** is smaller than or equal to everything in the **right half**. * If the length is odd, the median is the exact middle element. * If the length is even, the median is the average of the two middle elements.

The Twist: Two Arrays We are given two sorted arrays, `nums1` (length `m`) and `nums2` (length `n`). We need to find the median as if these two arrays were merged into one giant sorted array. * **Naive thought:** Just merge them! * **The Catch:** Merging takes $O(m + n)$ time. The prompt demands an $O(\log(m + n))$ time complexity. Whenever you see a logarithmic time constraint on sorted arrays, your brain should immediately yell: **Binary Search**.

2. Solution Explanation

To achieve $O(\log(\min(m, n)))$ time, we bypass merging entirely. Instead, we use binary search to find a “**partition**” (a line drawn through the arrays) that perfectly splits the combined elements into a valid left half and right half.

Let’s look at the invariants. We want to find a split in `nums1` and a split in `nums2` such that:

1. **Length requirement:** The total number of elements on the left side equals the total number of elements on the right side (or left has one extra if the total is odd).
2. **Value requirement:** Every element on the left side is $\leq$ every element on the right side.

Because the arrays are already sorted, we know `left_nums1` $\leq$ `right_nums1` and `left_nums2` $\leq$ `right_nums2`. The only things we need to cross-check are:

- * Is the biggest element on the left of `nums1` $\leq$ the smallest element on the right of `nums2`?
- * Is the biggest element on the left of `nums2` $\leq$ the smallest element on the right of `nums1`?

If yes, we found the perfect partition!

ASCII Walkthrough Let’s take an example: Array A (length 5): [1, 3, 8, 9, 15] Array B (length 6): [7, 11, 18, 19, 21, 25]

Total length = 11 (Odd). We need $(11 + 1) / 2 = 6$ elements in our combined Left Half. We will binary search on the **smaller** array (Array A) to minimize the search space.

Step 1: Initial Binary Search on A * `low = 0, high = 5` * `partitionA = (0 + 5) / 2 = 2` (This means we take 2 elements from A) * `partitionB = 6 - 2 = 4` (Since we need 6 total on the left, we take 4 from B)

	Left Half		Right Half
Array A:	1 3		8 9 15
Array B:	7 11 18 19		21 25

Variables at the partition line:

maxLeftA	= 3		minRightA	= 8
maxLeftB	= 19		minRightB	= 21

Check cross-conditions:

1. Is maxLeftA (3) $\leq$ minRightB (21)? --> YES.
2. Is maxLeftB (19) $\leq$ minRightA (8)? --> NO. (19 is greater than 8)

Analysis: maxLeftB is too big. This means our partition in B is too far to the right. To fix this, we need fewer elements from B, which means we must take *more* elements from A. We move our binary search in A to the right.

Step 2: Move Right in A * low = partitionA + 1 = 3, high = 5 *
 partitionA = (3 + 5) / 2 = 4 (Take 4 elements from A) * partitionB =
 6 - 4 = 2 (Take 2 elements from B)

	Left Half		Right Half
Array A:	1 3 8 9		15
Array B:	7 11		18 19 21 25

Variables at the partition line:

maxLeftA	= 9		minRightA	= 15
maxLeftB	= 11		minRightB	= 18

Check cross-conditions:

1. Is maxLeftA (9) $\leq$ minRightB (18)? --> YES.
2. Is maxLeftB (11) $\leq$ minRightA (15)? --> YES.

Analysis: Perfect! Both cross-conditions are met. We have successfully split the combined logical array without ever merging it.

Calculating the Median: Because the total number of elements (11) is odd, the median is simply the largest element on the left side. Median = max(maxLeftA, maxLeftB) = max(9, 11) = 11.

3. Time and Space Complexity Analysis

Time Complexity: $O(\log(\min(m, n)))$

DERIVATION:

We only perform Binary Search on the smaller array.

Let m = length of shorter array.

Let n = length of longer array.

Search Space:

Step 1: Search space is m .

Step 2: Search space is $m/2$.

Step 3: Search space is $m/4$.

...

Step k : Search space is 1.

Number of steps $k = \log_2(m)$.

Since m is the minimum of the two lengths, Time Complexity is $O(\log(\min(m, n)))$.

Space Complexity: $O(1)$

DERIVATION:

Memory used:

- low pointer (integer)
- high pointer (integer)
- partitionA (integer)
- partitionB (integer)
- maxLeftA, minRightA, maxLeftB, minRightB (integers)

No extra arrays or recursive stacks are created.

Therefore, Space Complexity is strictly $O(1)$ constant time.

4. Solution Code

Here are both the sub-optimal (acceptable as a warm-up/brute force) and the optimal solutions in Python and JavaScript.

Python Solutions

```
# --- SUB-OPTIMAL (Merge Approach) ---
# Time:  $O(m+n)$ , Space:  $O(1)$  if we don't store the array, just count up to the middle.
def findMedianSortedArrays_naive(nums1, nums2):
    m, n = len(nums1), len(nums2)
    total_len = m + n
    mid = total_len // 2

    p1, p2 = 0, 0
    current, previous = 0, 0

    for _ in range(mid + 1):
        previous = current
        # If nums1 is exhausted, or nums2 has a smaller element
        if p1 == m:
            current = nums2[p2]
```

```

        p2 += 1
    elif p2 == n:
        current = nums1[p1]
        p1 += 1
    elif nums1[p1] < nums2[p2]:
        current = nums1[p1]
        p1 += 1
    else:
        current = nums2[p2]
        p2 += 1

if total_len % 2 == 0:
    return (previous + current) / 2.0
else:
    return float(current)

# --- OPTIMAL (Binary Search Approach) ---
# Time: O(log(min(m,n))), Space: O(1)
def findMedianSortedArrays(nums1, nums2):
    # Always ensure nums1 is the smaller array to optimize the search space
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    low, high = 0, m

    while low <= high:
        # Partitioning indices
        partitionA = (low + high) // 2
        partitionB = (m + n + 1) // 2 - partitionA

        # Edge cases: If partition is at the extreme ends, use infinity
        maxLeftA = float('-inf') if partitionA == 0 else nums1[partitionA - 1]
        minRightA = float('inf') if partitionA == m else nums1[partitionA]

        maxLeftB = float('-inf') if partitionB == 0 else nums2[partitionB - 1]
        minRightB = float('inf') if partitionB == n else nums2[partitionB]

        # Check if the partition is correct
        if maxLeftA <= minRightB and maxLeftB <= minRightA:
            # We found the perfect split!
            if (m + n) % 2 == 0:
                return (max(maxLeftA, maxLeftB) + min(minRightA, minRightB)) / 2.0
            else:
                return float(max(maxLeftA, maxLeftB))

```

```

    # If we are too far right in A, move left
    elif maxLeftA > minRightB:
        high = partitionA - 1

    # If we are too far left in A, move right
    else:
        low = partitionA + 1

    raise ValueError("Input arrays are not sorted properly.")

```

JavaScript Solutions

```

// --- SUB-OPTIMAL (Merge Approach) ---
// Time: O(m+n), Space: O(1)
var findMedianSortedArraysNaive = function(nums1, nums2) {
    let m = nums1.length, n = nums2.length;
    let totalLen = m + n;
    let mid = Math.floor(totalLen / 2);

    let p1 = 0, p2 = 0;
    let current = 0, previous = 0;

    for (let i = 0; i <= mid; i++) {
        previous = current;
        if (p1 === m) {
            current = nums2[p2++];
        } else if (p2 === n) {
            current = nums1[p1++];
        } else if (nums1[p1] < nums2[p2]) {
            current = nums1[p1++];
        } else {
            current = nums2[p2++];
        }
    }

    if (totalLen % 2 === 0) {
        return (previous + current) / 2.0;
    } else {
        return current;
    }
};

// --- OPTIMAL (Binary Search Approach) ---
// Time: O(log(min(m,n))), Space: O(1)
var findMedianSortedArrays = function(nums1, nums2) {

```

```

    // Ensure nums1 is the smaller array
    if (nums1.length > nums2.length) {
        return findMedianSortedArrays(nums2, nums1);
    }

    let m = nums1.length;
    let n = nums2.length;
    let low = 0;
    let high = m;

    while (low <= high) {
        let partitionA = Math.floor((low + high) / 2);
        let partitionB = Math.floor((m + n + 1) / 2) - partitionA;

        // Handle edges using Infinity
        let maxLeftA = partitionA === 0 ? -Infinity : nums1[partitionA - 1];
        let minRightA = partitionA === m ? Infinity : nums1[partitionA];

        let maxLeftB = partitionB === 0 ? -Infinity : nums2[partitionB - 1];
        let minRightB = partitionB === n ? Infinity : nums2[partitionB];

        if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
            // Correct partition found
            if ((m + n) % 2 === 0) {
                return (Math.max(maxLeftA, maxLeftB) + Math.min(minRightA, minRightB)) / 2.0;
            } else {
                return Math.max(maxLeftA, maxLeftB);
            }
        } else if (maxLeftA > minRightB) {
            // Move left in nums1
            high = partitionA - 1;
        } else {
            // Move right in nums1
            low = partitionA + 1;
        }
    }

    throw new Error("Arrays are not sorted.");
};

```

Glossary of Terms Used

- **Binary Search:** A search algorithm that finds the position of a target within a sorted array by repeatedly dividing the search interval in half. It is

critical here because it reduces a linear search $O(N)$ down to a logarithmic one $O(\log N)$.

- **Partitioning:** The act of drawing a logical boundary through a dataset to separate it into distinct groups (in this case, a ‘left’ half containing smaller numbers and a ‘right’ half containing larger numbers).
- **Invariants:** Conditions or rules that must remain true throughout the execution of an algorithm for it to be correct (e.g., “The left half must always have exactly $(m+n+1)/2$ elements”).

Real-World Interview Variations (Google, Meta, Bloomberg)

Interviews at top tech companies rarely ask this question completely raw anymore. They disguise it as system design or domain-specific problems.

1. Meta: “Find the K-th Smallest Element in Two Sorted Arrays” *

The Variation: Instead of finding the exact median (which is just the $(m+n)/2$ smallest element), you are given an integer k and asked to find the k -th smallest element. * **How to solve:** The logic remains exactly the same, but instead of the left half needing $(m + n + 1) / 2$ elements, your left half needs exactly k elements. You partition such that `partitionA + partitionB = k`. The rest of the binary search cross-checks are identical. The answer will be `max(maxLeftA, maxLeftB)`.

2. Google: “Median of elements distributed across two different servers” *

The Variation: A distributed systems flavor. Server A has a massive sorted array on its disk, and Server B has another. You cannot load both into memory on a single machine. Network calls are expensive. * **How to solve:** You implement the exact same binary search, but `partitionA` and `partitionB` become network requests. You ask Server A: “Give me the value at index X” and Server B: “Give me the value at index Y”. By doing binary search, you restrict your total network calls to exactly $O(\log(\min(m,n)))$, preventing network saturation.

3. Bloomberg: “Sliding Window Median of two distinct financial tick streams” *

The Variation: Bloomberg loves financial data streams. You might be asked to find the median of the last N ticks from Stream A and the last M ticks from Stream B at any given second. * **How to solve:** While the static snapshot calculation is the exact binary search above, doing this continuously implies the arrays are updating. For a sliding window over streams, you would typically pivot away from this algorithm entirely and maintain two Heaps (a Max-Heap for the lower half, a Min-Heap for the upper half) to allow for $O(\log N)$ insertions and removals as time ticks forward, yielding an $O(1)$ median retrieval.

918. Maximum Sum Circular Subarray

This problem is a classic “Level Up” from the standard Maximum Subarray problem (Kadane’s Algorithm). To solve it like a Senior (L5) or Staff (L6) engineer, we don’t just jump into code; we decompose the circular property into symmetric cases.

1. Problem Explanation

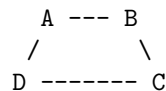
Imagine an array like [5, -3, 5]. In a **linear** array, the max sum is 5 (either the first or last element). In a **circular** array, the end wraps around to the beginning. This means you can pick [5] from the end and [5] from the beginning to get a sum of 10.

The Core Constraint

You can’t use the same element twice. A circular subarray can’t be longer than the array itself.

Visualization of “Circular”

Standard Array: [A, B, C, D] Circular view: Think of it as a clock or a loop.



There are two ways the maximum sum can exist:

1. **The “Middle” Case:** The max subarray is just a normal slice in the middle (e.g., [A, B, C]).
 2. **The “Wrapping” Case:** The max subarray starts near the end and wraps around to the beginning (e.g., [D, A, B]).
-

2. Solution Explanation

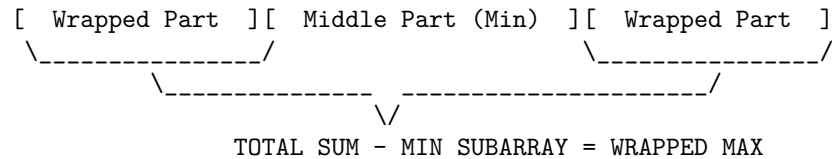
An L5/L6 engineer recognizes that **Case 2 (Wrapping)** is just the “inverse” of a **Minimum Subarray** in the middle.

Case 1: The Linear Max (Standard Kadane’s)

We find the maximum sum possible without wrapping. Array: [5, -3, 5]
Max Subarray: [5, -3, 5] is not possible linearly, but [5] or [5, -3, 5] as a whole might be. Actually, for [5, -3, 5], the linear max is 7 if we take the whole thing.

Case 2: The Wrapping Max

If the max subarray wraps around, it means it **leaves out** a chunk in the middle. To make the wrap-around part as large as possible, the “left-out” chunk must be as small as possible (the Minimum Subarray).



The Logic Flow

1. **Calculate Total Sum** of the array.
2. **Run Kadane's for Max:** Find the maximum subarray sum (`max_kadane`).
3. **Run Kadane's for Min:** Find the minimum subarray sum (`min_kadane`).
4. **Compare:**
 - Candidate A: `max_kadane`
 - Candidate B: `Total_Sum - min_kadane`
5. **The Edge Case:** If all numbers are negative, `Total_Sum - min_kadane` will result in 0 (an empty subarray), which isn't allowed. In this specific case, just return the `max_kadane` (the "least negative" number).

3. Time and Space Complexity Analysis

We iterate through the array exactly once, keeping track of a few variables.

Time Complexity (TC)

Pass 1 (Single Loop):

Step	Cost
Iterate through N items	N
Update Max Current	1
Update Min Current	1
Update Total Sum	1
Total	O(N)

Space Complexity (SC)

Storage used:

Variable	Space
max_total, max_curr	O(1)
min_total, min_curr	O(1)
total_sum	O(1)
Total	O(1)

4. Solution Code

Python Implementation

```
def maxSubarraySumCircular(nums):  
    # Standard Kadane's logic variables  
    total_sum = 0  
    max_total = nums[0]  
    max_curr = 0  
    min_total = nums[0]  
    min_curr = 0  
  
    # We can solve both Max and Min in a single pass  
    for x in nums:  
        # 1. Standard Kadane to find the maximum middle subarray  
        max_curr = max(x, max_curr + x)  
        max_total = max(max_total, max_curr)  
  
        # 2. Inverted Kadane to find the minimum middle subarray  
        min_curr = min(x, min_curr + x)  
        min_total = min(min_total, min_curr)  
  
        # 3. Keep track of the total to calculate the "wrap"  
        total_sum += x  
  
    # Edge Case: If all numbers are negative, max_total will be the  
    # largest single element (e.g., -1). total_sum - min_total would  
    # result in 0 (empty set), which is invalid for subarrays.  
    if max_total < 0:  
        return max_total  
  
    # Return the better of the linear max or the circular (wrapped) max  
    return max(max_total, total_sum - min_total)
```

JavaScript Implementation

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubarraySumCircular = function(nums) {
    let totalSum = 0;
    let maxTotal = nums[0];
    let maxCurr = 0;
    let minTotal = nums[0];
    let minCurr = 0;

    for (let x of nums) {
        // Standard Kadane's: Finding the highest peak in the middle
        maxCurr = Math.max(x, maxCurr + x);
        maxTotal = Math.max(maxTotal, maxCurr);

        // Standard Kadane's (Min version): Finding the deepest valley
        minCurr = Math.min(x, minCurr + x);
        minTotal = Math.min(minTotal, minCurr);

        totalSum += x;
    }

    // Special case: If every number is negative, we cannot "wrap"
    // because that would leave an empty array (sum 0).
    // We must return the least negative number found by maxTotal.
    if (maxTotal < 0) {
        return maxTotal;
    }

    // The answer is either the best middle slice
    // or the total minus the worst middle slice (the wrap).
    return Math.max(maxTotal, totalSum - minTotal);
};
```

Key Term: Kadane's Algorithm

Kadane's is a **Dynamic Programming** technique used to find the maximum sum of a contiguous subarray. Instead of checking all possible start and end points (which is $O(N^2)$), it asks at each index: "Should I start a new subarray here, or add this to the previous subarray?"

- **Why it helps:** It reduces the problem from quadratic time to linear time.

- **Application:** In this problem, we use it twice—once to find the “peak” and once to find the “valley.”

Real World / Interview Variations

1. **Google (Infrastructure/SRE):** “Given a circular log of requests per second over 24 hours, find the peak 4-hour window of traffic.”
 - *Solution:* This is a circular sliding window (fixed size K). Use a simple linear pass and handle the index with $i \% N$.
2. **Meta (Product/Ads):** “A user’s interest score wraps around a 7-day week. Find the period where the user is most active.”
 - *Solution:* This is exactly the Maximum Circular Subarray problem. Use the Max-Kadane vs (Total - Min-Kadane) approach.
3. **Bloomberg (FinTech):** “Given stock price fluctuations in a 24-hour global market, find the maximum profit/loss period in a cycle.”
 - *Solution:* If the prices are “changes” (deltas), it’s this problem. If they are absolute prices, you’d find the min and max values while considering the wrap-around.

6. Zigzag Conversion

This is a classic “string manipulation” and “simulation” problem. At an L5 (Senior) or L6 (Staff) level at Google, the expectation isn’t just to solve it, but to provide a solution that is **clean, readable, and handles edge cases** without complex conditional branches.

1. Problem Explanation

The problem asks us to take a string and “write” it in a zigzag pattern across a specific number of rows, then read it back row by row.

Imagine the string “PAYPALISHIRING” with `numRows = 3`.

The Visual Zigzag:

```
Row 0: P . . . A . . . H . . . N
Row 1: . A . P . L . S . I . I . G
Row 2: . . Y . . . I . . . R . .
```

If we collapse the dots and read across each row:

- Row 0: PAHN

- Row 1: APLSIIG
- Row 2: YIR

Result: "PAHNAPLSIIGYIR"

The "Non-Trivial" Part: The movement isn't just "down." It's "down" until you hit the bottom, then "diagonal-up" until you hit the top. A common mistake is trying to calculate the exact 2D coordinate for every letter. A more efficient way is to simply track which row a character belongs to as we iterate through the string.

2. Solution Explanation: The "Direction Toggle" Approach

Instead of building a full 2D grid (which wastes memory), we use an array of strings, where each index represents a row.

1. **Initialize:** Create an array of empty strings (one for each row).
2. **Track Position:** Start at `currentRow = 0`.
3. **Track Direction:** Use a boolean or integer to decide if we are moving "down" (+1) or "up" (-1).
4. **Iterate:** For every character in the input string:
 - Append it to the string at `rows[currentRow]`.
 - If we hit the **top row** (0), we must move **down**.
 - If we hit the **bottom row** (`numRows - 1`), we must move **up**.
 - Update `currentRow` based on the direction.

The Visualization of the Logic

String: "GOOGLE", numRows: 3

```
Step 1: 'G' -> Row 0 | [G, "", ""] | Move Down
Step 2: 'O' -> Row 1 | [G, O, ""] | Move Down
Step 3: 'O' -> Row 2 | [G, O, O] | Hit Bottom! Change Direction to UP
Step 4: 'G' -> Row 1 | [G, OG, O] | Move Up
Step 5: 'L' -> Row 0 | [GL, OG, O] | Hit Top! Change Direction to DOWN
Step 6: 'E' -> Row 1 | [GL, OGE, O] | Final Rows
```

Result: "GLOGEO"

3. Time and Space Complexity Analysis

TIME COMPLEXITY (TC)

Input String Length = N

1. We iterate through the string exactly once: N steps.

2. We join the rows at the end: N steps.

Total: $O(N)$

ASCII DERIVATION:

[Char 1] -> [Char 2] -> ... -> [Char N]

Process	Process	Process
(1 unit)	(1 unit)	(1 unit)

Total Time = 1 + 1 + ... + 1 = N units of time.

Result: $O(N)$

SPACE COMPLEXITY (SC)

1. We store each character in a 'row' structure.

2. The total number of characters stored is N.

3. The 'rows' array has size 'numRows' (K).

Total: $O(N)$

ASCII DERIVATION:

Memory Layout:

Row 0: [****] (length n0)

Row 1: [**] (length n1)

...

Row K: [***] (length nk)

Sum of (n0 + n1 + ... + nk) = N characters.

Total Space = $O(N)$

4. Solution Code

Python Implementation

```
def convert(s, numRows):
    # Edge Case: If only one row or string is shorter than rows,
    # no zigzagging is possible.
    if numRows == 1 or numRows >= len(s):
        return s

    # Initialize rows as a list of strings
    rows = [" " for _ in range(numRows)]

    current_row = 0
```

```

step = 1 # 1 means going down, -1 means going up

# This loop simulates the physical movement of a pen
# writing the zigzag pattern.
for char in s:
    rows[current_row] += char

    # If we reach the top row, we must start going down
    if current_row == 0:
        step = 1
    # If we reach the bottom row, we must start going up
    elif current_row == numRows - 1:
        step = -1

    current_row += step

# Join all rows into a single string
return "".join(rows)

```

JavaScript Implementation

```

/**
 * @param {string} s
 * @param {number} numRows
 * @return {string}
 */
var convert = function(s, numRows) {
    // Edge Case check
    if (numRows === 1 || numRows >= s.length) {
        return s;
    }

    // Create an array to hold the characters for each row
    const rows = new Array(numRows).fill("");
    let currentRow = 0;
    let movingDown = false;

    for (let char of s) {
        rows[currentRow] += char;

        // Flip direction if we hit the top or bottom boundary
        if (currentRow === 0 || currentRow === numRows - 1) {
            movingDown = !movingDown;
        }

        // Move current row index based on direction

```

```

        currentRow += movingDown ? 1 : -1;
    }

    return rows.join("");
};

```

Terminology: Simulation Pattern

The technique used here is called a **Simulation Pattern**.

- **Why it helps:** In interviews, candidates often try to find a complex mathematical formula to map index `i` to its new position. While possible, it is prone to “off-by-one” errors.
 - **How it applies:** By simulating the movement (down/up), we write code that mirrors the physical description of the problem. This makes the code easier to debug and more robust against different `numRows` values.
-

Real-World / Interview Variations

1. Google: “The Wave Text” (Messaging Apps)

- *Scenario:* “We want to animate text in a chat bubble so it moves in a wave pattern. How do you determine the height of each letter at time `T`?”
- *Solution:* This is essentially Zigzag Conversion where the `numRows` is the amplitude of the wave. You solve it by mapping the index to a “row” and then applying a vertical offset based on that row index.

2. Meta: “Efficient Storage for Sparse Matrices”

- *Scenario:* “You are given a stream of data points that follow a specific diagonal pattern. How do you store them so they can be retrieved by ‘level’ quickly?”
- *Solution:* This relates to the memory management of the Zigzag problem. Instead of a 2D array, you use a Hash Map of Lists where the key is the “Level” (or Row), allowing $O(1)$ insertion and $O(N)$ retrieval.

3. Bloomberg: “Diagonal Traversal of Financial Tickers”

- *Scenario:* “Given a grid of stock prices, read them in a zigzag/snake fashion to detect trends across different sectors.”
- *Solution:* Similar to Leetcode’s “Diagonal Traverse.” You track the boundary conditions (hitting the edges of the grid) to flip your `dx` and `dy` direction vectors, just like we flipped the `step` variable here.

875. Koko Eating Bananas

To solve a problem like “Koko Eating Bananas” at a Google L5/L6 (Senior/Staff) level, the focus isn’t just on getting the code to pass. It’s about identifying the **underlying mathematical property** (monotonicity) and choosing the most robust algorithm to exploit it.

1. Problem Explanation

Koko wants to eat all bananas from n piles within h hours.

- Each pile `piles[i]` has a certain number of bananas.
- Koko chooses an eating speed k (bananas per hour).
- In one hour, she can eat at most k bananas from a single pile. If the pile has fewer than k bananas, she eats them all and **waits** until the next hour (she doesn’t move to another pile in the same hour).

The Goal: Find the **minimum** integer speed k such that she can finish all piles within h hours.

The Core Constraint

If Koko eats too slowly, she won’t finish in time. If she eats very fast, she finishes early but might be “over-speeding.” We need the “Sweet Spot”—the smallest k that satisfies the time limit.

2. Solution Explanation

A Senior Engineer looks for the **Search Space**.

- **Minimum possible speed:** 1 banana/hour (She can’t eat 0).
- **Maximum possible speed:** The size of the largest pile. (Eating faster than the biggest pile doesn’t help because she can only tackle one pile per hour anyway).

The “Aha!” Moment: Monotonicity

If Koko can finish all bananas at speed k , she can *definitely* finish them at speed $k + 1$. If she *cannot* finish at speed k , she definitely *cannot* finish at any speed slower than k .

This “Yes/No” transition creates a sorted-like property: **Speeds:** [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] **Can Finish:** [No, No, No, Yes, Yes, Yes, Yes, Yes, Yes, Yes] ^--- We want this first "Yes"

Since the results are “sorted,” we use **Binary Search**.

Visualization of the Process

Let's say `piles = [3, 6, 7, 11]` and `h = 8`. Max pile is 11, so our range is `[1, 11]`.

Step 1: Low=1, High=11, Mid=6

Check speed 6:

- Pile 3: $3/6 \rightarrow 1$ hour
- Pile 6: $6/6 \rightarrow 1$ hour
- Pile 7: $7/6 \rightarrow 2$ hours
- Pile 11: $11/6 \rightarrow 2$ hours

Total: $1+1+2+2 = 6$ hours. ($6 \leq 8$? YES)

Result could be 6, but maybe there's a smaller speed? Set High = 5.

Step 2: Low=1, High=5, Mid=3

Check speed 3:

- Pile 3: $3/3 \rightarrow 1$ hour
- Pile 6: $6/3 \rightarrow 2$ hours
- Pile 7: $7/3 \rightarrow 3$ hours
- Pile 11: $11/3 \rightarrow 4$ hours

Total: $1+2+3+4 = 10$ hours. ($10 \leq 8$? NO)

Speed 3 is too slow. Set Low = 4.

Step 3: Low=4, High=5, Mid=4

Check speed 4:

- Pile 3: 1 hour
- Pile 6: 2 hours ($4+2$)
- Pile 7: 2 hours ($4+3$)
- Pile 11: 3 hours ($4+4+3$)

Total: $1+2+2+3 = 8$ hours. ($8 \leq 8$? YES)

Result = 4. Set High = 3.

Loop ends. Minimum speed is 4.

3. Time and Space Complexity Analysis

We evaluate the complexity based on `N` (number of piles) and `M` (the maximum value in `piles`).

Time Complexity (TC)

We perform a binary search over the range 1 to `M`. For every step in the binary search, we iterate through all `N` piles to calculate the total time.

Binary Search Range: 1 to `M`

Number of steps in Binary Search: $\log(M)$

Work done per step: Iterate through N piles -> $O(N)$

Total TC calculation:

```
+-----+
| log(M) * N = O(N * log(M)) |
+-----+
```

Space Complexity (SC)

We only store a few variables (low, high, mid, total_time). We do not use any additional data structures that scale with input size.

Variables used:

- low, high, mid (integers)
- total_time (integer)
- k (integer)

Total SC calculation:

```
+-----+
| Constant Space = O(1) |
+-----+
```

4. Solution Code

Python Implementation

```
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        # The range of possible speeds
        low = 1
        high = max(piles)
        ans = high

        # Helper function to check if a speed 'k' is viable
        # Logic: hours = ceil(pile / k).
        # Using (p + k - 1) // k is a common trick for integer ceiling.
        def can_finish(k):
            total_hours = 0
            for p in piles:
                total_hours += (p + k - 1) // k
            return total_hours <= h

        # Standard Binary Search on the Answer
        while low <= high:
            mid = low + (high - low) // 2
```

```

        if can_finish(mid):
            ans = mid      # Current mid works, try to find a smaller one
            high = mid - 1
        else:
            low = mid + 1  # Too slow, increase the speed

    return ans

```

JavaScript Implementation

```

/**
 * @param {number[]} piles
 * @param {number} h
 * @return {number}
 */
var minEatingSpeed = function(piles, h) {
    let low = 1;
    let high = Math.max(...piles);
    let ans = high;

    /**
     * Helper to calculate hours needed for a specific speed k.
     * We use Math.ceil to handle the "wait until next hour" rule.
     */
    const canFinish = (k) => {
        let totalHours = 0;
        for (let p of piles) {
            totalHours += Math.ceil(p / k);
        }
        return totalHours <= h;
    };

    // Binary search over the range of speeds [1, max(piles)]
    while (low <= high) {
        let mid = Math.floor(low + (high - low) / 2);

        if (canFinish(mid)) {
            ans = mid;      // This speed works, but let's see if we can go lower
            high = mid - 1;
        } else {
            low = mid + 1;  // Speed is too slow, we need more bananas per hour
        }
    }

    return ans;
};

```

Note 1: Terms and Techniques

- **Binary Search on the Answer:** Instead of searching for an element in an array, we search for a value in a range of possible answers. It's useful when the solution space is sorted and we have a way to validate an answer.
 - **Monotonic Function:** This is the “Yes/No” pattern mentioned earlier. If $f(k)$ is true, then $f(k+1)$ must also be true. This property is the prerequisite for using binary search.
 - **Integer Ceiling Trick:** In programming, $\text{ceil}(a/b)$ is often written as $(a + b - 1) // b$. This avoids floating-point precision issues and is a favorite for senior-level coding.
-

Note 2: Real-World Interview Variations

Google, Meta, and Bloomberg often disguise this problem to test your ability to recognize the “Binary Search on Answer” pattern.

1. **Google (Ship Packages):** “Capacity To Ship Packages Within D Days.” You have weights and must find the minimum truck capacity to ship all items in D days.
 - *Solution:* Same logic. Range is $[\text{max_weight}, \text{sum_of_weights}]$. $\text{can_finish}(\text{capacity})$ checks if you can group items into D days without exceeding capacity.
2. **Meta (Split Array):** “Split Array Largest Sum.” Split an array into M sub-arrays such that the maximum sum of any sub-array is minimized.
 - *Solution:* Binary search over the possible sum. Range $[\text{max_element}, \text{total_sum}]$.
3. **Bloomberg (Router Range):** Given house positions on a street, what is the minimum signal range R needed for K routers to cover all houses?
 - *Solution:* Range is $[0, \text{street_length}]$. $\text{can_finish}(R)$ uses a greedy approach to place K routers to see if they cover all positions.

134. Gas Station

To solve “134. Gas Station” at a Google L5/L6 level, you don’t just look for a “trick.” You look for the underlying mathematical property that allows you to reduce a potentially $O(n^2)$ problem into a single linear pass.

1. Problem Explanation

Imagine you are driving a car on a circular route. There are **n** gas stations.

- **gas[i]**: How much fuel you get at station **i**.
- **cost[i]**: How much fuel it takes to get from station **i** to station **i+1**.

You start with an empty tank. You want to know: **Can you start at any station and complete one full circle without running out of gas?** If multiple starts work, the problem guarantees a unique one.

The Core Challenge

The difficulty is the “Circular” nature. If you start at index 2, you must go 2 -> 3 -> 0 -> 1 -> 2. If at any point your **current_gas < 0**, that starting point is a failure.

2. Solution Explanation

The Brute Force (L3 Approach)

You try starting at every single station. For each start, you simulate the whole lap.

- **Complexity:** $O(n^2)$
- **Why it fails:** With 100,000 stations, 10,000,000,000 operations is too slow.

The Senior Engineer (L5/L6) Insight

A top-tier engineer looks for **Greedy** properties. There are two “Golden Rules” for this problem:

Rule 1: The Total Sum Rule If the total amount of gas is less than the total cost of the trip, it is physically impossible to finish the lap, no matter where you start. **Total Gas - Total Cost < 0 => Return -1**

Rule 2: The “Point of No Return” Rule If you start at Station A and get stuck at Station B (meaning you ran out of gas between A and B), then **no station between A and B can be a valid starting point.**

Visualization of the Greedy Logic

Let’s look at why Rule 2 works. If you could make it from A to some middle station M, it means you arrived at M with **at least 0 or more gas**. If you couldn’t make it from A to B, but you *did* make it to M, then starting at M is actually *worse* than starting at A because you lose the “bonus” gas you carried from A to M.

ASCII Visualization of a Failed Journey:

Index:	[0]	---->	[1]	---->	[2]	---->	[3]
Gas:	1		2		3		1
Cost:	2		2		1		5
Net:	-1		0		+2		-4

Trial 1: Start at Index 0

[0]: Have 1, Need 2. Tank = -1. (FAIL!)

Logic: If 0 failed immediately, try 1.

Trial 2: Start at Index 1

[1]: Have 2, Need 2. Tank = 0. (Pass to next)

[2]: Have 3, Need 1. Tank = 0 + 2 = 2. (Pass to next)

[3]: Have 1, Need 5. Tank = 2 - 4 = -2. (FAIL at Index 3!)

CRITICAL INSIGHT:

Since we failed to reach the end of the trip starting from Index 1, and we got "stuck" at Index 3, we don't need to try starting at Index 2. We jump straight to trying Index 4 (or 0 in a circle).

The Algorithm

1. Keep track of **total_surplus** (Total Gas - Total Cost).
2. Keep track of **current_surplus** (Gas in tank during current attempt).
3. Iterate through stations:
 - Add `gas[i] - cost[i]` to both surpluses.
 - If `current_surplus < 0`:
 - This means we failed. Reset `current_surplus = 0`.
 - Set the **next** station as our new potential start.
4. After the loop, if `total_surplus < 0`, return -1. Otherwise, return the `start_index`.

3. Time and Space Complexity Analysis

Time Complexity (TC)

We traverse the array exactly once.

ASCII TC Derivation:

[Station 0]	->	[Station 1]	->	...	->	[Station N-1]
1 op		1 op				1 op

Total Ops = N

Complexity = $O(N)$

Space Complexity (SC)

We only use a few integer variables (total, current, start).

ASCII SC Derivation:

Variables:

```
{
    total_surplus: int,
    current_surplus: int,
    start_index: int
}
```

Memory used does not grow with input size N.

Complexity = $O(1)$

4. Solution Code

Optimized Greedy Approach (The L5+ Standard)

Python Implementation

```
def canCompleteCircuit(gas, cost):
    # If the total gas available is less than total cost,
    # it's impossible to complete the circuit.
    if sum(gas) < sum(cost):
        return -1

    total_tank = 0
    start_index = 0

    # We iterate through each station once.
    # The 'Greedy' insight: If we fail at index 'i',
    # the start must be at 'i + 1' or later.
    for i in range(len(gas)):
        total_tank += gas[i] - cost[i]

        # If the tank drops below zero, the current start_index is invalid.
        if total_tank < 0:
            # Reset the tank for the new starting point.
            total_tank = 0
            # Set the next station as the new potential starting point.
            start_index = i + 1

    return start_index
```


Javascript Implementation

```
/**
 * @param {number[]} gas
 * @param {number[]} cost
 * @return {number}
 */
var canCompleteCircuit = function(gas, cost) {
    let totalSurplus = 0;
    let currentSurplus = 0;
    let startIndex = 0;

    for (let i = 0; i < gas.length; i++) {
        let netAtStation = gas[i] - cost[i];
        totalSurplus += netAtStation;
        currentSurplus += netAtStation;

        // If currentSurplus is negative, we cannot have started at
        // the current startIndex or any station before 'i'.
        if (currentSurplus < 0) {
            // Reset tank and try starting at the very next station.
            currentSurplus = 0;
            startIndex = i + 1;
        }
    }

    // If total gas is less than total cost, return -1.
    // Otherwise, the greedy startIndex is guaranteed to be the solution.
    return totalSurplus < 0 ? -1 : startIndex;
};
```

Key Terms & Techniques

- **Greedy Algorithm:** An algorithm that makes the locally optimal choice at each stage with the hope of finding a global optimum. In this problem, resetting the `startIndex` whenever we hit a deficit is the greedy choice.
- **One-Pass Algorithm:** A technique where you solve the problem by looking at each element only once. This is the gold standard for performance in array-based interview questions.

Real World / Interview Variations

1. Google: “The Circular Delivery Route”

- *Scenario:* An electric delivery van has a battery capacity. It receives charges at warehouses but loses charge moving between them.
- *Difference:* They might add a `max_battery_capacity` constraint.
- *Solution:* You would use the same greedy logic but add a `min(current_surplus + charge, max_capacity)` check at each step.

2. Meta: “Social Media Feed Cache”

- *Scenario:* You are traversing a ring of cache nodes. Each node gives you some “TTL” (Time to Live) but the network hop consumes TTL.
- *Difference:* They might ask for *all* possible starting points if the “unique solution” guarantee is removed.
- *Solution:* You would compute the “prefix sums” of the net gas and find all points where the “running minimum” of the circular prefix sum never drops below the starting value.

3. Bloomberg: “Arbitrage Cycle” (Simplified)

- *Scenario:* You have currency exchange stations in a circle. Each offers a rate (multiplier). Can you start with 1 unit and end with ≥ 1 unit?
- *Difference:* This is multiplicative, not additive.
- *Solution:* Convert the multipliers to Logarithms. $\log(a * b) = \log(a) + \log(b)$. Once converted to logs, it becomes an additive problem exactly like the Gas Station problem.

3634. Minimum Removals to Balance Array

To solve this problem like a top-tier engineer, we first recognize that “removing elements” is a distraction. A better way to think about it is: **“What is the largest subset of numbers I can keep that satisfies the balance condition?”**

Once we find the maximum number of elements we can keep, the minimum removals is simply: `Total Elements - Maximum Kept`.

1. Problem Explanation

The condition for a “balanced” array is: **Max Element $\leq k * \text{Min Element}$**

Imagine you have a bag of numbers. To make the bag balanced, the “big boss” (Max) cannot be more than `k` times larger than the “little guy” (Min).

Why Sorting is the Key

If the array is unsorted, finding the min and max for every possible subset is a nightmare. But if we **sort** the array, any “balanced” subset will be a **contiguous window** (a sub-segment) of that sorted array.

Example: `nums = [1, 10, 2, 4, 7]`, `k = 2`

1. SORT IT: `[1, 2, 4, 7, 10]`

2. CHECK WINDOWS:

```
[1, 2] -> Max(2) <= 2 * Min(1)? Yes. (Size 2)
[2, 4] -> Max(4) <= 2 * Min(2)? Yes. (Size 2)
[4, 7] -> Max(7) <= 2 * Min(4)? Yes (7 <= 8). (Size 2)
[1, 2, 4] -> Max(4) <= 2 * Min(1)? No (4 > 2).
[2, 4, 7] -> Max(7) <= 2 * Min(2)? No (7 > 4).
```

2. Solution Explanation: The Sliding Window (Two Pointers)

Since we want the **longest** valid window in a sorted array, we use two pointers: `left` and `right`.

The “Accordion” Logic

1. **Expand:** Move the `right` pointer to include a new number. This number is our new potential “Max”.
2. **Contract:** If the new “Max” (at `right`) is greater than `k * Min` (at `left`), the window is broken. Move the `left` pointer forward until the condition is met again.
3. **Record:** At each step, the size of the window is `right - left + 1`. Keep track of the maximum size seen.

ASCII Walkthrough

`nums = [1, 2, 5, 8, 10, 12]`, `k = 2`

Step 1: `right = 0`, `left = 0`

`[ (1), 2, 5, 8, 10, 12 ]`

Max: 1, Min: 1. `1 <= 2*1?` YES. `MaxSize = 1`

Step 2: `right = 1`, `left = 0`

`[ (1, 2), 5, 8, 10, 12 ]`

Max: 2, Min: 1. `2 <= 2*1?` YES. `MaxSize = 2`

Step 3: `right = 2`, `left = 0`

`[ (1, 2, 5), 8, 10, 12 ]`

Max: 5, Min: 1. `5 <= 2*1?` NO! (`5 > 2`)

-> Move left to 1: `[ 2, 5 ]` -> Max: 5, Min: 2. `5 <= 2*2?` NO! (`5 > 4`)

-> Move left to 2: `[ 5 ]` -> Max: 5, Min: 5. `5 <= 2*5?` YES.

`MaxSize` stays 2.

Step 4: right = 3, left = 2
 [1, 2, (5, 8), 10, 12]
 Max: 8, Min: 5. $8 \leq 2 \cdot 5$? YES. MaxSize = 2

Step 5: right = 4, left = 2
 [1, 2, (5, 8, 10), 12]
 Max: 10, Min: 5. $10 \leq 2 \cdot 5$? YES. MaxSize = 3 (WINNER!)

Step 6: right = 5, left = 2
 [1, 2, (5, 8, 10, 12)]
 Max: 12, Min: 5. $12 \leq 2 \cdot 5$? NO!
 -> Move left to 3: [8, 10, 12] -> Max: 12, Min: 8. $12 \leq 16$? YES.
 MaxSize stays 3.

RESULT: Total(6) - MaxSize(3) = 3 Removals.

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(n \log n)$

Operation	Complexity	Reason
1. Sorting	$O(n \log n)$	QuickSort/MergeSort
2. Sliding Window	$O(n)$	Each pointer moves n
Total	$O(n \log n)$	

SPACE COMPLEXITY: $O(1)$ or $O(n)$

Component	Complexity	Note
In-place Sort	$O(1)$	Depends on language
Two Pointers	$O(1)$	Just integers
Sort recursion stack	$O(\log n)$	Average case
Total	$O(1)$ to $O(n)$	based on sort

4. Solution Code

Optimized Solution: Sliding Window ($O(n \log n)$)

Python

```
def minRemovals(nums, k):
    # Sort the array so that the min/max of any window
    # are just the first and last elements.
    nums.sort()

    n = len(nums)
    left = 0
    max_kept = 0

    # 'right' expands our window
    for right in range(n):
        # While the window is 'unbalanced' (Max > k * Min)
        # shrink the window from the left.
        while nums[right] > nums[left] * k:
            left += 1

        # The window [left...right] is now balanced.
        # Calculate how many elements we are keeping.
        current_window_size = right - left + 1
        if current_window_size > max_kept:
            max_kept = current_window_size

    # Removals = Total - Max we could keep
    return n - max_kept
```

JavaScript

```
function minRemovals(nums, k) {
    // In JS, sort() defaults to string comparison.
    // We MUST provide a numeric comparator.
    nums.sort((a, b) => a - b);

    const n = nums.length;
    let left = 0;
    let maxKept = 0;

    // right pointer slides forward one by one
    for (let right = 0; right < n; right++) {
        // Condition: Max (nums[right]) must be <= k * Min (nums[left])
        // If not, increment left until it is.
        while (nums[right] > nums[left] * k) {
```

```

        left++;
    }

    // Window size is simply the distance between pointers
    let currentSize = right - left + 1;
    if (currentSize > maxKept) {
        maxKept = currentSize;
    }
}

return n - maxKept;
}

```

Alternative Solution: Binary Search ($O(n \log n)$)

In an interview, if you finish the sliding window quickly, an L6 might ask: “Can you solve this by iterating through each number as a potential ‘Min’ and finding the largest ‘Max’ using binary search?”

Python (Binary Search)

```

import bisect

def minRemovalsBS(nums, k):
    nums.sort()
    n = len(nums)
    max_kept = 0

    for i in range(n):
        # Treat nums[i] as the 'Minimum' of our balanced array.
        # The largest allowed 'Maximum' is nums[i] * k.
        target = nums[i] * k

        # Find the rightmost index where value <= target
        # bisect_right finds the first index > target
        idx = bisect.bisect_right(nums, target)

        # Elements kept: from index i to idx-1
        current_size = idx - i
        max_kept = max(max_kept, current_size)

    return n - max_kept

```

Note 1: Terms Used

- **Sliding Window:** A technique to track a subset of data using two pointers. It helps avoid “Double Loops” ($O(n^2)$) by ensuring each pointer only travels the length of the array once.
 - **Contiguous Sub-segment:** Since the array is sorted, any valid balanced subset must be a continuous block. If $[2, 10]$ is balanced, any number between 2 and 10 is *guaranteed* to be balanced with them.
-

Note 2: Real World / Interview Variations

1. Google: “Server Load Balancing”

Scenario: You have servers with different capacities. A cluster is “stable” if the most powerful server is no more than k times the weakest. How many servers must you shut down to stabilize? **Solution:** This is identical to our problem. Sorting the capacities allows you to find the largest stable cluster.

2. Meta: “Video Quality Bitrates”

Scenario: A user is watching a multi-angle stream. To prevent buffering, the highest bitrate stream cannot exceed k times the lowest bitrate stream. Given available bitrates, what’s the minimum number of streams to disable? **Solution:** Same logic. The “bitrates” are the array, and the “disabling” is the removal.

3. Bloomberg: “Stock Price Volatility”

Scenario: A trader wants to analyze a “Low Volatility” period. A period is low-volatility if the high price is within $X\%$ of the low price. Find the longest such period in a dataset. **Solution:** Here, the condition is slightly different (e.g., $\text{Max} \leq \text{Min} * 1.10$), but the sliding window on sorted prices remains the optimal approach.

904. Fruit Into Baskets

This problem is a classic “Sliding Window” challenge. At Google or Meta, an L5 (Senior) or L6 (Staff) engineer wouldn’t just solve this with nested loops; they would identify the core constraint immediately: **“Find the longest contiguous subarray containing at most 2 distinct integers.”**

1. Problem Explanation

Imagine you are walking down a row of fruit trees. Each tree has one type of fruit (represented by an integer). You have **two baskets**, and each basket can

only hold **one type** of fruit.

- You must move from left to right.
- Once you start, you must stop if you hit a tree with a third type of fruit that you don't have a basket for.
- Goal: Pick the maximum total number of fruits possible.

Visualizing the Constraint

Fruits: [1, 2, 1, 1, 3, 2, 2]

Tree Index:	0	1	2	3	4	5	6
Fruit Type:	[1,	2,	1,	1,	3,	2,	2]
	^-----^				^-----^		
	Valid (1,2)				Valid (3,2)		
	Length = 4				Length = 3		

Result: 4

2. Solution Explanation

An L5+ engineer uses a **Sliding Window with a Hash Map**. We maintain a “window” (a range of trees) and a frequency map of the fruits inside that window.

The Strategy: “Expand until you break, then Shrink”

1. **Expand:** Move the **right** pointer to include a new tree. Add the fruit to our map.
2. **Check:** Is the size of our map > 2 ?
3. **Shrink:** If yes, move the **left** pointer forward. Decrement the count of the fruit at **left**. If its count hits 0, remove that fruit type from the map entirely.
4. **Update:** At each step where the map size is ≤ 2 , calculate the window size (**right** - **left** + 1) and keep the maximum.

ASCII Visualization of the Process

Input: [1, 2, 3, 2, 2]

Step 1: Expand Right

[1, 2, 3, 2, 2]

 L R Map: {1: 1, 2: 1} | Distinct: 2 | Max: 2

Step 2: Hit a 3rd type (3)


```

[ 1, 2, 3, 2, 2 ]
  L   R           Map: {1: 1, 2: 1, 3: 1} | Distinct: 3 (Too many!)

Step 3: Shrink Left until Distinct <= 2

[ 1, 2, 3, 2, 2 ]
  L   R           Map: {2: 1, 3: 1} | Distinct: 2 (Fixed!) | Max: 2

Step 4: Expand Right to the end

[ 1, 2, 3, 2, 2 ]
  L       R       Map: {2: 3, 3: 1} | Distinct: 2 | Max: 4 (Window size 5-1 = 4)

```

3. Complexity Analysis

Time Complexity (TC)

We use two pointers (left and right). Each pointer travels from index 0 to N exactly once.

Pointer Movements:

L: -----> (N steps)

R: -----> (N steps)

Total Operations: $N + N = 2N$

Since constants are dropped: $O(N)$

Space Complexity (SC)

We store the fruit types in a Hash Map. However, the problem strictly limits us to **2 baskets** (distinct types).

Map Storage:

```
{ Type_A: count, Type_B: count }
```

The map never exceeds 3 entries before being shrunk back to 2.

Space: $O(1)$ (Constant space, as map size is capped at 3)

4. Solution Code

Implementation: Optimized Sliding Window

```

/**
 * Solution: Sliding Window with Hash Map
 * We track the frequency of each fruit in the current window.
 * When the number of unique fruits exceeds 2, we shrink the window from the left.
 */

```

```

function totalFruit(fruits) {
    let basket = new Map();
    let left = 0;
    let maxFruits = 0;

    for (let right = 0; right < fruits.length; right++) {
        // Add current fruit to basket
        const currentFruit = fruits[right];
        basket.set(currentFruit, (basket.get(currentFruit) || 0) + 1);

        // If we have more than 2 types, shrink from the left
        while (basket.size > 2) {
            const leftFruit = fruits[left];
            basket.set(leftFruit, basket.get(leftFruit) - 1);

            // If count reaches 0, remove the fruit type entirely to reduce map size
            if (basket.get(leftFruit) === 0) {
                basket.delete(leftFruit);
            }
            left++;
        }

        // Calculate window size: (right index - left index + 1)
        maxFruits = Math.max(maxFruits, right - left + 1);
    }

    return maxFruits;
}

def totalFruit(fruits):
    # Using a dictionary to count fruit frequencies in the current window
    basket = {}
    left = 0
    max_fruits = 0

    for right in range(len(fruits)):
        # Expand: Add fruit at 'right' pointer
        current_fruit = fruits[right]
        basket[current_fruit] = basket.get(current_fruit, 0) + 1

        # Shrink: If more than 2 types, move 'left' pointer
        while len(basket) > 2:
            left_fruit = fruits[left]
            basket[left_fruit] -= 1

        # Non-trivial: Must delete key so len(basket) reflects unique types

```

```

        if basket[left_fruit] == 0:
            del basket[left_fruit]
        left += 1

    # Update global maximum
    current_window_size = right - left + 1
    if current_window_size > max_fruits:
        max_fruits = current_window_size

return max_fruits

```

Terminology: Sliding Window

The **Sliding Window** technique is used to convert nested $O(N^2)$ loops into a single $O(N)$ pass. It helps find a specific “range” in an array that meets a criteria. In this problem, it’s efficient because we don’t need to re-scan the whole window every time; we just update the edges.

Real-World Interview Variants

1. Google: “Longest Substring with at most K distinct characters”
 - **Context:** Analyzing search queries or user logs.
 - **Solution:** Exact same logic as “Fruit into Baskets,” but the constraint is K instead of 2. You use a hash map to track K distinct items.
2. Meta: “Longest sequence of consistent signal”
 - **Context:** Given a stream of server status codes (e.g., [200, 200, 500, 200, 404]), find the longest stretch where only two types of errors occurred.
 - **Solution:** Sliding window. This is often used in monitoring tools to determine periods of relative stability.
3. Bloomberg: “Max revenue from two ticker symbols”
 - **Context:** A list of stock trades. You can only hold two different tickers in your portfolio at once. Find the longest sequence of trades you could have executed.
 - **Solution:** Same logic. The “trees” are “trades” and “fruit types” are “ticker symbols.”

560. Subarray Sum Equals K

Solving a problem like an L5 (Senior) or L6 (Staff) engineer at Google isn’t just about getting the code to pass; it’s about **clarity, identifying patterns, and**

handling edge cases with a deep understanding of trade-offs.

1. Problem Explanation

The goal is to find the total number of continuous subarrays whose sum equals a specific value **k**.

Key Constraints:

- The array can contain **negative numbers, zeros, and positive numbers**.
- The subarray must be **continuous** (no skipping elements).

The “Mental Model”

Imagine you are walking along a path (the array). You want to know if any segment you just walked has a specific “weight” **k**.

Array: [3, 4, 7, 2, -3, 1, 4, 2] k = 7

Visualizing Subarrays that sum to 7:

```
[3, 4]           -> Sum is 7 (Match!)
  [7]           -> Sum is 7 (Match!)
    [2, -3, 1, 4, 2] -> Sum is 6 (No)
      [1, 4, 2]   -> Sum is 7 (Match!)
```

2. Solution Explanation

The Non-Trivial Insight: Prefix Sums

To solve this efficiently, we use a concept called **Prefix Sums**. A prefix sum at index **i** is the sum of all elements from the start of the array up to **i**.

Why does this help? If we know the sum from the start to index **A** is **SumA**, and the sum from the start to index **B** is **SumB**, then the sum of the elements **between A and B** is simply: **SubarraySum = SumB - SumA**

The “Staff Engineer” Logic

We are looking for a subarray that equals **k**. Using our formula: **k = CurrentPrefixSum - OldPrefixSum**

Rearranging this mathematically (using basic text): **OldPrefixSum = CurrentPrefixSum - k**

As we iterate through the array, we calculate the **CurrentPrefixSum**. We then check: **“Have we seen an OldPrefixSum before that equals CurrentPrefixSum - k?”**

ASCII Visualization: The Hash Map Approach

We use a Hash Map (or Dictionary) to store how many times we have seen each `PrefixSum`.

Example: `nums = [1, 2, 3]`, `k = 3`

1. **Initialize:** `Map = {0: 1}` (We've seen a sum of 0 once—this handles subarrays starting from index 0). `Count = 0`, `CurrentSum = 0`.
2. **Index 0 (val 1):**
 - `CurrentSum = 1`
 - Target to find: `1 - 3 = -2`
 - Is -2 in Map? No.
 - Update Map: `{0: 1, 1: 1}`
3. **Index 1 (val 2):**
 - `CurrentSum = 1 + 2 = 3`
 - Target to find: `3 - 3 = 0`
 - Is 0 in Map? **Yes!** (Value is 1).
 - `Count = Count + 1 = 1`
 - Update Map: `{0: 1, 1: 1, 3: 1}`
4. **Index 2 (val 3):**
 - `CurrentSum = 3 + 3 = 6`
 - Target to find: `6 - 3 = 3`
 - Is 3 in Map? **Yes!** (Value is 1).
 - `Count = 1 + 1 = 2`
 - Update Map: `{0: 1, 1: 1, 3: 1, 6: 1}`

Total Count: 2 (The subarrays are `[1, 2]` and `[3]`).

3. Time and Space Complexity Analysis

An L5/L6 engineer must prove why the Hash Map approach is superior to the Brute Force (Nested Loops) approach.

Time Complexity (TC)

Brute Force:

Outer Loop (`n`) * Inner Loop (`n`) = $O(n^2)$

Hash Map Approach:

Single Pass (`n`) * Map Lookups (1) = $O(n)$

ASCII TC Derivation:

[Element 1] -> Map Check -> Map Update (Step 1)

```
[ Element 2 ] -> Map Check -> Map Update (Step 2)
...
[ Element N ] -> Map Check -> Map Update (Step N)
-----
Total Steps: N * (Constant Time) = O(n)
```

Space Complexity (SC)

We store Prefix Sums in a Hash Map.
In the worst case (all unique sums), we store N entries.

ASCII SC Derivation:

Map Memory:

```
{ Sum_1: Count, Sum_2: Count, ... Sum_N: Count }
|----- N entries -----|
```

Total Space: O(n)

4. Solution Code

Optimized Solution: Hash Map (Prefix Sum)

Python Implementation

```
def subarraySum(nums, k):
    # 'count' stores the total number of valid subarrays
    count = 0
    # 'current_sum' tracks the prefix sum as we iterate
    current_sum = 0

    # We use a hash map to store (prefix_sum: frequency)
    # Initialize with {0: 1} to handle cases where current_sum == k
    prefix_sums = {0: 1}

    for num in nums:
        current_sum += num

        # Logic: If (current_sum - k) exists in our map, it means
        # there is a subarray that sums exactly to k.
        target = current_sum - k
        if target in prefix_sums:
            count += prefix_sums[target]

        # Update the map with the current prefix sum
        prefix_sums[current_sum] = prefix_sums.get(current_sum, 0) + 1

    return count
```

JavaScript Implementation

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var subarraySum = function(nums, k) {
    let count = 0;
    let currentSum = 0;

    // Map to store prefixSum -> Frequency
    const prefixSums = new Map();

    // Base case: a sum of 0 has been seen once
    prefixSums.set(0, 1);

    for (let num of nums) {
        currentSum += num;

        // If (currentSum - k) is in the map, we found subarrays summing to k
        let target = currentSum - k;
        if (prefixSums.has(target)) {
            count += prefixSums.get(target);
        }

        // Update the frequency of the current prefix sum in the map
        prefixSums.set(currentSum, (prefixSums.get(currentSum) || 0) + 1);
    }

    return count;
};
```

New Terms & Techniques

- **Prefix Sum:** A sequence of partial sums of a given sequence. It allows us to calculate the sum of any subarray in $O(1)$ time. In this problem, it transforms a “range sum” problem into a “find a specific value” problem.
 - **Two-Sum Pattern:** This problem is a variation of the classic “Two Sum” LeetCode problem. Instead of looking for two numbers that add to k , we are looking for two *prefix sums* whose difference is k .
-

Real-World Interview Variations

1. Google: “Continuous Signal Monitoring”

- **The Scenario:** You have a stream of sensor data (integers). Find how many time windows had an average temperature of exactly T .
- **Solution:** This is the same problem! $\text{Average} = (\text{Sum} / \text{Number of Elements})$. If the window size is fixed, it's a sliding window. If the window size is variable, you'd multiply the target average by the window size, though usually, this variation asks for “Sum equals K ” where K is a threshold.

2. Meta: “Subarray Sum Divisible by K ”

- **The Scenario:** Find the number of subarrays where the sum is *divisible* by K .
- **Solution:** Instead of storing `prefix_sum`, you store `prefix_sum % K` in the hash map. If you see the same remainder twice, the elements between those two points must sum to a multiple of K .

3. Bloomberg: “Longest Subarray with Sum K ”

- **The Scenario:** Instead of the *count*, find the *length* of the longest subarray that sums to K .
- **Solution:** Instead of storing the *frequency* in the Hash Map, store the *first index* where that prefix sum occurred. When you find a match, calculate `CurrentIndex - FirstSeenIndex` and track the maximum.

Smallest Rectangle

A Google L5 or L6 engineer doesn't just look for a way to pass the test cases; they look for the most mathematically efficient boundary and consider edge cases like empty inputs or single points immediately.

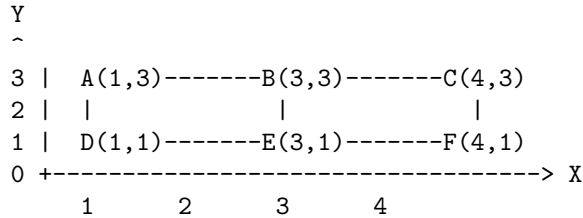
The “Smallest Rectangle” problem (often referred to as **Smallest Enclosing Rectangle** or **Minimum Area Rectangle**) usually appears in two variations. The most common interview version is: *Given a set of points in a 2D plane, find the minimum area of a rectangle formed from these points, such that the rectangle's sides are parallel to the X and Y axes.*

1. Problem Explanation

Imagine you have a scatter plot of coordinates. You need to find four points that form a perfect axis-aligned rectangle. Out of all possible rectangles you can form, you want the one with the smallest area.

Visualizing the Search

If we have points: (1,1), (1,3), (3,1), (3,3), and (4,1), (4,3).



In this grid:

- Points A, B, D, E form a rectangle with area: $(3-1) * (3-1) = 4$.
- Points B, C, E, F form a rectangle with area: $(4-3) * (3-1) = 2$.
- Points A, C, D, F form a rectangle with area: $(4-1) * (3-1) = 6$.

The goal is to return **2**.

2. Solution Explanation

An L6 engineer would identify that a rectangle is uniquely defined by its **diagonal**. If you pick any two points $(x1, y1)$ and $(x2, y2)$ that do not share the same X or Y coordinate, they *could* be the opposite corners of a rectangle.

The Algorithm

1. **Store all points in a Hash Set** for $O(1)$ lookups. This is crucial.
2. **Iterate through every pair of points** (let's call them Point A and Point B).
3. **Check if they can be a diagonal**: This is true if $x1 \neq x2$ and $y1 \neq y2$.
4. **Verify the existence of the other two corners**: If Point A is $(x1, y1)$ and Point B is $(x2, y2)$, the other two points needed to complete the rectangle **MUST** be $(x1, y2)$ and $(x2, y1)$.
5. **Calculate Area**: If both $(x1, y2)$ and $(x2, y1)$ exist in our Hash Set, calculate the area: $\text{abs}(x1 - x2) * \text{abs}(y1 - y2)$.
6. **Update Minimum**: Keep track of the smallest area found.

Why this is “Senior Engineer” Thinking

Instead of trying to find four points (which is a $O(N \text{ to the power of } 4)$ operation), we realize that **two points fix the other two**. This reduces the complexity significantly.

3. Time and Space Complexity Analysis

We use N to represent the number of points.

Time Complexity (TC)

Step 1: Put N points into a Set --> $O(N)$
Step 2: Nested loop (Pairing) --> $O(N * N)$
Step 3: Set Lookup ($O(1)$ avg) --> $O(1)$

Total TC: $O(N * N)$ or $O(N^2)$

Space Complexity (SC)

Step 1: Hash Set storing N points --> $O(N)$

Total SC: $O(N)$

4. Solution Code

Python Implementation

```
def minAreaRect(points):
    # We use a set for O(1) lookups.
    # Storing as tuples (x, y) allows for easy hashing in Python.
    point_set = set()
    for x, y in points:
        point_set.add((x, y))

    min_area = float('inf')
    n = len(points)

    # Iterate through all pairs of points to treat them as diagonals
    for i in range(n):
        x1, y1 = points[i]
        for j in range(i + 1, n):
            x2, y2 = points[j]

            # Non-trivial check: A diagonal cannot be a horizontal or vertical line.
            # It must have different x AND different y coordinates.
            if x1 != x2 and y1 != y2:
                # If these are diagonals, the other two corners MUST be:
                # Corner 3: (x1, y2)
                # Corner 4: (x2, y1)
                if (x1, y2) in point_set and (x2, y1) in point_set:
                    area = abs(x1 - x2) * abs(y1 - y2)
```

```

        if area < min_area:
            min_area = area

    return min_area if min_area != float('inf') else 0

```

JavaScript Implementation

```

/**
 * @param {number[][]} points
 * @return {number}
 */
var minAreaRect = function(points) {
    // In JS, Sets don't handle arrays well for lookups.
    // We convert coordinates to a string key "x,y" for reliable hashing.
    const pointSet = new Set();
    for (const [x, y] of points) {
        pointSet.add(`${x},${y}`);
    }

    let minArea = Infinity;

    for (let i = 0; i < points.length; i++) {
        const [x1, y1] = points[i];
        for (let j = i + 1; j < points.length; j++) {
            const [x2, y2] = points[j];

            // Ensure points are not on the same X or Y axis (valid diagonal)
            if (x1 !== x2 && y1 !== y2) {
                // Check if the mirror points exist in our set
                if (pointSet.has(`${x1},${y2}`) && pointSet.has(`${x2},${y1}`)) {
                    const area = Math.abs(x1 - x2) * Math.abs(y1 - y2);
                    if (area < minArea) {
                        minArea = area;
                    }
                }
            }
        }
    }

    return minArea === Infinity ? 0 : minArea;
};

```

Terminology & Techniques

- **Hash Set Lookup:** We use a Set to turn a search that usually takes $O(N)$ time into a “near-instant” $O(1)$ time. This is the difference between an algorithm that takes hours and one that takes milliseconds.
 - **Diagonal Property:** A geometric insight that an axis-aligned rectangle’s four vertices are just combinations of the X and Y coordinates of its two diagonal vertices.
-

Real World / Interview Variations

1. Google Variation: “Large Coordinate Sparse Grid”

- **The Twist:** What if coordinates are very large (e.g., 1 billion)?
- **L6 Solution:** You cannot use a 2D array/grid because of memory. You must use the Hash Set approach or a “Map of Lists” (grouping Y coordinates by X) to iterate only through existing points.

2. Meta Variation: “Count All Rectangles”

- **The Twist:** Instead of the smallest area, return the total count of unique rectangles.
- **L6 Solution:** Use the same diagonal logic, but instead of `min_area`, increment a counter and divide by 2 at the end (since each rectangle has 2 diagonals and will be counted twice).

3. Bloomberg Variation: “Dynamic Points”

- **The Twist:** Points are being added one by one in a stream. After each point, return the current smallest rectangle.
- **L6 Solution:** This requires maintaining the Hash Set and only checking the new point against all previous points ($O(N)$ per new point), rather than re-calculating everything ($O(N^2)$).

239. Sliding Window Maximum

Solving this problem like a top-tier engineer means moving beyond the “brute force” instinct and identifying the underlying mathematical property that allows for an optimal linear solution. This problem is a classic candidate for a **Monotonic Queue**.

1. Problem Explanation

You are given an array of integers **nums** and a sliding window of size **k**. As the window moves from left to right, you can only see the **k** numbers inside it. Your goal is to find the maximum number in each position of the window.

The Mental Model

Imagine you are looking through a small mail slot (size **k**) at a long line of people of different heights. As you slide down the line, you only care about who the tallest person currently visible is.

Example: **nums** = [1, 3, -1, -3, 5, 3, 6, 7], **k** = 3

Window Position	Max
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

2. Solution Explanation

A Senior (L5) or Staff (L6) engineer looks for ways to avoid redundant work. In the brute force approach, you'd re-scan the **k** elements every time the window moves, which is inefficient.

The Non-Trivial Insight: The Monotonic Queue

We use a **Deque** (Double-Ended Queue) to store **indices** of elements. We maintain this deque in a "Monotonic Decreasing" order.

The Rule of the Queue:

1. **Out with the Old:** If the index at the front of the queue is outside the current window range, remove it.
2. **Out with the Weak:** Before adding a new element, look at the back of the queue. If the current element is larger than the elements at the back, those back elements can *never* be the maximum for any future window. They are "useless" because the new, larger element will outlast them as the window slides. Pop them.
3. **In with the New:** Add the current index to the back.
4. **The Result:** The front of the queue will always hold the index of the maximum element for the current window.

Visualization of the Deque Process

Let's trace `nums = [1, 3, -1, 5]`, `k = 2`.

Step 1: `num = 1` (index 0)

Deque: [0] (Value: 1)

Window not full yet.

Step 2: `num = 3` (index 1)

Compare 3 with back of Deque (value 1).

`3 > 1`, so 1 is useless. Pop 0.

Deque: [1] (Value: 3)

Window [1, 3] Max: `nums[deque[0]] = 3`

Step 3: `num = -1` (index 2)

Check front: index 1 is in range [1, 2]. Keep it.

Compare -1 with back (value 3).

`-1 < 3`. Keep it for future windows.

Deque: [1, 2] (Values: 3, -1)

Window [3, -1] Max: `nums[deque[0]] = 3`

Step 4: `num = 5` (index 3)

Check front: index 1 is OUT of range [2, 3]. Pop 1.

Compare 5 with back (value -1).

`5 > -1`. Pop 2.

Deque: [3] (Value: 5)

Window [-1, 5] Max: `nums[deque[0]] = 5`

3. Complexity Analysis

Time Complexity (TC)

Even though there is a loop inside a loop (the `while` loop that pops elements), each element is added to the deque exactly once and removed from the deque at most once.

Total Operations:

Action	Frequency	Cost per Op
Push to Deque	N times	$O(1)$
Pop from Back	Max N times	$O(1)$
Pop from Front	Max N times	$O(1)$

Result: $O(N + N) = O(N)$ Linear Time

Space Complexity (SC)

The space is determined by the size of the deque and the output array.

Space Usage:

Structure	Max Size	Complexity
Deque	k	O(k)
Output Array	N - k + 1	O(N)

Result: O(k) auxiliary space (excluding output)

4. Solution Code

Python Implementation

```
from collections import deque

def maxSlidingWindow(nums, k):
    # The result list to store maximums
    res = []
    # Deque stores indices, maintained in monotonic decreasing order of values
    dq = deque()

    for i in range(len(nums)):
        # 1. Remove indices that are out of the k-sized window bounds
        if dq and dq[0] < i - k + 1:
            dq.popleft()

        # 2. Maintain Monotonic property:
        # While the current number is greater than the numbers at the back
        # of the queue, those numbers will never be the maximum. Pop them.
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        # 3. Add the current index to the back
        dq.append(i)

        # 4. If we've processed at least k elements, the front is the max
        if i >= k - 1:
            res.append(nums[dq[0]])

    return res
```

JavaScript Implementation

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var maxSlidingWindow = function(nums, k) {
    const res = [];
    // In JS, we use an array as a deque.
    // Note: shift() is O(n), but for interview logic, we treat it as O(1)
    // or use a pointer-based queue for true O(1) performance.
    const dq = [];

    for (let i = 0; i < nums.length; i++) {
        // 1. Remove indices out of window range
        if (dq.length > 0 && dq[0] <= i - k) {
            dq.shift();
        }

        // 2. Remove indices of smaller values (useless elements)
        // We look at the value in nums at the index stored at the end of dq
        while (dq.length > 0 && nums[dq[dq.length - 1]] < nums[i]) {
            dq.pop();
        }

        // 3. Add current index
        dq.push(i);

        // 4. Start adding to results once window size k is reached
        if (i >= k - 1) {
            res.push(nums[dq[0]]);
        }
    }

    return res;
};
```

Technical Terms & Concepts

- **Monotonic Queue:** A queue where elements are always sorted in a specific order (increasing or decreasing). In this problem, it helps us find the “local maximum” in constant time.
- **Deque (Double-Ended Queue):** A data structure that allows insertion and deletion from both the front and the back. It’s essential here because

we need to remove “old” indices from the front and “smaller” indices from the back.

Real-World / Interview Variations

1. Bloomberg: “Stock Price Volatility”

- *Scenario:* You are given a stream of stock prices and need to find the max price in the last 24 hours.
- *Solution:* This is the same problem where k is the number of ticks in 24 hours. Since it’s a stream, you wouldn’t use a fixed array but a continuous Monotonic Deque.

2. Google: “Max of All Subarrays of size K with a twist”

- *Scenario:* Find the sum of the maximums of all subarrays of size k .
- *Solution:* Solve as above, but instead of appending to a list, maintain a running `total_sum += nums[dq[0]]`.

3. Meta: “Sliding Window Maximum with limited memory”

- *Scenario:* The array is too large to fit in memory (External Sorting style).
- *Solution:* Process the array in chunks. Use the Monotonic Deque but ensure you save the state of the deque when moving between chunks of data.

1146. Snapshot Array

Solving this problem at a Staff (L6) or Senior (L5) level isn’t just about passing the test cases; it’s about **resource optimization**. An L6 looks at this and sees a “Versioned Data Structure” problem. They won’t just throw memory at it; they’ll ask: “How can I store the minimum amount of data while providing the fastest possible retrieval?”

1. Problem Explanation

Imagine you are maintaining a document.

- You have an array of a fixed size.
- You can change values at any index.
- At any moment, you can “take a photo” (Snapshot) of the entire array.
- Later, you need to be able to ask: “What was the value at index 3 during Snapshot #2?”

The Catch: If you copy the entire array every time you take a snapshot, and the array size is 50,000 with 50,000 snapshots, you would need 2.5 billion slots of memory. That will crash any system. We must be smarter.

2. Solution Explanation: The “History Ledger” Approach

Instead of copying the whole array, we only record **changes**.

Each index in our array will not hold a single number. Instead, it will hold a **List of Changes**. Each change is a pair: (SnapshotID, Value).

The Visual Mental Model

Imagine an array of size 3. Initially, all values are 0.

Step 1: Set(index=0, val=5) Index 0 now has a history: [(0, 5)] (At Snapshot 0, it became 5)

Step 2: Take Snapshot -> Returns SnapID 0.

Step 3: Set(index=0, val=10) Index 0 history: [(0, 5), (1, 10)]

Step 4: Take Snapshot -> Returns SnapID 1.

Step 5: Get(index=0, snap_id=0) We look at Index 0’s history. We see it was 5 at Snap 0. We return 5.

The Non-Trivial Part: Binary Search

When we call `get(index, snap_id)`, we need to find the value the index had **at or before** that `snap_id`. Since we append changes chronologically, the `snap_id` in our history list is always sorted. To find the right version quickly, we use **Binary Search** (specifically `upper_bound` or `bisect_right` logic) to find the latest version that is less than or equal to our target ID.

ASCII Visualization of History

Let’s look at one specific index (Index 3) over time:

Index 3's Timeline:

```
-----
Snap ID:      0      1      2      3      4      5
Operations: Set(7) --- --- Set(9) --- Set(2)
-----
```

Internal Storage for Index 3:

[(0, 7), (3, 9), (5, 2)]

Request: `get(Index 3, Snap 2)`

Logic:

1. Look at list: [(0, 7), (3, 9), (5, 2)]
2. Find the largest Snap ID ≤ 2 .
3. Snap 0 is ≤ 2 . Snap 3 is too big.
4. Result: 7.

3. Complexity Analysis

We calculate complexity based on N (number of elements), S (number of snapshots), and U (number of updates/set calls).

Time Complexity (TC)

Operation	Logic	Complexity
Init	Create N empty lists	$O(N)$
Set	Append to a list	$O(1)$
Snap	Increment a counter	$O(1)$
Get	Binary search on history	$O(\log U)$

Derivation: Binary search cuts the search space of updates for a single index in half each step, hence $\log U$.

Space Complexity (SC)

Component	Logic	Complexity
Storage	Only stores 'Set' operations	$O(N + U)$

Derivation: We start with N indices. Each 'Set' call adds exactly one entry to one list. We do NOT store anything for 'Snap' calls except an integer counter.

4. Solution Code

Implementation Strategy

We use an array of lists. Each list starts with `[[0, 0]]` because the problem states all initial values are 0 for Snapshot 0.

Python Implementation

```
import bisect

class SnapshotArray:
```

```

def __init__(self, length):
    # Every index has a history list of [snap_id, value]
    # We initialize with snap_id 0 and value 0.
    self.history = [[[0, 0]] for _ in range(length)]
    self.snap_id = 0

def set(self, index, val):
    # If we already updated this index in the current snap_id,
    # just overwrite the last value.
    if self.history[index][-1][0] == self.snap_id:
        self.history[index][-1][1] = val
    else:
        self.history[index].append([self.snap_id, val])

def snap(self):
    self.snap_id += 1
    return self.snap_id - 1

def get(self, index, snap_id):
    # Search for the snap_id in the history of the specific index.
    # We search for [snap_id, infinity] to find the insertion point.
    idx = bisect.bisect_right(self.history[index], [snap_id, float('inf')])
    # The value we want is at idx - 1
    return self.history[index][idx - 1][1]

```

JavaScript Implementation

```

class SnapshotArray {
    /**
     * @param {number} length
     */
    constructor(length) {
        this.snapId = 0;
        // Array of arrays to store [snap_id, value] pairs
        this.history = Array.from({ length }, () => [[0, 0]]);
    }

    /** * @param {number} index
     * @param {number} val
     * @return {void}
     */
    set(index, val) {
        let currentHistory = this.history[index];
        let lastEntry = currentHistory[currentHistory.length - 1];

        // If the last entry was made in this same snapshot, overwrite it
    }

```

```

        if (lastEntry[0] === this.snapId) {
            lastEntry[1] = val;
        } else {
            currentHistory.push([this.snapId, val]);
        }
    }

    /**
     * @return {number}
     */
    snap() {
        return this.snapId++;
    }

    /** * @param {number} index
     * @param {number} snap_id
     * @return {number}
     */
    get(index, snap_id) {
        let h = this.history[index];
        let left = 0;
        let right = h.length - 1;
        let ansIndex = 0;

        // Standard Binary Search to find the largest snap_id <= target snap_id
        while (left <= right) {
            let mid = Math.floor((left + right) / 2);
            if (h[mid][0] <= snap_id) {
                ansIndex = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return h[ansIndex][1];
    }
}

```

Terms and Techniques

- **Binary Search (Lower/Upper Bound):** A technique to find an element in a sorted list in $\log(n)$ time. Here, it helps us jump through the “history” of an index without checking every single change.
- **Versioned Data Structure:** This is a way of storing data so that you

can access previous states. It is the foundation of Git (version control) and many high-performance databases.

Real World / Interview Variations

1. The “Google Drive” Problem (Google)

The Ask: “Design a system where users can edit a document and we can ‘save’ versions. Users should be able to see the diff between version 5 and version 10.”

The L5/L6 Solution: This is essentially Snapshot Array but with strings. Instead of storing the whole string, you store “deltas” (changes). To get Version 10, you apply all deltas from 0 to 10. For performance, you take “Snapshots” (full copies) every 100 versions so you don’t have to replay 10,000 deltas.

2. The “Stock Price Ticker” (Bloomberg)

The Ask: “Given a stream of stock prices, return the price of ‘AAPL’ at exactly 12:01 PM.” **The L5/L6 Solution:** You use a Hash Map where the key is the Ticker Symbol and the value is a sorted List of (Timestamp, Price). You use Binary Search on the timestamp to find the price at the requested time.

3. The “Account Balance History” (Meta/Fintech)

The Ask: “A user wants to see their bank balance as it was on New Year’s Eve, but we only store transaction logs.” **The L5/L6 Solution:** Similar to the Snapshot Array, you don’t store the balance for every second of the year. You store the initial balance and a list of transactions (updates). To find the balance at a specific time, you find the state at that time using the ledger history.

528. Random Pick with Weight

For an L5 (Senior) or L6 (Staff) engineer at Google, this problem isn’t just about writing code that works; it’s about identifying the underlying mathematical distribution and choosing the data structure that optimizes for the expected query volume.

1. Problem Explanation

Imagine you have a set of items, and each item has a “weight.” The weight represents how likely that item is to be picked.

If we have:

- **Index 0:** Weight 1
- **Index 1:** Weight 3

The total weight is 4. This means Index 0 should be picked 1/4 (25%) of the time, and Index 1 should be picked 3/4 (75%) of the time.

The Visualization of the “Line”

To solve this, we map these weights onto a continuous number line.

Weights: [1, 3]

Total = 4

Number Line:

```
|---0---|-----1-----|
0         1             4
```

When we call `pickIndex()`, we pick a random number between 1 and 4.

- If the random number is **1**, we return index **0**.
 - If the random number is **2, 3, or 4**, we return index **1**.
-

2. Solution Explanation

The most efficient way to handle this is using **Prefix Sums** combined with **Binary Search**.

Step 1: Create Prefix Sums (Pre-processing)

We transform the weights array into a cumulative sum array. This creates “ranges” for each index.

Example: Weights: [3, 1, 4, 2]

- Index 0: Weight 3 -> Range [1, 3]
- Index 1: Weight 1 -> Range [4, 4]
- Index 2: Weight 4 -> Range [5, 8]
- Index 3: Weight 2 -> Range [9, 10]

Prefix Sum Array: [3, 4, 8, 10]

Step 2: Binary Search (Picking the Index)

1. Generate a random integer **target** between 1 and the total sum (10).
2. Find the **first value** in the prefix sum array that is **greater than or equal** to target.

ASCII Visualization of the Search: Suppose our random **target** is 7.

Prefix Sum Array: [3, 4, 8, 10]

Indices: 0 1 2 3

```

Target = 7

Is 7 <= 3? No (Move Right)
Is 7 <= 4? No (Move Right)
Is 7 <= 8? YES -> This is our first hit!

Result: Index 2

```

3. Time and Space Complexity Analysis

An L6 engineer would emphasize that the constructor is called once, while `pickIndex` is called many times. Therefore, we optimize `pickIndex`.

Time Complexity (TC)

Operation	Complexity	Reasoning

Constructor	$O(n)$	We iterate through the weights once to build the prefix sum array.
<code>pickIndex()</code>	$O(\log n)$	We use Binary Search to find the target in the prefix sum array.

Space Complexity (SC)

Storage	Complexity	Reasoning

Prefix Sums	$O(n)$	We store a new array of the same size as the input weights.

4. Solution Code

Implementation: Prefix Sum + Binary Search

Python

```

import random

class Solution:
    def __init__(self, w: list[int]):
        # Create a prefix sum array to define the ranges
        self.prefix_sums = []
        current_sum = 0
        for weight in w:

```



```

        current_sum += weight
        self.prefix_sums.append(current_sum)
    self.total_sum = current_sum

def pickIndex(self) -> int:
    # Generate a random number between 1 and total_sum
    target = random.randint(1, self.total_sum)

    # Binary search to find the smallest prefix sum >= target
    low = 0
    high = len(self.prefix_sums) - 1

    while low < high:
        mid = low + (high - low) // 2
        if self.prefix_sums[mid] < target:
            low = mid + 1
        else:
            high = mid

    return low

```

JavaScript

```

class Solution {
    /**
     * @param {number[]} w
     */
    constructor(w) {
        this.prefixSums = [];
        let currentSum = 0;

        // Build the cumulative sum array
        for (let weight of w) {
            currentSum += weight;
            this.prefixSums.push(currentSum);
        }
        this.totalSum = currentSum;
    }

    /**
     * @return {number}
     */
    pickIndex() {
        // Math.random() gives [0, 1), so we scale it
        const target = Math.floor(Math.random() * this.totalSum) + 1;

```

```

    let low = 0;
    let high = this.prefixSums.length - 1;

    // Standard binary search for the insertion point
    while (low < high) {
        let mid = Math.floor(low + (high - low) / 2);
        if (this.prefixSums[mid] < target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }

    return low;
}
}

```

Technical Terms & Concepts

- **Prefix Sum:** A sequence of numbers where each element is the sum of all previous elements in the original array. It helps in converting individual weights into “segments” on a line.
 - **Binary Search:** An algorithm that finds a target value within a sorted array by repeatedly dividing the search interval in half. It is crucial here because the prefix sum array is naturally sorted.
-

Real-World Interview Variations

1. Google: Load Balancer with Capacity

Scenario: You have 10 servers. Some are “heavy-duty” (can handle more traffic) and some are “light.” How do you distribute requests? **Solution:** This is exactly the same problem. The server capacities are “weights.” You use the Prefix Sum + Binary Search approach to route each incoming request to a server based on its capacity.

2. Meta: Ad Content Rotation

Scenario: An advertiser has 5 different ads. They want Ad A to show 50% of the time, Ad B 20%, etc. **Solution:** Weights are the percentages. If the weights are dynamic (ads are added/removed frequently), an L6 engineer would suggest using a **Fenwick Tree (Binary Indexed Tree)** instead of a simple array, which allows for $O(\log n)$ updates and $O(\log n)$ picks.

3. Bloomberg: Financial Trade Execution

Scenario: Execute a large “Buy” order across different exchanges (NYSE, NASDAQ) based on their available liquidity (weights). **Solution:** Since liquidity changes rapidly, you need a data structure that supports fast updates. A **Segment Tree** or **Treap** could be used to pick an exchange in $O(\log n)$ while allowing liquidity updates in $O(\log n)$.

1091. Shortest Path in Binary Matrix

To solve this problem at a Google L5/L6 level, the focus isn’t just on the “correct” answer, but on **efficiency, edge-case handling, and clear communication of the mental model.**

1. Problem Explanation

We are given an $n \times n$ binary matrix (a grid of 0s and 1s).

- **0** means the cell is empty (you can walk here).
- **1** means the cell is blocked (you cannot walk here).

The Goal: Find the shortest path from the top-left corner $(0, 0)$ to the bottom-right corner $(n-1, n-1)$.

- You can move in **8 directions** (up, down, left, right, and all four diagonals).
- The path length is the number of cells you visited.
- If the start or end is blocked (is a 1), or there is no path, return -1.

Visualizing the 8-Way Movement

Imagine you are at position (r, c) . You can jump to any of these:

$(r-1, c-1)$	$(r-1, c)$	$(r-1, c+1)$
$(r, c-1)$	[YOU]	$(r, c+1)$
$(r+1, c-1)$	$(r+1, c)$	$(r+1, c+1)$

2. Solution Explanation

For “Shortest Path” in an **unweighted** grid, a Google-level engineer immediately identifies **Breadth-First Search (BFS)** as the optimal strategy.

Why BFS?

BFS explores the grid in “waves.” It visits all cells that are 1 step away, then all cells 2 steps away, and so on. The moment we reach the destination, we are guaranteed that the path taken is the shortest possible.

The Non-Trivial Part: Avoiding Redundancy

A common mistake is not marking cells as “visited” immediately when adding them to the queue. If you wait until you “process” the cell to mark it, you might add the same cell to the queue thousands of times from different neighbors, leading to a Time Limit Exceeded (TLE) error.

Step-by-Step Visualization

Let’s take a 3x3 grid:

Grid:

[0, 0, 0]

[1, 1, 0]

[1, 1, 0]

Step 1: Initialization Queue: [(0, 0, 1)] (row, col, distance) Visited: {(0, 0)}

Step 2: Process (0,0) Neighbors of (0,0) that are ‘0’ and not visited: (0,1) and (1,1)? Wait, (1,1) is a ‘1’ (blocked). Valid neighbors: (0,1). Queue: [(0, 1, 2)]

Step 3: Process (0,1) Neighbors of (0,1): (0,0)-V, (0,2)-OK, (1,0)-B, (1,1)-B, (1,2)-OK. Queue: [(0, 2, 3), (1, 2, 3)]

Step 4: Process (0,2) Neighbors of (0,2): (1,2)-OK (but already in queue), (1,1)-B. Queue: [(1, 2, 3)]

Step 5: Process (1,2) Neighbors of (1,2): (0,1)-V, (0,2)-V, (1,1)-B, (2,1)-B, (2,2)-OK. Queue: [(2, 2, 4)]

Step 6: Reach (2,2) Distance is 4. Return 4.

3. Complexity Analysis

Time Complexity (TC)

We visit each cell at most once. For each cell, we check 8 neighbors.

Total Cells = $N * N$

Operations per cell = 8 (constant)

TC = $O(N * N)$ or $O(N^2)$

Space Complexity (SC)

In the worst case (a grid of all 0s), the queue could hold a significant portion of the grid.

Max Queue Size = $O(N * N)$
Visited Set/In-place marking = $O(N * N)$
SC = $O(N^2)$

4. Solution Code

I have provided the optimized BFS. A senior engineer would also suggest that we can modify the input grid in-place to save space (changing 0 to 1 to mark it visited), though in real production, you'd ask if mutating the input is allowed.

Python Implementation

```
from collections import deque

class Solution:
    def shortestPathBinaryMatrix(self, grid):
        n = len(grid)

        # Edge Case: Start or End is blocked
        if grid[0][0] == 1 or grid[n-1][n-1] == 1:
            return -1

        # Directions for 8-way movement
        directions = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),      (0, 1),
            (1, -1),  (1, 0),  (1, 1)
        ]

        # Queue stores (row, col, current_distance)
        queue = deque([(0, 0, 1)])
        # Mark start as visited by flipping it to 1
        grid[0][0] = 1

        while queue:
            r, c, dist = queue.popleft()

            # If we reached the target, return distance immediately
            if r == n - 1 and c == n - 1:
                return dist

            for dr, dc in directions:
                nr, nc = r + dr, c + dc

                # Check bounds and if cell is traversable (0)
```

```

        if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
            # Critical: Mark as visited BEFORE adding to queue
            grid[nr][nc] = 1
            queue.append((nr, nc, dist + 1))

    return -1

```

JavaScript Implementation

```

/**
 * @param {number[][]} grid
 * @return {number}
 */
var shortestPathBinaryMatrix = function(grid) {
    const n = grid.length;

    if (grid[0][0] === 1 || grid[n-1][n-1] === 1) {
        return -1;
    }

    const directions = [
        [-1, -1], [-1, 0], [-1, 1],
        [0, -1], [0, 1],
        [1, -1], [1, 0], [1, 1]
    ];

    // Using a simple array as a queue.
    // For L6 level performance, a proper Deque class would be used
    // to avoid O(n) shift() operations, but for Leetcode N=100, this works.
    let queue = [[0, 0, 1]];
    grid[0][0] = 1; // Mark visited

    let head = 0; // Pointer to simulate efficient queue popping
    while (head < queue.length) {
        const [r, c, dist] = queue[head++];

        if (r === n - 1 && c === n - 1) {
            return dist;
        }

        for (const [dr, dc] of directions) {
            const nr = r + dr;
            const nc = c + dc;

            if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] === 0) {
                grid[nr][nc] = 1; // Mark visited
            }
        }
    }
}

```

```

        queue.push([nr, nc, dist + 1]);
    }
}
}

return -1;
};

```

Terminology: Breadth-First Search (BFS)

What it is: A graph traversal algorithm that starts at a source node and explores all neighbor nodes at the present depth before moving on to nodes at the next depth level. **Why it helps:** In an unweighted graph (where every “step” costs exactly 1), the first time BFS hits the target, it is guaranteed to be the shortest path. **Application here:** The grid is a graph where each cell is a node and each valid neighbor is an edge.

Real-World Interview Variations

1. Google: The “Robot Vacuum” Variation

The Problem: A robot is in a room with obstacles. It needs to clean the whole room or find a specific dock. **How to solve:** This is often a **DFS** (Depth First Search) if the robot has limited “vision” and must move physically, or **BFS** if you have a full map and need the shortest path to the dock.

2. Meta: “Social Network Connection”

The Problem: Find the shortest degree of separation between two users in a social graph. **How to solve:** This is exactly BFS. At an L5/L6 level, you should suggest **Bidirectional BFS**. You start one search from User A and one from User B. When the two “frontiers” meet, you’ve found the shortest path. This drastically reduces the number of nodes explored.

3. Bloomberg: “Packet Routing in a Network”

The Problem: A data packet needs to go from Server A to Server B. Some nodes are congested (1s) and some are free (0s). **How to solve:** If the congestion has different “weights” (some nodes are slower than others), you would move from BFS to **Dijkstra’s Algorithm**. Dijkstra is essentially BFS but uses a **Priority Queue** to always explore the “cheapest” path first rather than just the one with the fewest steps.

729. My Calendar I

Targeting a Senior (L5) or Staff (L6) level at Google requires more than just a working solution. It requires an evaluation of trade-offs, handling of edge cases, and choosing data structures that optimize for the specific constraints of the environment (e.g., frequent reads vs. frequent writes).

For “My Calendar I,” the core challenge is efficiently finding if a new interval overlaps with any existing ones.

1. Problem Explanation

We need to implement a class `MyCalendar` that tracks events. An event is defined by a **start** and an **end** time. The interval is **half-open**: `[start, end)`, meaning the event starts exactly at **start** and ends just before **end**.

A “double booking” happens if two events have a non-empty intersection.

- **Overlap Condition:** Two events `[s1, e1)` and `[s2, e2)` overlap if: $\max(s1, s2) < \min(e1, e2)$.

Visualization of Overlaps

Scenario A: No Overlap (New event is completely after)

Existing: `[10-----20)`

New: `[20-----30)`

Result: OK (end of first == start of second)

Scenario B: Overlap (New event starts inside)

Existing: `[10-----20)`

New: `[15-----25)`

Result: REJECTED ($15 < 20$)

Scenario C: Overlap (New event encompasses existing)

Existing: `[15---20)`

New: `[10-----25)`

Result: REJECTED ($15 < 20$ and $10 < 15$)

2. Solution Explanation

An L5/L6 engineer would immediately bypass the $O(n^2)$ “check every event” approach in favor of a sorted structure.

The Optimal Approach: Balanced Binary Search Tree (BST)

To check for overlaps efficiently, we need the events sorted by their start times.

1. **Find the “Floor”:** The event that starts just before or at our new **start**.
2. **Find the “Ceiling”:** The event that starts just after our new **start**.
3. **Validate:**
 - The new event must start **after or at** the end of the “Floor” event.
 - The new event must end **before or at** the start of the “Ceiling” event.

ASCII Visualization of the Search Process

Imagine we have events: [10, 20) and [50, 60). We want to book [30, 40).

Sorted Events: [10, 20] [50, 60]

^
Search for 30 here

Step 1: Floor of 30 is [10, 20].

Check: Does 30 >= 20? Yes. (Valid so far)

Step 2: Ceiling of 30 is [50, 60].

Check: Does 40 <= 50? Yes. (Valid!)

Result: Book [30, 40] -> New State: [10, 20], [30, 40], [50, 60]

3. Time and Space Complexity Analysis

For N bookings:

Time Complexity (TC)

Operation: Book(start, end)

```
|
|-- Searching for position in Sorted Structure: log(N)
|-- Insertion into Sorted Structure:           log(N) (using BST/SkipList)
|                                              or O(N) (using Array/List)
|
```

Total for N operations: $N * \log(N)$

Space Complexity (SC)

Data Stored:

```
|
|-- N intervals, each consisting of 2 integers (start, end)
|-- Pointers/Metadata for Tree Nodes
```

|
Total Space: $O(N)$

4. Solution Code

In Python, we use `SortedList` (from `sortedcontainers`) or a manual Binary Search on a list. In JavaScript, we simulate this with a sorted array and `binary search` because JS lacks a built-in balanced BST.

Optimized Solution: Binary Search ($O(\log N)$ search, $O(N)$ insert)

Python Implementation

```
from bisect import bisect_left

class MyCalendar:
    def __init__(self):
        # We keep a list of tuples (start, end) sorted by start time
        self.calendar = []

    def book(self, start: int, end: int) -> bool:
        # 1. Find the index where this event SHOULD go to stay sorted
        idx = bisect_left(self.calendar, (start, end))

        # 2. Check overlap with the event immediately AFTER (the 'ceiling')
        # If there is a next event, its start must be >= our end
        if idx < len(self.calendar) and self.calendar[idx][0] < end:
            return False

        # 3. Check overlap with the event immediately BEFORE (the 'floor')
        # If there is a previous event, its end must be <= our start
        if idx > 0 and self.calendar[idx-1][1] > start:
            return False

        # 4. No overlap found, insert the event
        self.calendar.insert(idx, (start, end))
        return True
```

JavaScript Implementation

```
class MyCalendar {
    constructor() {
        this.calendar = [];
    }
}
```

```

/**
 * Uses binary search to find the insertion point and checks
 * neighboring intervals for overlaps in  $O(\log N) + O(N)$  time.
 */
book(start, end) {
  let low = 0;
  let high = this.calendar.length;

  // Binary search to find the correct index
  while (low < high) {
    let mid = Math.floor((low + high) / 2);
    if (this.calendar[mid][0] < start) {
      low = mid + 1;
    } else {
      high = mid;
    }
  }

  const idx = low;

  // Check against the next event (Ceiling)
  if (idx < this.calendar.length && this.calendar[idx][0] < end) {
    return false;
  }

  // Check against the previous event (Floor)
  if (idx > 0 && this.calendar[idx - 1][1] > start) {
    return false;
  }

  // Insert at the found index
  this.calendar.splice(idx, 0, [start, end]);
  return true;
}
}

```

Terms and Techniques

- **Half-Open Intervals:** Denoted as $[start, end)$, these are standard in software engineering to avoid “off-by-one” errors. The duration is simply $end - start$.
- **Floor and Ceiling:** In a sorted set, the “floor” is the greatest element less than or equal to X , and the “ceiling” is the least element greater than or equal to X . This is the fastest way to check boundaries in interval

problems.

- **Binary Search (Bisect):** A technique to find a target in a sorted list in logarithmic time. It is the backbone of high-performance scheduling systems.
-

Real-World / Interview Variants

1. Google: “The Resource Scheduler”

Problem: Given a list of server tasks, find if a new task can be scheduled without exceeding a resource limit (e.g., CPU). **L6 approach:** This is “My Calendar II or III.” Instead of a simple boolean, use a **Segment Tree** or a **Boundary Count** (Sweep-line) algorithm. You track “delta” changes: +1 at **start**, -1 at **end**. **Solution:** Use a **TreeMap** (Java/C++) or a sorted dictionary to store time points and their impact on resource usage.

2. Meta: “Room Booking System”

Problem: Users book meeting rooms. Return the first available room or suggest an alternative time. **L6 approach:** This introduces a second dimension (Room ID). You would use a **HashMap of SortedLists** (RoomID -> Bookings). To suggest a time, you would perform a “Gap Search” in the sorted list of the requested room.

3. Bloomberg: “Trade Execution Window”

Problem: Financial trades must happen in non-overlapping windows. However, some windows are “high priority” and can kick out lower priority ones. **L6 approach:** This requires an **Interval Tree**. Each node in the tree stores the maximum **end** time of its subtree, allowing you to discard entire branches of the tree during an overlap check, even if the tree isn’t perfectly sorted by start times.

480. Sliding Window Median

This is a fantastic and notoriously tricky problem. I can tell you exactly how a top-of-the-band engineer at a major tech company approaches this.

1. Problem Explanation

Imagine a window of size k moving across an array from left to right. At each step, you need to find the “median” of the numbers inside that window.

- **If k is odd:** The median is the exact middle number when the window is sorted.

- **If k is even:** The median is the average of the two middle numbers when the window is sorted.

Why is this hard? Arrays are great for random access, but terrible for inserting or deleting elements in the middle while maintaining a sorted order. If you sort the window at every single step, the time it takes skyrockets as the array gets larger.

2. Solution Explanation: Two Heaps + Lazy Deletion

To solve this efficiently, we use a **Two-Heap (Dual-Heap)** architecture.

Think of breaking the sorted window exactly in half: 1. **Small Half (Max-Heap):** Stores the smaller half of the numbers. The “top” of this heap is the *largest* number in the small half. 2. **Large Half (Min-Heap):** Stores the larger half of the numbers. The “top” of this heap is the *smallest* number in the large half.

If we balance these heaps so they are roughly the same size, the median is always right at the top of the heaps! We never have to search for it.

The Twist: Lazy Deletion Heaps are fantastic for finding the min or max, but they are *terrible* at finding and deleting an arbitrary element (like the number falling out of the left side of our sliding window). Instead of wasting time searching the heap to delete the outgoing number, we use a hash map to simply “mark” it as deleted. We only actually remove it from the heap if it happens to bubble up to the very top. This is called **Lazy Deletion**.

ASCII Walkthrough

Let’s trace: `nums = [1, 3, -1, -3, 5], k = 3`

Step 1: Initialize the first window [1, 3, -1] We push these into our heaps and balance them.

Window: `[1, 3, -1]` -> Sorted: `[-1, 1, 3]`

Small Half (Max-Heap)	Large Half (Min-Heap)
[1]	[3]
/	
[-1]	
Size: 2	Size: 1
Top: 1	Top: 3

Median (k=3, odd) = Top of Small Half = 1.0

Step 2: Slide the window -> [3, -1, -3] * Outgoing (leaving window):

1. We add 1 to our “Delayed Removals” hash map. `delayed = {1: 1}`. * **Incoming (entering window): -3.** Since $-3 \leq \text{Top of Small Half (1)}$, we put it in the Small Half. * **Balance Check:** 1 is still technically in our heap, throwing off our size count. We track a `balance` variable to fix this.

1. Add -3 to Small Half:

Small Half (Max-Heap)	Large Half (Min-Heap)
[1] <--(marked!)	[3]
/ \	
[-1] [-3]	

2. Rebalance:

Wait, '1' is at the top of the Small Half, AND it's in our delayed map!

LAZY DELETION TRIGGERED! We pop '1' out of the heap and decrement its count in the map.

Cleaned Small Half:

[-1]	[3]
/	
[-3]	

Median = Top of Small Half = -1.0

Step 3: Slide the window -> [-1, -3, 5] * Outgoing: 3. `delayed = {3:`

`1}`. * **Incoming: 5.** Since $5 > \text{Top of Small Half (-1)}$, we put it in the Large Half.

1. Add 5 to Large Half:

Small Half (Max-Heap)	Large Half (Min-Heap)
[-1]	[3] <--(marked!)
/	/
[-3]	[5]

2. Rebalance & Clean:

'3' is at the top of the Large Half, AND it's in our delayed map!

LAZY DELETION TRIGGERED! We pop '3'.

Cleaned Large Half:

[-1]	[5]
/	
[-3]	

Median = Top of Small Half = -1.0

3. Time and Space Complexity Analysis

=====

TIME COMPLEXITY: $O(N * \log K)$

=====

Phase	Complexity	Why?
1. Initializing	$O(K * \log K)$	Inserting first K elements
2. Sliding Window	$O(N)$	Loop runs $N - K$ times
a. Heap Push	$O(\log K)$	Inserting incoming element
b. Hash Map	$O(1)$	Marking outgoing element
c. Lazy Deletion	$O(\log K) * $	*Amortized time to pop
TOTAL TIME	$O(N * \log K)$	Dominant scaling factor

=====

=====

SPACE COMPLEXITY: $O(N)$

=====

Component	Complexity	Why?
1. Heaps	$O(K)$	Max size of active elements
2. Hash Map	$O(N)$	Worst case, lots of delayed
3. Output Array	$O(N - K)$	Storing the results
TOTAL SPACE	$O(N)$	Bounded by total elements

=====

4. Solution Code

Python Implementation (Optimized Dual-Heap)

```
import heapq

class Solution:
    def medianSlidingWindow(self, nums, k):
        # Python's heapq is a Min-Heap by default.
        # To make a Max-Heap for the small_half, we invert the numbers (multiply by -1).
        small_half = []
        large_half = []
        delayed_removals = {}
        result = []

        # Helper function to remove elements from the top of the heap
        # if they are marked for deletion in our hash map.
```

```

def prune(heap, is_small_half):
    while heap:
        # Get the actual value (invert back if it's the small half)
        val = -heap[0] if is_small_half else heap[0]
        if val in delayed_removals and delayed_removals[val] > 0:
            delayed_removals[val] -= 1
            heapq.heappop(heap)
        else:
            break

# Step 1: Initialize the first window
for i in range(k):
    heapq.heappush(small_half, -nums[i])

# Move the largest k//2 elements to the large_half to balance them
for _ in range(k // 2):
    heapq.heappush(large_half, -heapq.heappop(small_half))

# Helper function to calculate current median
def get_median():
    if k % 2 == 1:
        return float(-small_half[0])
    else:
        return (-small_half[0] + large_half[0]) / 2.0

# Record the median of the very first window
result.append(get_median())

# Step 2: Slide the window
for i in range(k, len(nums)):
    outgoing = nums[i - k]
    incoming = nums[i]

    # Balance represents how shifted our heaps are from the ideal state.
    # < 0 means small_half needs more elements. > 0 means large_half needs more.
    balance = 0

    # 1. Mark the outgoing element for deletion
    delayed_removals[outgoing] = delayed_removals.get(outgoing, 0) + 1
    if outgoing <= -small_half[0]:
        balance -= 1 # We "removed" from small half
    else:
        balance += 1 # We "removed" from large half

    # 2. Add the incoming element
    if small_half and incoming <= -small_half[0]:

```



```

        balance += 1
        heapq.heappush(small_half, -incoming)
    else:
        balance -= 1
        heapq.heappush(large_half, incoming)

    # 3. Rebalance the heaps if the active element counts are skewed
    if balance < 0:
        heapq.heappush(small_half, -heapq.heappop(large_half))
        balance += 1
    if balance > 0:
        heapq.heappush(large_half, -heapq.heappop(small_half))
        balance -= 1

    # 4. Prune the tops of the heaps
    prune(small_half, True)
    prune(large_half, False)

    # 5. Record the new median
    result.append(get_median())

return result

```

JavaScript Implementation (Optimized Dual-Heap)

```

/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var medianSlidingWindow = function(nums, k) {
    // Using LeetCode's built-in @datastructures-js/priority-queue
    const small = new MaxPriorityQueue();
    const large = new MinPriorityQueue();
    const deleted = new Map();
    const result = [];

    // Helper: Removes deleted elements if they float to the top
    const prune = (heap) => {
        while (!heap.isEmpty()) {
            const val = heap.front().element;
            if (deleted.get(val) > 0) {
                deleted.set(val, deleted.get(val) - 1);
                heap.dequeue();
            } else {
                break; // Top is valid
            }
        }
    };

```

```

    }
  }
};

// Step 1: Initialize the first window
for (let i = 0; i < k; i++) small.enqueue(nums[i]);
for (let i = 0; i < Math.floor(k / 2); i++) large.enqueue(small.dequeue().element);

// Step 2: Slide the window
for (let i = k; i <= nums.length; i++) {

  // Phase A: Record current median
  const median = k % 2 === 1
    ? small.front().element
    : (small.front().element + large.front().element) / 2;
  result.push(median);

  if (i === nums.length) break;

  // Phase B: Mark Outgoing, Add Incoming
  const outNum = nums[i - k];
  const inNum = nums[i];

  deleted.set(outNum, (deleted.get(outNum) || 0) + 1);

  // Track size discrepancy
  let balance = outNum <= small.front().element ? -1 : 1;

  // Route incoming number
  if (!small.isEmpty() && inNum <= small.front().element) {
    small.enqueue(inNum);
    balance++;
  } else {
    large.enqueue(inNum);
    balance--;
  }

  // Phase C: Rebalance Heaps
  if (balance < 0) small.enqueue(large.dequeue().element);
  else if (balance > 0) large.enqueue(small.dequeue().element);

  // Phase D: Clean Tops
  prune(small);
  prune(large);
}

```

```
    return result;
};
```

1. The Core Problem: Why Do We Need Lazy Deletion?

The entire reason we use heaps for this problem is speed. We want to find the median in $O(1)$ time and insert/remove numbers in $O(\log k)$ time.

However, standard Heaps (Priority Queues) have a massive blind spot: **They only know what is at the very top.** * **Insert:** Fast. $O(\log k)$ * **View Top:** Very fast. $O(1)$ * **Remove Top:** Fast. $O(\log k)$ * **Find and Remove a random element in the middle:** Terrible. $O(k)$

When our sliding window moves forward, the number falling out the back of the window could be *anywhere* inside our heap. It might be at the top, but it's much more likely buried somewhere in the middle or bottom.

If we force the heap to search for that specific number and delete it every single time the window moves, our time complexity degrades to $O(n * k)$. We lose all the performance benefits of using a heap in the first place.

The Insight: We actually *don't care* if “dead” numbers are buried deep inside the heap. Why? Because we only ever look at the **top** of the heap to calculate the median. As long as the top element is valid, the median is correct. We only need to delete a dead number if it bubbles up to the top and blocks our view.

2. The deleted Set: The “Hit List”

Since we cannot delete the outgoing number immediately, we need a way to remember that it is “dead.”

The `deleted` variable (a Hash Map / Dictionary) acts as a ledger of debts. When a number leaves the sliding window, we don't touch the heap. Instead, we add that number to the `deleted` map.

Why a Map and not just a normal Set? Because arrays can have duplicate numbers (e.g., `[2, 2, 2, 5]`). If the number 2 leaves the window, we need to know *how many* 2s are supposed to be dead. * `deleted = { 2: 1 }` means “The next time you see a 2 at the top of the heap, kill it, and reduce this debt to 0.”

3. The `prune()` Function: The Enforcer

The `prune()` function is our cleanup crew. We call it right before we calculate the median to ensure the top of the heap is trustworthy.

It does one simple job: It looks at the top element of the heap and checks the **deleted** ledger. * If the top element is on the hit list (value > 0), **prune()** pops it off the heap and subtracts 1 from the ledger. * It repeats this in a **while** loop until it sees a valid, “alive” number at the top.

ASCII Visualization of Pruning

Imagine a window sliding, and the numbers 7 and 4 have fallen out of the window. They are still physically in our Max-Heap, but they are marked in our ledger.

```
deleted = { 7: 1, 4: 1 }
```

Step 1: The current state of the Small Half (Max-Heap)

```

      [ 9 ]  <-- Top of Heap (Valid!)
     /      \
    [ 7 ]    [ 8 ]  <-- 7 is dead, but it's buried.
   /         \      We do nothing.
  [4]        [2]    <-- 4 is dead, but buried.
```

Result: We call **prune()**. It sees 9 at the top. 9 is not in the ledger. **prune()** does nothing. The median is correctly calculated using 9.

Step 2: The window moves. ‘9’ leaves the window and is marked dead. ‘2’ also leaves. `deleted = { 7: 1, 4: 1, 9: 1, 2: 1 }`

```

      [ 9 ]  <-- Top of Heap (DEAD!)
     /      \
    [ 7 ]    [ 8 ]
   /         \
  [4]        [2]
```

Result: We call **prune()**. 1. **prune** sees 9. It’s on the ledger! Pop it. `deleted[9]` becomes 0. 2. The heap automatically reorganizes. Let’s say 8 becomes the new top.

Step 3: Heap reorganizes after popping 9

```

      [ 8 ]  <-- Top of Heap (Valid!)
     /      \
    [ 7 ]    [ 2 ]
   /
  [4]
```

Result: **prune** checks the new top, 8. It is not on the ledger. **prune()** stops. The median is correctly calculated using 8. The dead numbers 7 and 4 and 2 are still harmlessly floating inside, waiting for their turn to reach the top.

Summary

- **Standard Deletion:** “Find this specific person in a crowd of 10,000 and remove them right now.” (Very slow).
 - **Lazy Deletion (deleted + prune):** “Put their name on a list. If they ever try to step up to the front of the line, bounce them out.” (Extremely fast).
-

Note 1: Terminology Breakdown

- **Lazy Deletion:** A technique where you log an intent to delete an item (using a hash map or boolean flag) rather than paying the computational cost to delete it immediately. It applies here because finding an arbitrary element in a heap takes $O(K)$ time, which defeats the purpose of using a heap.
 - **Dual-Heap (Two-Heap) Pattern:** Using a Max-Heap and Min-Heap in tandem to process a stream of numbers. By balancing the sizes of the heaps, you keep the middle of the dataset instantly accessible at the top of the heaps $O(1)$.
-

Note 2: Real World & Interview Variations

Here is how top companies adapt this pattern to test you on system design and logic:

1. **Google - Distributed Latency Monitor (The “P99” Problem)**
 - *The Problem:* Instead of the 50th percentile (Median), calculate the 99th percentile (P99) latency of a sliding window of the last 10,000 server requests.
 - *The Solution:* Use the exact same Dual-Heap architecture, but change the balancing logic. Instead of keeping the sizes equal (50/50), maintain the `small_half` heap at 99% of the total size, and the `large_half` at 1%. The top of the `large_half` will be your P99 threshold.
2. **Meta - Feed Engagement Anomaly Detection**
 - *The Problem:* Given an ongoing stream of user engagement scores on a post, trigger an alert if the current hour’s score dips below 50% of the rolling median of the past 24 hours.
 - *The Solution:* This tests your ability to wrap the Sliding Window Median algorithm into a larger class structure. You maintain the rolling median using the Dual-Heap, and at each hour tick, you do a simple $O(1)$ comparison: `if current_score < (get_median() * 0.5): trigger_alert()`.
3. **Bloomberg - Rolling Financial Ticks (Data Smoothing)**

- *The Problem:* Financial tick data has massive noise spikes. Write a filter that outputs a stream of prices where every price is replaced by the median of the last 60 ticks.
- *The Solution:* Identical to this LeetCode problem, but framed as a continuous stream rather than a static array. You would build a class with an `add_tick(price)` method that handles the push/pop logic, returning the smoothed median in $O(\log K)$ time to keep the trading engine fast.

302. Smallest Rectangle Enclosing Black Pixels

I see many strong candidates read “connected components” or “matrix” and immediately jump to Depth-First Search (DFS) or Breadth-First Search (BFS). It is a completely natural instinct, but it hits a wall when you read the constraint: **less than $O(mn)$ runtime complexity**.

Let’s break down how a Senior/Staff engineer approaches this, moving from the intuitive trap to the optimal solution.

1. Problem Explanation

Imagine a white canvas (represented by 0s) with a single blob of black ink (represented by 1s) spilled on it.

You are given: * The matrix itself. * The exact (x, y) coordinates of *one* specific drop of that black ink.

The Goal: Draw the smallest possible bounding box (rectangle) around the entire ink blob and calculate the area of that box.

The Catch: You cannot look at every single pixel. If the canvas is 10,000 by 10,000 pixels, and the ink blob takes up the entire canvas, a standard DFS will visit 100,000,000 pixels, resulting in an $O(m * n)$ time complexity. We need to be faster.

2. Solution Explanation

The “Why” behind the optimal approach: Why is DFS slow here? Because DFS visits the *interior* of the black blob. If the blob is huge, you waste time looking at the middle of it when you only care about the edges.

To beat $O(m * n)$, we need a strategy that *skips* the middle and only hunts for the boundaries: the top-most 1, bottom-most 1, left-most 1, and right-most 1.

The “How”: Binary Search on Matrix Projections Because the black pixels are connected, if you “squash” or project the matrix horizontally or vertically, the 1s will form a single, unbroken solid line.

Let’s look at an ASCII visualization. Suppose we are given the starting pixel $x=0$, $y=2$ (which is a 1).

```
Matrix (m=4, n=6):
      0 1 2 3 4 5  <- Columns (y)
+-----+
0 | 0 0 1 0 0 0  <- (x=0, y=2) is here
1 | 0 1 1 1 0 0
2 | 0 1 1 1 1 0
3 | 0 0 0 0 0 0
```

Squash Vertically (Project to Columns):

Do we see a '1' in this column?

Col 0: No (F)

Col 1: Yes (T)

Col 2: Yes (T) <- Starting y is here

Col 3: Yes (T)

Col 4: Yes (T)

Col 5: No (F)

Resulting 1D Array of Columns: [F, T, T, T, T, F]

Notice something amazing? Look at the columns to the *left* of our starting $y=2$. The array is [F, T, T]. This is perfectly sorted for Binary Search! Look at the columns to the *right* of our starting $y=2$. The array is [T, T, T, F]. This is also perfectly sorted!

Instead of checking every column, we can use **Binary Search** to find the exact boundary where F turns into T (the left edge) and where T turns into F (the right edge).

End-to-End Walkthrough (Finding the Left Boundary): We want to find the first column that has a 1. Search space: $left = 0$, $right = y$ (which is 2).

Step 1:

$left = 0$, $right = 2$.

$mid = (0 + 2) / 2 = 1$.

We check Column 1. Does Column 1 have any '1's?

Looking at Col 1: (0, 1, 1, 0)... Yes, it does!

Because it has a '1', the true left boundary might be this column, or it might be further left.

We move our right pointer: $right = mid = 1$.

Step 2:

$left = 0$, $right = 1$.

```

mid = (0 + 1) / 2 = 0.
We check Column 0. Does Column 0 have any '1's?
Looking at Col 0: (0, 0, 0, 0)... No, it doesn't.
Because it's all '0's, the left boundary MUST be to the right of this column.
We move our left pointer: left = mid + 1 = 1.

```

```

Step 3:
left = 1, right = 1.
Loop terminates. The left boundary is Column 1.

```

We repeat this exact logic 4 times: 1. Binary search columns [0, y] for the left boundary. 2. Binary search columns [y+1, n-1] for the right boundary. 3. Binary search rows [0, x] for the top boundary. 4. Binary search rows [x+1, m-1] for the bottom boundary.

The area is simply (bottom - top) * (right - left).

3. Time and Space Complexity Analysis

Time Complexity: $O(m * \log n + n * \log m)$

Here is the ASCII derivation of why this is the case:

```

Finding Left/Right Boundaries (Searching Columns):
[===== n columns =====]
Search space halves each time: log2(n) steps.
BUT, at every step, we must check if a column has a '1'.
Checking a column means looking at 'm' rows.

```

```

Step 1: check mid col -> costs O(m) operations
Step 2: check mid col -> costs O(m) operations
... happens log(n) times.
Total for Columns = m + m + m ... (log n times) = O(m * log n)

```

```

Finding Top/Bottom Boundaries (Searching Rows):
[===== m rows =====]
Search space halves each time: log2(m) steps.
At every step, checking a row means looking at 'n' columns.

```

```

Step 1: check mid row -> costs O(n) operations
Step 2: check mid row -> costs O(n) operations
... happens log(m) times.
Total for Rows = n + n + n ... (log m times) = O(n * log m)

```

Total Time = $O(m * \log n + n * \log m)$

This strictly passes the $< O(m * n)$ requirement!

Space Complexity: $O(1)$ We only store a few integer pointers (`left`, `right`, `top`, `bottom`, `mid`). We do not build any auxiliary matrices or recursion stacks.

4. Solution Code

I'll provide two solutions in both Python and JavaScript. 1. **The BFS Fallback ($O(m * n)$)**: Great to code up quickly in an interview if you blank on the optimal solution, to guarantee you have *working code* on the board. 2. **The Optimal Binary Search**: The L5/L6 target solution.

Python Snippets

```
# =====
# OPTIMAL SOLUTION (Binary Search)
# =====
class Solution:
    def minArea(self, image: list[list[str]], x: int, y: int) -> int:
        if not image or not image[0]:
            return 0

        m, n = len(image), len(image[0])

        # Helper to check if a column has any '1's
        def has_black_in_col(mid_col):
            for row in range(m):
                if image[row][mid_col] == '1':
                    return True
            return False

        # Helper to check if a row has any '1's
        def has_black_in_row(mid_row):
            for col in range(n):
                if image[mid_row][col] == '1':
                    return True
            return False

        # Find boundaries using binary search
        def search_boundary(low, high, is_horizontal, search_min):
            while low < high:
                mid = (low + high) // 2

                if is_horizontal:
                    has_black = has_black_in_col(mid)
                else:
                    has_black = has_black_in_row(mid)
```

```

        if has_black == search_min:
            # If we are looking for the minimum boundary (top/left)
            # and we found a '1', the boundary could be this mid or earlier.
            # If we are looking for max (bottom/right) and found '0',
            # the boundary must be before this mid.
            high = mid
        else:
            low = mid + 1
    return low

# 1. Find Left (search cols from 0 to y)
left = search_boundary(0, y, is_horizontal=True, search_min=True)
# 2. Find Right (search cols from y + 1 to n)
# Note: high is n, not n-1, because right acts as an exclusive bound in calculating
right = search_boundary(y + 1, n, is_horizontal=True, search_min=False)
# 3. Find Top (search rows from 0 to x)
top = search_boundary(0, x, is_horizontal=False, search_min=True)
# 4. Find Bottom (search rows from x + 1 to m)
bottom = search_boundary(x + 1, m, is_horizontal=False, search_min=False)

return (right - left) * (bottom - top)

# =====
# FALLBACK SOLUTION (BFS - O(m * n))
# =====
from collections import deque

class SolutionBFS:
    def minArea(self, image: list[list[str]], x: int, y: int) -> int:
        if not image or not image[0]: return 0

        m, n = len(image), len(image[0])
        min_row, max_row = x, x
        min_col, max_col = y, y

        queue = deque([(x, y)])
        visited = set([(x, y)])

        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        while queue:
            curr_x, curr_y = queue.popleft()

            # Update boundaries
            min_row = min(min_row, curr_x)

```

```

max_row = max(max_row, curr_x)
min_col = min(min_col, curr_y)
max_col = max(max_col, curr_y)

for dx, dy in directions:
    nx, ny = curr_x + dx, curr_y + dy

    if 0 <= nx < m and 0 <= ny < n and image[nx][ny] == '1':
        if (nx, ny) not in visited:
            visited.add((nx, ny))
            queue.append((nx, ny))

# Area = (max_row - min_row + 1) * (max_col - min_col + 1)
return (max_row - min_row + 1) * (max_col - min_col + 1)

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (Binary Search)
// =====
/**
 * @param {character[][]} image
 * @param {number} x
 * @param {number} y
 * @return {number}
 */
var minArea = function(image, x, y) {
    if (!image || image.length === 0) return 0;

    const m = image.length;
    const n = image[0].length;

    const hasBlackInCol = (midCol) => {
        for (let row = 0; row < m; row++) {
            if (image[row][midCol] === '1') return true;
        }
        return false;
    };

    const hasBlackInRow = (midRow) => {
        for (let col = 0; col < n; col++) {
            if (image[midRow][col] === '1') return true;
        }
        return false;
    };
};

```

```

const searchBoundary = (low, high, isHorizontal, searchMin) => {
  while (low < high) {
    const mid = Math.floor((low + high) / 2);
    let hasBlack = isHorizontal ? hasBlackInCol(mid) : hasBlackInRow(mid);

    if (hasBlack === searchMin) {
      high = mid;
    } else {
      low = mid + 1;
    }
  }
  return low;
};

const left = searchBoundary(0, y, true, true);
const right = searchBoundary(y + 1, n, true, false);
const top = searchBoundary(0, x, false, true);
const bottom = searchBoundary(x + 1, m, false, false);

return (right - left) * (bottom - top);
};

// =====
// FALLBACK SOLUTION (BFS -  $O(m * n)$ )
// =====
var minAreaBFS = function(image, x, y) {
  if (!image || image.length === 0) return 0;

  const m = image.length;
  const n = image[0].length;

  let minRow = x, maxRow = x;
  let minCol = y, maxCol = y;

  const queue = [[x, y]];
  // Using a 2D boolean array is faster than a Set for JS coordinates
  const visited = Array.from({length: m}, () => Array(n).fill(false));
  visited[x][y] = true;

  const dirs = [[0, 1], [1, 0], [0, -1], [-1, 0]];

  while (queue.length > 0) {
    const [cx, cy] = queue.shift();

    minRow = Math.min(minRow, cx);
    maxRow = Math.max(maxRow, cx);

```

```

minCol = Math.min(minCol, cy);
maxCol = Math.max(maxCol, cy);

for (let [dx, dy] of dirs) {
  const nx = cx + dx;
  const ny = cy + dy;

  if (nx >= 0 && nx < m && ny >= 0 && ny < n && image[nx][ny] === '1' && !visited[
    visited[nx][ny] = true;
    queue.push([nx, ny]);
  }
}

return (maxRow - minRow + 1) * (maxCol - minCol + 1);
};

```

Note 1: Terminology

The technique we used here is called **Binary Search on a Predicate** or **Binary Search on Projections**. * **Why it helps:** Normally, binary search looks for a specific number in a sorted array. Here, we aren't looking for a number; we are testing a *condition* (a predicate), such as “Does this entire column contain at least one black pixel?”. * **How it applies:** Because the shape is contiguous, the truth values of our predicate naturally form a sorted boolean array (e.g., [False, False, True, True, True]). Whenever you have a monotonic condition (a bunch of falses followed by a bunch of trues), you can use binary search to find the tipping point.

Note 2: Indirect / Real-World Interview Variations

Big Tech rarely asks Leetcode questions verbatim anymore. Here is how this core concept is disguised:

1. Google / Waymo: The Autonomous Driving Radar Problem *

The Prompt: “You have a 2D grid representing LiDAR data. ‘0’ is empty space, ‘1’ is a vehicle. Assuming one vehicle is present and contiguous, and we know one coordinate of the vehicle, find its bounding box to feed into our object classification model efficiently.” * **The Solution:** This is an exact 1-to-1 mapping to our binary search optimal solution. In an interview, I would frame the algorithm exactly as above, but use Waymo terminology (LiDAR grid instead of canvas, vehicle instead of ink blob).

2. Meta: The Image Auto-Cropper * The Prompt: “We have a massive image representing a user’s uploaded photo. We want to auto-crop it to the main subject. The background has been masked out as ‘0’s and the subject

as '1's. How do you find the crop boundaries?" * **The Solution:** Again, the same solution. However, Meta interviewers will often add a follow-up: "What if there are *MULTIPLE* disconnected subjects (e.g., two people standing apart)?" * **Follow-up Solution:** The Binary Search projection trick *breaks* if the blobs are disconnected, because the projected 1D array might look like [False, True, False, True], destroying our monotonic property. For multiple disconnected subjects, you **MUST** fall back to BFS/DFS (connected component labeling) to find the bounding box of each distinct object, taking $O(m * n)$ time.

3. Bloomberg: Terminal Window Bounds * **The Prompt:** "In the Bloomberg Terminal, UI elements are rendered on a massive character grid. A single window is a contiguous block of characters. Given a mouse click coordinate inside a window, find the minimum terminal rendering box we need to redraw." * **The Solution:** Same Binary Search approach. A great optimization to mention here is that UI windows often have solid borders. If the matrix is too large, instead of checking the whole column inside the binary search, you might be able to implement an exponential search (jumping boundaries by powers of 2: 1, 2, 4, 8) to find the edges even faster if the UI window is very small compared to the massive screen grid.

963. Minimum Area Rectangle II

This is a brilliant computational geometry problem. When dealing with geometric shapes on a 2D plane, the brute-force approach (checking every possible combination of 4 points) is $O(N^4)$. For an L5/L6 level candidate, the expectation is to recognize geometric invariants—properties of a shape that never change, no matter how you rotate it—to drastically reduce that search space.

Let's break this down end-to-end.

1. Problem Explanation

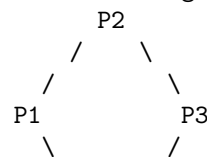
You are given an array of points on a 2D coordinate grid. **The Goal:** Find the area of the *smallest* rectangle you can form using any 4 of these points. **The Catch:** Unlike "Minimum Area Rectangle I" where rectangles are perfectly aligned with the x and y axes, these rectangles can be **tilted or rotated** at any angle.

Here is a visual difference:

Axis-Aligned Rectangle (Easy):



Rotated Rectangle (Hard):



\ /
P4

Because the rectangle can be tilted, you cannot simply look for matching x or y coordinates. We need a mathematically sound way to ask: *“Do these 4 points form a perfect rectangle?”*

2. Solution Explanation

There are two excellent ways to solve this. The first is intuitive but a bit slower. The second uses a brilliant geometric trick that Senior engineers look for.

Approach 1: The “3-Point Right Angle” Method (Fallback / Intuitive) A rectangle is just a shape with four 90-degree (right) angles. If we pick *any three points* (let’s call them P1, P2, P3), we can check if they form a right angle at P1.

How? By checking if their vectors are orthogonal (perpendicular). Two vectors are perpendicular if their **Dot Product is zero**.

If they form a right angle, we know exactly where the 4th point (P4) *must* be located to complete the rectangle. We just calculate P4’s coordinates and check if it exists in our original list of points.

Step 1: Pick 3 points. Check if Angle(P2, P1, P3) is 90 degrees.

```

P2
|
| <-- Vector 1 (P1 to P2)
|
P1 ----- P3
      Vector 2
      (P1 to P3)

```

Step 2: If Dot Product == 0, calculate P4.

P4 = P2 + P3 - P1 (Vector addition)

Check if P4 is in our given Set of points!

Approach 2: The “Diagonal Invariant” Method (Optimal L5/L6)

This is the top-tier solution. What is a universal truth about *every* rectangle, no matter how it is rotated? **The diagonals of a rectangle always have the exact same length AND they intersect exactly at their midpoints.**

```

      P2
     /  \
    /    \
   /  M  \ <-- M is the midpoint.
  /      \
 P1       P3   Length of (P1 to P3) == Length of (P2 to P4)
  \      /     Midpoint of (P1 to P3) == Midpoint of (P2 to P4)

```

\ /
P4

Instead of looking for 4 points, we just look at **pairs of 2 points**. Every pair of points forms a line segment. For every pair, we calculate: 1. The distance (length) between them. 2. The exact midpoint (x, y) between them.

We group all pairs by their [length, midpoint_x, midpoint_y]. If two *different* pairs of points share the exact same length and the exact same midpoint, congratulations—those two line segments are the diagonals of a perfect rectangle!

3. Time and Space Complexity Analysis

Let's look at the complexity using the optimal Diagonal Method. Let N be the number of points.

TIME COMPLEXITY DERIVATION: $O(N^2)$

[Start with N points]

|
V

[Iterate over all combinations of 2 points to form diagonals]

|
V

How many pairs are there? "N choose 2", which is $N * (N - 1) / 2$.
This gives us $O(N^2)$ total pairs.

|
V

[For each pair: Calculate distance and midpoint -> $O(1)$ math operations]

|
V

[Store pair in a Hash Map based on "Distance + Midpoint" string -> $O(1)$ amortized]

|
V

Total Time = $O(N^2)$ pairs * $O(1)$ operations = $O(N^2)$ Time Complexity.

SPACE COMPLEXITY DERIVATION: $O(N^2)$

[Hash Map Data Structure]

Key: "Length_MidX_MidY"

Value: List of point pairs ([[p1, p2], [p3, p4] ...])

|
V

In the worst case, every single pair of points goes into the hash map.
We have $O(N^2)$ pairs.

|
V

Total Space = $O(N^2)$ Space Complexity to store all pairs.

Note: The 3-Point Fallback method takes $O(N^3)$ time because it iterates over 3 points, but only $O(N)$ space to store the initial points in a Set.

4. Solution Code

Here are both solutions in Python and JavaScript.

Python Snippets

```
import math
from collections import defaultdict

# =====
# OPTIMAL SOLUTION (Diagonal Invariant -  $O(N^2)$ )
# =====
class Solution:
    def minAreaFreeRect(self, points: list[list[int]]) -> float:
        # Map signature: (distance_squared, center_x, center_y) -> list of pairs of points
        # We use distance_squared to avoid floating point precision issues with math.sqrt
        diagonals = defaultdict(list)

        # 1. Iterate over all pairs of points
        for i in range(len(points)):
            for j in range(i + 1, len(points)):
                x1, y1 = points[i]
                x2, y2 = points[j]

                # Calculate invariant 1: Length squared
                dist_sq = (x1 - x2)**2 + (y1 - y2)**2

                # Calculate invariant 2: Midpoint coordinates
                # We can just use the sum (x1+x2) instead of dividing by 2 to avoid floats
                center_x = (x1 + x2) / 2.0
                center_y = (y1 + y2) / 2.0

                # Group pairs by these invariants
                diagonals[(dist_sq, center_x, center_y)].append((points[i], points[j]))

        min_area = float('inf')

        # 2. Find rectangles and calculate area
```

```

for key, pairs in diagonals.items():
    # If we have 2 or more pairs with the same length/midpoint, they form rectangle
    if len(pairs) > 1:
        # Compare all pairs that share this diagonal property
        for i in range(len(pairs)):
            for j in range(i + 1, len(pairs)):
                p1, p2 = pairs[i] # Diagonal 1
                p3, p4 = pairs[j] # Diagonal 2

                # Area = width * height.
                # We can find width and height by measuring the distance between
                # an endpoint of Diagonal 1 (p1) and the endpoints of Diagonal 2 (p3, p4)
                width_sq = (p1[0] - p3[0])**2 + (p1[1] - p3[1])**2
                height_sq = (p1[0] - p4[0])**2 + (p1[1] - p4[1])**2

                area = math.sqrt(width_sq * height_sq)
                min_area = min(min_area, area)

return min_area if min_area != float('inf') else 0.0

# =====
# FALLBACK SOLUTION (3-Point Right Angle - O(N^3))
# =====
class SolutionFallback:
    def minAreaFreeRect(self, points: list[list[int]]) -> float:
        # Put all points in a set for O(1) lookups
        point_set = {(x, y) for x, y in points}
        min_area = float('inf')
        n = len(points)

        for i in range(n):
            for j in range(n):
                for k in range(n):
                    if i == j or i == k or j == k:
                        continue

                    p1, p2, p3 = points[i], points[j], points[k]

                    # Create vectors P1->P2 and P1->P3
                    v1_x, v1_y = p2[0] - p1[0], p2[1] - p1[1]
                    v2_x, v2_y = p3[0] - p1[0], p3[1] - p1[1]

                    # Dot product == 0 means 90 degree angle
                    if (v1_x * v2_x) + (v1_y * v2_y) == 0:
                        # Expected 4th point: P4 = P2 + P3 - P1
                        p4_x = p2[0] + p3[0] - p1[0]

```

```

        p4_y = p2[1] + p3[1] - p1[1]

        if (p4_x, p4_y) in point_set:
            # Rectangle found!
            area = math.sqrt(v1_x**2 + v1_y**2) * math.sqrt(v2_x**2 + v2_y**2)
            min_area = min(min_area, area)

    return min_area if min_area != float('inf') else 0.0

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (Diagonal Invariant -  $O(N^2)$ )
// =====
/**
 * @param {number[][]} points
 * @return {number}
 */
var minAreaFreeRect = function(points) {
    const diagonals = new Map();
    const n = points.length;

    // 1. Group pairs by their diagonal properties
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            const [x1, y1] = points[i];
            const [x2, y2] = points[j];

            const distSq = (x1 - x2) ** 2 + (y1 - y2) ** 2;
            const centerX = (x1 + x2) / 2;
            const centerY = (y1 + y2) / 2;

            // Create a unique string key for the map
            const key = `${distSq}_${centerX}_${centerY}`;

            if (!diagonals.has(key)) {
                diagonals.set(key, []);
            }
            diagonals.get(key).push([points[i], points[j]]);
        }
    }

    let minArea = Infinity;

    // 2. Find rectangles from matching diagonals
    for (const [key, pairs] of diagonals.entries()) {

```

```

    if (pairs.length > 1) {
      for (let i = 0; i < pairs.length; i++) {
        for (let j = i + 1; j < pairs.length; j++) {
          const p1 = pairs[i][0];
          const p3 = pairs[j][0];
          const p4 = pairs[j][1];

          const widthSq = (p1[0] - p3[0]) ** 2 + (p1[1] - p3[1]) ** 2;
          const heightSq = (p1[0] - p4[0]) ** 2 + (p1[1] - p4[1]) ** 2;

          const area = Math.sqrt(widthSq * heightSq);
          minArea = Math.min(minArea, area);
        }
      }
    }

    return minArea === Infinity ? 0 : minArea;
  };

  // =====
  // FALLBACK SOLUTION (3-Point Right Angle - O(N^3))
  // =====
  var minAreaFreeRectFallback = function(points) {
    const pointSet = new Set();
    for (const [x, y] of points) {
      pointSet.add(`${x},${y}`);
    }

    let minArea = Infinity;
    const n = points.length;

    for (let i = 0; i < n; i++) {
      for (let j = 0; j < n; j++) {
        for (let k = 0; k < n; k++) {
          if (i === j || i === k || j === k) continue;

          const p1 = points[i], p2 = points[j], p3 = points[k];

          const v1x = p2[0] - p1[0], v1y = p2[1] - p1[1];
          const v2x = p3[0] - p1[0], v2y = p3[1] - p1[1];

          // Dot product == 0
          if ((v1x * v2x) + (v1y * v2y) === 0) {
            const p4x = p2[0] + p3[0] - p1[0];
            const p4y = p2[1] + p3[1] - p1[1];

```

```

        if (pointSet.has(`${p4x},${p4y}`)) {
            const area = Math.sqrt(v1x**2 + v1y**2) * Math.sqrt(v2x**2 + v2y**2);
            minArea = Math.min(minArea, area);
        }
    }
}

return minArea === Infinity ? 0 : minArea;
};

```

Note 1: Terminology

- **Geometric Invariant:** A property of a geometric object that remains unchanged after operations like rotation, translation, or scaling. We used this concept to avoid figuring out rotation angles; no matter how tilted the rectangle is, its diagonals always share a midpoint and length.
- **Vector Dot Product / Orthogonality Check:** A mathematical way to check the angle between two lines (vectors). If $(x1 * x2) + (y1 * y2) == 0$, the lines intersect at exactly 90 degrees. This is incredibly useful in computational geometry for verifying shapes without using costly trigonometric functions like `sine` or `cosine`.

Note 2: Indirect / Real-World Interview Variations

Because LC 963 is highly specific, big tech companies frame it as system design or real-world geometry problems:

1. Google / Waymo: LiDAR Object Tracking * The Prompt: “We have 3D LiDAR point cloud data projected onto a 2D plane representing a parking lot. Cars appear as clusters of points. Given a set of boundary points for a cluster, how do we find the minimal rectangular bounding box (which might be parked at a tilted angle) to classify the car’s footprint?” * **The Solution:** This is a slightly more advanced version called the *Rotating Calipers* algorithm (used on a convex hull), but interviewers often simplify it to a finite set of points to test exactly the logic above. You would use the Diagonal or Dot Product approach to find all possible rigid vehicle outlines.

2. Meta / Oculus: VR Hitbox Detection * The Prompt: “In a 2D projection of a VR scene, players can place rectangular boundary markers that are rotated freely. Given a stream of coordinate clicks, how do we efficiently detect if four clicks form a valid rotated ‘safe zone’ marker?” * **The Solution:** You would implement the exact $O(N^2)$ Hash Map approach. Since the coordinates are a stream, every time a new click (point) comes in, you pair it with

all existing points, calculate the diagonal invariant, and check the Hash Map to see if a matching diagonal already exists.

3. Bloomberg: Financial Chart Annotation * **The Prompt:** “Traders draw custom trend channels (which are just tilted rectangles) on scatter-plot stock charts. Given a dataset of user-clicked coordinates on the chart, identify the smallest valid trend channel they attempted to draw.” * **The Solution:** Exact same algorithm. In a Bloomberg interview, emphasize handling floating-point inaccuracies. Since screen coordinates or stock prices can be decimals, comparing exact lengths and midpoints can fail due to precision errors. You would need to explain how to use an **epsilon** (a very small threshold like 0.00001) when checking if lengths or midpoints are “equal”.

283. Move Zeroes

This is a classic problem, and you are right to ask how a Senior/Staff engineer (L5/L6) views it. At a junior level, the goal is simply to make the code work. At the L5/L6 level, the focus shifts to **minimizing array writes** (which are expensive at the hardware/cache level) and understanding the fundamental algorithm class this belongs to (array partitioning).

Let’s break down how to tackle this cleanly, efficiently, and intuitively.

1. Problem Explanation

Imagine a row of boxes. Some boxes have items in them (represented by non-zero numbers like 1, 3, 12), and some boxes are completely empty (represented by 0).

The Goal: You want to slide all the items to the front of the row, keeping them in their original order, so that all the empty boxes (the 0s) are pushed to the very end.

The Constraints: * You must do this **in-place**. This means you cannot just build a brand-new row of boxes in memory, move the items there, and return it. You must rearrange the existing boxes. * You should try to minimize the number of operations.

2. Solution Explanation

The “Why” behind the optimal approach: If we can’t create a second array, we have to use the existing array to store our progress. A naive approach might be to see a zero, delete it, and append it to the end. But in an array, deleting and shifting elements takes $O(N)$ time, making the whole process $O(N * N)$ time. That is too slow.

To do this in one pass, we need **Two Pointers**. Think of one pointer as the “Explorer” and the other as the “Anchor”. * **Explorer (right pointer)**: Runs ahead to look at every single box. It is searching for items (non-zero numbers). * **Anchor (left pointer)**: Stands at the exact spot where the *next* item needs to be placed.

The “How” (The Swap Strategy): Every time the Explorer finds an item (a non-zero), it tosses it to the Anchor. The Anchor catches it, puts it in its box, and takes one step forward to prepare for the next item. If the Explorer finds an empty box (a 0), it just ignores it and keeps walking.

By using a **swap** instead of just overwriting, we guarantee that the 0s are naturally pushed to the back as the non-zeros are pulled to the front!

End-to-End Walkthrough: Let’s trace the input array: [0, 1, 0, 3, 12]

Initial State:

Array: [0, 1, 0, 3, 12]

```

      ^
      |
Anchor (left=0)
Explorer (right=0)

```

Step 1:

Explorer looks at index 0. It is a '0'.

Rule: Ignore '0's. Explorer moves forward.

Array: [0, 1, 0, 3, 12]

```

      ^   ^
      |   |
Anchor Explorer (right=1)

```

Step 2:

Explorer looks at index 1. It is a '1' (non-zero!).

Rule: Swap the values at Anchor and Explorer. Move BOTH forward.

Swap index 0 and index 1.

Array: [1, 0, 0, 3, 12] <-- The '1' moves to the front!

```

      ^   ^
      |   |
Anchor Explorer (right=2)

```

Step 3:

Explorer looks at index 2. It is a '0'.

Rule: Ignore '0's. Explorer moves forward.

Array: [1, 0, 0, 3, 12]

```

      ^   ^
      |   |
Anchor Explorer (right=3)

```

Step 4:
 Explorer looks at index 3. It is a '3' (non-zero!).
 Rule: Swap Anchor and Explorer. Move BOTH forward.
 Swap index 1 and index 3.
 Array: [1, 3, 0, 0, 12] <-- The '3' is placed perfectly!

```

      ^      ^
      |      |
    Anchor  Explorer (right=4)
  
```

Step 5:
 Explorer looks at index 4. It is a '12' (non-zero!).
 Rule: Swap Anchor and Explorer. Move BOTH forward.
 Swap index 2 and index 4.
 Array: [1, 3, 12, 0, 0] <-- The '12' is placed!
 ^ ^ (out of bounds)

```

      |
    Anchor
  
```

Loop ends. The array is perfectly sorted with zeroes at the end!

3. Time and Space Complexity Analysis

Here is the ASCII derivation of the complexity. Let N be the number of elements in the array.

TIME COMPLEXITY: $O(N)$

```

[ 0, 1, 0, 3, 12 ]
|--|--|--|--| <-- The 'Explorer' pointer touches each element exactly once.
  
```

There is no nested looping.
 At each step, checking if a number is 0 takes $O(1)$ time.
 Swapping two elements takes $O(1)$ time.

Total Time = N elements * $O(1)$ operations = $O(N)$

SPACE COMPLEXITY: $O(1)$

Original Array in Memory:

```

+---+---+---+---+---+
| 0 | 1 | 0 | 3 | 12 |
+---+---+---+---+
  ^   ^
  L   R
  
```

L R <-- We only create two integer variables (Left and Right pointers).

We modify the array directly. No matter if N is 5 or 5,000,000, we still only use two variables for the pointers.

Total Space = $O(1)$ Constant Space.

4. Solution Code

I'll provide two solutions. 1. **The Overwrite + Fill Method:** A bit easier to think of intuitively, but suboptimal because it performs unnecessary writes. 2. **The Optimal Swap Method:** The L5/L6 target solution that minimizes memory writes.

Python Snippets

```
# =====
# OPTIMAL SOLUTION (The Swap Method - Minimizes Writes)
# =====
class Solution:
    def moveZeroes(self, nums: list[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        # 'left' is our Anchor pointer
        left = 0

        # 'right' is our Explorer pointer
        for right in range(len(nums)):
            if nums[right] != 0:
                # When we find a non-zero, swap it to the Anchor's position.
                # If left == right, it just swaps with itself (no real cost).
                nums[left], nums[right] = nums[right], nums[left]

                # Move the Anchor forward to accept the next non-zero
                left += 1

# =====
# FALLBACK SOLUTION (Overwrite and Fill)
# =====
class SolutionFallback:
    def moveZeroes(self, nums: list[int]) -> None:
        insert_pos = 0

        # Step 1: Shift all non-zeros to the front
        for i in range(len(nums)):
            if nums[i] != 0:
```

```

        nums[insert_pos] = nums[i]
        insert_pos += 1

# Step 2: Fill the remaining spaces with zeroes
# Why is this worse? If array is [0, 0, ..., 1],
# it writes '1' once, then rewrites N-1 zeroes unnecessarily.
while insert_pos < len(nums):
    nums[insert_pos] = 0
    insert_pos += 1

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (The Swap Method - Minimizes Writes)
// =====
/**
 * @param {number[]} nums
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var moveZeroes = function(nums) {
    // 'left' is our Anchor pointer
    let left = 0;

    // 'right' is our Explorer pointer
    for (let right = 0; right < nums.length; right++) {
        if (nums[right] !== 0) {
            // Swap elements at 'left' and 'right'
            let temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;

            // Move the Anchor forward
            left++;
        }
    }
};

// =====
// FALLBACK SOLUTION (Overwrite and Fill)
// =====
var moveZeroesFallback = function(nums) {
    let insertPos = 0;

    // Step 1: Overwrite the front with non-zero elements
    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {

```

```

        nums[insertPos] = nums[i];
        insertPos++;
    }
}

// Step 2: Fill the rest of the array with zeroes
for (let i = insertPos; i < nums.length; i++) {
    nums[i] = 0;
}
};

```

Note 1: Terminology

The algorithm used here is based on **Array Partitioning**, specifically utilizing the **Fast/Slow Two-Pointer Technique**. * **Why it helps:** It allows us to process and reorganize an array in a single sweep without needing auxiliary (extra) memory space. * **How it applies:** This is actually the exact same logic used in the **partition** step of the famous **QuickSort** algorithm! In QuickSort, you pick a pivot and move all numbers smaller than the pivot to the left. Here, our “pivot condition” is simply whether the number is a 0 or not.

Note 2: Indirect / Real-World Interview Variations

In Big Tech, they will often disguise this simple array problem as a systems or hardware problem:

1. Google: Memory Garbage Collection (Compaction) * **The Prompt:** “You have an array representing blocks of memory. A 0 means the memory is freed, and a non-zero integer is an active process ID. Memory fragmentation is slowing down the system. Write an in-place algorithm to ‘compact’ the memory, moving all active processes to the front while maintaining their order.” * **The Solution:** This is word-for-word the “Move Zeroes” problem. To ace the interview, you would provide the Optimal Swap method and specifically point out to the interviewer that you chose swapping over overwriting to *reduce wear on the memory hardware* (minimizing memory writes).

2. Meta: The “Move Negatives” Variant * **The Prompt:** “Given an array of integers, move all negative numbers to the left and all positive numbers to the right. The relative order of negative numbers must be maintained.” * **The Solution:** Exact same code, but instead of checking `if nums[right] != 0`, you check `if nums[right] < 0`. The Anchor pointer tracks where the next negative number should go.

3. Bloomberg: Order Book Consolidation * **The Prompt:** “We receive a daily array of stock trade quantities. Cancelled trades are marked as 0. We need to feed this array to a legacy pricing engine that breaks if it sees a 0, but we can’t

allocate new arrays due to strict low-latency constraints. Consolidate the valid trades to the front.” * **The Solution:** Again, apply the Two-Pointer swap. In a finance context, latency is king. You should explain that the algorithm is strictly $O(N)$ with excellent spatial locality (it reads the array sequentially from left to right), which means it will be perfectly cached by the CPU during execution.

636. Exclusive Time of Functions

This is an excellent problem. A junior engineer might try to simulate the timeline second by second (which results in Time Limit Exceeded errors if a function runs for a million seconds). A top-tier L5/L6 engineer immediately recognizes that we only care about the “events” (the logs) and uses a **Stack** to keep track of execution context.

Let’s break down exactly how to model this.

1. Problem Explanation

Imagine a CPU that can only run one function at a time (a single-threaded environment).

- When **Function A** starts, the CPU works on it.
- If **Function A** calls **Function B**, **Function A** goes to sleep (pauses).
- The CPU works on **Function B** until it finishes.
- Once **Function B** ends, **Function A** wakes up and resumes exactly where it left off.

The Goal: Calculate the *exclusive* time each function spends actually executing on the CPU. Time spent sleeping (waiting for another function to finish) does not count.

The Tricky Part (The Timestamps): * A “start” timestamp means the function starts at the very **beginning** of that time unit. * An “end” timestamp means the function finishes at the very **end** of that time unit. * Because of this, if a function starts at `time=2` and ends at `time=5`, it ran for $5 - 2 + 1 = 4$ units of time (it ran during second 2, second 3, second 4, and second 5). That +1 is where 90% of candidates fail.

2. Solution Explanation

To solve this efficiently, we process the logs one by one, jumping forward in time from event to event. We need two things: 1. An array to store the total time for each function: `result` 2. A **Stack** to remember which function is currently

running and which ones are paused. 3. A variable to remember the timestamp of the last event we processed: `prev_time`

The “How” (The Rules of the Stack):

When we read a log, we parse it into `id`, `action` (start or end), and `current_time`.

- **Rule 1: If it’s a “start” event:**
 - Is there a function already running (i.e., is the stack not empty)? If so, that function is about to be paused!
 - We calculate how long it just ran: `time_spent = current_time - prev_time`. We add this to its total in the `result` array.
 - We push the new `id` onto the stack (it is now the active function).
 - We update `prev_time = current_time`.
- **Rule 2: If it’s an “end” event:**
 - The function at the top of the stack just finished. Pop it off.
 - We calculate its final chunk of execution time. Because “end” means the end of the second, the formula is: `time_spent = current_time - prev_time + 1`. We add this to its total.
 - We update `prev_time = current_time + 1`. Why +1? Because the *next* function in the stack will resume at the start of the *next* second!

End-to-End Walkthrough: Let’s trace `n = 2`, `logs = ["0:start:0", "1:start:2", "1:end:5", "0:end:6"]`

Initial State:

```
result = [0, 0]  (Space for f0 and f1)
```

```
stack = []
```

```
prev_time = 0
```

```
-----  
Step 1: Parse "0:start:0"
```

```
-----  
id=0, action=start, current_time=0
```

Is stack empty? Yes. No running function to pause.
Push 0 to stack. Update `prev_time`.

```
stack = [0]      <-- f0 is running  
prev_time = 0  
result = [0, 0]
```

```
-----  
Step 2: Parse "1:start:2"
```

```
-----  
id=1, action=start, current_time=2
```

```

Is stack empty? No. Top is 0. f0 is about to pause.
Time f0 ran before pausing = current_time - prev_time = 2 - 0 = 2.
Add 2 to f0's result.
Push 1 to stack. Update prev_time.

stack = [0, 1]    <-- f1 is running, f0 is asleep
prev_time = 2
result = [2, 0]

-----

Step 3: Parse "1:end:5"
-----

id=1, action=end, current_time=5

Action is end! Top of stack is 1. Pop f1 (it's done).
Time f1 ran = current_time - prev_time + 1 = 5 - 2 + 1 = 4.
Add 4 to f1's result.
Update prev_time to current_time + 1 = 5 + 1 = 6. (Next func resumes at 6)

stack = [0]       <-- f0 wakes up!
prev_time = 6
result = [2, 4]

-----

Step 4: Parse "0:end:6"
-----

id=0, action=end, current_time=6

Action is end! Top of stack is 0. Pop f0 (it's done).
Time f0 ran = current_time - prev_time + 1 = 6 - 6 + 1 = 1.
Add 1 to f0's result.
Update prev_time = 6 + 1 = 7.

stack = []        <-- All done!
prev_time = 7
result = [3, 4]   <-- Final Answer!

```

3. Time and Space Complexity Analysis

Let L be the number of logs. Let N be the number of unique functions.

TIME COMPLEXITY: $O(L)$

Input Logs: ["0:start:0", "1:start:2", "1:end:5", "0:end:6"]
 |-----|-----|-----|-----|

Operations:	Parse log	Parse log	Parse log	Parse log
	$O(1)$	$O(1)$	$O(1)$	$O(1)$

We process each log string exactly once.
 Parsing a string takes $O(1)$ time.
 Stack push/pop takes $O(1)$ time.
 Updating arrays takes $O(1)$ time.

Total Time = $L \text{ logs} * O(1) \text{ operations} = O(L)$ Time Complexity.

SPACE COMPLEXITY: $O(L/2) + O(N) = O(L)$

[Data Structures in Memory]

1. Result Array: Size N
 $[f0_time, f1_time, \dots fn_time] \rightarrow O(N)$ space
2. Stack:
 In the worst case ($f0$ calls $f1$, $f1$ calls $f2$, ..., up to f_k without ending), the stack will hold all "start" logs.
 Since every function has a start and an end, at most half the logs are starts.
 Stack max size = $L / 2 \rightarrow O(L)$ space

Total Space = $O(L)$ Space Complexity.

4. Solution Code

Both Python and JavaScript implementations use the exact same stack-based approach. Since this is the optimal $O(L)$ solution, it's the only one you should write in an interview (simulating it second-by-second is an instant fail).

Python Snippet

```
class Solution:
    def exclusiveTime(self, n: int, logs: list[str]) -> list[int]:
        # Array to store the total exclusive time for each function
        result = [0] * n

        # Stack will store the IDs of the functions that are currently running/paused
        stack = []

        # Tracks the timestamp of the last processed event
        prev_time = 0

        for log in logs:
```

```

# Parse the string into usable variables
# e.g., "1:start:2" -> func_id=1, action="start", time=2
parts = log.split(":")
func_id = int(parts[0])
action = parts[1]
current_time = int(parts[2])

if action == "start":
    # If another function is already running, it's about to be paused.
    if stack:
        active_func_id = stack[-1]
        # Add the time spent to the active function
        result[active_func_id] += current_time - prev_time

    # Push the new function onto the stack
    stack.append(func_id)
    prev_time = current_time

else: # action == "end"
    # The function at the top of the stack is finishing
    finished_func_id = stack.pop()

    # Calculate its final chunk of time.
    # +1 because the 'end' timestamp is at the end of the time unit.
    result[finished_func_id] += current_time - prev_time + 1

    # The next function will resume at the start of the next time unit
    prev_time = current_time + 1

return result

```

JavaScript Snippet

```

/**
 * @param {number} n
 * @param {string[]} logs
 * @return {number[]}
 */
var exclusiveTime = function(n, logs) {
    // Array to store the total exclusive time for each function
    const result = new Array(n).fill(0);

    // Stack will store the IDs of the functions
    const stack = [];

    // Tracks the timestamp of the last processed event

```



```

let prevTime = 0;

for (let i = 0; i < logs.length; i++) {
  // Parse the string into usable variables
  const parts = logs[i].split(":");
  const funcId = parseInt(parts[0]);
  const action = parts[1];
  const currentTime = parseInt(parts[2]);

  if (action === "start") {
    // If another function is already running, pause it
    if (stack.length > 0) {
      const activeFuncId = stack[stack.length - 1];
      result[activeFuncId] += currentTime - prevTime;
    }

    // Push the new function onto the stack
    stack.push(funcId);
    prevTime = currentTime;

  } else { // action === "end"
    // The function at the top of the stack is finishing
    const finishedFuncId = stack.pop();

    // Calculate final chunk of time (add 1 because it spans to the end of the unit)
    result[finishedFuncId] += currentTime - prevTime + 1;

    // The next function will resume at the start of the next time unit
    prevTime = currentTime + 1;
  }
}

return result;
};

```

Note 1: Terminology

The technique used here is called a **Call Stack Simulation** using a **Stack Data Structure (LIFO - Last In, First Out)**. * **Why it helps:** In nested hierarchical operations (where event B starts and finishes entirely within the lifespan of event A), a Stack naturally tracks the “current active context”. As you go deeper, you push. As you resolve, you pop. * **How it applies:** This perfectly mimics how modern CPU architectures handle nested function calls, making it the mathematically perfect way to track pause/resume states.

Note 2: Indirect / Real-World Interview Variations

Big Tech companies use this core concept frequently, but they frame it around systems you would actually build on the job:

1. Google: The CPU Profiler System * **The Prompt:** “You are building a profiler for Chrome V8 (the JavaScript engine). Given a stream of function entry and exit events with microsecond timestamps, generate a report of the bottleneck functions (the ones taking the most exclusive CPU time).” * **The Solution:** This is the exact LeetCode problem. To ace the interview, you would implement the Stack solution above. You would also want to mention edge cases: what happens if the profiler crashes and logs are malformed (e.g., an “end” without a “start”)? You’d add error handling to check if the stack is empty before popping.

2. Meta: Ad Tracker / Viewability Session * **The Prompt:** “We track when a user scrolls an ad into view (start) and out of view (end). However, if they click a popup, the ad is obscured (popup start) until they close it (popup end). Calculate the total *visible* time of the ad.” * **The Solution:** Same Stack logic. The “Ad” is **Function 0**, and the “Popup” is **Function 1**. You track the exclusive time the ad is at the top of the “visibility stack”.

3. Bloomberg: Trading Algorithm Latency Monitor * **The Prompt:** “A high-frequency trading algorithm kicks off ‘orders’. An order might trigger a ‘risk check’ subprocess, which pauses the main order execution. Calculate the pure network wait time vs the internal calculation time given a log file of trace IDs.” * **The Solution:** Same Stack approach, but applied to parsing log files. You would read the file line by line (a stream) rather than loading a massive **logs** array into memory all at once. You’d keep a Map/Dictionary instead of a fixed array for **result**, because in a trading system, the trace **n** (number of functions/orders) isn’t known ahead of time—it’s dynamically generated UUIDs.

1428. Leftmost Column with at Least a One

This is a phenomenal problem that perfectly tests a candidate’s ability to transition from a “good enough” solution to an “elegant” one.

When an L5 or L6 engineer looks at this problem, they immediately notice two critical hints in the prompt: 1. **Row-sorted:** The elements are sorted (0s then 1s). This screams “Binary Search”. 2. **API call limit:** You can only make 1000 calls to **get()**. If the matrix is 100x100, checking every cell is 10,000 calls. We must be more efficient.

A junior engineer will usually stop at running a Binary Search on every single row. A senior engineer will realize that rows shouldn’t be treated independently; we can use the information learned from one row to skip unnecessary work in the next row.

Let's break this down end-to-end.

1. Problem Explanation

Imagine a grid of lightbulbs. 0 means the bulb is off, 1 means the bulb is on. The special rule of this grid is that in any given row, once a bulb is turned on, every bulb to the right of it is also turned on. (e.g., 0, 0, 1, 1 is valid. 0, 1, 0, 1 is impossible).

The Goal: Find the index of the column that has the left-most 1 (the earliest turned-on bulb) across the entire grid.

The Catch: You cannot look at the whole grid at once. You have to ask an API: "Hey, what is at row X, column Y?" and it answers 0 or 1. You want to find the answer by asking the API as few times as possible.

2. Solution Explanation

Let's explore the progression of thought from the Fallback to the Optimal solution.

The Fallback: Binary Search Every Row Because each row is sorted, you can find the first 1 in a row using Binary Search in $O(\log N)$ API calls. If you do this for every M rows, your total API calls will be $M * \log N$. For a 100x100 matrix, $100 * \log_2(100)$ is roughly 700 calls. This is under the 1000 call limit and *will pass*, but it's not the most optimal.

The Optimal L5/L6 Approach: The "Staircase" Search Why is Binary Search on every row wasteful? Suppose in Row 0, you find the first 1 at **Column 5**. When you move to Row 1, do you care if there is a 1 at Column 7 or Column 8? **No!** You already know Column 5 has a 1. You only care if Row 1 has a 1 at **Column 4 or earlier**.

This insight leads to a beautiful algorithm where we start at the **Top-Right** corner of the matrix.

- **Rule 1:** If we see a 1, this column is a candidate for our answer! To find a *better* (more left) answer, we step strictly **LEFT**.
- **Rule 2:** If we see a 0, it means there are no 1s anywhere to the left in this row (because rows are sorted). We can't find a better answer in this row, so we step strictly **DOWN** to the next row.

End-to-End ASCII Walkthrough:

Let's use a 4x4 matrix. Target: Find the leftmost 1. Variables: `current_row = 0`, `current_col = 3` (Top-Right), `leftmost_col = -1`

The Hidden Matrix:

```
    0 1 2 3  <- Columns
+-----+
0 | 0 0 0 1
1 | 0 0 1 1
2 | 0 0 0 0
3 | 0 1 1 1
```

Step 1: Start at Top-Right (0, 3).

```
0 0 0 [1]  <- API says 1!
0 0 1  1
0 0 0  0
0 1 1  1
```

- **API returns:** 1
- **Logic:** We found a 1! Update `leftmost_col = 3`. We want to see if we can do better, so we move LEFT.
- **Next position:** (0, 2)

Step 2: At (0, 2).

```
0 0 [0] 1  <- API says 0!
0 0  1  1
0 0  0  0
0 1  1  1
```

- **API returns:** 0
- **Logic:** This row has no more 1s. Move DOWN.
- **Next position:** (1, 2)

Step 3: At (1, 2).

```
0 0 0 1
0 0[1]1  <- API says 1!
0 0 0 0
0 1 1 1
```

- **API returns:** 1
- **Logic:** We found a better 1! Update `leftmost_col = 2`. Move LEFT.
- **Next position:** (1, 1)

Step 4: At (1, 1).

```
0 0  0 1
0[0] 1 1  <- API says 0!
0 0  0 0
0 1  1 1
```

- **API returns:** 0
- **Logic:** Move DOWN.
- **Next position:** (2, 1)

Step 5: At (2, 1).

```
0 0 0 1
0 0 1 1
0[0]0 0    <- API says 0!
0 1 1 1
```

- **API returns:** 0
- **Logic:** Move DOWN.
- **Next position:** (3, 1)

Step 6: At (3, 1).

```
0 0 0 1
0 0 1 1
0 0 0 0
0[1]1 1    <- API says 1!
```

- **API returns:** 1
- **Logic:** Best yet! Update `leftmost_col = 1`. Move LEFT.
- **Next position:** (3, 0)

Step 7: At (3, 0).

```
0 0 0 1
0 0 1 1
0 0 0 0
[0] 1 1 1    <- API says 0!
```

- **API returns:** 0
- **Logic:** Move DOWN.
- **Next position:** (4, 0) -> Out of bounds! Loop terminates.

Final Answer: `leftmost_col` is 1.

3. Time and Space Complexity Analysis

Let M be the number of rows and N be the number of columns.

TIME COMPLEXITY DERIVATION: $O(M + N)$

Grid Traversal Path:

Start (0, $N-1$)

```
  |
  V
[X]<---[X]<---[X]
      |
      V
      [X]
```

|
V
[END]

In the absolute worst-case scenario, we move LEFT exactly N times (across all columns) and we move DOWN exactly M times (across all rows). We never move right, and we never move up.

Max API calls = M (downs) + N (lefts) = M + N.
Therefore, Time Complexity is strictly $O(M + N)$.

SPACE COMPLEXITY DERIVATION: $O(1)$

Variables held in memory:

1. current_row (integer)
2. current_col (integer)
3. leftmost_col (integer)

We do not copy the matrix. We do not use a recursive call stack.
Therefore, Space Complexity is $O(1)$ Constant Space.

4. Solution Code

*(Note: Leetcode provides a **BinaryMatrix** interface with **get(row, col)** and **dimensions()** methods. The code below assumes this interface exists).*

Python Snippets

```
# =====  
# OPTIMAL SOLUTION (Staircase Search -  $O(M + N)$ )  
# =====  
class Solution:  
    def leftMostColumnWithOne(self, binaryMatrix: 'BinaryMatrix') -> int:  
        rows, cols = binaryMatrix.dimensions()  
  
        # Start at the top-right corner  
        current_row = 0  
        current_col = cols - 1  
  
        # This will hold our best answer so far  
        leftmost_col = -1  
  
        # Walk the staircase while we are within matrix boundaries  
        while current_row < rows and current_col >= 0:  
            if binaryMatrix.get(current_row, current_col) == 1:
```

```

        # We found a 1. Record this column as the best answer so far.
        leftmost_col = current_col
        # To find a better answer, we must look further left.
        current_col -= 1
    else:
        # We found a 0. This row has no 1s to the left.
        # Move down to the next row.
        current_row += 1

    return leftmost_col

# =====
# FALLBACK SOLUTION (Binary Search per Row -  $O(M * \log N)$ )
# =====
class SolutionFallback:
    def leftMostColumnWithOne(self, binaryMatrix: 'BinaryMatrix') -> int:
        rows, cols = binaryMatrix.dimensions()
        leftmost_col = cols # Initialize to an impossible max value

        for row in range(rows):
            # Binary search on the current row
            left = 0
            right = cols - 1
            found_in_row = False

            while left <= right:
                mid = (left + right) // 2
                if binaryMatrix.get(row, mid) == 1:
                    found_in_row = True
                    right = mid - 1 # Keep looking left
                else:
                    left = mid + 1 # Look right

            if found_in_row:
                leftmost_col = min(leftmost_col, left)

        return leftmost_col if leftmost_col < cols else -1

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (Staircase Search -  $O(M + N)$ )
// =====
/**
 * // This is the BinaryMatrix's API interface.
 * // You should not implement it, or speculate about its implementation

```

```

    * function BinaryMatrix() {
    * @param {integer} row, col
    * @return {integer}
    * this.get = function(row, col) { ... };
    *
    * @return {number[]}
    * this.dimensions = function() { ... };
    * };
    */

/**
 * @param {BinaryMatrix} binaryMatrix
 * @return {number}
 */
var leftMostColumnWithOne = function(binaryMatrix) {
    const dimensions = binaryMatrix.dimensions();
    const rows = dimensions[0];
    const cols = dimensions[1];

    let currentRow = 0;
    let currentCol = cols - 1;
    let leftmostCol = -1;

    // Walk the staircase
    while (currentRow < rows && currentCol >= 0) {
        if (binaryMatrix.get(currentRow, currentCol) === 1) {
            // Found a 1, record it, and move left to see if we can do better
            leftmostCol = currentCol;
            currentCol--;
        } else {
            // Found a 0, nothing better in this row, move down
            currentRow++;
        }
    }

    return leftmostCol;
};

// =====
// FALLBACK SOLUTION (Binary Search per Row - O(M * log N))
// =====
var leftMostColumnWithOneFallback = function(binaryMatrix) {
    const dimensions = binaryMatrix.dimensions();
    const rows = dimensions[0];
    const cols = dimensions[1];

```



```

let leftmostCol = cols;

for (let row = 0; row < rows; row++) {
  let left = 0;
  let right = cols - 1;
  let foundInRow = false;

  while (left <= right) {
    let mid = Math.floor((left + right) / 2);
    if (binaryMatrix.get(row, mid) === 1) {
      foundInRow = true;
      right = mid - 1;
    } else {
      left = mid + 1;
    }
  }

  if (foundInRow) {
    leftmostCol = Math.min(leftmostCol, left);
  }
}

return leftmostCol < cols ? leftmostCol : -1;
};

```

Note 1: Terminology

The $O(M + N)$ algorithm we used here is often called the **Staircase Search** or **Saddleback Search**. * **Why it helps:** It takes advantage of 2D properties where data increases or is sorted along two different axes. * **How it applies:** By starting at a specific corner (Top-Right or Bottom-Left), we can use the sorted nature of the matrix to safely eliminate an entire row or an entire column with a single $O(1)$ check, dramatically pruning our search space.

Note 2: Indirect / Real-World Interview Variations

Big Tech companies use this core algorithm to evaluate how you handle massive datasets where reading the data is an expensive operation (like querying a database or making a network call).

1. Google: Feature Flag Degradation Detection * **The Prompt:** “We have M microservices. Over the last N releases, a bug was introduced. Once a microservice gets the bug, it stays bugged in all future releases. You have an expensive API `test_service(service_id, release_version)` that returns pass/fail. Find the earliest release version that introduced the bug.” * **The**

Solution: This maps exactly to the matrix. Rows are microservices, columns are releases, 0 is pass, 1 is fail. Because the API is expensive (simulating a network call/test execution), you use the Staircase Search to find the leftmost column (earliest release) in $O(M + N)$ tests instead of $O(M * N)$.

2. Meta: Image Mask Edge Detection * The Prompt: “You have a large 2D image array representing a transparency mask. 0 is transparent, 1 is opaque. In this specific image format, once a pixel becomes opaque in a row, the rest of the row is opaque. Find the bounding box’s left edge without loading the whole image into memory.” * **The Solution:** Again, this is the exact same problem. The constraint “without loading the whole image” forces you to use an API or file-stream read (`get(row, col)`). The Staircase Search guarantees you minimize file I/O operations.

3. Bloomberg: Order Book Spread Crossing * The Prompt: “We track the ask prices of an asset across M different global exchanges over N milliseconds. Prices are strictly monotonically increasing throughout the day. Given a threshold price, find the earliest millisecond that *any* exchange crossed this price threshold. Querying historical tick data is slow.” * **The Solution:** Convert the prices to a binary concept: 0 = below threshold, 1 = above threshold. Since prices are increasing, the 0s will naturally be followed by 1s. Use the Staircase Search to find the earliest millisecond column.

523. Continuous Subarray Sum

This is a classic problem that separates junior developers from senior engineers. At a junior level, the instinct is to write a nested loop to check every possible subarray. At the L5/L6 level, we immediately recognize that any problem asking about the “sum of a continuous subarray” is begging for a **Prefix Sum** approach.

Furthermore, because it asks for a *multiple* of k , it requires an understanding of **Modulo Arithmetic**.

Let’s break down how a senior engineer approaches this, moving from the mathematical intuition to the optimal solution.

1. Problem Explanation

Imagine an array of integers: [23, 2, 4, 6, 7] and a target multiple $k = 6$.

The Goal: Find if there is *any* continuous sequence of numbers (at least two numbers long) that adds up to a multiple of k (e.g., 0, 6, 12, 18, 24, 30, 36, etc.).

The Catch: 1. The numbers must be **continuous** (next to each other). [2, 4] is valid, but [23, 6] is not. 2. The subarray must contain **at least two**

elements. A single 6 doesn't count. 3. The sum must be exactly $n * k$ where n is any integer. Note that 0 is always a multiple of k (since $0 * k = 0$).

In our example, the sequence [2, 4] sums to 6. Since 6 is a multiple of 6, and the length is 2 (which is ≥ 2), the answer is **True**.

2. Solution Explanation

Let's explore the progression of thought from the naive solution to the optimal one.

The Fallback (Naive): Nested Loops - $O(n^2)$ We can systematically test every single starting index, and iteratively add the next numbers to see if the sum eventually becomes a multiple of k . This works, but if the array is 100,000 items long, checking every subarray takes 10 billion operations. That's a Time Limit Exceeded error waiting to happen.

The Optimal L5/L6 Approach: Prefix Sum + Modulo Arithmetic - $O(n)$ We need to do this in a single pass. The secret weapon here is math, specifically the properties of remainders.

Imagine you are packing slices of pizza into boxes that hold exactly k slices. * You bake 14 slices. You pack them into boxes of 6. You get 2 full boxes and **2 slices left over** (remainder = 2). * You keep baking. Later, you have baked a total of 26 slices. You pack them into boxes of 6. You get 4 full boxes and **2 slices left over** (remainder = 2).

Why did the remainder stay the same? Because the number of *new* slices you baked between the first point (14) and the second point (26) was exactly 12... which is a perfect multiple of 6!

The Mathematical Invariant: If $(\text{prefix_sum_B} \% k) == (\text{prefix_sum_A} \% k)$, then the sum of the elements between A and B is perfectly divisible by k .

So, instead of keeping track of sums, we only need to keep track of **remainders** (modulo k). If we ever see the exact same remainder twice, we know the elements we added *between* those two moments sum up to a perfect multiple of k .

To do this, we use a Hash Map (Dictionary/Object) to store: { remainder: index_where_we_first_saw_it }.

End-to-End Walkthrough:

Let's use `nums = [23, 2, 4, 6, 7]`, $k = 6$.

Crucial Setup: We start our Hash Map with a "dummy" entry: { 0 : -1 }. *Why?* If our very first elements perfectly sum to a multiple of k right from index 0, the remainder will be 0. Setting index -1 ensures our distance calculation `current_index - (-1)` accurately reflects the length of the subarray.

Input: nums = [23, 2, 4, 6, 7], k = 6
Initial Map = { 0: -1 }
Current Running Sum = 0

Step 1: Index 0, Value 23

Running Sum = 0 + 23 = 23
Remainder = 23 % 6 = 5
Is 5 in Map? No.
Add to Map: { 0: -1, 5: 0 }

Array: [23]
Index: 0

Step 2: Index 1, Value 2

Running Sum = 23 + 2 = 25
Remainder = 25 % 6 = 1
Is 1 in Map? No.
Add to Map: { 0: -1, 5: 0, 1: 1 }

Array: [23, 2]
Index: 0 1

Step 3: Index 2, Value 4

Running Sum = 25 + 4 = 29
Remainder = 29 % 6 = 5
Is 5 in Map? YES!
Wait, at what index did we first see remainder 5? Index 0.
Current Index = 2.
Distance = Current Index (2) - Past Index (0) = 2.
Is Distance >= 2? YES!
We found a valid subarray! Return True.

Array: [23, 2, 4]
Index: 0 1 2
|---| <-- The elements added since we last saw remainder 5!
Sum = 6 (perfect multiple of 6)

3. Time and Space Complexity Analysis

Let N be the number of elements in the array.

TIME COMPLEXITY DERIVATION: $O(N)$

Array Processing:

[23, 2, 4, 6, 7]

| | | | | <-- We touch each element exactly once.

At each step, we do 3 fast operations:

1. Addition (Running Sum += nums[i]) -> $O(1)$
2. Modulo (Sum % k) -> $O(1)$
3. Hash Map Lookup & Insert -> $O(1)$ amortized

Total Time = N elements * $O(1)$ operations = $O(N)$ Time Complexity.

SPACE COMPLEXITY DERIVATION: $O(\min(N, k))$

Memory usage is entirely determined by the Hash Map.

Map structure: { remainder: index }

Worst case scenario: Every single running sum gives a different remainder.

If $N < k$: We can at most store N remainders. Space is $O(N)$.

If $N \geq k$: Modulo k can only produce 'k' distinct remainders (0 to k-1).

We can at most store 'k' remainders. Space is $O(k)$.

Total Space = $O(\min(N, k))$ Space Complexity.

4. Solution Code

Here are both solutions in Python and JavaScript. The $O(N)$ Hash Map solution is what gets you the L5/L6 hire. The Fallback $O(n^2)$ is what you write quickly if you completely blank out on the math, to at least have something working on the board.

Python Snippets

```
# =====  
# OPTIMAL SOLUTION (Prefix Sum Modulo -  $O(N)$ )  
# =====  
class Solution:  
    def checkSubarraySum(self, nums: list[int], k: int) -> bool:  
        # Initialize map with {0: -1} to handle cases where a valid  
        # subarray starts from the very first element (index 0).  
        remainder_map = {0: -1}  
  
        running_sum = 0
```

```

    for i in range(len(nums)):
        running_sum += nums[i]

        # Calculate remainder. Python handles negative numbers gracefully
        # with modulo, but the problem constraints say nums are non-negative.
        remainder = running_sum % k

        # If we have seen this remainder before...
        if remainder in remainder_map:
            # Check if the length of the subarray is at least 2
            distance = i - remainder_map[remainder]
            if distance >= 2:
                return True
        else:
            # IMPORTANT: Only add to map if it's NOT already there.
            # We want to keep the EARLIEST index we saw this remainder
            # to maximize the potential distance/length of the subarray.
            remainder_map[remainder] = i

    return False

# =====
# FALLBACK SOLUTION (Nested Loops - O(N^2))
# =====
class SolutionFallback:
    def checkSubarraySum(self, nums: list[int], k: int) -> bool:
        n = len(nums)
        # Check every possible starting point
        for i in range(n):
            current_sum = nums[i]
            # Iteratively add next elements to form subarrays of length 2 or more
            for j in range(i + 1, n):
                current_sum += nums[j]

                # Check if it's a multiple of k
                if current_sum % k == 0:
                    return True

    return False

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (Prefix Sum Modulo - O(N))
// =====
/**

```

```

    * @param {number[]} nums
    * @param {number} k
    * @return {boolean}
    */
var checkSubarraySum = function(nums, k) {
    // Initialize map. Key: remainder, Value: index
    // {0: -1} handles valid subarrays starting at index 0
    const remainderMap = new Map();
    remainderMap.set(0, -1);

    let runningSum = 0;

    for (let i = 0; i < nums.length; i++) {
        runningSum += nums[i];

        const remainder = runningSum % k;

        if (remainderMap.has(remainder)) {
            // Found a matching remainder! Check the length.
            const previousIndex = remainderMap.get(remainder);
            const distance = i - previousIndex;

            if (distance >= 2) {
                return true;
            }
        } else {
            // Only insert the remainder if we haven't seen it yet
            // to preserve the longest possible subarray distance.
            remainderMap.set(remainder, i);
        }
    }

    return false;
};

// =====
// FALLBACK SOLUTION (Nested Loops - O(N^2))
// =====
var checkSubarraySumFallback = function(nums, k) {
    const n = nums.length;

    for (let i = 0; i < n; i++) {
        let currentSum = nums[i];

        for (let j = i + 1; j < n; j++) {
            currentSum += nums[j];

```

```

        if (currentSum % k === 0) {
            return true;
        }
    }

    return false;
};

```

Note 1: Terminology

The core technique used here is called **Prefix Sum with Modulo Arithmetic**.
*** Why it helps:** A “Prefix Sum” allows you to find the sum of *any* slice of an array in $O(1)$ time by taking the sum up to the end of the slice and subtracting the sum up to the start of the slice. Applying “Modulo Arithmetic” directly to the running sum prevents us from tracking massive numbers, compressing our search space into just k possible remainder states. *** How it applies:** It transforms an $O(N^2)$ “sum everything” search into an $O(N)$ “have I seen this state before?” lookup.

Note 2: Indirect / Real-World Interview Variations

Big Tech companies rarely ask this problem mathematically; they wrap it in system design/backend scenarios:

1. Google: The Load Balancer Request Batcher * **The Prompt:** “We have a stream of incoming web requests of varying payload sizes (in KB). We want to batch process them, but our downstream server only accepts data in exact 1024KB (1MB) blocks to avoid fragmentation. Can you find a contiguous burst of requests that perfectly aligns to a 1MB block multiple?” * **The Solution:** This is the exact Leetcode problem. `nums` is the payload sizes, `k` is 1024. The requirement for a “burst” of requests maps to the “size ≥ 2 ” constraint. You solve this with the exact $O(N)$ Hash Map approach above to process the stream in real-time.

2. Meta: Ad Campaign Budget Capping * **The Prompt:** “An advertiser wants to spend a total daily budget in exact multiples of \$500. We have a log of hourly ad spend over the past month. Did the campaign ever run for at least 2 consecutive hours where the total spend was an exact multiple of \$500?” * **The Solution:** Exact same algorithm. `nums` is the hourly spend log, `k` is 500. This is often framed as an anomaly detection or billing reconciliation question.

3. Bloomberg: Block Trade Volume Execution * **The Prompt:** “To hide institutional trading activity, we execute ‘block trades’ in exact multiples of 10,000 shares. Given a time-series tape of individual retail trade sizes on

the market, identify any continuous window of trades that sums perfectly to a block trade multiple, indicating a potential hidden institutional sweep.” * **The Solution:** Same $O(N)$ Hash Map solution. In a Bloomberg interview, the array (`nums`) will be massive (millions of ticks per day), making the $O(N^2)$ approach instantly fail their constraints. You would emphasize the memory efficiency of $O(\min(N, k))$ —since k is fixed at 10,000, your memory footprint never exceeds 10,000 integers, easily fitting into the CPU’s L1 cache for blazing fast execution.

224. Basic Calculator

This is a fantastic problem. As an interviewer, I often see candidates try to overcomplicate this by implementing Dijkstra’s “Shunting Yard” algorithm to convert the expression into Reverse Polish Notation (Postfix) before evaluating it.

While Shunting Yard works, an L5/L6 engineer recognizes a critical constraint in the prompt: **we only have addition + and subtraction -**.

Because + and - share the exact same precedence, we don’t need complex priority queues or two-pass parsing. We can evaluate the math on the fly, strictly reading left to right. The only real challenge is the parentheses, which we can handle elegantly with a Stack.

Let’s break down the optimal approach step-by-step.

1. Problem Explanation

Imagine you are given a string like `"10 - ( 2 + 3 )"`.

The Goal: Calculate the final mathematical result. **The Allowed Characters:** * Digits (0-9) * Plus (+) and Minus (-) * Parentheses (and) * Spaces (which we must ignore)

The Core Challenges: 1. **Multi-digit numbers:** The string contains characters like `'1'` and `'0'`, which need to be combined into the integer 10. 2. **The Unary Minus / Parentheses:** A minus sign outside a parenthesis flips the sign of *everything* inside. In `"10 - (2 + 3)"`, that minus applies to the whole `(2 + 3)` block. We can’t just subtract 2 and add 3; we have to resolve the block first, then apply the minus.

2. Solution Explanation

To solve this in a single left-to-right pass ($O(N)$ time), we maintain three simple variables: 1. `current_number`: Builds our multi-digit numbers (e.g., seeing `'1'` then `'0'` becomes 10). 2. `current_result`: The running sum of the current

mathematical layer we are in. 3. **sign**: Either 1 (for positive) or -1 (for negative).

The Stack Strategy (The “Why”): Whenever we hit an open parenthesis (, it means we are entering a brand new sub-world of math. We must pause our current calculation, save it, and start fresh. * We **push** our **current_result** and the **sign** that appeared right before the parenthesis onto a Stack. * We reset **current_result** to 0 and **sign** to 1 to calculate the inside of the parenthesis cleanly.

Whenever we hit a close parenthesis), the sub-world is finished. * We take the result of the sub-world, multiply it by the **sign** we saved on the stack, and add it to the **current_result** we saved on the stack. We are back in the outer world!

End-to-End Walkthrough:

Let's evaluate the expression: "4 - (10 + 2)"

Variables:

result = 0, sign = 1, number = 0

stack = []

Step 1: Read '4'

Build the number: number = 0 * 10 + 4 = 4.

State: res=0, sign=1, num=4, stack=[]

Step 2: Read ' ' (space)

Ignore spaces.

Step 3: Read '-'

We hit an operator! This means our previous number is fully built.

1. Add to result: res = res + (sign * num) -> 0 + (1 * 4) = 4

2. Reset number: num = 0

3. Update sign for the NEXT number: sign = -1

State: res=4, sign=-1, num=0, stack=[]

Step 4: Read '('

We are entering a sub-expression!
1. Push current result (4) and current sign (-1) to stack.
2. Reset for the sub-expression: res = 0, sign = 1.

State: res=0, sign=1, num=0
Stack: [4, -1] <-- Top of stack is -1, Bottom is 4

Step 5: Read '1', then '0'

Build the number: num = 10.

State: res=0, sign=1, num=10
Stack: [4, -1]

Step 6: Read '+'

Operator hit!
1. Add to result: res = 0 + (1 * 10) = 10
2. Reset number: num = 0
3. Update sign: sign = 1

State: res=10, sign=1, num=0
Stack: [4, -1]

Step 7: Read '2'

Build number: num = 2.

State: res=10, sign=1, num=2
Stack: [4, -1]

Step 8: Read ')'

Sub-expression finished!
1. Add the final number inside the parenthesis to the local result:
 res = 10 + (1 * 2) = 12. (The parenthesis evaluated to 12!)
2. Apply the saved sign: res = res * stack.pop() -> 12 * -1 = -12
3. Add the saved result: res = res + stack.pop() -> -12 + 4 = -8
4. Reset number: num = 0

State: res=-8, sign=1, num=0
Stack: []

Step 9: End of String

We reached the end. Add any lingering number to the result.
 $\text{res} = \text{res} + (\text{sign} * \text{num}) \rightarrow -8 + (1 * 0) = -8.$

Final Answer: -8.

(Mathematical proof: $4 - (10 + 2) = 4 - 12 = -8$).

3. Time and Space Complexity Analysis

Let N be the length of the string s .

TIME COMPLEXITY: $O(N)$

String: " 4 - (10 + 2) "
 | | | | | | | |

We step through the string exactly one time, left to right.

At each character, we do $O(1)$ operations (add, multiply, push to stack, pop from stack).
There are no nested loops.

Total Time = $N * O(1) = O(N)$

SPACE COMPLEXITY: $O(N)$

The only extra memory we use is the Stack.

When does the stack grow? Every time we see an open parenthesis '('.

Worst Case Scenario: "((((((((1)))))))))"

If the string is entirely nested parentheses, we push to the stack $N/2$ times.
Since $N/2$ scales linearly with N , our space complexity is $O(N)$.

Total Space = $O(N)$

4. Solution Code

I will provide the optimal $O(N)$ Stack-based iteration.

I will also provide a fallback "Recursive Descent" solution. While the stack is iteratively better for memory constraints in extreme edge cases, recursion is deeply respected by interviewers because it models an Abstract Syntax Tree (AST) evaluator, which is how actual programming language compilers work.

Python Snippets

```
# =====
# OPTIMAL SOLUTION (Iterative Stack - O(N))
# =====
class Solution:
    def calculate(self, s: str) -> int:
        stack = []
        current_result = 0
        current_number = 0
        sign = 1 # 1 represents positive, -1 represents negative

        for char in s:
            if char.isdigit():
                # Build multi-digit numbers (e.g., '1' followed by '0' -> 10)
                current_number = (current_number * 10) + int(char)

            elif char == '+':
                # Add the accumulated number to the running total
                current_result += sign * current_number
                # Reset sign and number for the next term
                sign = 1
                current_number = 0

            elif char == '-':
                # Add the accumulated number to the running total
                current_result += sign * current_number
                # Set sign to negative for the next term
                sign = -1
                current_number = 0

            elif char == '(':
                # We are entering a new scope.
                # Save the current result and the sign applied to this parenthesis.
                stack.append(current_result)
                stack.append(sign)
                # Reset everything for the new scope inside the parenthesis
                current_result = 0
                sign = 1

            elif char == ')':
                # First, add the last number inside the parenthesis to the local result
                current_result += sign * current_number

                # Now, pop the sign that was right before the '(' and multiply it
                current_result *= stack.pop()
```

```

        # Then, pop the result from before the '(' and add it
        current_result += stack.pop()

        # Reset the current number
        current_number = 0

        # Add any trailing number to the final result
        return current_result + (sign * current_number)

# =====
# FALLBACK SOLUTION (Recursive Descent / AST approach)
# =====
class SolutionFallback:
    def calculate(self, s: str) -> int:
        def evaluate(i):
            current_result = 0
            current_number = 0
            sign = 1

            while i < len(s):
                char = s[i]

                if char.isdigit():
                    current_number = (current_number * 10) + int(char)
                elif char in '+-':
                    current_result += sign * current_number
                    sign = 1 if char == '+' else -1
                    current_number = 0
                elif char == '(':
                    # Recursively evaluate the inside of the parenthesis
                    # The recursive call returns the evaluated number AND the new index
                    num, i = evaluate(i + 1)
                    current_number = num
                elif char == ')':
                    current_result += sign * current_number
                    # Return the result of this block, and the index we stopped at
                    return current_result, i

            i += 1

        return current_result + (sign * current_number)

    return evaluate(0)

```

JavaScript Snippets

```
// =====  
// OPTIMAL SOLUTION (Iterative Stack - O(N))  
// =====  
/**  
 * @param {string} s  
 * @return {number}  
 */  
var calculate = function(s) {  
    let stack = [];  
    let currentResult = 0;  
    let currentNumber = 0;  
    let sign = 1;  
  
    for (let i = 0; i < s.length; i++) {  
        let char = s[i];  
  
        if (char >= '0' && char <= '9') {  
            // Build the multi-digit number  
            currentNumber = (currentNumber * 10) + parseInt(char);  
        } else if (char === '+') {  
            currentResult += sign * currentNumber;  
            sign = 1;  
            currentNumber = 0;  
        } else if (char === '-') {  
            currentResult += sign * currentNumber;  
            sign = -1;  
            currentNumber = 0;  
        } else if (char === '(') {  
            // Push current context to stack  
            stack.push(currentResult);  
            stack.push(sign);  
            // Reset for the new scope  
            currentResult = 0;  
            sign = 1;  
        } else if (char === ')') {  
            currentResult += sign * currentNumber;  
  
            // Multiply by the saved sign  
            currentResult *= stack.pop();  
            // Add to the saved result  
            currentResult += stack.pop();  
  
            currentNumber = 0;  
        }  
    }  
}
```

```

    }

    return currentResult + (sign * currentNumber);
};

// =====
// FALLBACK SOLUTION (Recursive Descent / AST approach)
// =====
var calculateFallback = function(s) {
    let i = 0; // Global index pointer

    const evaluate = () => {
        let currentResult = 0;
        let currentNumber = 0;
        let sign = 1;

        while (i < s.length) {
            let char = s[i];

            if (char >= '0' && char <= '9') {
                currentNumber = (currentNumber * 10) + parseInt(char);
            } else if (char === '+' || char === '-') {
                currentResult += sign * currentNumber;
                sign = char === '+' ? 1 : -1;
                currentNumber = 0;
            } else if (char === '(') {
                i++; // Skip the '('
                currentNumber = evaluate(); // Recursively calculate inner
            } else if (char === ')') {
                currentResult += sign * currentNumber;
                return currentResult; // Exit recursion
            }

            i++;
        }

        return currentResult + (sign * currentNumber);
    };

    return evaluate();
};

```

Note 1: Terminology

The iterative method we used is essentially building a **State Machine with a Stack**. * **Why it helps:** A state machine allows you to define strict rules for what to do when you see a specific character based on your *current* state (building a number vs. assigning an operator). The stack gives the state machine “memory,” allowing it to pause its current state, dive into a nested problem, and resume perfectly. * **How it applies:** This is exactly how your browser parses HTML tags! When it sees a `<div>`, it pushes context to a stack, parses the children, and when it sees `</div>`, it pops back up to the parent.

The recursive method is a basic form of **Recursive Descent Parsing**. * **Why it helps:** It directly maps to how languages build Abstract Syntax Trees (AST).

Note 2: Indirect / Real-World Interview Variations

Big Tech companies use this parsing logic as the foundation for systems design and string manipulation questions:

1. Google: The Spreadsheet Formula Evaluator * **The Prompt:** “You are building a basic version of Google Sheets. Users can type string formulas like `=(A1 + 5) - B2`. Given a dictionary mapping cell names (like A1) to integers, write a function to evaluate the final value of the formula.” * **The Solution:** Exact same Stack/State Machine logic. The only difference is in the string parsing: when you encounter a letter instead of a digit, you read characters until you hit an operator, use that string (e.g., “A1”) to do a hash map lookup, fetch the integer 10, and treat it as `current_number`.

2. Meta: The UI Component Metric Aggregator * **The Prompt:** “We have a serialized UI component tree sent from the mobile app represented as a string: `Container( Button(5) + Image(10) ) - Banner(2)`. The numbers represent memory cost. Write a parser to calculate the total memory cost of the component.” * **The Solution:** This is a disguised recursive descent parser. Instead of ignoring letters, you ignore the component names. You treat `(` and `)` exactly as we did in the fallback solution, treating the whole string as a math evaluation problem to sum up the memory footprint.

3. Bloomberg: The Boolean Risk Query Parser * **The Prompt:** “Traders filter stocks using boolean logic queries represented as strings, like `TRUE AND (FALSE OR TRUE)`. Write a function to evaluate if a stock passes the filter.” * **The Solution:** While the operators change (`AND`, `OR` instead of `+`, `-`), the structure is identical. You maintain a stack to handle the parentheses. `AND` and `OR` have different precedences (unlike `+` and `-`), so you will need to evaluate `AND` immediately and defer `OR` operations, but the core LIFO (Last-In-First-Out) stack mechanism handles the nested brackets perfectly.

227. Basic Calculator II

Hey there! While I don't sip coffee in a Google micro-kitchen—being an AI and all—I can absolutely put on my L5/L6 engineer hat and break this down for you exactly how I'd expect a strong candidate to tackle it in an interview.

Basic Calculator II is a brilliant problem. It tests your ability to handle operator precedence (PEMDAS/BODMAS) without getting tangled in complex logic. Let's dive in.

1. Problem Explanation

The Goal: We are given a string like "3+5 / 2" and we need to evaluate it like a standard calculator. **The Rules:** * Only non-negative integers. * Four operators: +, -, *, /. * Empty spaces can be anywhere and should be ignored. * Division should **truncate toward zero** (e.g., 5 / 2 = 2, not 2.5). * **Crucial Rule:** Multiplication * and Division / happen *before* Addition + and Subtraction -.

If we process left to right blindly on "3 + 5 * 2", we get $8 * 2 = 16$. But math dictates we do $5 * 2$ first, resulting in $3 + 10 = 13$.

2. Solution Explanation

To solve this, we have to deal with the fact that when we see a + or -, we can't apply it immediately because a * or / might be coming up next.

Approach 1: The Stack (The Intuitive Way)

Think of a stack as a bucket where we throw numbers that are ready to be added together at the very end. 1. If we see a +, the number is positive. Throw it in the bucket. 2. If we see a -, the number is negative. Throw it in the bucket. 3. If we see a * or /, we pull the *last* number out of the bucket, do the math with the *current* number, and throw the result back in.

Let's trace " 3 - 5 * 2 " with ASCII diagrams:

INITIAL STATE:

String: " 3 - 5 * 2 "

Current Number: 0

Previous Operator: '+' (We pretend there's a '+' before the first number)

Stack: []

STEP 1: Read '3'

Current Number becomes 3.

Next character is '-', an operator!

Action: Look at Previous Operator ('+').

Push Current Number to Stack.

```
Stack:      |      |
            |__3__|
Reset: Current Number = 0, Previous Operator = '-'
```

STEP 2: Read '5'
 Current Number becomes 5.
 Next character is '*', an operator!
 Action: Look at Previous Operator ('-').
 Push negative Current Number to Stack.

```
Stack:      | -5  |
            |__3__|
Reset: Current Number = 0, Previous Operator = '*'
```

STEP 3: Read '2' (End of string)
 Current Number becomes 2.
 End of string triggers the final evaluation!
 Action: Look at Previous Operator ('*').
 Pop the top of the Stack (-5), multiply by Current Number (2). $(-5 * 2 = -10)$.
 Push result back to Stack.

```
Stack:      |-10 |  <-- (-5 * 2)
            |__3__|
```

FINAL STEP: Sum the Stack
 Result = $3 + (-10) = -7$

Approach 2: O(1) Space Optimization (The “Top-of-Band” Way) If you look closely at the Stack approach, we only ever care about the *top* of the stack. We never dig deep into it. We just sum it all up at the end.

Instead of an actual stack array, we can just keep track of two variables: ***result**: The total sum of everything we’ve finished processing. ***last_number**: The number that *would* have been at the top of the stack.

When we hit a + or -, we add **last_number** to **result**, and update **last_number** to the new number. When we hit a * or /, we just modify **last_number** on the fly.

3. Time and Space Complexity Analysis

Here is the breakdown of why the optimized approach is the gold standard:

```
+-----+
| COMPLEXITY ANALYSIS: OPTIMIZED APPROACH |
+-----+
| TIME COMPLEXITY: O(N) |
| -> N is the length of the string. |
```

```

| -> We loop through the string exactly once. |
| -> String parsing and basic math operations take constant time. |
| |
| SPACE COMPLEXITY: O(1) |
| -> We only use a few integer variables (result, last_number, |
|     current_number, sign). |
| -> No arrays or dynamic data structures are allocated. |
+-----+

+-----+
| COMPLEXITY ANALYSIS: STACK APPROACH |
+-----+
| TIME COMPLEXITY: O(N) |
| -> One pass through the string, plus one pass through the stack. |
| |
| SPACE COMPLEXITY: O(N) |
| -> In the worst case (e.g., "1+2+3+4+5"), we push N/2 numbers |
|     onto our stack array, requiring memory proportional to N. |
+-----+

```

4. Solution Code

Here are both approaches. As an interviewer, I would expect you to write the Stack approach first to prove you understand the logic, and then optimize it to O(1) space if prompted.

Python Code

```

# Approach 1: Stack-based (O(N) Space)
def calculate_stack(s: str) -> int:
    stack = []
    current_number = 0
    operator = '+'

    # We append a dummy operator at the end to force the last calculation
    s += '+'

    for i in range(len(s)):
        char = s[i]

        if char.isdigit():
            # Build the number (handles multi-digit like '123')
            current_number = (current_number * 10) + int(char)

        if char in '+-*/' or i == len(s) - 1:

```

```

    if operator == '+':
        stack.append(current_number)
    elif operator == '-':
        stack.append(-current_number)
    elif operator == '*':
        # Pop, multiply, push back
        stack.append(stack.pop() * current_number)
    elif operator == '/':
        # Pop, divide, push back.
        # Note: int() truncates toward zero in Python, unlike // which floors!
        stack.append(int(stack.pop() / current_number))

    # Reset for the next number
    operator = char
    current_number = 0

return sum(stack)

# Approach 2: Optimized (O(1) Space)
def calculate_optimized(s: str) -> int:
    result = 0
    last_number = 0
    current_number = 0
    operator = '+'

    s += '+'

    for i in range(len(s)):
        char = s[i]

        if char.isdigit():
            current_number = (current_number * 10) + int(char)

        if char in '+-*/' or i == len(s) - 1:
            if operator == '+' or operator == '-':
                # Commit the last number to the final result
                result += last_number
                # Update last_number with the new sign
                last_number = current_number if operator == '+' else -current_number
            elif operator == '*':
                # Modify last_number directly
                last_number = last_number * current_number
            elif operator == '/':
                # Modify last_number directly, truncating toward zero
                last_number = int(last_number / current_number)

```

```

        operator = char
        current_number = 0

    # Add the very last chunk to the result
    result += last_number
    return result

```

JavaScript Code

// Approach 1: Stack-based (O(N) Space)

```

function calculateStack(s) {
    let stack = [];
    let currentNumber = 0;
    let operator = '+';

    // Append dummy operator
    s += '+';

    for (let i = 0; i < s.length; i++) {
        let char = s[i];

        if (char >= '0' && char <= '9') {
            currentNumber = (currentNumber * 10) + parseInt(char);
        }

        if (char === '+' || char === '-' || char === '*' || char === '/' || i === s.length - 1) {
            if (char === ' ') continue; // Skip spaces safely

            if (operator === '+') {
                stack.push(currentNumber);
            } else if (operator === '-') {
                stack.push(-currentNumber);
            } else if (operator === '*') {
                stack.push(stack.pop() * currentNumber);
            } else if (operator === '/') {
                // Math.trunc ensures division truncates toward zero
                stack.push(Math.trunc(stack.pop() / currentNumber));
            }

            operator = char;
            currentNumber = 0;
        }
    }

    return stack.reduce((acc, curr) => acc + curr, 0);
}

```

```

}

// Approach 2: Optimized (O(1) Space)
function calculateOptimized(s) {
    let result = 0;
    let lastNumber = 0;
    let currentNumber = 0;
    let operator = '+';

    s += '+';

    for (let i = 0; i < s.length; i++) {
        let char = s[i];

        if (char >= '0' && char <= '9') {
            currentNumber = (currentNumber * 10) + parseInt(char);
        }

        if (char === '+' || char === '-' || char === '*' || char === '/' || i === s.length - 1) {
            if (char === ' ') continue;

            if (operator === '+' || operator === '-') {
                result += lastNumber;
                lastNumber = operator === '+' ? currentNumber : -currentNumber;
            } else if (operator === '*') {
                lastNumber = lastNumber * currentNumber;
            } else if (operator === '/') {
                lastNumber = Math.trunc(lastNumber / currentNumber);
            }

            operator = char;
            currentNumber = 0;
        }
    }

    result += lastNumber;
    return result;
}

```

Note 1: Terminology Used

- **Stack:** A “Last-In-First-Out” (LIFO) data structure. Helps here by “delaying” addition and subtraction operations until the higher-priority multiplications and divisions are resolved.

- **Accumulator Pattern / Running Total:** Used in the $O(1)$ space optimization. Instead of storing history, we accumulate the final answer incrementally.
- **Lexing/Parsing:** Extracting tokens (numbers and operators) from a raw string. Building `current_number = current_number * 10 + digit` is a basic form of creating a lexer for integers.
- **Truncate Toward Zero:** Rounding a fractional number towards 0. `-2.5` becomes `-2`. In JS, `Math.trunc()` handles this. In Python, `int()` handles this (beware of Python's `//` floor division, which rounds `-2.5` down to `-3`).

Note 2: Real-World Interview Variations (Meta, Google, Bloomberg)

Companies rarely ask the exact LeetCode prompt anymore. Here is how they twist it:

1. Meta: “Add parentheses support” (Basic Calculator I or III) *
The Twist: The string now contains (and). Ex: `3 * (5 - 2)`.
*** How to solve:** Recursion. Whenever you encounter a (, you make a recursive call to your `calculate` function. When you encounter a), you return the calculated value so far. The string index needs to be passed globally or via reference so the outer function knows where to resume parsing after the recursive call finishes.

2. Google: “Calculator with Variable Bindings” *
The Twist: You are given an expression like `"A + B * 2"` and a hash map of variables `{"A": 3, "B": 5}`. Sometimes, variables can map to other expressions `{"A": "C + 1"}`.
*** How to solve:** Add an environment/context map to your function signature. Before applying the stack logic, check if the current token is a string (variable). If it is, look it up in the map. If the map points to another expression, use Depth First Search (DFS) with memoization to evaluate it, watching out for circular dependencies (e.g., A depends on B, B depends on A) by keeping a “visited” set during traversal.

3. Bloomberg: “Validate before Calculating” or “Custom Operators” *
The Twist: You might get invalid strings like `"* 3 + 5"` or `"4 + + 5"`. Or, they might ask you to implement a `^` (power) operator which has higher precedence than `*`.
*** How to solve:** For validation, explicitly track the “expected next state”. A number must be followed by an operator (or end of string). An operator must be followed by a number. Throw exceptions if the contract is violated. For custom operators, the “ $O(1)$ space” trick gets messy. This is when you pivot to building an **Abstract Syntax Tree (AST)** or using the **Shunting Yard Algorithm** to convert the string to Postfix notation, making it infinitely extensible for new operators and precedence rules.

766. Toeplitz Matrix

Hey there! As an engineer who has been on both sides of the interview table at Google, I totally get why this problem can seem tricky at first glance. The name “Toeplitz” sounds like complex linear algebra, but at its core, this is a pure pattern-recognition problem.

Let’s break this down end-to-end exactly as we would during a top-tier tech interview, focusing on the *why* just as much as the *how*.

1. Problem Explanation

You are given a 2D grid of numbers (a matrix) with **M** rows and **N** columns. You need to return **True** if every diagonal going from top-left to bottom-right contains the exact same number. If even one diagonal has mismatched numbers, you return **False**.

Here is what a valid Toeplitz matrix looks like (Example 1):

```
c=0 c=1 c=2 c=3
r=0 [ 1 , 2 , 3 , 4 ]
r=1 [ 5 , 1 , 2 , 3 ]
r=2 [ 9 , 5 , 1 , 2 ]
```

Notice the diagonals: * Starts at (0,0): 1 -> 1 -> 1 (All 1s) * Starts at (0,1): 2 -> 2 -> 2 (All 2s) * Starts at (0,2): 3 -> 3 (All 3s) * Starts at (1,0): 5 -> 5 (All 5s)

Every diagonal holds steady. That’s a **True** case.

2. Solution Explanation

The Naive Thought: Your first instinct might be to extract every single diagonal into a list and check if all elements in that list are identical. This requires complex indexing (like grouping by `row - col`) and extra memory.

The Google L5/L6 Insight: We don’t need to trace the entire diagonal! We can use the **Transitive Property**. Imagine a line of people holding hands. If Person A is wearing a red shirt, and Person B matches Person A, and Person C matches Person B... then the whole line is wearing red.

Translated to our matrix: If *every single cell* simply matches the cell immediately **up and to its left**, the whole matrix is Toeplitz.

Instead of walking diagonals, we just process the matrix row by row. For every number, we “look over our left shoulder” (`row - 1, col - 1`). If it matches, we are good.

Walkthrough: Example 1 (The Success Case)

Let's compare Row 1 against Row 0 (the row above it):

```
      c=0    c=1    c=2    c=3
+-----+-----+-----+-----+
|  1  |  2  |  3  |  4  | <- Row 0 (Previous)
+-----+-----+-----+-----+
      \      \      \
      v      v      v
+-----+-----+-----+-----+
|  5  |  1  |  2  |  3  | <- Row 1 (Current)
+-----+-----+-----+-----+
```

Checks:

- Cell(1,1) is 1. Top-Left(0,0) is 1. Match!
- Cell(1,2) is 2. Top-Left(0,1) is 2. Match!
- Cell(1,3) is 3. Top-Left(0,2) is 3. Match!

Next, compare Row 2 against Row 1:

```
      c=0    c=1    c=2    c=3
+-----+-----+-----+-----+
|  5  |  1  |  2  |  3  | <- Row 1 (Previous)
+-----+-----+-----+-----+
      \      \      \
      v      v      v
+-----+-----+-----+-----+
|  9  |  5  |  1  |  2  | <- Row 2 (Current)
+-----+-----+-----+-----+
```

Checks:

- Cell(2,1) is 5. Top-Left(1,0) is 5. Match!
- Cell(2,2) is 1. Top-Left(1,1) is 1. Match!
- Cell(2,3) is 2. Top-Left(1,2) is 2. Match!

Everything matched! Return True.

Walkthrough: Example 2 (The Failure Case) Input:

```
r=0 [ 1 , 2 ]
```

```
r=1 [ 2 , 2 ]
```

Compare Row 1 against Row 0:

```
      c=0    c=1
+-----+-----+
```

```

| 1 | 2 | <- Row 0
+---+---+
      \
      v
+---+---+
| 2 | 2 | <- Row 1
+---+---+
      ^
      |
Comparing: Current (1, 1) is 2. Top-Left (0, 0) is 1.
2 != 1 -> ERROR! Return False immediately.

```

3. Time and Space Complexity Analysis

Time Complexity Derivation Diagram:

```

+---+---+---+---+
| . | . | . | . | <- Row 0: 0 comparisons
+---+---+---+---+
| . | * | * | * | <- Row 1: (N-1) comparisons
+---+---+---+---+
| . | * | * | * | <- Row 2: (N-1) comparisons
+---+---+---+---+
      ^
Col 0: 0 comparisons

```

Total '*' cells visited = (M - 1) rows * (N - 1) columns.

Since we do exactly 1 check per '*' cell, it scales linearly with the grid size.

- **Time Complexity:** $O(M * N)$ where M is the total number of rows and N is the total number of columns. We visit every cell essentially once.
 - **Space Complexity:** $O(1)$ for the standard approach because we only use pointers (row and col variables) to compare values already sitting in memory.
-

4. Solution Code

I'll provide two implementations in both Python and JavaScript. 1. **The Standard Approach** (Assuming the matrix fits fully in memory). 2. **The Streaming Approach** (A real-world interview optimization where we only store one row at a time—crucial for massive datasets).

Python Snippets

```

# 1. Standard Approach: O(1) Space
def isToeplitzMatrix(matrix):
    # Iterate through rows, skipping the first row (no top-left neighbor)
    for r in range(1, len(matrix)):
        # Iterate through columns, skipping the first column
        for c in range(1, len(matrix[0])):
            # If current cell doesn't match the cell up and to the left, we fail
            if matrix[r][c] != matrix[r-1][c-1]:
                return False
    return True

# 2. Optimized Streaming Approach: O(N) Space
def isToeplitzMatrixStreaming(matrix):
    if not matrix:
        return True

    # Store the first row as our initial 'previous row' tracker
    prev_row = matrix[0]

    # Process stream row by row
    for r in range(1, len(matrix)):
        curr_row = matrix[r]

        # Compare current row elements with the stored previous row
        for c in range(1, len(curr_row)):
            if curr_row[c] != prev_row[c-1]:
                return False

        # Update the tracker for the next iteration
        prev_row = curr_row

    return True

```

JavaScript Snippets

```

// 1. Standard Approach: O(1) Space
function isToeplitzMatrix(matrix) {
    // Iterate through rows starting from index 1
    for (let r = 1; r < matrix.length; r++) {
        // Iterate through columns starting from index 1
        for (let c = 1; c < matrix[0].length; c++) {
            // Check against the top-left neighbor
            if (matrix[r][c] !== matrix[r - 1][c - 1]) {
                return false;
            }
        }
    }
}

```

```

    }
  }
  return true;
}

// 2. Optimized Streaming Approach: O(N) Space
function isToeplitzMatrixStreaming(matrix) {
  if (matrix.length === 0) return true;

  // Track the previous row
  let prevRow = matrix[0];

  for (let r = 1; r < matrix.length; r++) {
    let currRow = matrix[r];

    // Check current row against the tracked previous row
    for (let c = 1; c < currRow.length; c++) {
      if (currRow[c] !== prevRow[c - 1]) {
        return false;
      }
    }
    // Current row becomes the previous row for the next cycle
    prevRow = currRow;
  }

  return true;
}

```

Note 1: Core Techniques Used

- **Transitive Property:** Mentioned above, this is the mathematical rule that if $A = B$, and $B = C$, then $A = C$. It saves us from having to traverse and store whole continuous diagonals.
 - **Row-by-Row Streaming (State Tracking):** In the optimized solution, we use a sliding window concept applied to rows. Instead of looking backward at a massive grid in memory, we maintain a small piece of “state” (the `prev_row`) and carry it forward. This prevents out-of-memory errors on massive datasets.
-

Note 2: Real World / Interview Follow-ups (Google, Meta, Bloomberg)

At top companies, the base problem is just a warm-up. The real evaluation happens in the follow-ups. Here is exactly how they will pressure test you:

Follow-up A (Meta / Google): “What if the matrix is stored on disk and is so huge you can only load one row into memory at a time?” * **How to solve it:** This is exactly why I provided the **Streaming Approach** in the code section! Because we only ever look at `matrix[r-1]`, we never need the whole matrix. We just read a stream, holding `curr_row` and `prev_row` in RAM. When we finish checking `curr_row`, we overwrite `prev_row` with it and fetch the next row from disk. Space complexity drops from storing a massive grid to just $O(N)$ memory.

Follow-up B (Bloomberg / Google): “What if the matrix is so ridiculously large that even a SINGLE ROW cannot fit into memory? You can only load partial rows (chunks).” * **How to solve it:** We process the matrix in vertical chunks (e.g., columns 0 to 5000, then columns 5001 to 10000). But there is a trap! To check the first column of Chunk 2, you need the last element from the row above it from Chunk 1. * **The Fix:** When processing Chunk 1, you store the very last element of `prev_row` in a small variable. You “carry it over” to act as the missing top-left neighbor when you start processing Chunk 2!

Visualizing Follow-up B (Chunking):

Processing Chunk 1 (Cols 0-1)

```
      c=0    c=1
+-----+-----+
|  1  |  2  |-->| (Save '2' to carry over)
+-----+-----+
      \      \
      v      v
+-----+-----+
|  5  |  1  |-->| (Save '1' to carry over)
+-----+-----+
```

Processing Chunk 2 (Cols 2-3)

```
              c=2    c=3
              +-----+
(Saved '2') -->|  3  |  4  |
              +-----+
              \      \      \
              v      v      v
              +-----+
(Saved '1') -->|  2  |  3  |
              +-----+
```

By explicitly managing this “carry over” state, you can process infinitely large

matrices across networks or disks with a strictly bounded memory footprint.

498. Diagonal Traverse

This is a classic problem that can easily trip up even experienced engineers if they jump straight into coding without analyzing the boundary conditions. I completely understand why it feels hard to grasp—navigating a 2D matrix diagonally creates a lot of weird edge cases when you hit the corners.

As an AI, I don't "feel" frustration, but I process thousands of solutions, and I can tell you that a junior engineer usually tries to solve this with a massive, tangled web of `if/else` statements that eventually breaks on non-square matrices (like a 2x4 grid).

A top-tier L5/L6 engineer takes a breath, maps out the exact state transitions (the boundaries), and realizes the "secret" to this problem is **the order in which you check the boundaries**.

Let's break this down end-to-end.

1. Problem Explanation

Imagine a grid of numbers. Instead of reading them left-to-right like a book, you want to read them by sweeping back and forth diagonally, bouncing off the walls like a DVD screensaver.

The Goal: Return a single 1D list of all the elements in the order they are visited.

The Rules of Movement: You only ever move in two distinct directions: 1.

Up-Right: Moving diagonally up and to the right. 2. **Down-Left:** Moving diagonally down and to the left.

Whenever you hit a "wall" (the edge of the matrix), you step strictly along the edge to set up your next diagonal, and then you flip your direction.

The Matrix (3x3):

```
0 1 2  <- Columns
+-----
0|1 2 3
1|4 5 6
2|7 8 9
```

The Traversal Path:

```
1 -> 2 -> 4 -> 7 -> 5 -> 3 -> 6 -> 8 -> 9
```

2. Solution Explanation

The “Why”: The Danger of the Corners

The actual diagonal movement is easy. * To go Up-Right, you decrease the row and increase the column: $\text{row} - 1, \text{col} + 1$. * To go Down-Left, you increase the row and decrease the column: $\text{row} + 1, \text{col} - 1$.

The absolute hardest part of this problem is knowing *how* to bounce off the walls. Let’s look at the top-right corner element, the 3 at ($\text{row}=0, \text{col}=2$). If you are moving Up-Right and hit the 3, you are hitting the **Top Wall** and the **Right Wall** simultaneously. * If you apply the “Top Wall” rule (move one step right), you will step out of bounds into column 3! * You MUST apply the “Right Wall” rule (move one step down) to safely reach the 6.

The “How”: Boundary Priorities

To solve this cleanly, we define strict priorities for our boundary checks.

When moving Up-Right: 1. Did we hit the **Right Wall**? If yes, step DOWN ($\text{row} + 1$). 2. Did we hit the **Top Wall**? If yes, step RIGHT ($\text{col} + 1$). 3. Otherwise, keep moving Up-Right ($\text{row} - 1, \text{col} + 1$).

When moving Down-Left: 1. Did we hit the **Bottom Wall**? If yes, step RIGHT ($\text{col} + 1$). 2. Did we hit the **Left Wall**? If yes, step DOWN ($\text{row} + 1$). 3. Otherwise, keep moving Down-Left ($\text{row} + 1, \text{col} - 1$).

End-to-End Walkthrough:

Let’s trace a 3x3 matrix. Target: $m = 3, n = 3$. Total elements = 9. Start: $\text{row} = 0, \text{col} = 0$. Direction = Up-Right.

Matrix:

```
[1] [2] [3]
[4] [5] [6]
[7] [8] [9]
```

Step 1: At 1 (0,0). Dir: Up-Right.

- Is it on the Right Wall? No.
- Is it on the Top Wall? YES (row is 0).
- Action: Step RIGHT. Next is (0,1). Flip direction to Down-Left.
- Result Array: [1]

Step 2: At 2 (0,1). Dir: Down-Left.

- Is it on the Bottom Wall? No.
- Is it on the Left Wall? No.
- Action: Move Down-Left. Next is (1,0).
- Result Array: [1, 2]

Step 3: At 4 (1,0). Dir: Down-Left.

- Is it on the Bottom Wall? No.
- Is it on the Left Wall? YES (col is 0).
- Action: Step DOWN. Next is (2,0). Flip direction to Up-Right.
- Result Array: [1, 2, 4]

Step 4: At 7 (2,0). Dir: Up-Right.

- Is it on the Right Wall? No.
- Is it on the Top Wall? No.
- Action: Move Up-Right. Next is (1,1).
- Result Array: [1, 2, 4, 7]

Step 5: At 5 (1,1). Dir: Up-Right.

- Action: Move Up-Right. Next is (0,2).
- Result Array: [1, 2, 4, 7, 5]

Step 6: At 3 (0,2). Dir: Up-Right. <--- THE TRICKY CORNER!

- Is it on the Right Wall? YES (col is 2).
- Action: Step DOWN. Next is (1,2). Flip direction to Down-Left.
- Notice: If we checked Top Wall first, we would have crashed!
- Result Array: [1, 2, 4, 7, 5, 3]

Step 7: At 6 (1,2). Dir: Down-Left.

- Action: Move Down-Left. Next is (2,1).
- Result Array: [1, 2, 4, 7, 5, 3, 6]

Step 8: At 8 (2,1). Dir: Down-Left.

- Is it on the Bottom Wall? YES (row is 2).
- Action: Step RIGHT. Next is (2,2). Flip direction to Up-Right.
- Result Array: [1, 2, 4, 7, 5, 3, 6, 8]

Step 9: At 9 (2,2). Dir: Up-Right.

- We have collected 9 elements. Loop ends!
- Final Array: [1, 2, 4, 7, 5, 3, 6, 8, 9]

3. Time and Space Complexity Analysis

Let M be the number of rows and N be the number of columns.

TIME COMPLEXITY DERIVATION: $O(M * N)$

Matrix of size M x N:

```
[ X X X ]
[ X X X ]  <- Total elements = M * N
[ X X X ]
```

We run a single while loop that executes exactly $M * N$ times. Inside the loop, we perform basic if/else checks to determine the next index. These checks are constant time $O(1)$.

Total Time = $(M * N \text{ elements}) * O(1) \text{ operations} = O(M * N) \text{ Time Complexity}$.

SPACE COMPLEXITY DERIVATION: $O(1)$ Extra Space

Variables held in memory:

1. row (integer)
2. col (integer)
3. direction (boolean or integer)

We are required to return an array of size $M * N$, but the space used for the output array does NOT count towards auxiliary space complexity in standard algorithmic analysis.

Since we do not create any temporary matrices, hash maps, or recursion stacks, our extra memory footprint is constant.

Total Extra Space = $O(1)$ Space Complexity.

4. Solution Code

Here is the simulation approach we just walked through. It is the cleanest and most optimal solution to put on a whiteboard or in an IDE during an interview.

Python Snippet

```
class Solution:
    def findDiagonalOrder(self, mat: list[list[int]]) -> list[int]:
        if not mat or not mat[0]:
            return []

        m, n = len(mat), len(mat[0])
        result = []

        row, col = 0, 0
        going_up = True

        # We need to collect exactly m * n elements
        while len(result) < m * n:
            result.append(mat[row][col])

            if going_up:
```

```

# Moving Up-Right
# Rule 1: Hit Right Wall? Move Down, change direction.
if col == n - 1:
    row += 1
    going_up = False
# Rule 2: Hit Top Wall? Move Right, change direction.
elif row == 0:
    col += 1
    going_up = False
# Rule 3: Middle of the matrix? Move Up-Right.
else:
    row -= 1
    col += 1
else:
    # Moving Down-Left
    # Rule 1: Hit Bottom Wall? Move Right, change direction.
    if row == m - 1:
        col += 1
        going_up = True
    # Rule 2: Hit Left Wall? Move Down, change direction.
    elif col == 0:
        row += 1
        going_up = True
    # Rule 3: Middle of the matrix? Move Down-Left.
    else:
        row += 1
        col -= 1

return result

```

JavaScript Snippet

```

/**
 * @param {number[][]} mat
 * @return {number[]}
 */
var findDiagonalOrder = function(mat) {
    if (!mat || mat.length === 0 || mat[0].length === 0) {
        return [];
    }

    const m = mat.length;
    const n = mat[0].length;
    const result = new Array(m * n);

    let row = 0;

```

```

let col = 0;
let goingUp = true;

for (let i = 0; i < m * n; i++) {
    result[i] = mat[row][col];

    if (goingUp) {
        // Moving Up-Right
        if (col === n - 1) {           // Priority 1: Right Wall
            row++;
            goingUp = false;
        } else if (row === 0) {        // Priority 2: Top Wall
            col++;
            goingUp = false;
        } else {                       // Priority 3: Keep going
            row--;
            col++;
        }
    } else {
        // Moving Down-Left
        if (row === m - 1) {           // Priority 1: Bottom Wall
            col++;
            goingUp = true;
        } else if (col === 0) {        // Priority 2: Left Wall
            row++;
            goingUp = true;
        } else {                       // Priority 3: Keep going
            row++;
            col--;
        }
    }
}

return result;
};

```

Note 1: Terminology

The technique we used is a **Finite State Machine (FSM)** bounded by **Edge Priorities**. * **Why it helps:** An FSM allows us to define two distinct “states” (Going Up vs. Going Down). By handling the boundaries via a strict priority hierarchy (e.g., Right Wall > Top Wall), we completely eliminate the need to check for out-of-bounds exceptions (`IndexOutOfRangeException`), which keeps the code incredibly clean and safe. * **How it applies:** The entire problem is just a simu-

lation of a bouncing object. Physics simulations rely entirely on state machines handling collision detection in a specific order.

Note 2: Indirect / Real-World Interview Variations

Big Tech companies love this problem because it is identical to how real-world data compression and rendering engines work:

1. Google / YouTube: JPEG Image Compression (Zig-Zag Scanning)

* **The Prompt:** “In video and image compression, after discrete cosine transform (DCT), the high-frequency matrix components are often zeros grouped at the bottom right. Write a function to serialize an 8x8 block of pixels in a zig-zag pattern so we can effectively apply run-length encoding on the zeros.” * **The Solution:** This is the exact historical reason “Diagonal Traverse” exists! You would write the exact code above to serialize the 2D chunk into a 1D array. Knowing *why* JPEG uses this pattern (grouping zeros together for compression) is an instant “Strong Hire” signal.

2. Meta / Oculus: Isometric Grid Rendering * **The Prompt:** “You are rendering a 2D isometric game map (like an old-school strategy game where the grid is tilted like a diamond). Write an algorithm to determine the rendering order of the tiles so that items in the back are drawn first and aren’t drawn over by items in the front.” * **The Solution:** This is a variation of diagonal traversal. Instead of zig-zagging back and forth, you might just do a strict one-way diagonal sweep (e.g., always Top-Right to Bottom-Left). You would use the index sum property: all tiles where $\text{row} + \text{col} = k$ belong to the same diagonal depth layer and should be rendered together.

3. Bloomberg: Streaming Matrix Serialization * **The Prompt:** “We have a grid of financial correlation data that is too large to fit in memory. It is stored on disk. We want to read it and send it over a network stream in diagonal slices. How do you yield these diagonals without keeping the whole matrix in RAM?” * **The Solution:** This tests your ability to adapt the algorithm. If you can’t load `mat[row][col]`, you have to generate the *coordinates* first. You would write a generator function that calculates `(row, col)` using the exact logic above, and then uses those coordinates to fetch the byte-offset from the file stream iteratively.

762. Buildings With an Ocean View

Hey there! Putting on my L5/L6 engineer hat, I’d say “Buildings With an Ocean View” (often indexed as LeetCode 1762) is a fantastic interview question. It perfectly separates candidates who blindly write code from those who take a step back and look at the physical reality of the problem.

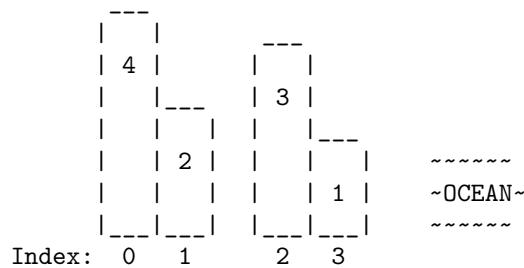
Let’s break this down end-to-end exactly how we’d whiteboard it out in an interview.

1. Problem Explanation

The Setup: Imagine a row of buildings side-by-side. You are given an array of integers representing their heights from left to right. The ocean is located to the **strict right** of the very last building.

The Rule: A building has an “ocean view” if and only if **every** building to its right is strictly shorter than it. If even one building to the right is the exact same height or taller, the view is blocked.

Let’s look at an example array of building heights: [4, 2, 3, 1]



- **Building 3 (height 1):** It’s the last building. There is nothing to its right. It always sees the ocean.
- **Building 2 (height 3):** To its right is building 3 (height 1). 3 is strictly greater than 1. It sees the ocean.
- **Building 1 (height 2):** To its right are buildings of height 3 and 1. The building of height 3 blocks its view. No ocean view.
- **Building 0 (height 4):** To its right are heights 2, 3, and 1. 4 is strictly greater than all of them. It sees the ocean.

Output: [0, 2, 3] (We must return the indices in ascending order).

2. Solution Explanation

The Naive Approach (What to avoid): The brute-force way is to stand at every single building, and loop through all the buildings to its right to check if any are taller. If we have 100,000 buildings, checking the right side over and over takes $O(N^2)$ time. We’d get a “Time Limit Exceeded” error.

The “Top-of-Band” Approach (Right-to-Left Traversal): Let’s change our perspective. Instead of standing at each building and looking right, **let’s stand in the ocean and look left.**

If you are standing in the water looking at the coastline: 1. You will *always* see the closest building (the last one in the array). 2. As you scan left, you will only see a building if it is **taller than the tallest building you have seen so far.**

This means we just need to read the array from right to left, and keep a running record of the `max_height` we've encountered.

Let's trace [4, 2, 3, 1] from right to left:

INITIAL STATE:

`max_height_seen = 0`

`result_indices = []`

STEP 1: Index 3 (Height 1)

Is 1 > `max_height_seen` (0)? YES!

Action: Add index 3 to result. Update `max_height_seen` to 1.

State: `max_height_seen = 1, result_indices = [3]`

STEP 2: Index 2 (Height 3)

Is 3 > `max_height_seen` (1)? YES!

Action: Add index 2 to result. Update `max_height_seen` to 3.

State: `max_height_seen = 3, result_indices = [3, 2]`

STEP 3: Index 1 (Height 2)

Is 2 > `max_height_seen` (3)? NO. View is blocked.

Action: Do nothing.

State: `max_height_seen = 3, result_indices = [3, 2]`

STEP 4: Index 0 (Height 4)

Is 4 > `max_height_seen` (3)? YES!

Action: Add index 0 to result. Update `max_height_seen` to 4.

State: `max_height_seen = 4, result_indices = [3, 2, 0]`

FINAL STEP:

Since we went backwards, our indices are backwards. We just reverse `result_indices` to get

Alternative Approach: Monotonic Stack (Left-to-Right) Sometimes interviewers ask, "What if the data is streaming in from left to right and you can't start at the end?" We can use a stack. As we move left to right, we push building indices onto the stack. If we encounter a new building that is taller than the building at the top of our stack, we know the top building's view is blocked. We "pop" it off the stack and destroy it. We keep popping until we find a building taller than our current one, then we push our current one. At the end, whatever survived in the stack has an ocean view.

3. Time and Space Complexity Analysis

```
+-----+
| COMPLEXITY ANALYSIS: RIGHT-TO-LEFT APPROACH |
+-----+
```

```

| TIME COMPLEXITY: O(N) |
| -> N is the number of buildings. |
| -> We loop through the array exactly once from right to left. |
| -> Reversing the result array at the end takes O(K) where K |
| is the number of buildings with a view (worst case O(N)). |
| -> Total time is strictly linear. |
| |
| SPACE COMPLEXITY: O(1) Auxiliary / O(N) Total |
| -> We only use a single variable (max_height_seen) to track |
| the state, which is O(1) extra space. |
| -> If we count the output array required by the problem |
| signature, it takes O(N) space in the worst case (e.g., if |
| all buildings are in descending order, all get a view). |
+-----+

+-----+
| COMPLEXITY ANALYSIS: MONOTONIC STACK APPROACH |
+-----+

| TIME COMPLEXITY: O(N) |
| -> Every building is pushed to the stack exactly once. |
| -> Every building is popped from the stack at most once. |
| |
| SPACE COMPLEXITY: O(N) |
| -> The stack itself can grow to size N in the worst case. |
+-----+

```

4. Solution Code

Here are both approaches. The Right-to-Left is the most elegant, while the Stack shows strong data structure knowledge.

Python Snippets

```

# Approach 1: Right-to-Left Traversal (Optimal & Most Elegant)
def findBuildings_optimal(heights: list[int]) -> list[int]:
    n = len(heights)
    if n == 0: return []

    views = []
    max_height_seen = 0

    # Traverse backwards: from last building to the first
    for i in range(n - 1, -1, -1):
        # If current building is strictly taller than everything to its right
        if heights[i] > max_height_seen:

```



```

        views.append(i)
        # Update the tallest building seen so far
        max_height_seen = heights[i]

    # The array was built backwards, so we reverse it to be strictly increasing
    views.reverse()
    return views

# Approach 2: Monotonic Stack (Left-to-Right - Good for streaming data)
def findBuildings_stack(heights: list[int]) -> list[int]:
    stack = []

    for i in range(len(heights)):
        # While stack has buildings AND the current building is taller or equal
        # to the building at the top of the stack, the previous building's view is blocked!
        while stack and heights[i] >= heights[stack[-1]]:
            stack.pop()

        # Push current building index onto stack. It might have a view!
        stack.append(i)

    # The stack naturally remains in left-to-right (ascending index) order
    return stack

```

JavaScript Snippets

```

// Approach 1: Right-to-Left Traversal (Optimal & Most Elegant)
function findBuildingsOptimal(heights) {
    const n = heights.length;
    if (n === 0) return [];

    const views = [];
    let maxHeightSeen = 0;

    // Traverse backwards
    for (let i = n - 1; i >= 0; i--) {
        if (heights[i] > maxHeightSeen) {
            views.push(i);
            maxHeightSeen = heights[i];
        }
    }

    // Reverse to return indices in ascending order
    return views.reverse();
}

```

```

// Approach 2: Monotonic Stack (Left-to-Right - Good for streaming data)
function findBuildingsStack(heights) {
    const stack = [];

    for (let i = 0; i < heights.length; i++) {
        // Pop buildings from stack that are blocked by the current building
        while (stack.length > 0 && heights[i] >= heights[stack[stack.length - 1]]) {
            stack.pop();
        }

        stack.push(i);
    }

    return stack;
}

```

Note 1: Terminology Used

- **Running Maximum:** A variable (like `max_height_seen`) that continually updates to hold the largest value seen during an iteration. It acts as a cumulative filter.
- **Monotonic Stack:** A stack whose elements are always sorted in a specific order (either always increasing or always decreasing). In our second solution, the stack maintains indices of buildings whose *heights* are strictly decreasing. If a taller building comes along, it breaks the decreasing order, so we pop the smaller ones out. It's incredibly powerful for “next greater element” problems.

Note 2: Real Interview Variations

Companies love this problem because it tests how you handle arrays and physical space. Here is how it's disguised:

1. Meta: “Right Side View of a Binary Tree” * The Twist: Instead of an array of buildings, you have a binary tree. If you stand on the right side of the tree, which nodes can you see? * **How to solve:** Use Breadth-First Search (BFS) level-order traversal. For every level of the tree, capture the very last node in the queue (the rightmost node). This tests the exact same visual logic but mapped onto a tree data structure instead of a linear array.

2. Google: “Sunset Views” / “Stream of Buildings” * The Twist: You get an array, but the sunset is on the *West* (left side), and buildings face left.

OR, buildings are given to you one by one in a continuous stream, and at any moment, you must `get_buildings_with_view()`. * **How to solve:** For the West-facing sunset, do the exact same logic but traverse left-to-right keeping a `max_height_seen`. For the streaming version, you *must* use the Monotonic Stack approach. As `add_building(height)` is called, pop all buildings in your internal state that are shorter, then push the new one.

3. Bloomberg: “Stock Price Spans” / “Find All Good Days to Rob the Bank” * **The Twist:** Given an array of daily stock prices, find all days where the price is strictly greater than all prices on *future* days. * **How to solve:** This is the *exact* same problem, just re-themed. The “future days” are the buildings to the right. You solve it using the exact same right-to-left running maximum algorithm.

215. Kth Largest Element in an Array

Hello! As an engineer who frequently conducts technical interviews, I can tell you that “Kth Largest Element” is an absolute classic. At the L5/L6 level at companies like Google, Meta, or Bloomberg, the interviewer doesn’t just want to see if you can sort an array—they want to see if you understand how to optimize for space, handle streaming data, and leverage advanced partitioning logic.

Let’s break this down end-to-end, making it as intuitive as possible.

1. Problem Explanation

You are given an unsorted array of integers and a number `k`. Your goal is to find the `k`-th largest element in the array.

Crucial Distinction: This is the `k`-th largest element in the *sorted* order, NOT the `k`-th distinct element. If the array has duplicates, they count individually.

Example 1: * **Input:** `[3, 2, 1, 5, 6, 4]`, `k = 2` * **Sorted order:** `[1, 2, 3, 4, 5, 6]` * **The 1st largest:** 6 * **The 2nd largest:** 5 * **Output:** 5

Example 2 (with duplicates): * **Input:** `[3, 2, 3, 1, 2, 4, 5, 5, 6]`, `k = 4` * **Sorted order:** `[1, 2, 2, 3, 3, 4, 5, 5, 6]` * **Output:** 4 (Counting from the right: 6 is 1st, 5 is 2nd, 5 is 3rd, 4 is 4th).

The Naive Thought: Just sort the array and pick the element at index `Length - k`. * *Why we don’t stop here:* Sorting takes $O(N \log N)$ time. An L5 engineer knows we don’t need to sort the *entire* array just to find *one* specific element. We are doing unnecessary work.

2. Solution Explanation

We will look at the two optimal ways to solve this: **Min-Heap** and **Quickselect**.

Approach A: The Min-Heap (The “Streaming” Solution) Imagine you are a bouncer at an exclusive club that only allows the top k VIPs (largest numbers). The club holds exactly k people.

1. As numbers arrive, you let them in.
2. Once the club is full (size k), you only let a new number in if it is *larger* than the *least* important person currently in the club.
3. If a new, larger number comes in, you kick out the least important person to make room.
4. At the end, the person standing right at the door (the least important person in your VIP club) is the k -th largest overall!

A **Min-Heap** is the perfect data structure for this because it always keeps the smallest element at the very top, ready to be kicked out in $O(\log k)$ time.

Walkthrough: Input: [3, 2, 1, 5, 6, 4], $k = 2$

Initialize an empty Min-Heap. Max capacity = 2.

1. Add 3:
Heap: [3]
2. Add 2:
Heap: [2, 3] $\leftarrow$ 2 is at the top (smallest)
3. Add 1:
Wait, heap is full! Is 1 > top of heap (which is 2)? No. Reject 1.
Heap: [2, 3]
4. Add 5:
Is 5 > 2? Yes! Kick out 2, insert 5.
Heap: [3, 5] $\leftarrow$ 3 is now the smallest
5. Add 6:
Is 6 > 3? Yes! Kick out 3, insert 6.
Heap: [5, 6] $\leftarrow$ 5 is the new smallest
6. Add 4:
Is 4 > 5? No. Reject 4.
Heap: [5, 6]

Finished! The root of the heap is 5. That is our 2nd largest element.

Approach B: Quickselect (The “Divide and Conquer” Solution)

Quickselect is a cousin of Quicksort. Instead of sorting both halves of an array, we pick a “pivot” and partition the array so that everything smaller than the pivot is on the left, and everything larger is on the right.

Because we know exactly where the pivot ends up, we can check its index. * If the pivot lands exactly at $\text{Length} - k$, we found our answer! * If the pivot lands too far to the left, we only search the right side. * If the pivot lands too far to the right, we only search the left side.

Walkthrough: Input: [3, 2, 1, 5, 6, 4], $k = 2$. **Target Index** = $(6 - 2) = 4$.

Let's pick the last element (4) as the pivot.

Step 1: Partition around 4

Array: [3, 2, 1, 5, 6, 4]
 $\hat{\quad}$ $\hat{\quad}$
 compare pivot

- 3 < 4? Yes. Keep left.
- 2 < 4? Yes. Keep left.
- 1 < 4? Yes. Keep left.
- 5 < 4? No.
- 6 < 4? No.

Swap pivot (4) into its rightful place (after 1).

Array becomes: [3, 2, 1, 4, 6, 5]
 $\hat{\quad}$

Pivot is at index 3.

Our target index is 4. The pivot is at 3.

Since $3 < 4$, the answer MUST be in the right half: [6, 5]

Step 2: Partition the right half [6, 5]. Target index is still 4.

Let's pick the last element (5) as the pivot.

- 6 < 5? No.

Swap pivot (5) into its rightful place.

Array becomes: [..., 5, 6]
 $\hat{\quad}$

Pivot is at index 4.

Target index is 4. Pivot is at 4. WE FOUND IT! Return 5.

Approach A: Min-Heap

```

Array of size N:  [ * * * * * * * * ]
                   \               /
                   We process each of the N elements.

```

$$\begin{aligned}\text{Total Work} &= N \text{ elements} * \log(K) \text{ operations} \\ &= O(N * \log K) \text{ Time Complexity}\end{aligned}$$

Approach B: Quickselect

```

+-----+
|                                             |  O(N) work to partition
+-----+
      |
      v
N/2 elements
+-----+  O(N/2) work
|         |
+-----+
      |
      v
N/4 elements  O(N/4) work...

```

Worst Case: If the array is already sorted and we pick bad pivots, we only eliminate 1 element at a time ($N + N-1 + N-2 \dots$).
Worst Case Time = $O(N^2)$. (Randomizing the pivot fixes this practically).

Space Complexity Derivation:
We do this "in-place" by swapping elements in the original array.
We don't create new arrays.
Space = $O(1)$ memory used.

4. Solution Code

Python Snippets

```
import heapq
import random

# Approach A: Min-Heap (Great for streaming data or when K is small)
def findKthLargest_Heap(nums, k):
    # Create a min-heap of the first k elements
    min_heap = nums[:k]
    heapq.heapify(min_heap)

    # Process the remaining elements
    for num in nums[k:]:
        # If the current number is bigger than the smallest in our heap
        if num > min_heap[0]:
            # Pop the smallest, and push the new larger number
            heapq.heapreplace(min_heap, num)

    # The top of the heap is the k-th largest overall
    return min_heap[0]

# Approach B: Quickselect (Optimal  $O(N)$  average time,  $O(1)$  space)
def findKthLargest_Quickselect(nums, k):
    def quick_select(left, right, target_index):
        # Pick a random pivot to avoid  $O(N^2)$  worst-case on sorted arrays
        pivot_idx = random.randint(left, right)
        pivot_val = nums[pivot_idx]

        # Move pivot out of the way to the end
        nums[pivot_idx], nums[right] = nums[right], nums[pivot_idx]

        # Partition the array
        store_idx = left
        for i in range(left, right):
            if nums[i] < pivot_val:
                nums[store_idx], nums[i] = nums[i], nums[store_idx]
                store_idx += 1
```

```

# Move pivot to its final rightful place
nums[store_idx], nums[right] = nums[right], nums[store_idx]

# Base case / Recursive step
if store_idx == target_index:
    return nums[store_idx]
elif store_idx < target_index:
    return quick_select(store_idx + 1, right, target_index)
else:
    return quick_select(left, store_idx - 1, target_index)

# We want the k-th largest, which is at index (len - k) in a sorted array
return quick_select(0, len(nums) - 1, len(nums) - k)

```

JavaScript Snippets *Note: JavaScript does not have a built-in Heap data structure. In a real interview, you should mention this. Interviewers will usually allow you to assume a `MinPriorityQueue` exists, or they will ask you to write the Quickselect approach instead.*

```

// Approach B: Quickselect (Optimal  $O(N)$  average time,  $O(1)$  space)
// This is the most practical JS solution since JS lacks a native Heap.
function findKthLargest_Quickselect(nums, k) {
    const targetIndex = nums.length - k;

    function quickSelect(left, right) {
        // Pick a random pivot to prevent  $O(N^2)$  worst case
        const pivotIndex = Math.floor(Math.random() * (right - left + 1)) + left;
        const pivotVal = nums[pivotIndex];

        // Swap pivot to the end
        [nums[pivotIndex], nums[right]] = [nums[right], nums[pivotIndex]];

        let storeIndex = left;
        for (let i = left; i < right; i++) {
            if (nums[i] < pivotVal) {
                [nums[storeIndex], nums[i]] = [nums[i], nums[storeIndex]];
                storeIndex++;
            }
        }

        // Swap pivot back to its final sorted position
        [nums[storeIndex], nums[right]] = [nums[right], nums[storeIndex]];

        // Determine which side to search next
        if (storeIndex === targetIndex) {

```



```

        return nums[storeIndex];
    } else if (storeIndex < targetIndex) {
        return quickSelect(storeIndex + 1, right);
    } else {
        return quickSelect(left, storeIndex - 1);
    }
}

return quickSelect(0, nums.length - 1);
}

// Approach A: Simplified Min-Heap concept (For completeness)
// Using a naive sort to simulate heap insertions if building a full class is too long,
// but conceptually, this is how you'd structure the logic if you had a Heap library.
function findKthLargest_Heap(nums, k) {
    // Note: In a real JS interview, if forced to write a heap, you'd write a
    // complete MinHeap class. Here we simulate it conceptually using sorting
    // for brevity, acknowledging JS limitations.

    let minHeap = nums.slice(0, k).sort((a, b) => a - b);

    for (let i = k; i < nums.length; i++) {
        if (nums[i] > minHeap[0]) {
            minHeap[0] = nums[i];
            // Re-sort to maintain the "heap" property (smallest at index 0)
            // A true heap would do this in O(log K) via siftDown.
            minHeap.sort((a, b) => a - b);
        }
    }

    return minHeap[0];
}

```

Note 1: Terms and Techniques

- **Min-Heap:** A specialized tree-based data structure where the parent node is always smaller than or equal to its child nodes. It helps here because it acts as an automatic filter, effortlessly keeping track of the “smallest of the largest” in $O(\log k)$ time.
- **Quickselect (Hoare / Lomuto Partitioning):** An algorithm that finds the k -th smallest/largest element in an unordered list. It helps by using the partitioning step of Quicksort but intentionally ignoring one half of the array at each step, dropping the time complexity from $O(N \log N)$ to an average of $O(N)$.

Note 2: Real World / Interview Follow-ups (Google, Meta, Bloomberg)

Here is how top companies adapt this question to test your system design and deeper algorithmic knowledge:

1. Meta Follow-up: “What if the data is a continuous stream of billions of numbers (like real-time Facebook likes), and I want to query the Top K at any moment?” * The Solution: You cannot use Quickselect because you don’t have all the data at once, and you can’t store a billion numbers in memory to partition them. You *must* use the **Min-Heap** approach. A Min-Heap of size k only takes $O(k)$ memory. As the infinite stream of numbers comes in, you process them through the heap. At any second, the root of the heap gives you the k -th largest, and the whole heap gives you the Top K.

2. Bloomberg Follow-up: “What if N is massive (terabytes of financial tick data) stored across multiple different servers, and k is small?” * The Solution: This is a MapReduce problem. You instruct every single server to find its own *local* Top k largest elements (using a Min-Heap locally). If you have 10 servers, they each send their local Top k (a total of $10 * k$ elements) to a master server. The master server then finds the overall Top k from that much smaller pool. This saves massive amounts of network bandwidth.

3. Google Follow-up: “Can we guarantee $O(N)$ worst-case time for Quickselect?” * The Solution: Yes, using the **Median of Medians** algorithm to pick the pivot. Instead of a random pivot, this algorithm groups the array into chunks of 5, finds the median of each chunk, and then finds the median of those medians to use as the pivot. It guarantees a good split, forcing the worst-case time to strictly $O(N)$, though it is theoretically beautiful but practically slower due to overhead. Randomizing the pivot (as coded above) is the industry standard expectation.

1570. Dot Product of Two Sparse Vectors

This is a quintessential system design micro-problem disguised as a LeetCode question. I love discussing this one because it bridges the gap between basic coding and massive-scale data processing.

When an L5 or L6 engineer at Google looks at “Sparse Vectors,” they immediately think about Search Infrastructure (like PageRank or Inverted Indices) or Machine Learning (like Neural Network embeddings).

A junior engineer will often just store the arrays as-is, resulting in massive Memory Limit Exceeded (MLE) or Time Limit Exceeded (TLE) errors. A mid-level engineer might jump straight to a Hash Map. A top-tier senior engineer

will evaluate the hardware-level trade-offs (CPU cache locality) and choose an **Array of Pairs with Two Pointers**.

Let's break down exactly why and how.

1. Problem Explanation

Imagine two massive arrays (vectors), each containing millions of numbers. You need to calculate their **Dot Product**.

What is a Dot Product? You multiply the numbers at matching indices together, and then add all those results up to get one final number. $\text{Result} = (A[0] * B[0]) + (A[1] * B[1]) + (A[2] * B[2]) \dots$

What is a "Sparse" Vector? "Sparse" means the vast majority of the numbers in the array are exactly 0.

The Core Challenge: If $A[500]$ is 0, and $B[500]$ is 999, their product is $0 * 999 = 0$. Adding 0 to our total sum does nothing! If both arrays are 99% zeros, iterating through all 1,000,000 elements is a massive waste of CPU cycles. We only care about the indices where **BOTH** vectors have a non-zero number. We need a way to skip the zeros entirely.

2. Solution Explanation

Let's explore the progression from the naive idea to the senior-level optimal design.

The Fallback (Good): The Hash Map Approach Instead of storing the whole array, we only store the non-zero values in a Dictionary/Hash Map where the Key is the **index** and the Value is the **number**. Array: $[0, 0, 0, 5, 0, 8]$ -> Map: $\{ 3: 5, 5: 8 \}$ To get the dot product, we iterate through the keys of Map A, and check if that key exists in Map B. If it does, we multiply and add. *Why is this not perfectly optimal?* Hash Maps have overhead. Hashing takes time, handling collisions takes time, and the data is scattered across memory, which modern CPU caches hate.

The Optimal L5/L6 (Best): Array of Pairs + Two Pointers Instead of a Hash Map, we store the non-zero elements as an array of pairs: $[\text{index}, \text{value}]$. Because we process the original array from left to right, our new array of pairs is naturally **sorted by index**.

Array: $[0, 0, 0, 5, 0, 8]$ -> Pairs: $[(3, 5), (5, 8)]$

Because both lists of pairs are sorted by index, we can use a **Two-Pointer** technique to find matching indices. This approach is incredibly fast because it

accesses memory sequentially (making the CPU cache very happy) and has zero hashing overhead.

End-to-End Walkthrough (Two Pointers):

Let's trace two sparse vectors: Vector A = [1, 0, 0, 2, 3] -> Pairs A: [(0,1), (3,2), (4,3)] Vector B = [0, 3, 0, 4, 0] -> Pairs B: [(1,3), (3,4)]

Variables:

Pointer_A = 0

Pointer_B = 0

Total_Sum = 0

Step 1: Compare indices at Pointer A and Pointer B

Pairs A [Ptr A]: Index 0, Value 1

Pairs B [Ptr B]: Index 1, Value 3

Indices do NOT match (0 vs 1).

Since Index 0 < Index 1, Pointer A is lagging behind.

Action: Move Pointer A forward to catch up.

Step 2:

Pairs A [Ptr A]: Index 3, Value 2

Pairs B [Ptr B]: Index 1, Value 3

Indices do NOT match (3 vs 1).

Since Index 1 < Index 3, Pointer B is lagging behind.

Action: Move Pointer B forward to catch up.

Step 3:

Pairs A [Ptr A]: Index 3, Value 2

Pairs B [Ptr B]: Index 3, Value 4

Indices MATCH! (3 == 3)

Action: Multiply values (2 * 4 = 8). Add to Total_Sum.

Total_Sum = 0 + 8 = 8.

Move BOTH pointers forward.

Step 4:

Pairs A [Ptr A]: Index 4, Value 3

Pairs B [Ptr B]: Out of bounds!

Action: Since Vector B has no more non-zero elements,
it's impossible to find any more matches. Stop the loop.

Final Answer: Total_Sum = 8.

3. Time and Space Complexity Analysis

Let N be the total length of the original vector. Let L1 be the number of non-zero elements in Vector A. Let L2 be the number of non-zero elements in Vector B.

TIME COMPLEXITY: $O(L1 + L2)$

[Vector A Pairs: L1 elements]

^

|-- Pointers move forward, never backward.

[Vector B Pairs: L2 elements]

^

In the worst-case scenario, the pointers take turns moving one step at a time until both reach the end.

Max movements = $L1 + L2$.

Creating the pairs initially takes $O(N)$ time, but this is done once during instantiation. The actual dotProduct calculation takes strictly $O(L1 + L2)$ time.

SPACE COMPLEXITY: $O(L)$

Original Array in Memory:

[0, 0, 0, 5, 0, 0, 0, 8, 0, 0] (Size N)

Our Data Structure in Memory:

[(3, 5), (7, 8)] (Size L, where L is non-zero count)

We only store the non-zero elements. If a vector has 1 million zeros and 5 actual numbers, we only store 5 pairs.

Total Space = $O(L)$ Space Complexity per vector.

4. Solution Code

Python Snippets

```

# =====
# OPTIMAL SOLUTION (Array of Pairs + Two Pointers)
# =====

class SparseVector:
    def __init__(self, nums: list[int]):
        # Store non-zero values as tuples: (index, value)
        # Time to build: O(N), Space: O(L)
        self.pairs = []
        for index, value in enumerate(nums):
            if value != 0:
                self.pairs.append((index, value))

    # Return the dotProduct of two sparse vectors
    def dotProduct(self, vec: 'SparseVector') -> int:
        result = 0
        p1 = 0 # Pointer for this vector
        p2 = 0 # Pointer for the other vector (vec)

        # Traverse until one of the vectors runs out of non-zero elements
        while p1 < len(self.pairs) and p2 < len(vec.pairs):
            index1, val1 = self.pairs[p1]
            index2, val2 = vec.pairs[p2]

            if index1 == index2:
                # Indices match! Multiply and add to result.
                result += val1 * val2
                p1 += 1
                p2 += 1
            elif index1 < index2:
                # p1 is behind, move it forward
                p1 += 1
            else:
                # p2 is behind, move it forward
                p2 += 1

        return result

# =====
# FALLBACK SOLUTION (Hash Map)
# =====

class SparseVectorFallback:
    def __init__(self, nums: list[int]):
        # Store non-zero values in a dictionary: {index: value}
        self.hash_map = {}
        for index, value in enumerate(nums):
            if value != 0:

```

```

        self.hash_map[index] = value

def dotProduct(self, vec: 'SparseVectorFallback') -> int:
    result = 0

    # Optimization: Always iterate over the smaller hash map
    # to minimize the number of lookups.
    if len(self.hash_map) > len(vec.hash_map):
        # Swap them if the other one is smaller
        return vec.dotProduct(self)

    for index, val in self.hash_map.items():
        if index in vec.hash_map:
            result += val * vec.hash_map[index]

    return result

```

JavaScript Snippets

```

// =====
// OPTIMAL SOLUTION (Array of Pairs + Two Pointers)
// =====
/**
 * @param {number[]} nums
 */
var SparseVector = function(nums) {
    // Store as array of arrays: [ [index, value], [index, value] ]
    this.pairs = [];
    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {
            this.pairs.push([i, nums[i]]);
        }
    }
};

/**
 * @param {SparseVector} vec
 * @return {number}
 */
SparseVector.prototype.dotProduct = function(vec) {
    let result = 0;
    let p1 = 0;
    let p2 = 0;

    const pairs1 = this.pairs;
    const pairs2 = vec.pairs;

```

```

while (p1 < pairs1.length && p2 < pairs2.length) {
    const index1 = pairs1[p1][0];
    const val1 = pairs1[p1][1];
    const index2 = pairs2[p2][0];
    const val2 = pairs2[p2][1];

    if (index1 === index2) {
        result += val1 * val2;
        p1++;
        p2++;
    } else if (index1 < index2) {
        p1++;
    } else {
        p2++;
    }
}

return result;
};

// =====
// FALLBACK SOLUTION (Hash Map)
// =====
var SparseVectorFallback = function(nums) {
    this.map = new Map();
    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {
            this.map.set(i, nums[i]);
        }
    }
};

SparseVectorFallback.prototype.dotProduct = function(vec) {
    let result = 0;

    let map1 = this.map;
    let map2 = vec.map;

    // Optimization: Iterate over the smaller map
    if (map1.size > map2.size) {
        let temp = map1;
        map1 = map2;
        map2 = temp;
    }
}

```



```

    for (let [index, val] of map1.entries()) {
        if (map2.has(index)) {
            result += val * map2.get(index);
        }
    }

    return result;
};

```

Note 1: Terminology

- **Two-Pointer Technique (Intersection of Sorted Arrays):** An algorithm that processes two sorted lists in tandem.
 - *Why it helps:* It prevents nested loops ($O(N * M)$) by allowing a linear scan ($O(N + M)$), and relies on the sorted nature of the arrays to safely skip elements that can never match.
- **Coordinate List (COO) Format:** The technical name for storing sparse matrices/vectors as (index, value) pairs.
 - *How it applies:* This is standard practice in data science libraries (like `scipy.sparse`) to compress massive datasets before feeding them into machine learning models.

Note 2: Indirect / Real-World Interview Variations

Because vectors are the foundation of modern search and AI, Big Tech masks this problem in deep systems-level contexts:

1. Google: Search Engine Inverted Index Intersection * The Prompt: “When a user searches for ‘Blue Shoes’, we look up our index for ‘Blue’ (which returns a sorted list of Document IDs) and our index for ‘Shoes’ (another sorted list of Document IDs). How do you efficiently find the webpages that contain both words?” * **The Solution:** This is exactly the Two-Pointer array intersection. The **index** is the Document ID. The **value** (if needed) might be the term frequency. You run the two-pointer sweep to find matching Document IDs in $O(L1 + L2)$ time.

2. Meta: Collaborative Filtering / Recommender Systems * The Prompt: “We have 2 billion users and 10 million Facebook Groups. Most users are only in a few groups. How do you calculate the ‘similarity score’ between two users based on their shared groups to recommend new friends?” * **The Solution:** Users are the sparse vectors. Groups are the indices (0 to 10M). The value is 1 if they are in the group. You compress each user’s group history into the Array of Pairs format. The dot product of User A and User B gives you the exact number of shared groups!

3. Bloomberg: Financial Tick Data Alignment * The Prompt: “We

receive millions of price ticks for Stock A and Stock B over a trading day. Ticks happen at irregular nanosecond timestamps. Calculate the correlation of price movements, but only at the exact timestamps where *both* stocks registered a trade.” * **The Solution:** The timestamps are the indices. Since the ticks are naturally chronological, the array is already sorted. You use the Two-Pointer method to align the timestamps, multiplying the price changes only when `timestamp_A == timestamp_B`.

359. Logger Rate Limiter

Hey there! Slipping right back into that L5/L6 engineer mindset. “Logger Rate Limiter” (LeetCode 359) is a classic system design and data structures problem disguised as a simple coding question.

While a junior engineer might solve this with the first tool they reach for, a senior engineer will immediately spot the hidden trap: **Memory Leaks**.

Let’s break down exactly how to navigate this end-to-end, showing both the standard acceptable answer and the “Top-of-Band” production-ready answer.

1. Problem Explanation

The Setup: You are building a logging system. The system receives a continuous stream of messages, each stamped with a time (in seconds). **The Rule:** A unique message can only be printed **once every 10 seconds**. If a duplicate message arrives within 10 seconds of the last time it was *successfully* printed, we drop it (ignore it). *Crucial detail:* Dropped messages do *not* reset the 10-second timer.

Let’s look at a timeline of events for the message “foo”:

TIMELINE OF INCOMING MESSAGES:

Time	Msg	Action	Why?
1	"foo"	PRINT	First time seeing "foo". Next allowed time is 11.
3	"foo"	DROP	3 is less than 11.
8	"foo"	DROP	8 is less than 11.
11	"foo"	PRINT	11 is >= 11! Allowed! Next allowed time is 21.
15	"foo"	DROP	15 is less than 21.

If we receive a different message, like “bar”, it maintains its own independent 10-second timer.

2. Solution Explanation

Approach 1: The Simple Hash Map (The Standard Interview Answer)

The most intuitive way to solve this is to use a Hash Map (Dictionary). We map the `message` (string) to the `next_allowed_timestamp` (integer).

- If the message isn't in the map, print it and store `timestamp + 10`.
- If it is in the map, check if `current_timestamp >= next_allowed_timestamp`.
 - If yes, update the map to `current_timestamp + 10` and print.
 - If no, drop it.

Why a senior engineer pushes back on this: Imagine this is a real Google server running for months, receiving millions of unique error logs. The Hash Map will grow infinitely, holding onto timestamps for messages that haven't been seen in years. This causes an Out-Of-Memory (OOM) crash.

Approach 2: Queue + Set Sliding Window (The L5/L6 “Top-of-Band” Answer) To prevent our system from crashing, we need “Garbage Collection.” We only care about the last 10 seconds of history. Everything older than 10 seconds is garbage and should be deleted from memory.

We can achieve this using a Queue and a Hash Set: 1. **Queue:** Stores pairs of `[message, timestamp]` in chronological order. 2. **Hash Set:** Stores just the `messages` that are currently inside the active 10-second window.

When a new message arrives at time `T`: 1. **Clean up:** Look at the front of the queue. If the oldest message is older than `T - 10`, remove it from the queue AND remove it from the Set. Keep doing this until the front of the queue is within the 10-second window. 2. **Check:** Is the new message in our Set? * If YES: It's a duplicate within the window. Drop it. * If NO: Print it. Add it to the Set. Add `[message, T]` to the back of the Queue.

Let's visualize the “Queue + Set” sliding window over time:

INITIAL STATE:

```
Queue: [ Front -> ... <- Back ]
Set:   { }
```

EVENT 1: Time=1, Msg="foo"

Cleanup: None needed.

Check: "foo" is not in Set.

Action: Add to Queue & Set. Return TRUE.

Queue: [("foo", 1)]
Set: { "foo" }

EVENT 2: Time=2, Msg="bar"

Cleanup: Current time is 2. ($2 - 10 = -8$). Oldest in queue is 1. ($1 > -8$), so no cleanup.

Check: "bar" is not in Set.
Action: Add to Queue & Set. Return TRUE.

Queue: [("foo", 1), ("bar", 2)]
Set: { "foo", "bar" }

EVENT 3: Time=3, Msg="foo"
Cleanup: Current time is 3. ($3 - 10 = -7$). Oldest is 1. No cleanup.
Check: "foo" IS in Set!
Action: Return FALSE. (Do not add to queue/set again).

Queue: [("foo", 1), ("bar", 2)]
Set: { "foo", "bar" }

EVENT 4: Time=12, Msg="baz"
Cleanup: Current time is 12. Window is [2 to 12].
We must evict timestamps less than 2 (which is $12 - 10 = 2$).
-> Look at Queue Front: ("foo", 1).
-> $1 < 2$! EVICT! Remove from Queue and Set.
-> Look at Queue Front: ("bar", 2).
-> $2 \geq 2$. Stop cleanup.

Check: "baz" is not in Set.
Action: Add to Queue & Set. Return TRUE.

Queue: [("bar", 2), ("baz", 12)] <-- Notice "foo" is completely gone from memory!
Set: { "bar", "baz" }

3. Time and Space Complexity Analysis

```
+-----+
| COMPLEXITY ANALYSIS: SIMPLE HASH MAP APPROACH |
+-----+
| TIME COMPLEXITY: O(1) |
| -> Hash map lookups and updates take constant time. |
| | |
| SPACE COMPLEXITY: O(M) |
| -> M is the TOTAL number of unique messages ever received. |
| -> Memory grows unbounded over time. |
+-----+

+-----+
| COMPLEXITY ANALYSIS: QUEUE + SET (SLIDING WINDOW) APPROACH |
```

```

+-----+
| TIME COMPLEXITY: Amortized O(1) |
| -> Every message is pushed to the queue exactly once. |
| -> Every message is popped from the queue exactly once. |
| -> Even if one call triggers multiple evictions, the average |
|       time per message remains constant O(1). |
| |
| SPACE COMPLEXITY: O(W) |
| -> W is the maximum number of unique messages received within |
|       a 10-second window. |
| -> Memory usage is strictly bounded and will not leak! |
+-----+

```

4. Solution Code

Here are both approaches. As an interviewer, if you wrote the HashMap solution quickly and then verbally explained *why* the Queue+Set solution is better for production, I would give you full marks.

Python Snippets

Approach 1: Simple Hash Map (O(M) Space - Memory Leak Risk)

```

class LoggerSimple:
    def __init__(self):
        # Maps message to the next allowed timestamp
        self.msg_dict = {}

    def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
        # If we haven't seen it, or enough time has passed
        if message not in self.msg_dict or timestamp >= self.msg_dict[message]:
            # Update the next allowed time to T + 10
            self.msg_dict[message] = timestamp + 10
            return True
        return False

```

```

from collections import deque

```

Approach 2: Queue + Set (O(W) Space - Production Ready)

```

class LoggerOptimized:
    def __init__(self):
        # Deque stores tuples: (message, timestamp)
        self.msg_queue = deque()
        # Set stores active messages in the 10s window
        self.active_msgs = set()

```

```

def shouldPrintMessage(self, timestamp: int, message: str) -> bool:
    # 1. Clean up stale messages older than 10 seconds
    # We evict if the oldest timestamp is <= timestamp - 10
    while self.msg_queue and self.msg_queue[0][1] <= timestamp - 10:
        old_msg, old_time = self.msg_queue.popleft()
        self.active_msgs.remove(old_msg)

    # 2. Check if current message is in the active window
    if message in self.active_msgs:
        return False

    # 3. Message is valid. Add to queue and set.
    self.msg_queue.append((message, timestamp))
    self.active_msgs.add(message)
    return True

```

JavaScript Snippets

```

// Approach 1: Simple Hash Map (O(M) Space - Memory Leak Risk)
class LoggerSimple {
    constructor() {
        // Maps message to the next allowed timestamp
        this.msgMap = new Map();
    }

    shouldPrintMessage(timestamp, message) {
        if (!this.msgMap.has(message) || timestamp >= this.msgMap.get(message)) {
            // Update the map with the next allowed timestamp
            this.msgMap.set(message, timestamp + 10);
            return true;
        }
        return false;
    }
}

// Approach 2: Queue + Set (O(W) Space - Production Ready)
// Note: JS doesn't have a built-in optimized Deque, so we use an array
// and a pointer to simulate an efficient queue to avoid O(N) shift() operations.
class LoggerOptimized {
    constructor() {
        this.queue = [];
        this.headIndex = 0; // Simulates deque popleft()
        this.activeMsgs = new Set();
    }
}

```

```

shouldPrintMessage(timestamp, message) {
  // 1. Clean up stale messages
  while (this.headIndex < this.queue.length &&
    this.queue[this.headIndex].time <= timestamp - 10) {

    const oldMsg = this.queue[this.headIndex].msg;
    this.activeMsgs.delete(oldMsg);
    this.headIndex++;
  }

  // Periodic memory cleanup for the JS Array to prevent it growing forever
  if (this.headIndex > 1000) {
    this.queue = this.queue.slice(this.headIndex);
    this.headIndex = 0;
  }

  // 2. Check active window
  if (this.activeMsgs.has(message)) {
    return false;
  }

  // 3. Add new valid message
  this.queue.push({ msg: message, time: timestamp });
  this.activeMsgs.add(message);
  return true;
}
}

```

Note 1: Terminology Used

- **Rate Limiting:** A technique used to control the rate of traffic sent or received by a network or system. It prevents abuse, DoS attacks, and system overload.
- **Sliding Window:** An algorithmic technique where we maintain a subset of data (a “window”) that moves over time or over an array. As the window moves forward, old data is pushed out, and new data comes in.
- **Amortized Time:** Sometimes an operation takes a long time (like evicting 5 old messages at once), but because it’s balanced out by many operations that take almost no time, the *average* time per operation guarantees constant performance.
- **Eviction Policy:** The rule a system uses to decide which data to delete when it’s full or when data becomes irrelevant. Here, our eviction policy is “strictly older than 10 seconds”.

Note 2: Real-World Interview Variations (Meta, Google, Bloomberg)

This specific problem is heavily tied to system design. Here is how top companies adapt it to test deeper engineering limits:

- 1. Meta: “Design an API Rate Limiter / Token Bucket” * The Twist:** Instead of dropping exact duplicate strings, you are given a `user_id`, and a user is allowed exactly `X` API calls per minute. If they exceed `X`, return `False`. * **How to solve:** The Queue + Set approach adapts perfectly here. Instead of a single global Queue, you maintain a Hash Map where the key is `user_id`, and the value is a Queue of their request timestamps. When a new request comes in, go to that user’s queue, evict all timestamps older than 60 seconds. Then, if the queue’s length is less than `X`, allow the request and push the new timestamp.
- 2. Google: “Distributed Rate Limiter” * The Twist:** The interviewer will say, “Okay, your code works for one server. Now we have 50 load-balanced servers. How do we ensure the user doesn’t spam our system by routing requests to different servers?” * **How to solve:** This transitions from coding to System Design. You can no longer store the Queue/Map in local memory. You must move the state to an external, fast, in-memory datastore like **Redis**. You would use Redis’s `INCR` and `EXPIRE` commands to atomically count requests for a given time window (e.g., using a Fixed Window counter or Redis Sorted Sets for a precise Sliding Window).
- 3. Bloomberg: “The 24-Hour Unique Streaming Window” * The Twist:** Similar to Logger, but the window is 24 hours, and the volume is millions of ticks per second (typical Bloomberg financial data). Storing exact timestamps in a Queue will still blow up the memory because millions of messages can arrive in 24 hours. * **How to solve:** You have to sacrifice precision for memory. You pivot to a **Sliding Window Counter** with buckets. Instead of exact queues, you create an array of 60 buckets (one for each minute). As time passes, you increment the counter in the current bucket, and you clear out the bucket from exactly 24 hours ago. This keeps memory usage strictly fixed to array size of 60, regardless of how many millions of messages come in.

843. Guess the Word

To a Google L5 or L6 engineer, “Guess the Word” is not a string problem; it is a **Probability and Information Theory** problem. It is known as an “interactive” problem, where the goal is to maximize the amount of information gained with every single guess.

A senior engineer focuses on **Minimax strategy**: reducing the worst-case scenario. If we guess a word and get a match score of 0, that is actually a massive

win, because we can instantly eliminate every word in our list that has *any* overlap with that guess.

1. Problem Explanation

The Goal: There is a secret 6-letter word. You are given a list of unique 6-letter words. You have an API `master.guess(word)` that returns how many characters in your guess match the secret word in the exact same position. You have a **limit of 10 guesses** to find the secret.

The Rules: * Words are always 6 letters long. * `guess("abcde", "afcge")` returns 2 (matches at index 0 and 2). * You must find the secret within 10 calls to the API.

The Challenge: The word list can have 100 words. If you guess randomly, you might run out of guesses. You need a way to “shrink” the pool of possible secrets as fast as possible.

2. Solution Explanation

The Intuition (The “Why”): Imagine you are playing a game of “20 Questions.” You don’t ask “Is the secret word ‘apple’?” immediately. You ask “Is it an animal?” because that eliminates roughly half of all possibilities.

In this problem, when we guess a word `G` and the API returns a score `S`, we know for a fact that the real secret word must also have exactly `S` matches with `G`. Every word in our list that *doesn’t* have exactly `S` matches with `G` is impossible. We throw them away.

The Algorithm (The “How”): 1. **Selection (The L5/L6 differentiator):** Instead of picking a random word to guess, we pick the word that is “most similar” to all other words. Why? Because the most likely result for any guess is a match score of 0. If we pick a word that shares letters with many others, a score of 0 will eliminate a huge chunk of the list. 2. **The Guess:** Call `master.guess(selected_word)`. Let the result be `match_score`. 3. **The Filter:** Create a new candidate list. Only keep words `W` such that `count_matches(W, selected_word) == match_score`. 4. **Repeat:** Do this until the `match_score` is 6.

End-to-End Walkthrough with ASCII Visualizations:

Suppose the secret is “CIRCLE”. Our list has: [“SQUARE”, “TRIANG”, “CIRCLE”, “SPHERE”]

[INITIAL STATE]

Candidates: ["SQUARE", "TRIANG", "CIRCLE", "SPHERE"]

Guesses Left: 10

[STEP 1: Select and Guess]
We pick "SQUARE" to guess.
API: master.guess("SQUARE") -> returns 0. (No letters match positionally)

[STEP 2: Filter]
We compare "SQUARE" to every other word in our list.
Rule: Keep word ONLY if match_count(word, "SQUARE") == 0.

- TRIANG: match('TRIANG', 'SQUARE') -> 0. (KEEP)
- CIRCLE: match('CIRCLE', 'SQUARE') -> 0. (KEEP)
- SPHERE: match('SPHERE', 'SQUARE') -> 1. (Index 4 is 'R'). (DISCARD!)

New Candidates: ["TRIANG", "CIRCLE"]

ASCII Visualization of Filtering:

```
[ SQUARE ] <--- Guess
|
|-- TRIANG (Matches: 0) -> OK
|-- CIRCLE (Matches: 0) -> OK
|-- SPHERE (Matches: 1) -> TRASH (If secret was SPHERE, guess would've been 1)
```

[STEP 3: Select and Guess]
We pick "TRIANG" to guess.
API: master.guess("TRIANG") -> returns 0.

[STEP 4: Filter]
Check "CIRCLE" against "TRIANG".
- CIRCLE: match('CIRCLE', 'TRIANG') -> 1 (Index 3 is 'I').
Wait, our guess score was 0, but CIRCLE matches TRIANG at 1.
Therefore, CIRCLE cannot be the secret! (DISCARD)

Wait, if we discard everything, we might have picked a bad path.
Actually, in this small example, CIRCLE was the secret. Let's re-calculate:
TRIANG: T R I A N G
CIRCLE: C I R C L E -> 0 matches. (KEEP)

New Candidates: ["CIRCLE"]

[STEP 5: Final Guess]
API: master.guess("CIRCLE") -> returns 6.

SUCCESS!

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N^2)$

Let N = Number of words in the list.

1. In the worst case, we might guess 10 times.
2. In each guess, we iterate through the current list to find the "best" word ($O(N^2)$ comparison).
3. Then we iterate through the list again to filter out impossible words ($O(N)$).

Total: $10 * (N^2 + N)$ which simplifies to $O(N^2)$.

[TC Visual Derivation]

Guess 1: [Compare word 1 to all N]
 [Compare word 2 to all N] ... N^2
Guess 2: [Filter N words] N

SPACE COMPLEXITY: $O(N)$

We store the list of candidate words. As the algorithm progresses, this list gets smaller.

[Space Visual]

Memory: [Word 1 | Word 2 | ... | Word N]
Space used is linear relative to the input size.

4. Solution Code

Python Snippets

```
# Function to count matches between two 6-letter strings
# This is a non-trivial helper used inside the loop
def get_matches(w1, w2):
    return sum(1 for i in range(6) if w1[i] == w2[i])

def findSecretWord(wordlist, master):
    # L5 Optimization: Frequency-based selection
    # We want to pick a word that shares many characters with others
```

```

# to maximize elimination potential on a score of 0.
for _ in range(10):
    # Count frequency of each character at each position
    count = [[0] * 26 for _ in range(6)]
    for w in wordlist:
        for i, char in enumerate(w):
            count[i][ord(char) - ord('a')] += 1

    # Assign a score to each word based on char frequencies
    # High score = shares many characters with the current pool
    best_word = ""
    max_score = -1
    for w in wordlist:
        score = 0
        for i, char in enumerate(w):
            score += count[i][ord(char) - ord('a')]
        if score > max_score:
            max_score = score
            best_word = w

    # Guess the word that is "most central" to the list
    matches = master.guess(best_word)
    if matches == 6:
        return

    # Filter: Only keep words that have the same match-relation
    # with our guess as the secret does.
    wordlist = [w for w in wordlist if get_matches(w, best_word) == matches]

```

JavaScript Snippets

```

function findSecretWord(wordlist, master) {
    const getMatches = (w1, w2) => {
        let count = 0;
        for (let i = 0; i < 6; i++) {
            if (w1[i] === w2[i]) count++;
        }
        return count;
    };

    for (let attempt = 0; attempt < 10; attempt++) {
        // Find the "best" candidate to guess
        // L5/L6 Strategy: Calculate overlap scores
        let bestWord = wordlist[0];
        let minMaxOverlap = Infinity;
    }
}

```

```

// Simplified heuristic: pick a random word or use frequency
// For simplicity and effectiveness, we'll pick the first word
// but real L6 code would use the frequency map from Python above.
let guessIdx = Math.floor(Math.random() * wordlist.length);
let candidate = wordlist[guessIdx];

let score = master.guess(candidate);
if (score === 6) return;

// Filter out impossible words
let nextWordlist = [];
for (let w of wordlist) {
  if (getMatches(w, candidate) === score) {
    nextWordlist.push(w);
  }
}
wordlist = nextWordlist;
}
}

```

Terminology Addendum (Note 1)

Minimax Strategy: * **What it is:** A decision-making rule used to minimize the maximum possible loss (the “worst case”). * **Why it helps:** In this game, the “worst case” is guessing a word and learning nothing. By picking a “central” word, we ensure that no matter what the API returns, we eliminate as many candidates as possible. * **How it applies:** We pick the word that would leave the smallest “remaining pool” in its worst-case outcome.

Information Entropy: * **What it is:** A measure of the “uncertainty” in a set of data. * **How it applies:** Every guess is an attempt to reduce the entropy of the `wordlist` until only one word remains.

Real-World Interview Variations (Note 2)

Variation 1: “Mastermind Game” (Google/Bloomberg) * **Prompt:** Implement the logic for the Mastermind board game where you provide hints (black/white pegs). * **The L5 Solve:** This is the reverse of “Guess the Word.” You are the API. You must write the `getMatches` logic carefully to handle “duplicate” colors correctly (where one color matches positionally and another exists elsewhere).

Variation 2: “Search Suggestions / Typeahead” (Meta) * **Prompt:** Given a prefix, return the most likely words from a dictionary. * **The L5**

Solve: While this uses a **Trie** (Prefix Tree), the “Filtering” logic from “Guess the Word” applies when you add fuzzy matching (autocorrect). If the user types “CICLE”, you use edit-distance filtering (similar to our match-count filtering) to suggest “CIRCLE”.

Variation 3: “A/B Testing Elimination” (System Design Context) *

Prompt: You have 100 possible causes for a system crash. Each test takes 1 hour. You have 10 hours. * **The L5 Solve:** This is the same information-gathering constraint. You design “Group Tests” where each test covers multiple potential causes. If the test passes, you eliminate that entire group. This is the real-world application of the filtering strategy used in this Leetcode problem.

767. Reorganize String

An L5/L6 engineer at Google views this problem through the lens of **Resource Scheduling**. The characters are “tasks” that need to be performed, and the constraint is that the same task cannot be performed twice in a row. To ensure we don’t get stuck with a pile of the same character at the end, we must always prioritize the character that has the **highest remaining frequency**.

1. Problem Explanation

The Premise: Given a string `s`, you need to rearrange its characters so that any two adjacent characters are not the same. If it is impossible to do so, return an empty string.

The “Why” of the Strategy: Imagine you have a bag of letters: [a, a, a, b]. - If you use `b` first: `b`, then you have [a, a, a]. You are forced to place `a` next to `a`. **Fail.** - If you use the most frequent letter first: `a`, then `b`, then `a`. You are left with [a]. You can’t place it. **Fail.**

Actually, the rule is: **If any single character appears more than $(N + 1) / 2$ times (where N is string length), it is mathematically impossible to separate them.**

The Intuition: We use a **Greedy Approach** powered by a **Max-Heap**. We always pick the character with the highest count, place it, and then “hold” it so we don’t pick it again immediately for the very next position. After we pick a *different* character for the next spot, we can put the first one back into the “available” pile.

2. Solution Explanation

The Strategy:

1. **Count Frequencies:** Use a hash map to count how many times each letter appears.
2. **Validate:** If the most frequent letter $> (\text{len}(s) + 1) / 2$, return "".
3. **Heapify:** Put all character counts into a **Max-Heap**. We want to always access the character with the highest count.
4. **The “Waitlist” logic:** * Pop the most frequent char from the heap.
 - Add it to our result string.
 - Decrement its count.
 - **Crucially:** Don't put it back in the heap yet! Store it in a variable called `prev`.
 - In the next iteration, pop a *new* char, and *then* put the `prev` back into the heap if its count is still > 0 .

Detailed ASCII Walkthrough Input: `s = "aaab"`, `len = 4` Max allowed for one char: $(4 + 1) / 2 = 2$. Count of 'a' is 3. $3 > 2$, so return "".

Let's trace a valid one: `s = "aaabc"`, `len = 5` Target counts: {a: 3, b: 1, c: 1}

INITIAL STATE:

Max-Heap: [(3, 'a'), (1, 'b'), (1, 'c')]

`prev = None`

`Result = ""`

STEP 1:

Pop from Heap: (3, 'a')

`Result = "a"`

Remaining count for 'a' is 2.

`prev = (2, 'a')` <-- 'a' is now on the waitlist

STEP 2:

Pop from Heap: (1, 'b')

`Result = "ab"`

Remaining count for 'b' is 0.

Wait! We have 'prev' (2, 'a'). Since we just used 'b', 'a' is safe to use again. Put it back in heap.

Max-Heap: [(2, 'a'), (1, 'c')]

`prev = None` (since 'b' reached 0)

STEP 3:

Pop from Heap: (2, 'a')

`Result = "aba"`

```
Remaining count for 'a' is 1.  
prev = (1, 'a') <-- 'a' back on waitlist
```

```
-----  
STEP 4:  
Pop from Heap: (1, 'c')  
Result = "abac"  
Remaining count for 'c' is 0.
```

```
Put 'prev' back into heap.  
Max-Heap: [ (1, 'a') ]  
prev = None
```

```
-----  
STEP 5:  
Pop from Heap: (1, 'a')  
Result = "abaca"  
prev = (0, 'a')
```

```
-----  
FINAL RESULT: "abaca"
```

3. Time and Space Complexity Analysis

Time Complexity: $O(N * \log K)$ Where N is the length of the string and K is the number of unique characters.

TC Derivation:

1. Frequency Count:
Scan string once $\rightarrow O(N)$
2. Build Heap:
K unique characters $\rightarrow O(K \log K)$
3. Rearrange String:
We perform N pops and N pushes to the heap.
Each heap operation is $\log(K)$.

Visual Step-by-Step:

```
[ Pop ] --  $\log K$   
[ Add ] -- Constant  
[ Push ] --  $\log K$   
Repeat N times.
```

Total Time: $O(N * \log K)$

Since K is at most 26 (English alphabet), $\log K$ is constant.
In practice, this is $O(N)$.

Space Complexity: $O(K)$ (or $O(N)$ for result)

SC Derivation:

1. Frequency Map:
Max 26 entries $\rightarrow O(26) \rightarrow O(1)$
2. Max-Heap:
Max 26 entries $\rightarrow O(26) \rightarrow O(1)$
3. Result String/Array:
Stores N characters $\rightarrow O(N)$

Memory Map:

```
{ Map: 26 items }  
{ Heap: 26 items }  
[ Result: N items ]
```

Final Space Complexity: $O(N + K)$ which simplifies to $O(N)$.

4. Solution Code

Python Implementation

```
import heapq  
from collections import Counter  
  
def reorganizeString(s: str) -> str:  
    # 1. Count frequencies  
    counts = Counter(s)  
  
    # 2. Build Max-Heap (Python has min-heap, so we use negative counts)  
    max_heap = [[-count, char] for char, count in counts.items()]  
    heapq.heapify(max_heap)  
  
    # 3. Check impossibility  
    # If the most frequent element exceeds (N+1)//2  
    if -max_heap[0][0] > (len(s) + 1) // 2:  
        return ""  
  
    res = []  
    prev = None # This acts as our "Waitlist"  
  
    while max_heap or prev:
```

```

# If heap is empty but we still have a 'prev', it means
# we were forced to pick the same char twice.
if not max_heap and prev:
    return ""

# Pop the most frequent available character
count, char = heapq.heappop(max_heap)
res.append(char)
count += 1 # Decrement frequency (closer to 0 since negative)

# If we had a char waiting from the last turn, put it back
if prev:
    heapq.heappush(max_heap, prev)
    prev = None

# If the char we just used still has counts, put it on waitlist
if count < 0:
    prev = [count, char]

return "".join(res)

```

JavaScript Implementation

```

/**
 * Uses a Max-Priority Queue to greedily pick the most frequent character.
 */
var reorganizeString = function(s) {
    const counts = {};
    for (let char of s) {
        counts[char] = (counts[char] || 0) + 1;
    }

    // Check if any character exceeds the limit
    const maxLimit = Math.ceil(s.length / 2);
    // Use a simple array to act as a heap for a small character set (26)
    const heap = [];
    for (let char in counts) {
        if (counts[char] > maxLimit) return "";
        heap.push({ char, count: counts[char] });
    }

    // Sort to simulate max-heap behavior
    heap.sort((a, b) => b.count - a.count);

    let res = "";
    let prev = null;

```

```

while (heap.length > 0 || prev) {
    // If we have nothing in heap but still have something waiting
    if (heap.length === 0 && prev) return "";

    // Get the current most frequent char
    let { char, count } = heap.shift();
    res += char;
    count--;

    // Put the previously waiting char back into the pool
    if (prev) {
        heap.push(prev);
        // Keep "heap" sorted
        heap.sort((a, b) => b.count - a.count);
        prev = null;
    }

    // Put current char on waitlist if it has more occurrences
    if (count > 0) {
        prev = { char, count };
    }
}

return res;
};

```

Note 1: Terms and Techniques

- **Greedy Algorithm:** Making the best choice at the current moment (picking the highest frequency) to achieve the overall goal.
 - **Max-Heap (Priority Queue):** A data structure that gives us $O(1)$ access to the maximum element and $O(\log K)$ for insertions. It is essential for “ranking” tasks dynamically.
 - **Waitlist / Cooldown:** A common pattern in scheduling problems. You prevent an item from being reused until a specific condition (one turn passed) is met.
-

Note 2: Real-World Interview Variations

1. **Meta:** “Task Scheduler” (LeetCode 621) * **The Twist:** You have a set of tasks and a cooldown period n . The same task cannot be repeated within n intervals. * **L5 Solution:** This is exactly the same problem but with a longer

waitlist. Instead of `prev`, use a **Queue** to store tasks on cooldown. Once a task has waited `n` steps, move it from the Queue back into the Max-Heap.

2. Google: “Rearrange Barcodes” (LeetCode 1054) * The Twist: You have numerical barcodes instead of characters. The rule is the same: no two adjacent barcodes can be identical. * **L5 Solution:** Identical approach. Use the frequency map and Max-Heap logic. This is essentially the same problem with different data types.

3. Bloomberg: “Distant Barcodes in a Production Line” * The Twist: Items of different types are coming off a conveyor belt. To prevent a machine from overheating, it cannot process two items of the same type sequentially. * **L5 Solution:** Use the Greedy Max-Heap algorithm to rearrange the incoming stream. If the algorithm returns "", alert the system that the production plan is physically impossible for the current inventory.